ROBOT KIT MANUAL

Mobile Robot Setup Manual

Raspberry Pi 5 · ROS 2 Jazzy · from a blank SD card to mapping and autonomous navigation

A blank SD card to a robot that drives from a phone, logs its encoders, tunes its own PID, maps a room and takes navigation goals.

Built from the NarrowAisleBot project (Aritra Das, IIT Bombay), rewritten so the motors, encoders, microcontroller and drive type are inputs you provide rather than things you inherit.

Version 1.0 · 12 November 2026 · ships with `robot_kit/`

Contents

1. Before you start

1.1 What you have

1.2 What has been tested, and what hasn't

1.3 Habits that saved us time

2. How the system fits together

2.1 Two computers, one cable

2.2 How a command reaches the wheels

2.3 Axes: use REP-103 and nothing else

2.4 Frames

3. Describe your robot: robot_config.yaml

3.1 robot:

3.2 wheels:

3.3 driver:, mcu:

3.4 lidar:, teleop:, slam:, nav:, system:

3.5 What generate.py writes

4. Choosing and wiring the hardware

4.1 The Pi

4.2 The microcontroller, and how fast your encoders tick

4.3 Velocity resolution: coarse encoders need a window

4.4 Motor drivers and wiring

5. Ubuntu on the Pi: flash and first boot

5.1 Which image

5.2 Flash the card

5.3 First login

5.4 The install, end to end

5.5 The shell environment

6. Network: let the robot bring its own Wi-Fi

6.1 Why

6.2 Turn it on

6.3 Going online and coming back

6.4 On Ubuntu Server (no NetworkManager)

7. The microcontroller

7.1 What the firmware does

7.2 The protocol

7.3 Flash it from the laptop (Arduino IDE)

7.4 Or flash from the Pi (arduino-cli)

7.5 First contact, and which way is forward for each motor

7.6 Encoder direction and the true counts per revolution

7.7 First closed-loop spin

8. Stable device names (udev)

8.1 Find each device

8.2 When two devices look identical

9. Encoder logging and PID calibration

9.1 What's in the log, and why the timestamps matter

9.2 Practise on the simulator first

9.3 Test 1: the plant (do this first)

9.4 Test 2: the staircase, in the air and then on the floor

9.5 Test 3: steps, the verdict on a gain set

9.6 Test 4: the sweep, and why you shouldn't trust the formula

9.7 Make the numbers permanent

9.8 Picking max_wheel_speed: leave the PID some room

9.9 Your own firmware: raw mode

9.10 Looking at the results

9.11 Logging while driving under ROS

10. The ROS 2 base: bring-up, driving, odometry

10.1 Build, then start by hand the first time

10.2 Drive it

10.3 Does every direction mean what it says?

10.4 Calibrate odometry with a tape measure

10.5 Start at boot

11. The dashboard

12. The LiDAR

12.1 Choosing and mounting it

12.2 Driver parameters

12.3 Bring it up and check it

12.4 Is it pointing the right way?

12.5 Masking the robot's own body

13. Mapping

13.1 The order that works
13.2 What a good map needs from you
13.3 Checking the result
13.4 Scan matching on or off?

14. Autonomous navigation (work in progress)

14.1 What starts
14.2 Order of operations
14.3 Parameters that matter first
14.4 How to debug a goal that doesn't arrive
14.5 What we've seen on NarrowAisleBot
14.6 The ladder
14.7 What comes next

15. Troubleshooting

16. Daily use

16.1 A session, start to finish
16.2 The health check
16.3 Seeing the robot from a laptop
16.4 Command reference

Appendix A. Serial protocol reference

Appendix B. How this kit was checked

Appendix C. Source listings

Your robot
Generator
Setup scripts
Microcontroller firmware
ROS 2 package robot_base
Dashboard
Tools

1. Before you start

You'll need a Raspberry Pi 5, a microSD card, your robot (motors, encoders, a microcontroller, a motor driver, a 2D LiDAR) and a laptop on the same desk. Put the robot on blocks so the wheels hang free. Almost every test in chapters 7 and 9 assumes the wheels are in the air, and the first time a sign is wrong you'll be glad they were.

1.1 What you have

The kit is a folder, `robot_kit/`. Everything specific to your robot lives in one file inside it, `robot_config.yaml`. A script, `generate.py`, reads that file and writes every other robot-specific file: the URDF, the ROS parameter files, the Nav2 and SLAM configs, the udev rule and the microcontroller's header. You never edit those by hand. Change the YAML, run the script, rebuild.

Folder	What's in it
`robot_config.yaml`	your robot, the one file you edit (chapter 3)
`generate.py`	turns it into everything else
`examples/`	a 4WD skid steer on an Arduino Mega, and NarrowAisleBot itself, for reference
`setup/`	numbered install scripts for a fresh Pi, a USB device lister, a health check
`firmware/robot_mcu/`	the microcontroller sketch, for ESP32, AVR, RP2040 or Teensy
`ros2/robot_base/`	the ROS 2 package: bridge, kinematics, odometry, scan filter, dashboard, launch files
`tools/`	the encoder logger (PID calibration), the CSV analyser, the firmware simulator
`tests/`	the checks that were run on all of the above (Appendix B)

Copy the folder to the Pi's home directory so it sits at `~/robot_kit`. If you got it as a zip, on your laptop:

```
scp robot_kit.zip <user>@<pi-address>:~
```

and on the Pi:

```
cd ~ && unzip robot_kit.zip && cd robot_kit && ls
```

1.2 What has been tested, and what hasn't

I'd rather you know this now than discover it on the bench.

Part	Status
Kinematics (differential, skid steer, mecanum, omni)	Tested in software. Reproduces NarrowAisleBot's asymmetric mecanum equations exactly.

Part	Status
`generate.py`	Tested on three different robots, including bad configs it must reject.
Firmware logic (PID, feedforward, parser, telemetry, watchdog, E-STOP, trips)	Compiled from the real sketch with g++ and run against simulated motors with real quadrature edges, 60 checks on three robots. **Not yet flashed on hardware.** The ESP32-only lines (PCNT, ledc, FreeRTOS tasks) are NarrowAisleBot's, which do run on hardware.
Encoder logger	Run end to end against that simulator over a pseudo-terminal. Recovers the simulated motor's constants (Kff within 0.3 %, Kstat within 0.1 PWM).
Dashboard	Run in headless Chromium in mock mode: 21 checks, no console errors.
ROS 2 nodes and launch files	Written against Jazzy APIs, compiled and lint-clean, structure copied from what runs on NarrowAisleBot. **Not run under ROS here**, because the machine that built this kit had no ROS. First bring-up on your Pi is the real test.
Nav2 configuration	A starting point. Ours doesn't reach goals reliably yet (chapter 14).

So: the parts you can't easily debug (the control maths, the logger, the protocol) were checked hard. The ROS glue is simple and close to code we run every day, but it hasn't met a real ROS install in this exact form. Chapter 10 walks through bringing it up one node at a time for that reason.

1.3 Habits that saved us time

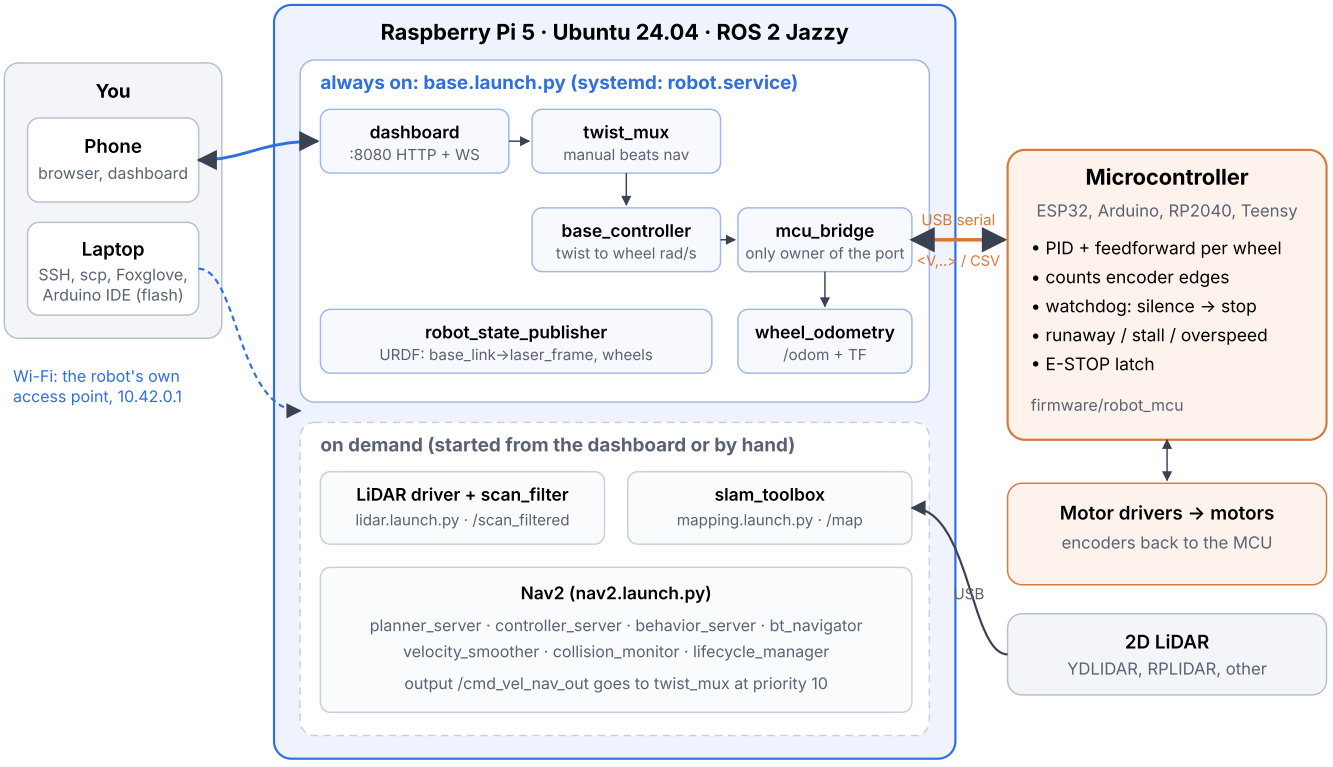
These come from a year of NarrowAisleBot sessions. Each one is here because ignoring it once cost a day.

1. One step at a time, and look at the actual output before the next step. Never assume a command worked because it printed nothing.
2. Ask the live system, not a file. `ros2 param get /node name` tells you what is running; a YAML file tells you what someone meant to run. We once drove for three days on SLAM defaults because a tuned file never reached the Pi. `setup/check_robot.sh` does this for you.
3. When you copy a file to the Pi, hash it on both ends (`sha256sum file`), per file. Batches hide the one that didn't arrive.
4. Write your prediction down before a test or a drive. "It should reach the goal, 60 % sure" takes ten seconds and turns a surprise into information.
5. Keep a dated log of what you changed and what you measured. Your future self is the main reader.
6. The Pi doesn't need internet for routine work. Get it online for installs, then go back to the robot's own Wi-Fi (chapter 6).

2. How the system fits together

2.1 Two computers, one cable

The microcontroller does the fast, hard real-time work: it counts encoder edges, runs a velocity loop on every wheel at 50 to 100 Hz, and stops the motors by itself if the Pi goes quiet. The Pi does everything that needs a map, a plan or a person: kinematics, odometry, SLAM, navigation, the phone dashboard. Between them is one USB serial cable and a small text protocol.

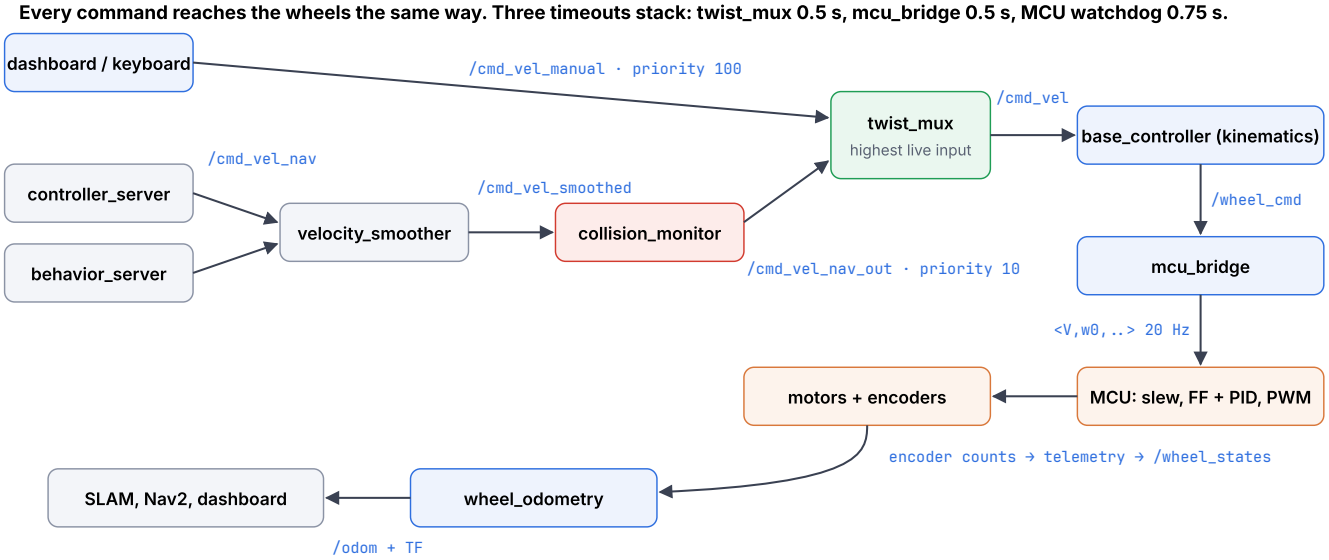


The whole system. Blue is the Pi, orange is the microcontroller and what it drives. Dashed boxes start only when you ask.

That split is deliberate. If the Pi crashes, reboots or just gets slow, the wheels don't care for 750 ms and then they stop. If you need to change how the robot plans a path, you never touch the firmware.

2.2 How a command reaches the wheels

Every way of driving, whether it's your thumb on the phone or Nav2, ends at the same node, `base_controller`, through `twist_mux`. Manual input has priority 100 and navigation has 10, so the moment you touch the joystick you own the robot, and you keep it until 0.5 s after you let go.



The command chain. The collision monitor sits on the autonomous path only; manual driving is your responsibility.

Topic names, so you can find them with `ros2 topic list`:

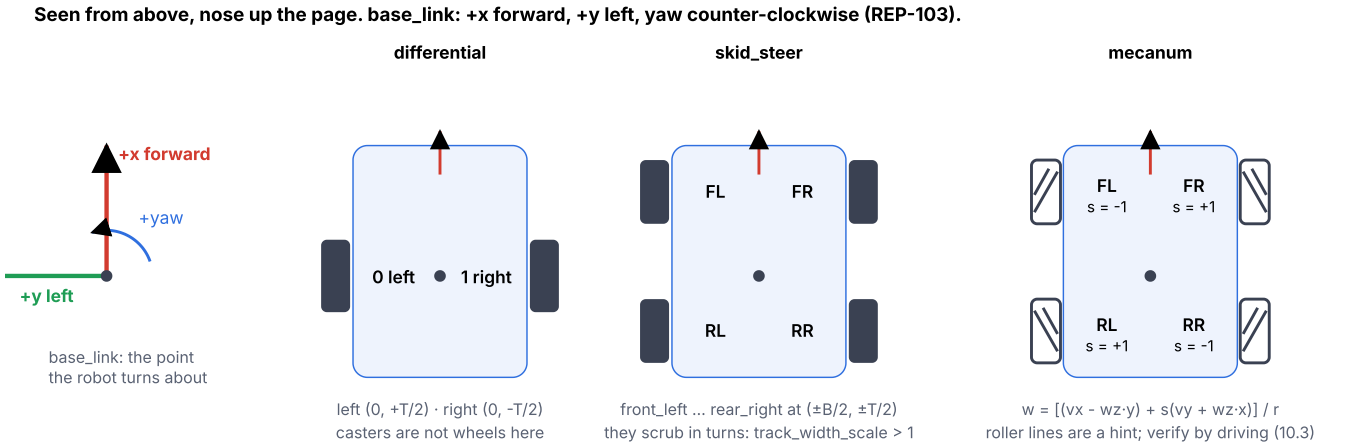
Topic	Type	From	To
`/cmd_vel_manual`	Twist	dashboard, keyboard teleop	twist_mux
`/cmd_vel_nav_out`	Twist	collision_monitor (Nav2)	twist_mux
`/cmd_vel`	Twist	twist_mux	base_controller
`/wheel_cmd`	Float64MultiArray	base_controller	mcu_bridge
`/wheel_states`	JointState (position rad, velocity rad/s, effort = PWM)	mcu_bridge	wheel_odometry
`/mcu/telemetry`	Float64MultiArray `[mcu_ms, values...]`	mcu_bridge	dashboard, telemetry_logger
`/mcu/status`	String (every `[...]` line from the firmware)	mcu_bridge	dashboard
`/mcu/command`	String, raw protocol	dashboard	mcu_bridge
`/odom` + TF `odom → base_link`	Odometry	wheel_odometry	SLAM, Nav2, dashboard
`/odom/reset`	Empty	dashboard	wheel_odometry
`/scan` → `/scan_filtered`	LaserScan	LiDAR driver → scan_filter	SLAM, costmaps, dashboard
`/map` + TF `map → odom`	OccupancyGrid	slam_toolbox	Nav2, dashboard
`/goal_pose`	PoseStamped	dashboard	bt_navigator

2.3 Axes: use REP-103 and nothing else

The ROS convention (REP-103) is +x forward, +y left, +z up, and yaw counter-clockwise when you look down on the robot. Every Nav2 plugin, RViz, Foxglove, slam_toolbox and every recovery behaviour assumes it.

On NarrowAisleBot we didn't follow it. Our `base_link` has +x to the robot's right and +y forward, a choice made early so the LiDAR calibration we already had would line up. It looked like a one-line decision. It has since needed an adapter node on the velocity path, a second one for goals, swapped values throughout the Nav2 file, two MPPI critics switched off, and a recovery behaviour (BackUp) that strafes sideways instead of reversing. The first autonomous goal went 0.956 m at 88° to the direction asked for. Our own config file calls it "the stack's deepest open issue".

Don't repeat it. This kit is REP-103 from the URDF down, and the generator writes stock-shaped Nav2 values. If your LiDAR is mounted backwards or reports angles the wrong way round, that's fixed where it belongs: a yaw in the URDF mount, or the `mirror` flag in the scan filter (chapter 12), never by redefining the robot's axes.



Axes and the three common wheel layouts. The wheel index is the order in robot_config.yaml, which is also the order on the wire.

2.4 Frames

Parent → child	Published by	Notes
`map → odom`	slam_toolbox	only while mapping. AMCL would publish it in saved-map mode; never run both.
`odom → base_link`	wheel_odometry	the robot's dead-reckoned pose; the only pose source until an IMU and EKF arrive
`base_link → laser_frame`	robot_state_publisher (URDF)	the measured LiDAR mount. No other node may publish it.
`base_link → chassis, wheel_*`	robot_state_publisher	fixed joints, so they always exist in TF

`base_link` in this kit sits on the floor directly under the point the robot turns about. For a differential drive that's the middle of the axle. Putting it on the floor means every height in the URDF is a height you can measure with a ruler.

3. Describe your robot: robot_config.yaml

Open `~/robot_kit/robot_config.yaml` in an editor. The example values are for a small two-wheel ESP32 robot and they are wrong for yours. Go through it top to bottom with a tape measure and the datasheets beside you. When you finish:

```
cd ~/robot_kit
python3 generate.py --check
```

It prints a table of wheel positions, counts per revolution and the top speed your limits allow, plus any warning. A `CONFIG ERROR` means it refused the file and told you why. When `--check` is clean, run it without the flag to write the files.

3.1 robot:

`drive_type` is one of four:

drive_type	Wheels	Can strafe	Notes
`differential`	exactly 2 driven	no	casters don't count, leave them out
`skid_steer`	4 or 6	no	wheels scrub when turning; `track_width_scale` corrects odometry
`mecanum`	exactly 4	yes	`lateral_scale` corrects strafe odometry
`omni`	any, usually 3 or 4	yes	every wheel needs explicit x, y and `drive_angle_deg`

`wheel_radius` matters more than anything else in this file, because every distance the robot reports is proportional to it. Don't trust the number printed on the box. Mark the tyre, roll the robot (loaded, on the real floor) ten full turns in a straight line, measure the distance, and divide by 20π. You'll refine it again in section 10.4.

`track_width` is centre-to-centre between a left wheel and the matching right wheel. `wheelbase` is front axle to rear axle; leave it at 0 for a two-wheel differential.

`body` is the box that must never touch anything. Measure the widest point including the wheels. `center_x` is where that box's centre sits relative to `base_link`: for a differential robot with the axle at the back, the box centre is forward of the axle and `center_x` is positive. `footprint_margin` is padding Nav2 adds on every side; 3 cm is a sensible start, and NarrowAisleBot uses 6.

3.2 wheels:

One entry per driven wheel, **in the order your microcontroller numbers its motors**. That order is fixed forever: it's the order of every velocity command and every telemetry column, and the bridge refuses to drive if the firmware reports different names.

`x` and `y` can be `auto` if you name the wheels `left`/`right` (differential) or `front_left`, `front_right`, `rear_left`, `rear_right` and optionally `middle_left`/`middle_right` (skid steer, mecanum). The short forms `fl`, `fr`, `rl`, `rr` work too. Anything else needs numbers: the wheel's contact point in `base_link`, in metres. NarrowAisleBot's asymmetric layout is in `examples/nab_mecanum_reference.yaml` if you want to see explicit positions in use.

The encoder fields deserve care, because they fail silently.

`counts_per_rev` is counts per **wheel** revolution, as the firmware counts them. With `auto` the generator multiplies `encoder_ppr × encoder_decoding × gear_ratio`:

- `encoder_ppr`: pulses per motor-shaft revolution on one channel. Datasheets call this PPR, CPR, "lines" or "resolution" and they don't agree on what any of those mean. Treat it as a guess until section 7.6 measures it.
- `encoder_decoding`: 4 means the firmware counts every rising and falling edge of both A and B (the most resolution). 2 counts both edges of A only, half the counts and half the interrupt load.
- `gear_ratio`: motor turns per wheel turn.

A story about why this matters. NarrowAisleBot's front motors got new encoders with twice the resolution of the rear ones, while the firmware still used one counts-per-rev number for all four. The front wheels therefore reported double their real speed, the PID believed them, and drove the front wheels at half the commanded speed. Every telemetry plot looked perfect, because the loop was tracking exactly what it was told. The only thing that caught it was a bench test comparing raw front and rear counts. Since then `counts_per_rev` is per wheel, and so is it here.

`motor_dir` and `encoder_dir` are +1 or -1. They're set by two separate bench tests in sections 7.5 and 7.6, so leave them at your best guess for now. (Right-side motors are usually mounted mirrored and need -1 for both.)

`pins` depends on your motor driver type, below. Encoder pins are `enc_a` and `enc_b` for every type.

`mecanum_sign` is optional for mecanum wheels named in the standard way; section 10.3 tells you whether the default is right. `drive_angle_deg` (omni only) is the direction the wheel pushes the robot, measured counter-clockwise from +x.

3.3 driver:, mcu:

`driver.type` describes how one motor is controlled; section 4.4 has the wiring. `pwm_dir` uses keys `pwm` and `dir`; `pwm_in1_in2` uses `pwm`, `in1`, `in2`; `dual_pwm` uses `pwm_fwd` and `pwm_rev`.

The `mcu` block, in the order you'll care about it:

Key	Meaning	Start with
`board`	`esp32`, `avr`, `rp2040`, `teensy`, `other`	what you have
`encoder_backend`	`pcnt` (ESP32 hardware counters) or `isr` (interrupts, any board)	`pcnt` on a classic ESP32, else `isr`
`baud`	serial speed	921600 ESP32, 115200 AVR
`device_name`	the `/dev/` name udev will create	`robot_mcu`

Key	Meaning	Start with
`usb_*`	the board's USB identity, from `setup/list_usb_serial.py` (chapter 8)	fill in later
`control_hz`	PID rate	100 ESP32, 50 AVR
`telemetry_hz`	default telemetry rate	20
`watchdog_ms`	silence longer than this stops the wheels	750
`max_wheel_speed`	rad/s clamp in the firmware	from your motor; see 9.8
`max_wheel_accel`	rad/s² setpoint slew limit	10 to 20
`pwm_bits`, `pwm_freq_hz`	PWM resolution and frequency	8 bit, 20 kHz (AVR ignores the frequency)
`pid`, `kff`, `kstat`	controller gains	the soft defaults, until chapter 9 replaces them

3.4 lidar:, teleop:, slam:, nav:, system:

`lidar.model` picks the driver (`ydlidar`, `rplidar`, `none`, `custom`). `mount` is the laser's centre in `base_link` (height above the floor for z) and its yaw. `mirror` and `mask_deg` are explained in chapter 12; leave them alone until then.

`teleop.max_linear` and `max_angular` cap the phone joystick. Start slow (0.15 m/s) and raise it when you trust the robot.

`slam` and `nav` are explained in chapters 13 and 14. `nav.controller: auto` picks Regulated Pure Pursuit for differential and skid steer, and MPPI for holonomic bases.

`system.wifi_ap` turns the Pi into its own access point (chapter 6). `system.autostart` starts the base stack at every boot.

3.5 What generate.py writes

File	Used by
`ros2/robot_base/config/robot.yaml`	every kit node (bridge, controller, odometry, scan filter, dashboard)
`ros2/robot_base/config/nav2_params.yaml`	`nav2.launch.py`
`ros2/robot_base/config/slam_toolbox.yaml`	`mapping.launch.py`
`ros2/robot_base/config/lidar.yaml`, `kit.yaml`	`lidar.launch.py`
`ros2/robot_base/config/twist_mux.yaml`	`base.launch.py`
`ros2/robot_base/urdf/robot.urdf`	robot_state_publisher
`firmware/robot_mcu/robot_config.h`	the sketch. Reflash after it changes.
`setup/generated/99-robot.rules`, `robot.env`	udev, the setup scripts

Every double-valued ROS parameter is written with a decimal point. That sounds pedantic until you hit it: ROS 2 refuses to start a node whose parameter file says `10` where the node declared a double, with an exception that doesn’t mention which file.

4. Choosing and wiring the hardware

4.1 The Pi

A Raspberry Pi 5 with 8 GB, the official Active Cooler (SLAM plus Nav2 keeps all four cores busy; without the cooler it throttles), and a 64 GB A2 microSD card or an NVMe drive on a HAT.

Power is where Pi robots die quietly. The Pi 5 wants 5 V at 5 A. If it can't confirm a 5 A supply (any supply that isn't a USB-C PD source offering 5 A, including a buck converter wired to the 5 V pins) it limits the USB ports to 600 mA *in total*. A LiDAR motor and a microcontroller can exceed that, and the symptom is a LiDAR that stutters or a board that resets under load, not an error message. If your supply really delivers 5 A, add `usb_max_current_enable=1` to `/boot/firmware/config.txt` and reboot. Then check for undervoltage after a drive: `vcgencmd get_throttled` should print `throttled=0x0` (if the command isn't installed on your Ubuntu image, `dmesg | grep -i voltage` shows the same warnings).

Keep the motors on their own battery. The Pi and the microcontroller's logic run from the Pi's supply; motor current never flows through the Pi. Join the grounds, not the 5 V rails.

A clock that survives power-off helps more than you'd expect, because every log file is named by time. NarrowAisleBot uses a DS3231 module. The Pi 5 also has an onboard RTC that needs a battery on its J5 connector; check `timedatectl` after a power cycle with no network to see whether yours keeps time.

4.2 The microcontroller, and how fast your encoders tick

The firmware needs to see every encoder edge. How many edges per second that is decides which board you can use:

$$\text{edges/s per wheel} = \text{counts_per_rev} \times \frac{\text{max_wheel_speed}}{2\pi}$$

`generate.py` prints exactly this number for each wheel. Then:

Backend	Board	Comfortable total edge rate
`pcnt`	classic ESP32 (WROOM, WROVER)	effectively unlimited: counted in hardware, no CPU cost
`isr`	ESP32, RP2040, Teensy	a few hundred thousand per second across all wheels
`isr`	AVR (Uno, Mega, Nano) at 16 MHz	a few tens of thousands per second across all wheels

Two examples. A JGB37-style motor with an 11-pulse hall encoder, x4 decoding and a 30:1 gearbox gives 1320 counts per wheel turn, about 2100 edges/s per wheel at 10 rad/s: any board copes. NarrowAisleBot's front wheels give 186 264 counts per turn, 154 000 edges/s per wheel: only the ESP32's hardware counters handle four of those.

On AVR, every encoder pin that needs an interrupt must be an external-interrupt pin. On an Uno that's pins 2 and 3 only; on a Mega 2, 3, 18, 19, 20 and 21. With x4 decoding each wheel needs two such pins, with x2 only `enc_a` does. The firmware checks this at boot and prints `[ERR,ENC_PIN,<pin>,not an interrupt pin]`. The ESP32-C3 has no pulse counter at all, so use `isr` there.

My recommendation, if you haven't bought the board yet: a classic ESP32 DevKit. It's what NarrowAisleBot runs, it has hardware counters for up to eight encoders, two cores, and it's cheap.

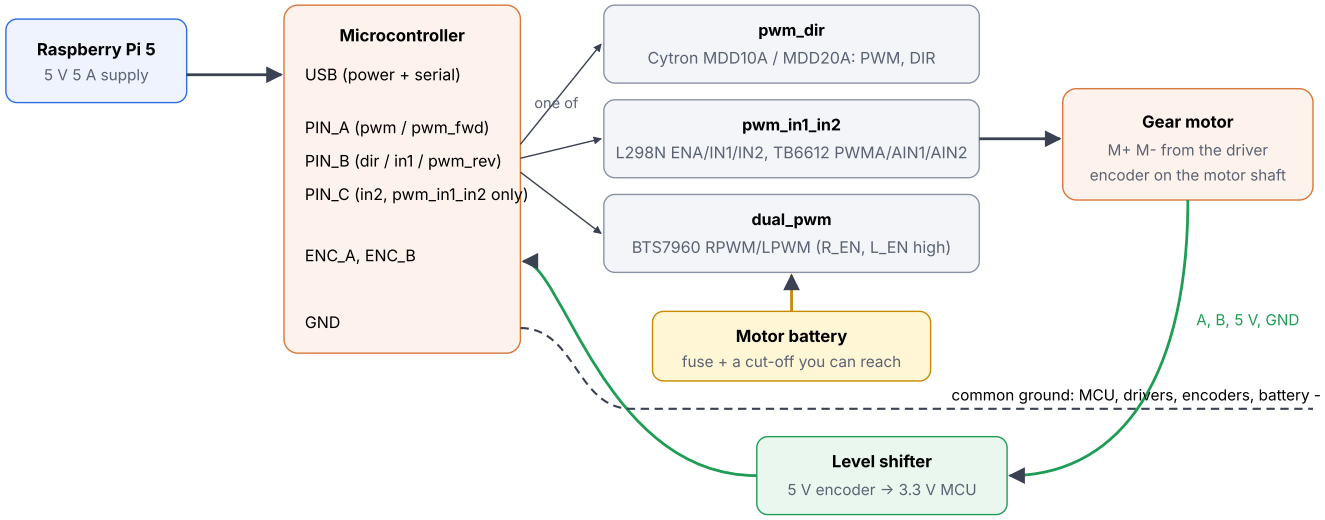
4.3 Velocity resolution: coarse encoders need a window

The firmware measures speed by counting edges per control tick. One count in one tick is a speed quantum of 2π / (counts_per_rev × tick length). At 100 Hz, 1320 counts per turn makes one count worth 0.48 rad/s. A PID chasing a signal that jumps in steps of 0.48 rad/s will chatter.

So the firmware averages over a window of `VEL_WINDOW` ticks, chosen by `generate.py` to bring the quantum to about 0.05 rad/s, capped at 8 ticks. The header records the trade it made, for example: one count is 0.476 rad/s in one tick and 0.06 over 8 ticks, at the cost of about 35 ms of extra lag. NarrowAisleBot's encoders are fine enough that the window is 1.

If the generator warns that your encoders are coarse even after the window, the real fixes are mechanical or electrical: x4 decoding, encoders on the motor shaft rather than the wheel, or a lower `control_hz`.

4.4 Motor drivers and wiring



The Pi and the MCU logic run from the Pi's own 5 V supply; motors from their own battery. Grounds are joined; the 5 V rails are not. ESP32 inputs are 3.3 V only (GPIO 34 to 39 are input-only, no pull-ups). A 5 V AVR board takes 5 V encoders directly.

Wiring. Pick one driver style; the firmware handles all three.

driver.type	Typical boards	Pins per motor	Notes
`pwm_dir`	Cytron MDD10A, MDD20A, most "smart" drivers	PWM, DIR	what NarrowAisleBot uses
`pwm_in1_in2`	L298N, TB6612FNG	PWM (enable), IN1, IN2	both low = coast. L298N drops about 2 V, it gets hot; TB6612 is better below 1.2 A
`dual_pwm`	BTS7960 (IBT-2), DRV8833	one PWM per direction	tie the BTS7960's R_EN and L_EN high

Rules that apply whichever you choose:

- Common ground between the microcontroller, the drivers, the encoders and the battery negative.
- A fuse at the battery, and a switch you can reach without leaning over the robot. With the Pi's dashboard as the only control there is no other way to stop a robot whose Pi has hung, except the firmware

watchdog (which does stop it after 750 ms).

- ESP32 inputs are 3.3 V. Most hall encoders run on 5 V and output 5 V; put a level shifter (a BSS138 board is fine) between them. GPIO 34 to 39 on the ESP32 are input-only and have no pull-ups, which suits encoders with push-pull outputs; open-collector encoders need external pull-ups to 3.3 V.
- Twist each encoder’s A and B with its ground, and keep them away from motor wires. Noise on an encoder line looks exactly like a slipping wheel.

5. Ubuntu on the Pi: flash and first boot

5.1 Which image

Ubuntu 24.04 LTS, 64-bit, because ROS 2 Jazzy is built for it. Two flavours:

- Ubuntu Desktop 24.04, which is what we run. It comes with NetworkManager, which the Wi-Fi access point in chapter 6 needs, and a desktop if you ever plug in a screen. Heavier.
- Ubuntu Server 24.04 is lighter and faster to boot, no desktop. Needs NetworkManager installed before the access point will work (section 6.4).

For a first robot I'd take Desktop. You can always strip it later; adding NetworkManager to a headless Server over Wi-Fi is the one step in this manual that can lock you out.

5.2 Flash the card

On the laptop, install Raspberry Pi Imager from raspberrypi.com/software. Then:

1. Device: Raspberry Pi 5.
2. Operating system: Other general-purpose OS, Ubuntu, then Ubuntu Desktop 24.04.x LTS (64-bit) or Ubuntu Server 24.04.x LTS (64-bit).
3. Storage: your card. Everything on it is erased.
4. If Imager offers "OS customisation" (it does for Server images), set the hostname, a username and password, your Wi-Fi network and country, and enable SSH. Desktop images usually skip this and ask the same questions on first boot instead, so you'll need a screen and keyboard once.
5. Write, then put the card in the Pi and power it.

The first boot takes a few minutes. Desktop runs a setup wizard; pick the same username you'll use everywhere (the scripts use whatever account runs them).

5.3 First login

From the laptop, on the same network:

```
ssh <user>@<hostname>.local
```

If `.local` names don't resolve on your network, find the address on the Pi's screen (`hostname -I`) or in your router's client list. On Desktop, SSH may not be installed yet; on the Pi's own terminal:

```
sudo apt update && sudo apt install -y openssh-server && sudo systemctl enable --now ssh
```

Then set the clock zone and confirm the basics:

```
sudo timedatectl set-timezone Asia/Kolkata
uname -m          # aarch64
lsb_release -a    # Ubuntu 24.04 ... noble
df -h ~           # how much card is left
```

5.4 The install, end to end

With the kit at `~/robot_kit` and `robot_config.yaml` filled in (chapter 3; the USB fields can wait):

```
cd ~/robot_kit
python3 generate.py --check
./setup/install_all.sh
```

That runs five scripts in order. Each one is safe to run again, prints `ok` or `FAIL` per step, and appends everything to `~/robot_kit_install.log`. It takes 30 to 45 minutes on a Pi 5, most of it in step 2. If a step fails, read the last lines of the log, fix the cause and run the same numbered script again; you never need to start over.

Script	What it does
`01_base_system.sh`	checks for Ubuntu 24.04 and internet, `apt upgrade`, build tools, Python serial/YAML/numpy/matplotlib, locale, adds you to `dialout` (serial ports), removes `brltty`, disables `ModemManager`, enables SSH, pip-installs the dashboard's web server (fastapi, uvicorn, websockets, pinned)
`02_ros_jazzy.sh`	adds the ROS 2 apt repository the current official way (the `ros2-apt-source` package), installs `ros-jazzy-ros-base` plus Nav2, slam_toolbox, twist_mux, robot_state_publisher, foxglove_bridge, CycloneDDS, robot_localization, colcon, rosdep
`03_lidar_driver.sh`	YDLIDAR: builds YDLidar-SDK and clones `ydlidar_ros2_driver` (branch `humble`, which builds on Jazzy). RPLIDAR: `apt install ros-jazzy-rplidar-ros`.
`04_workspace.sh`	runs `generate.py`, copies `ros2/robot_base` into `~/ros2_ws/src`, `rosdep install`, `colcon build --symlink-install`, prints hashes of what it deployed
`05_system.sh`	installs the udev rule, adds the ROS environment and `rk-*` aliases to `~/.bashrc`, installs `robot.service` (enabled at boot if `system.autostart` is true)

Two of those lines deserve an explanation. `brltty` is a braille-display service that ships with Ubuntu Desktop and claims CH340 and CP210x USB serial adapters; your board's `/dev/ttyUSB0` appears and vanishes a few seconds later. `ModemManager` sends modem AT commands to every new `/dev/ttyACM*`, which is what an Arduino or a Pico looks like. Both have cost people days. The script removes the first and disables the second.

Want RViz on the Pi itself? Run step 2 as `ROS_PKG=desktop ./setup/02_ros_jazzy.sh`. I wouldn't: Foxglove on the laptop does the same job without loading the Pi (section 16.3).

Log out and back in after the install. The `dialout` group only applies to new logins, and until then every serial port says "permission denied".

5.5 The shell environment

`05_system.sh` adds these to `~/.bashrc`:

```
source /opt/ros/jazzy/setup.bash
source ~/ros2_ws/install/setup.bash
export ROS_DOMAIN_ID=42
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
export CYCLONEDDS_URI='<CycloneDDS><Domain><General><Interfaces><NetworkInterface name="lo"
priority="default" multicast="false"/></Interfaces></General></Domain></CycloneDDS>'
```

The last line pins DDS to the loopback interface. Every node runs on the Pi, and on a headless boot with no LAN cable CycloneDDS can otherwise fail to choose an interface and nodes don't see each other. The cost: a laptop on the network can't see the Pi's topics through DDS. Use Foxglove (it goes through a WebSocket) or SSH instead. If you really want DDS across the network, delete that line on both machines and use the same `ROS_DOMAIN_ID`.

The aliases:

Alias	Does
`rk-gen`	`python3 ~/robot_kit/generate.py`
`rk-build`	regenerate, copy, rebuild (`04_workspace.sh`) and re-source
`rk-start`, `rk-stop`, `rk-restart`, `rk-status`	the systemd service
`rk-log`	follow `~/robot_boot.log`
`rk-check`	the live health check (section 16.2)
`rk-ports`	show `/dev/robot_mcu` and `/dev/lidar`

6. Network: let the robot bring its own Wi-Fi

6.1 Why

Campus Wi-Fi changes addresses, blocks devices from each other, and isn't there in the corridor you want to map. NarrowAisleBot's Pi is its own access point instead: power it on, join its network from the phone, open `http://10.42.0.1:8080`. That address never changes, because NetworkManager's shared mode always gives the host 10.42.0.1 and hands out 10.42.0.x to everyone else.

6.2 Turn it on

In `robot_config.yaml`:

```
system:
  wifi_ap:
    enable: true
    ssid: MyRobot-Pi
    password: something-8-or-more
    country: IN
```

then

```
cd ~/robot_kit && python3 generate.py && ./setup/06_wifi_ap.sh
```

It creates a NetworkManager connection called `robot-ap` with autoconnect priority 10, higher than any normal network, so the AP wins at every boot whatever else is in range. If you ran it over Wi-Fi SSH, your session drops when the radio switches. That's expected. Join `MyRobot-Pi` and reconnect with `ssh <user>@10.42.0.1`.

6.3 Going online and coming back

The Pi has one radio. While it's an access point it has no internet. For an `apt` install or a `git pull`:

```
sudo nmcli con up "<your-wifi-name>"     # SSH drops; reconnect on that network
# ... do the online work ...
sudo nmcli con up robot-ap               # SSH drops again; reconnect on 10.42.0.1
```

`nmcli -t -f NAME,DEVICE con show --active` shows which one is live. An Ethernet cable is the painless alternative: plug it into a router and the Pi has internet on `eth0` while `wlan0` stays the AP.

Moving files without internet works fine. From the laptop, joined to the robot's network:

```
scp robot_config.yaml <user>@10.42.0.1:~/robot_kit/
scp <user>@10.42.0.1:~/robot_logs/*.csv .
```

Then hash both ends (`sha256sum` on the Pi, `certutil -hashfile <file> SHA256` in Windows PowerShell). We lost a day once to a file that "copied" and wasn't the file we thought.

6.4 On Ubuntu Server (no NetworkManager)

Server images use systemd-networkd through netplan. Do this with a screen and keyboard attached, or over Ethernet, never over the Wi-Fi you're about to reconfigure:

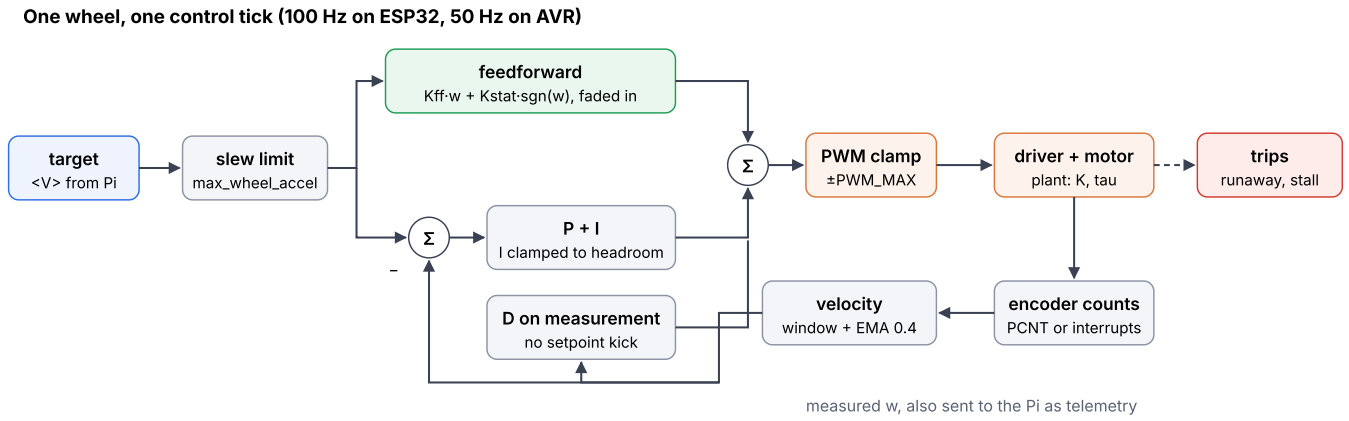
```
sudo apt install -y network-manager
echo 'network: {config: disabled}' | sudo tee /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg
sudo nano /etc/netplan/50-cloud-init.yaml      # add the line   renderer: NetworkManager   under
network:
sudo netplan apply
nmcli device status                            # wlan0 should now be listed as managed
```

The cloud-init line stops it from rewriting netplan at the next boot. After that, `06_wifi_ap.sh` works as on Desktop.

7. The microcontroller

7.1 What the firmware does

`firmware/robot_mcu/robot_mcu.ino` is NarrowAisleBot's ESP32 v3.0 controller, generalised to 1 to 6 motors, three driver styles, two encoder backends and four families of board. Per wheel, per tick:



Inside one control tick.

The target first moves toward what the Pi asked for at no more than `max_wheel_accel`. Asking a motor for an acceleration it can't produce only winds up the integrator, so the slew limit sits in front of everything else.

Feedforward then supplies most of the PWM straight away: `Kff·w` for the viscous part, plus `Kstat·sgn(w)` to break static friction, faded in between 0.05 and 0.20 rad/s so that a tiny command doesn't make the output jump. A PI loop corrects whatever is left. Its integral is clamped to the PWM headroom that actually remains after feedforward and P, recomputed every tick, so it can't wind up. (Our old firmware had a fixed clamp that allowed an integral worth 23 times the whole PWM range. It existed. It never bound.) The derivative term acts on the measured speed rather than the error, so a new setpoint from the Pi doesn't kick the output, and it's small by design.

Three safety trips latch the E-STOP and print `[TRIP,...]`. Overspeed fires beyond 1.3 × the speed limit for 300 ms. Stall fires when PWM sits at 98 % and the wheel hasn't turned for 2 s. Runaway is the important one: PWM saturated one way, the wheel turning the other way, and not slowing down, for 500 ms. That's the signature of a `motor_dir`/`encoder_dir` pair that disagrees, which turns the loop into positive feedback and pins the wheel at full speed backwards. NarrowAisleBot's 60 RPM motors can't even reach the overspeed threshold, so without the runaway check nothing would stop them.

Finally the watchdog. If no motion command (`V`, `B`, `M` or `T`) arrives for `watchdog_ms`, the targets go to zero. Status queries like `<?>` don't count, on purpose, because a monitoring script must never be able to keep a stale velocity alive. Our v3.0 fed the watchdog on any command; this firmware is stricter.

On an ESP32 the control loop runs in its own FreeRTOS task on core 1 and the serial side on core 0, as on NarrowAisleBot. On other boards both run from `loop()`, and all serial output goes through a non-blocking buffer so a long telemetry line can't stall a control tick (on an AVR, a blocking 200-byte line at 115200 baud would freeze the loop for about 17 ms).

7.2 The protocol

Everything the Pi sends is framed `<...>`. Everything the board sends is either a telemetry line (plain CSV starting with a digit) or a reply in square brackets. Appendix A has the full table; the ones you'll type yourself

are:

Send	Effect
`<P>`	`[PONG]`
`<I>`	configuration: `[INFO,...]`, `[MOTORS,N,names]`, gains, Kff, Kstat, CPR, limits
`<?>`	live state of every motor
`<E1>` / `<E0>`	enable (and clear an E-STOP) / disable
`<S>`	E-STOP, latches until `<E1>`
`<V,w0,w1,...>`	wheel speeds in rad/s, one per motor, closed loop
`<M,p0,p1,...>`	raw PWM, open loop
`<W0>` / `<W1>`, `<Y0>` / `<Y1>`	watchdog off/on, trips off/on
`<L1,20>` / `<L2,50>` / `<L0>`	telemetry basic / extended / off, at a rate
`<G,kp,ki,kd>`, `<F,...>`, `<K,...>`	gains, Kff per motor, Kstat per motor (lost at reset)
`<O>` / `<R>`	print / zero the accumulated wheel angles

Telemetry, basic (`<L1>`): `ms, then target, actual, pwm for each motor` (1 + 3N columns). Extended (`<L2>`) appends `error, ff, iterm, pos_rad` for each motor (1 + 7N). `pos_rad` is the exact integral of encoder counts, unfiltered, and it's what the odometry uses.

If you write your own firmware instead, implement at least `<E1>`, `<S>`, `<V,...>`, `<L2,hz>` with this telemetry format, `<I>` with a `[MOTORS,N,names]` line, and the watchdog. Everything on the Pi side will then just work. For a board that only prints its own CSV, the encoder logger's raw mode (section 9.9) still gets you calibration data.

7.3 Flash it from the laptop (Arduino IDE)

1. On the Pi, after `generate.py`, the sketch folder has two files: `robot_mcu.ino` and `robot_config.h`. Copy the folder to the laptop:

   ```
   scp -r <user>@10.42.0.1:~/robot_kit/firmware/robot_mcu .
   ```

2. Install Arduino IDE 2. For an ESP32: Boards Manager, install "esp32 by Espressif Systems" (core 3.x; 2.x also compiles). For AVR, "Arduino AVR Boards" is already there.
3. Open `robot_mcu/robot_mcu.ino`. Select the board (for a typical ESP32 DevKit: "ESP32 Dev Module") and the port.
4. Upload. If an ESP32 never connects, hold the BOOT button while the upload starts.

Every time `robot_config.h` changes (new pins, gains, limits, CPR), reflash.

7.4 Or flash from the Pi (arduino-cli)

The board is plugged into the Pi anyway, so you can skip the laptop. This needs internet once, for the tools and the ESP32 core (I couldn't run these commands on the machine that built the kit; they're the standard arduino-cli ones):

```
curl -fsSL https://raw.githubusercontent.com/arduino/antigravity-cli/master/install.sh |
BINDIR=$HOME/.local/bin sh
arduino-cli config init
arduino-cli config add board_manager.additional_urls https://espressif.github.io/arduino-
esp32/package_esp32_index.json
arduino-cli core update-index
arduino-cli core install esp32:esp32        # or: arduino-cli core install arduino:avr
```

and each time you flash:

```
rk-stop                                     # the bridge must let go of the port
arduino-cli compile --fqbn esp32:esp32:esp32 ~/robot_kit/firmware/robot_mcu
arduino-cli upload  --fqbn esp32:esp32:esp32 -p /dev/robot_mcu ~/robot_kit/firmware/robot_mcu
rk-start
```

For a Mega the FQBN is `arduino:avr:mega`, for an Uno `arduino:avr:uno`. Before the udev rule exists, use `/dev/ttyUSB0` or `/dev/ttyACM0`.

7.5 First contact, and which way is forward for each motor

Wheels in the air. Stop anything that might hold the port (`rk-stop`), then open a serial terminal on the Pi:

```
python3 -m serial.tools.miniterm /dev/ttyUSB0 921600      # Ctrl+] quits
```

(Use your port and your `mcu.baud`. Arduino IDE's Serial Monitor works too.) Press the board's reset button. You should see `[BOOT,...]`, the `[INFO]` block and `[READY]`. Type `<P>` and get `[PONG]`. Type `<I>` and check that `[MOTORS,...]` lists your wheels in your order, `[CPR,...]` has your counts per revolution, and there's no `[ERR,ENC_PIN,...]` line.

Now each motor, one at a time. For wheel 0 on a two-motor robot:

```
<E1>
<W0>
<Y0>
<M,40,0>
```

Wheel 0 should turn the way that would drive the robot **forward**. (Forward for the robot, not clockwise for the motor.) Then stop it:

```
<M,0,0>
```

If it turned backwards, set that wheel's `motor_dir` to the opposite sign. Repeat for every wheel, then `<W1>` and `<Y1>` to put the watchdog and trips back. Use a PWM that turns the wheel slowly; 40 of 255 suits most gear motors, more if yours doesn't move. `<W0>` is needed because `<M>` is a single command and the watchdog would otherwise stop it after 750 ms.

7.6 Encoder direction and the true counts per revolution

This one test settles both the sign and the number the datasheet got wrong.

```
<E0>
<R>
```

`<E0>` disables the motors so the wheels turn freely by hand. Put a piece of tape on wheel 0. Turn it exactly 10 revolutions **forward** (the way that drives the robot forward), slowly, then:

```
<O>
```

The reply is `[ODOM,angle0,angle1,...]` in radians. Ten forward turns should read +62.83 (10 × 2π).

- Negative: flip that wheel's `encoder_dir`.
- Wrong size: your `counts_per_rev` is off. The true value is `old_value × reported / 62.83`. If it's out by a factor of 2 or 4, it's the PPR versus CPR confusion or the wrong `encoder_decoding`; if it's out by something like 1.03, your gear ratio isn't exactly the nominal one (NarrowAisleBot's "47:1" gearbox is 46.566:1).

Put the measured value in `counts_per_rev` (as a number, replacing `auto`), regenerate, reflash, and repeat until ten turns read 62.8 ± 0.2 on every wheel.

7.7 First closed-loop spin

Still in the air:

```
<E1>
<L1,5>
<V,2,2>
```

(one value per motor). Both wheels should spin forward, and within a second or two the `actual` columns should sit near 2.0. They'll wobble and overshoot, because the gains are the soft defaults; chapter 9 fixes that. After 750 ms without a new `<V>` the watchdog stops them. That's correct, and it's why the bridge resends commands at 20 Hz. `<L0>` to stop the telemetry.

If a wheel runs away and you see `[TRIP,RUNAWAY,...]`, its `motor_dir` and `encoder_dir` disagree. Redo 7.5 and 7.6 for that wheel.

8. Stable device names (udev)

8.1 Find each device

Linux names serial devices in the order they appear: the board might be `/dev/ttyUSB0` today and `/dev/ttyUSB1` after you plug in the LiDAR. The udev rule gives each one a permanent name instead (`/dev/robot_mcu`, `/dev/lidar`). Plug in **one device at a time** and run:

```
python3 ~/robot_kit/setup/list_usb_serial.py
```

```
/dev/ttyUSB0   Silicon Labs CP2102 USB to UART Bridge Controller
  (looks like: Silicon Labs CP210x (many ESP32 boards, YDLIDAR and RPLIDAR adapters))
  paste into robot_config.yaml (mcu: or lidar:):
    usb_vendor_id: "10c4"
    usb_product_id: "ea60"
    usb_serial: "0001"
    usb_port_path: ""          # this socket is "4-1"
```

Copy the values into the `mcu:` block, plug in the LiDAR alone, run it again and fill the `lidar:` block. Then `rk-build` (which regenerates the rule) and `./setup/05_system.sh` (which installs it), unplug and replug, and:

```
ls -l /dev/robot_mcu /dev/lidar
```

8.2 When two devices look identical

NarrowAisleBot's ESP32 board and its YDLIDAR adapter use the same CP2102 chip, with the same vendor and product ID and the same factory serial number, `0001`. udev cannot tell them apart by identity, and for a while our logger opened the LiDAR about half the time and waited forever for a reply.

If `list_usb_serial.py` prints the "SAME vendor, product and serial" warning, or `generate.py` warns that the MCU and LiDAR identities match, fill `usb_port_path` for both with the socket values the lister printed. The rule then matches the physical USB socket. The price: those two cables must never swap sockets. Label them.

9. Encoder logging and PID calibration

This chapter replaces “copy the serial monitor into a spreadsheet”. `tools/encoder_logger.py` runs on the Pi, drives a scripted test over the serial port, timestamps every telemetry line, writes a CSV and prints the numbers you need before you’ve unplugged anything.

The serial port can only have one reader, so stop the ROS side first:

```
rk-stop
cd ~/robot_kit
./tools/encoder_logger.py --help
```

It reads `robot_config.yaml` for the port and baud, then asks the firmware for its motor names (`<I>`), so the CSV columns are named after your wheels. Every test except `record` and `raw` asks you to press Enter first; Ctrl+C at any point sends `<S>`, `<L0>` and `<E0>`.

9.1 What’s in the log, and why the timestamps matter

Every CSV starts `pi_time_s, mcu_ms`, then `<wheel>_target_rads, <wheel>_actual_rads, <wheel>_pwm` for each wheel, then `<wheel>_error, _ff, _iterm, _pos_rad` for each wheel. A `.json` file beside it records the test, the commanded steps and their times, and the firmware’s identity, so `tools/analyze_log.py` can redo any calculation later.

`pi_time_s` is the moment the microcontroller *took* the sample, expressed on the Pi’s clock. It isn’t the moment the line arrived. That distinction cost us: NarrowAisleBot’s original logger read the port 512 bytes at a time with a 100 ms timeout, so lines arrived in bursts and were stamped up to 100 ms late. Testing this kit’s logger against a simulated motor with a known 0.090 s time constant, the burst-stamped version measured 0.238 s. Reading promptly, stamping on the MCU’s clock and measuring from the unfiltered wheel angle brought it to 0.110 s; the remaining 20 ms is one control tick of dead time plus sampling, and it’s documented in the output.

9.2 Practise on the simulator first

`tools/firmware_sim` compiles the **real** firmware against simulated motors (first-order, with static friction) and real quadrature edges, and exposes it as a serial port. Every test below runs against it exactly as against your board, so you can learn the procedure, and see what good numbers look like, before a motor turns:

```
cd ~/robot_kit/tools/firmware_sim && make && ./sim --pty
```

It prints `PTY /dev/pts/3` (some number) and the simulated motor’s true constants. In a second terminal:

```
cd ~/robot_kit && ./tools/encoder_logger.py --test plant --port /dev/pts/3 --open-delay 0.3
```

The simulator’s motors default to needing about 65 % PWM at `max_wheel_speed`, Kstat 3 % of full PWM, and tau 0.09 s, with the motors 3 % apart. `./sim --pty --kff 38 --kstat 8 --tau 0.12` changes them.

9.3 Test 1: the plant (do this first)

```
./tools/encoder_logger.py --test plant
./tools/encoder_logger.py --test plant --only left       # one wheel at a time, if you prefer
```

Open loop, PID bypassed: for each of three PWM levels (30, 45 and 65 % of full scale by default) it steps forward and backward and holds each for 2.5 s, logging at 50 Hz. About 25 seconds. For each wheel it prints:

```
    left: Kff  16.08  Kstat   7.6  Ki  107.2  tau 0.110 s  Kp  11.79   (filter adds +0.058 s)
```

`Kff` (PWM per rad/s) and `Kstat` (PWM to break friction) come from a straight-line fit through the steady-state speeds. `tau` is the motor's mechanical time constant, measured from the wheel angle; "filter adds" is the extra lag the firmware's velocity filter puts in front of the PID, reported separately so you don't mistake it for the motor. `Ki` is Kff / λ, where λ (default 0.15 s, `--lam`) is how fast you want the closed loop to respond; smaller is faster and less forgiving. `Kp` is τ · Ki, the textbook direct-synthesis value. Treat that last one as a candidate, not an answer (9.6 explains why).

Where these formulas come from, in two lines: a geared DC motor behaves like a first-order plant, wheel speed = K·PWM through a lag τ, with K = 1/Kff. Choosing a PI controller that cancels that lag and gives a closed-loop time constant λ yields Ki = 1/(K·λ) and Kp = τ·Ki. The useful property is that Ki doesn't depend on τ at all, so Ki is trustworthy even when τ is noisy.

Try the numbers live without reflashing, then check them (9.5):

```
<F,16.08,16.58>
<K,7.6,7.6>
<G,11.8,107,0>
```

9.4 Test 2: the staircase, in the air and then on the floor

```
./tools/encoder_logger.py --test staircase --max-pwm 90
```

PWM rises from 1 % in 1 % steps, each held 1.5 s from rest, up to `--max-pwm` (default 35 % of full scale). It prints the speed at every step and fits a line through the points that moved: Kff is one over the slope, Kstat is where the line crosses zero. It ends with the two live commands to try, ready to paste:

```
    Live:  <F,16.10,16.61>
           <K,7.7,7.6>
```

Run it once with the wheels in the air and once on the floor. **The difference is the ground load.** On NarrowAisleBot we expected feedforward to rise 10 to 30 % on the ground and Kstat by more. On the floor this test drives the robot in a straight line for the whole ramp, several metres by the end. If you haven't got the space:

```
./tools/encoder_logger.py --test staircase --mode rotate
```

turns the robot on the spot instead, using the sign pattern your own kinematics give for a pure rotation. The wheels still roll on the floor under load, at a slightly different angle to the tyres than in a straight run, so treat it as a close estimate.

9.5 Test 3: steps, the verdict on a gain set

```
./tools/encoder_logger.py --test steps --kff 16.1,16.6 --kstat 7.7,7.6 --gains 12,107,0
```

Closed loop, four velocity steps by default (20, 40, 60 and 85 % of `max_wheel_speed`), each from rest. `--kff`, `--kstat` and `--gains` are sent live first, so you can test fitted numbers before making them permanent. The table it prints:

```
    setpt  motor          ss      err   overshoot  settle(s)  sat%
     2.00  left          2.017  +0.017     +0.689       0.51     0
```

`ss` is the steady speed over the last half second of the step and `err` its error. Under 0.05 rad/s is good; above 0.15 means the feedforward is off or Ki is too low. `overshoot` is peak minus target, and under 5 % of the setpoint is what we aim for. `settle` is the time until the speed stays within ±5 % for the rest of the step (`none` means it never did). `sat%` is the share of samples at full PWM; more than a few percent means that setpoint is beyond what the motor can hold with headroom to spare (9.8).

`--no-slew` turns the setpoint slew limiter off (`<A,0>`) so the controller sees a true step, useful when you want to see the raw loop.

9.6 Test 4: the sweep, and why you shouldn't trust the formula

```
./tools/encoder_logger.py --test sweep --gains "45,250,0.5 22,250,0.5 26,250,0.5" --kff ... --kstat
...
```

Runs the step test once per gain set, one CSV each, and ends with a comparison of mean overshoot, settle time and worst error.

What happened on NarrowAisleBot is worth knowing before you run yours. We measured τ ≈ 0.09 s with the plant test, and the formula said Kp should drop from the 45 we'd been running to 22 or 26. We predicted, in writing, that both would settle faster with less overshoot. The sweep said the opposite, in **16 rows out of 16**: Kp = 45 had a mean overshoot of 0.107 rad/s, 22 had 0.164 and 26 had 0.153. The likely reason is that the formula assumes PI acting on the bare motor, but with good feedforward most of the PWM arrives instantly and Kp's real job is damping the transient around it; lower Kp, less damping. We kept 45.

So use the plant test to get Kff, Kstat and Ki, and use the sweep to pick Kp. Prefer the lowest overshoot with a settle under about 0.3 s and no saturation.

9.7 Make the numbers permanent

Live values (`<F>`, `<K>`, `<G>`) are lost when the board resets. Once a set wins, put it in `robot_config.yaml`:

```yaml
mcu:
  pid: {kp: 45.0, ki: 250.0, kd: 0.5}
  kff: [37.3, 38.4]
  kstat: [8.0, 8.0]
```

then `python3 generate.py`, reflash (7.3 or 7.4), and check with `<I>` that the board reports them. Run `--test steps` once more with no `--kff`/`--gains` options at all, to confirm the flashed values behave like the live ones did.

9.8 Picking max_wheel_speed: leave the PID some room

The feedforward alone needs `Kff × w + Kstat` PWM to hold speed w. Whatever is left up to full scale is all the PID has for correcting errors. On NarrowAisleBot, in the air, the worst motor needed 38.4 × 5.2 + 8 = 208

of 255 at the 5.2 rad/s limit, leaving 47 (18 %). On the ground, with feedforward 10 to 30 % higher, that margin disappears and the controller has nothing left to regulate with. So the rule we use:

$$\text{max_wheel_speed} \le \frac{0.85 \times \text{PWM_MAX} - \text{Kstat}}{\text{Kff}_{\text{ground}}}$$

with the ground Kff from 9.4. `generate.py` warns when a configured limit breaks it. On NarrowAisleBot that means about 4.2 rad/s on the floor instead of 5.2 in the air.

9.9 Your own firmware: raw mode

If your board runs its own code and just prints CSV, the logger still timestamps and saves it:

```
./tools/encoder_logger.py --test raw --port /dev/ttyACM0 --baud 115200 \
    --columns t_ms,left_ticks,right_ticks \
    --ticks left_ticks:1320,right_ticks:1320 --time-col t_ms --duration 30
```

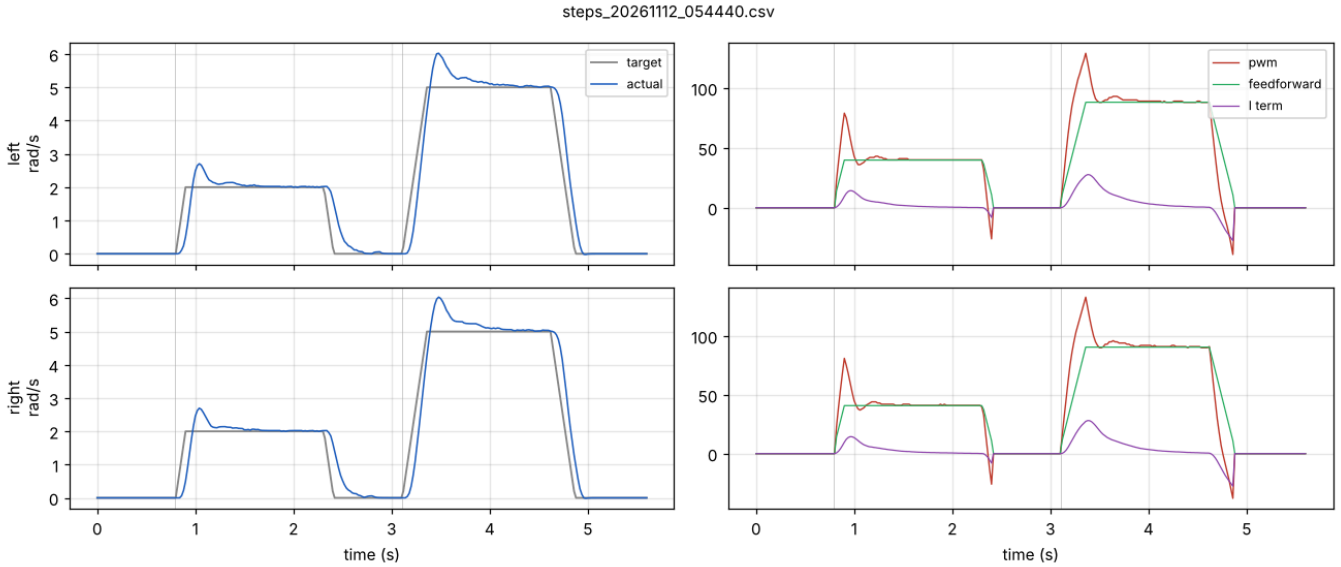
- `--columns` names the columns. If the board prints a header line starting with `#` (for example `# t_ms,left_ticks,right_ticks`), that's used instead.
- `--ticks name:counts_per_rev` adds `name_rad` and `name_rads` columns: the angle, and the speed from successive angles.
- `--time-col` uses the board's own millisecond column for those speeds, which is more accurate than the Pi's arrival time.
- `--send '<L1,20>'` sends commands first (repeatable), for boards that need waking up.

Lines that aren't all numbers, or have the wrong column count, are skipped and counted.

9.10 Looking at the results

```
python3 tools/analyze_log.py ~/robot_logs/steps_20261112_101500.csv --radius 0.05
```

prints, per wheel, the RMS tracking error while moving, the share of time at full PWM, how often the wheel turned against its target (a wiring or slip symptom), and the distance travelled; and writes a plot beside the CSV.



The simulator, a two-wheel robot, step test with fitted feedforward and Kp 20, Ki 100. Left: target (grey) and measured speed (blue). Right: PWM (red) split into feedforward (green) and the integral term (purple). The overshoot is what a sweep would then reduce.

To pull logs to a laptop on the robot's network: `scp <user>@10.42.0.1:~/robot_logs/*.csv .` (and hash them).

9.11 Logging while driving under ROS

The logger needs the port to itself, so it can't run while ROS drives. Two ways to log then, both writing the same CSV format:

- The dashboard's **REC** button (chapter 11), which writes `~/robot_logs/dash_<time>.csv`.
- From a terminal (Ctrl+C or `duration` to stop):

```
ros2 run robot_base telemetry_logger --ros-args \
    --params-file $(ros2 pkg prefix robot_base)/share/robot_base/config/robot.yaml -p
duration:=60.0
```

Both record what `mcu_bridge` publishes, which is the extended telemetry at 20 Hz. Use these for ground runs: drive a route by hand and look at saturation and tracking error under real load.

10. The ROS 2 base: bring-up, driving, odometry

10.1 Build, then start by hand the first time

```
rk-build                          # regenerate, copy to ~/ros2_ws/src, colcon build
rk-stop                           # make sure the service isn't holding the port
ros2 launch robot_base base.launch.py
```

Run it in the foreground the first time so you can read every line. You want to see `mcu_bridge: /dev/robot_mcu @ ...`, then `connected: robot_kit_mcu 1.0,<name>`, and no red. If the bridge says the firmware's motors don't match `wheel_names`, it refuses to drive, on purpose: regenerate and reflash.

In a second terminal:

```
ros2 node list
ros2 topic hz /wheel_states        # about 20 Hz
ros2 topic echo /wheel_states --once
ros2 run tf2_ros tf2_echo odom base_link
```

`tf2_echo` must print a translation. If it says the frame doesn't exist, odometry isn't running, and slam_toolbox will later wait at "Activating" forever without a single error message. That silence cost NarrowAisleBot an afternoon. The usual cause is no telemetry: check `/wheel_states` first.

10.2 Drive it

Wheels on the ground, robot in open space, and your hand near the battery switch. From a laptop or a second SSH session:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args -r cmd_vel:=cmd_vel_manual
```

The remap matters: `twist_mux` listens to `/cmd_vel_manual` for manual input. Press `z` a few times first to slow it down. Or open the dashboard (chapter 11) and use the joystick.

10.3 Does every direction mean what it says?

Write down what should happen, then check each with the dashboard's pose readout or `ros2 topic echo /odom --field pose.pose:`

Command	Robot should	`/odom` should
forward	drive forward	x increases
turn left (counter-clockwise)	rotate left	yaw increases
strafe left (mecanum/omni only)	move left	y increases

A robot that drives forward but turns the wrong way has its wheels in the wrong order in `wheels:`. A robot whose odometry disagrees with what it physically did has a sign wrong in `encoder_dir` (go back to 7.6). On mecanum, if strafing goes the wrong way *and* turning on the spot is weak and scrubby, the wheels are mounted in the wrong diagonal pattern: swap the left and right wheels physically. If only the strafe direction is wrong, set `mecanum_sign` explicitly with all four signs reversed.

10.4 Calibrate odometry with a tape measure

Odometry is the robot's only sense of where it is until there's an IMU, and SLAM draws the map wherever odometry says the robot is. Ten minutes here saves hours of chasing "bad maps".

Mark a start line on the floor. For each test, re-zero (dashboard RE-ZERO, or `ros2 topic pub --once /odom/reset std_msgs/msg/Empty`), drive, read `/odom`, measure, then correct:

Test	Measure	Correct
straight 2 m	true distance d, odometry x	`wheel_radius` × d / x
3 full turns on the spot	1080°, odometry yaw (count the wraps)	`track_width_scale` × reported / 1080
strafe 1 m (mecanum/omni)	true y, odometry y	`lateral_scale` × true / reported

The two scales can be tried live, without a restart that would lose the pose:

```
ros2 param set /wheel_odometry track_width_scale 1.08
ros2 param set /wheel_odometry lateral_scale 0.92
```

then made permanent in `robot_config.yaml` and `rk-build`. Do the straight test first, since the other two depend on the wheel radius.

What to expect. NarrowAisleBot's straight-line odometry reads 1.503 m for a 1.50 m tape distance, and a 9.5 m three-lap route closed to within 2 mm. Strafing is a different story: the rollers scrub, and the same robot over-reported strafe by 25 % on one floor (scale 0.80) and 8 % on another floor in the same building (0.92). Calibrate on the floor you'll actually map, and expect to redo it on a new surface. Skid-steer bases typically land `track_width_scale` between 1.1 and 1.5, because the tyres scrub sideways in every turn.

10.5 Start at boot

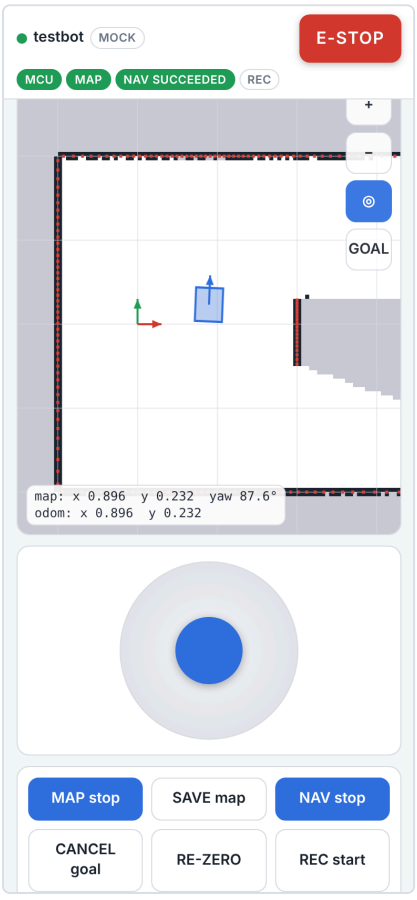
Once the foreground launch behaves:

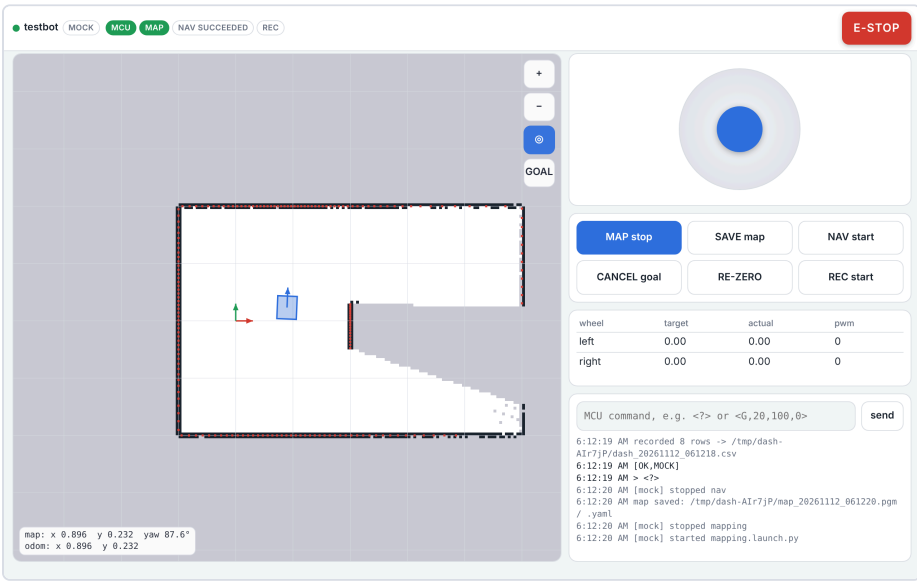
```
rk-start            # now
sudo systemctl enable robot     # at every boot (05_system.sh does this if autostart is true)
rk-status
rk-log              # ~/robot_boot.log
```

The service waits up to 30 s for `/dev/robot_mcu`, then starts `base.launch.py`; if the device never appears it exits and systemd retries every 10 s. It stops with SIGINT and a 30 s timeout, so the dashboard has time to save an open map and shut SLAM down cleanly. (NarrowAisleBot once lost a 24-minute map to a service restart that killed everything at once.)

11. The dashboard

With the base stack running, open `http://<pi-address>:8080` on a phone (on the robot's own Wi-Fi, `http://10.42.0.1:8080`).





On a laptop, the same page laid out in two columns.

On a phone.

Both in mock mode (no robot), mapping a simulated room. The red dots are the laser, the grey is unexplored, the blue box is the robot's footprint and the red and green arrows mark the map origin.

Control	What it does
E-STOP (top right)	zero velocity, `<S>` to the firmware (it latches), cancel every Nav2 goal. Mapping keeps running, because a stop shouldn't cost you the map. The button then reads **CLEAR**, which sends `<E1>`. Space bar does the same on a laptop.
joystick	up/down is forward/back. On a differential or skid-steer base left/right turns; on mecanum/omni it strafes and the **turn** slider rotates. Release and it sends zero immediately; if the phone loses the connection the server stops the robot within 0.3 s.
map	the live SLAM map, laser points in red, your footprint in blue with a heading arrow. Drag to pan, pinch or scroll to zoom, ◎ to follow the robot.

Control	What it does
GOAL	then tap the map for a goal (heading: facing the goal) or press and drag (heading: the drag direction). Needs NAV running.
MAP start / stop	starts `mapping.launch.py` (LiDAR, scan filter, slam_toolbox). Stop saves the map first, into `~/maps/map_<time>.pgm` and `.yaml`.
SAVE map	saves now and keeps mapping
NAV start / stop	starts `nav2.launch.py`. Refused unless MAP is running.
CANCEL goal	cancels every navigation goal
RE-ZERO	odometry (0, 0, 0) where the robot stands. Refused while MAP or NAV runs.
REC	log the motor telemetry to `~/robot_logs/dash_<time>.csv`
wheel table	per wheel: target, measured speed, PWM
console	type a raw firmware command (`<?>`, `<I>`, `<G,20,100,0>`) and see every `[...]` reply

The chips at the top read MCU (green when telemetry is flowing, red ESTOP when latched), MAP, NAV (with the goal status) and REC (row count).

Launch files started from the dashboard write their output to `~/robot_logs/mapping_<time>.log` and `nav_<time>.log`. When a map or Nav2 misbehaves, that's where the error is.

Preview without a robot

The server has a mock mode with a simulated robot, room and LiDAR. On any machine with Python, fastapi and uvicorn:

```
cd ~/robot_kit/ros2/robot_base
python3 -m robot_base.dashboard.server --mock --port 8090
```

and open `http://localhost:8090`. Add `--holonomic` to see the strafe controls.

If you change the page

It's three plain files in `robot_base/dashboard/static/`. Keep it that way. NarrowAisleBot's dashboard holds its HTML and JavaScript inside a Python string, and on 15 Sep 2026 one escaped apostrophe (`\'`, valid in Python) became a bare apostrophe in the JavaScript Python actually produced. The whole page died: no joystick, no map, no E-STOP. `curl` returned 200, the service log was clean, `ros2 node list` looked normal. Only the browser's console showed the error. So: after any change, run `node tests/test_dashboard_browser.js` (it opens the page in Chromium and fails on any console error), and after deploying, open the page on the real robot with the browser console visible before calling it done.

12. The LiDAR

12.1 Choosing and mounting it

Any 2D LiDAR with a ROS 2 driver works. The kit has `lidar.launch.py` entries for YDLIDAR and RPLIDAR, and a `custom` entry for anything else.

Mount it where it sees as much of the circle as possible, level, and rigid (vibration shows up as noise in every scan). Remember that it sees one horizontal plane only. NarrowAisleBot's scan plane is 0.35 m above the floor, so a 30 cm box is invisible to it; test obstacles must be taller than your scan plane. Anything the robot carries above the laser (a mast, an arm) blocks a sector, which you mask out (12.5).

Measure the mount from `base_link` (on the floor under the turning centre): x forward, y left, z up. Put it in `lidar.mount`. The URDF is then the only source of the `base_link → laser_frame` transform. Don't use the driver's own launch file: YDLIDAR's, for one, also publishes a hard-coded placeholder transform for the same frames, which then fights the measured one.

12.2 Driver parameters

For `model: ydlidar` the generator writes NarrowAisleBot's X4 Pro parameters (verified byte for byte against the running robot) with your port, baud, frame and range. Other YDLIDAR models need different values; set them in `lidar.driver_overrides`:

YDLIDAR model	baud	lidar_type	isSingleChannel	sample_rate
X4 Pro	128000	1	true	5
X4	128000	1	false	5
X2, X2L	115200	1	true	3
G4	230400	1	false	9

Check those against the YDLIDAR wiki for your exact unit; the table comes from their published examples, and only the X4 Pro row has run on our robot. For RPLIDAR the driver needs mostly the baud: 115200 for A1 and A2M8, 256000 for A2M12, A3 and S1.

Three things about the X4 Pro, measured on NarrowAisleBot:

- In the parameter file, the values must be nested under the node name and `ros__parameters:`. A flat file loads without complaint and binds nothing; the driver runs on its compiled defaults.
- The `frequency` parameter does nothing on this unit. With `support_motor_dtr: false` the driver never commands the motor and the head free-runs at about 11.4 Hz. We measured 430 points per scan at that rate.
- It is rated to 10 m. We had `range_max: 12` for months, inherited from a different sensor, and everything between 10 and 12 m was noise with a number attached. Set the driver to what the sensor can do, and cap what SLAM uses separately (13.4).

12.3 Bring it up and check it

```
ros2 node list | grep -i -e lidar -e rplidar      # must be empty first: two drivers on one port is not a soft failure
ros2 launch robot_base lidar.launch.py
```

then, in another terminal:

```
ros2 topic hz /scan
ros2 topic hz /scan_filtered
ros2 topic echo /scan_filtered --once --field ranges | head
```

`scan_filter` republishes the driver's best-effort `/scan` as reliable `/scan_filtered`. That's not cosmetic: slam_toolbox and the Nav2 costmaps subscribe reliably, and a reliable subscriber never connects to a best-effort publisher. No error appears; there's just no data. Everything in the kit reads `/scan_filtered`. If you add a consumer, point it there too.

12.4 Is it pointing the right way?

Don't trust a map until this passes. With the base and LiDAR running, open the dashboard (or Foxglove):

1. Put a box, taller than the scan plane, about 1 m directly in front of the robot. The red points should appear in front of the blue footprint's arrow.
2. Move it 1 m to the robot's left. The points should move to the left of the footprint.

If the front is right but left and right are swapped, the scanner reports angles in the opposite sense: set `lidar.mirror: true`. If everything is rotated by some angle, set `lidar.mount.yaw_deg` (positive is counter-clockwise). If it's mirrored and rotated, set both; mirror is applied first, then the yaw through TF. After each change: `rk-gen`, `rk-build`, restart the LiDAR launch, repeat the test.

The mirror exists because TF can rotate a frame but cannot reflect one, so no URDF value can fix a mirrored scanner. NarrowAisleBot's X4 Pro needed mirror plus a rotation.

12.5 Masking the robot's own body

If part of the robot is in the laser's view, its points appear as a permanent obstacle stuck to the robot. Find the bearings (the dashboard shows them as red points against the footprint; in base_link degrees, 0 is forward and +90 is left) and list them:

```
lidar:
  mask_deg: [[135, 225]]       # e.g. a mast directly behind the laser
```

Masked beams become NaN, not 0 and not infinity. Zero would mark an obstacle on top of the sensor, and infinity would clear the costmap along that ray. NaN means "no information" to every consumer. NarrowAisleBot masks 107 beams behind its mast, which leaves a blind wedge behind the robot; chapter 14 comes back to that.

13. Mapping

13.1 The order that works

1. Base stack running, telemetry flowing (`rk-check`).
2. Park the robot on a start mark you can find again (tape on the floor).
3. RE-ZERO odometry.
4. MAP start. Wait about 30 s for the first `/map`.
5. Drive slowly. 0.10 to 0.15 m/s; turn gently; stay within a few metres of walls.
6. Come back to the start mark at the end, then MAP stop (it saves).

Re-zeroing must happen before step 4, never during mapping. slam_toolbox fixes the `map → odom` relationship at its first scan, so moving odometry's origin afterwards shifts the whole map. The dashboard refuses RE-ZERO while MAP runs for that reason.

From a terminal instead of the dashboard:

```
ros2 launch robot_base mapping.launch.py
ros2 run nav2_map_server map_saver_cli -f ~/maps/lab_$(date +%Y%m%d_%H%M) --ros-args -p
map_subscribe_transient_local:=true
```

13.2 What a good map needs from you

Slow driving, because the LiDAR takes a scan every 90 ms or so and the robot moves during it. Overlap: drive past the same walls more than once, and return to the start, which gives SLAM the chance to close a loop. Texture: a long featureless corridor gives the scan matcher nothing to hold on to along its length (two parallel walls constrain sideways position and nothing else).

13.3 Checking the result

Drive back to the start mark and read the dashboard's pose: how far from (0, 0) the robot believes it is, when it's physically on the mark, is the most honest single number about a mapping run. Then look at the saved `.pgm` for doubled walls (the same wall drawn twice, a few centimetres apart), which is what a pose jump between two passes looks like. NarrowAisleBot's repo has `tools/map_integrity.py` and `tools/run_analyzer.py`, which measure both from the saved files; they work on any map.

13.4 Scan matching on or off?

slam_toolbox does two things with each scan: it estimates where the robot is by matching the scan against the map so far (scan matching), then draws the scan there. With `use_scan_matching: false` it skips the first step and trusts odometry completely.

The kit defaults to `true`, slam_toolbox's own default, because with a good LiDAR and decent odometry matching is supposed to help. On NarrowAisleBot it didn't, and the evidence is clear enough that you should know it:

- Over the same 21.85 m drive, wheel odometry alone closed to 0.229 m. Odometry plus scan matching closed to 0.706 m. The matcher was three times worse.
- Turned off (Stage G), the map stopped folding, and every `map → odom` correction over two runs and 18.5 m was exactly zero.

- Turned back on with a tighter setup (Stage H), 14 of 17 corrections were 0.15 m or more, the map folded, and the robot thought it was 206.7 mm from its start mark when wheel odometry alone, on the same drive, said 16.2 mm. Reverted.

The likely reasons are ours: an entry-level LiDAR (about 430 points per scan, many of them weak) and small, symmetric test spaces. Your sensor and building may well behave differently. Test it rather than assuming either way: the same route twice, once each way, one variable changed, and compare the return-to-mark distance. Whichever is closer wins.

`max_laser_range` is SLAM's choice of how far to trust the scanner, separate from the driver's `range_max` (what the scanner can do). NarrowAisleBot uses 5 m against a 10 m sensor, because the long returns were the weakest and noisiest. Start around 6 m and raise it if your scanner is good.

14. Autonomous navigation (work in progress)

This is where NarrowAisleBot is right now, so this chapter is half instructions and half lab notebook. The kit's Nav2 setup brings up cleanly on our structure; reaching goals reliably is still being worked out.

14.1 What starts

`nav2.launch.py` starts the planner, controller, smoother, behaviour server, velocity smoother, collision monitor, BT navigator, waypoint follower and the lifecycle manager, with parameters from the generated `nav2_params.yaml`. It is navigation **while mapping**: slam_toolbox provides `map` and `map → odom`, so no AMCL. Some decisions baked in, each from something that broke:

- Nodes are listed explicitly instead of including `nav2_bringup`. Recent `nav2_bringup` also starts a docking server, which refuses to configure without dock plugins, and because the lifecycle manager brings nodes up all-or-nothing, that one node we never needed aborted the whole bringup.
- The behaviour tree XML paths are resolved in the launch file. An empty string in the YAML did not fall back to the default on our Nav2 build; the first goal returned "Empty Tree".
- `behavior_server` is remapped onto the same path as the controller, so recovery Spin and BackUp go through the velocity smoother and the collision monitor too. Left on its default, it publishes straight to `/cmd_vel` and bypasses both.
- `odom_topic: /odom` is set on every node that takes one. On NarrowAisleBot the controller had none set, listened on a topic nobody published, and ran with no measured velocity for weeks before anyone noticed.

14.2 Order of operations

1. Base running, robot on the start mark, RE-ZERO.
2. MAP start, wait for the map (30 s).
3. NAV start. Watch `~/robot_logs/nav_<time>.log` until the lifecycle manager reports everything active.
4. GOAL, tap a point a metre away in open, mapped space. Hand on E-STOP.

Starting Nav2 before the map exists makes the global costmap reject the missing map once a second until one arrives, and the planner can't plan meanwhile.

From a terminal:

```
ros2 action send_goal --feedback /navigate_to_pose nav2_msgs/action/NavigateToPose \
  "{pose: {header: {frame_id: map}, pose: {position: {x: 1.0, y: 0.0}, orientation: {w: 1.0}}}}"
```

Ctrl+C there cancels the goal.

14.3 Parameters that matter first

Parameter	Where in robot_config.yaml	Why it matters
footprint	`robot.body` + `footprint_margin`	too small means confident collisions; too big means "no path" in a doorway
`inflation_radius`	`nav.inflation_radius`	auto is the padded footprint's circumscribed radius plus 0.10 m

Parameter	Where in robot_config.yaml	Why it matters
controller	`nav.controller`	RPP (differential, skid steer): rotate to the path, then follow it; simple and predictable. MPPI (holonomic): samples many trajectories, heavy on a Pi.
speed and acceleration limits	`nav.max_vx`, `max_wz`, `accel`, `decel`	start at 0.10 to 0.15 m/s
`controller_hz`	`nav.controller_hz`	ask for what the Pi can actually do (14.5)
goal tolerances	`nav.goal_xy_tolerance`, `goal_yaw_tolerance`	10 cm and 0.15 rad to start; tighten once it works

Everything else in the file is stock Nav2, laid out the way NarrowAisleBot's runs.

14.4 How to debug a goal that doesn't arrive

Record before you test, so a failure gives you data:

```
ros2 bag record -s mcap -o ~/robot_logs/bags/test1 \
  /cmd_vel_nav /cmd_vel_smoothed /cmd_vel_nav_out /cmd_vel /collision_monitor_state \
  /odom /tf /tf_static /plan /local_costmap/published_footprint
```

Stop it with Ctrl+C (or `pkill -INT -f "ros2 bag record"`) and wait for `metadata.yaml` to appear; a bag that wasn't closed may not be readable. Then compare speeds stage by stage along the chain in section 2.2. The first stage that shows small numbers is where the speed was lost. NarrowAisleBot's `tools/bag_cmd_chain.py` prints exactly that table from a bag.

Two cost queries tell you whether the robot thinks it's already in collision. Get its pose from the dashboard first; a cost query at the wrong place means nothing:

```
ros2 service call /local_costmap/get_cost_local_costmap nav2_msgs/srv/GetCost "{x: 0.0, y: 0.0,
theta: 0.0, use_footprint: true}"
ros2 service call /global_costmap/get_cost_global_costmap nav2_msgs/srv/GetCost "{x: 0.0, y: 0.0,
theta: 0.0, use_footprint: true}"
```

253 or 254 means the footprint is touching an inflated obstacle. The planner uses the global costmap; the controller and the collision monitor use the local one.

After Nav2 exits, `ros2 node list` and `ros2 service list` may still show its nodes for a while; that's the ROS daemon's cache, not a running Nav2. `ros2 daemon stop` clears it.

14.5 What we've seen on NarrowAisleBot

Measured on our robot (holonomic, MPPI, Pi 5 with SLAM running), October 2026:

The controller can't hold its rate. We asked MPPI for 20 Hz with 300 samples × 40 steps and it ran at 5.5 to 8.9 Hz, which is why the kit generates 10 Hz and a model step of 1/controller_hz. Measure yours during a run: `ros2 topic hz /cmd_vel_nav`, with `top` and the CPU temperature open.

A 0.6 m goal crept at about 4 mm/s and aborted. Every stage of the command chain showed the same 4.4 mm/s, which cleared the smoother, the collision monitor, the mux and the firmware: MPPI itself was asking

for almost nothing. Our leading suspect is an acceleration limit we had set to 0.3 m/s² (at a 0.05 s step that allows a change of only 0.015 m/s per step), not yet confirmed. If your robot creeps, compare MPPI's `ax_max`/`ay_max` with the velocity smoother's before anything else.

- The collision monitor has no memory; it sees only the live scan. The local costmap does remember obstacles, until a laser ray clears them or they leave its 3 × 3 m window, and we measured an object held for about 100 s after the robot turned away from it.
- The rear can be blind. With a mast behind the LiDAR, nothing directly behind the robot is ever seen, and on a REP-103 robot Nav2's BackUp recovery reverses straight into that wedge. If your scanner can't see behind you, consider taking BackUp out of the behaviour tree, or add rear sensing.
- The first planning attempts sometimes fail right after Nav2 starts ("Failed to create plan with tolerance"), then succeed. We haven't established why.

14.6 The ladder

Climb it one rung at a time, each with a written prediction and a bag: 1 m forward in open space; 1 m sideways (holonomic only); a 90° turn; a 1 m reverse with something behind the robot; a goal around a corner; then goals out of sight of the start. Don't move up a rung until the one below works three times running.

14.7 What comes next

In the order we plan to do it on NarrowAisleBot:

1. An IMU and an EKF. A BNO085 on the microcontroller's I²C, fused with wheel odometry in `robot_localization` (already installed by step 2). Yaw from a gyro is far better than yaw from wheels, especially on skid steer and mecanum. Exactly one node may publish `odom → base_link`: when the EKF does, set `wheel_odometry`'s `publish_tf` to false.
2. Saved maps and AMCL: map once, then localise on the saved map. Our own AMCL block had no `scan_topic` set and would have read the raw `/scan`; set `scan_topic: /scan_filtered`.
3. Exploration. Frontier exploration (`m-explore-ros2`, or `frontier_exploration_ros2` for Jazzy) sends the robot toward the edge of the known map by itself. Only once single goals are dependable.
4. Rear sensing, if your LiDAR has a blind sector: a second small LiDAR, or time-of-flight sensors along the body.

15. Troubleshooting

Symptom	Likely cause	Check
`/dev/ttyUSB0` appears, then vanishes	`brltty` grabbed it	`sudo apt remove brltty` (step 01 does this)
“Permission denied” on the serial port	not in `dialout` yet	log out and in; `groups`
`/dev/robot_mcu` missing	udev rule not installed, or wrong IDs	`list_usb_serial.py`; `05_system.sh`; replug
Logger opens the port, never gets a reply	wrong port (another device with the same chip) or wrong baud	8.2; `mcu.baud` matches the sketch
Bridge: “firmware motors … != wheel_names”	config changed, board not reflashed	`rk-gen`, reflash, `<I>`
Bridge connected, “no telemetry for 10 s”	firmware isn’t the kit’s, or baud mismatch	miniterm, `<I>`, `<L1,5>`
`[TRIP,RUNAWAY,<wheel>]`	`motor_dir` and `encoder_dir` disagree on that wheel	redo 7.5 and 7.6
`[TRIP,STALL,...]`	wheel jammed, or PWM too low to move it	free it; check Kstat, the battery
`[ERR,ENC_PIN,<pin>,...]`	encoder pin can’t take an interrupt (AVR)	4.2
Wheels chatter at constant speed	coarse encoders, or Kp too high	the header’s VEL_WINDOW note; sweep
Odometry drifts sideways on a straight line	wheel radii differ, or one encoder’s counts per rev is wrong	7.6 for each wheel; 10.4
A node refuses to start: InvalidParameterTypeException	an integer where a double is expected in a hand-edited YAML	regenerate rather than hand-edit
slam_toolbox stuck at “Activating”, no error	no `odom → base_link` TF	10.1: `tf2_echo odom base_link`; is `/wheel_states` flowing?
Map is empty, `/scan` has data	QoS: something reads `/scan` reliably	everything must read `/scan_filtered` (12.3)
Scan rotated or mirrored against the room	mount yaw or mirror	12.4
Two copies of every node	two launches, e.g. service plus a manual launch	`rk-stop`; `ros2 node list | sort | uniq -d`
Joystick does nothing, MCU chip red	E-STOP latched or the bridge isn’t connected	CLEAR; `rk-log`
Dashboard won’t load	service down, wrong address, firewall	`rk-status`; `curl localhost:8080/health`; `sudo ufw status`
Dashboard loads but nothing works	a JavaScript error	the browser console (chapter 11)

Symptom	Likely cause	Check
Nav2 aborts every goal	see 14.4	bag the chain; cost queries
LiDAR stutters, board resets under load	Pi USB current limit	4.1: `usb_max_current_enable`, a 5 A supply
Nodes can’t see each other after a network change	DDS interface	the loopback `CYCLONEDDS_URI` in `~/.bashrc` (5.5)

16. Daily use

16.1 A session, start to finish

1. Robot on blocks or in open space, battery connected, switch within reach.
2. Power the Pi. Join its Wi-Fi from the phone (if you use the AP).
3. `ssh <user>@10.42.0.1`, then `rk-check`. Fix anything red before driving.
4. Open the dashboard. Check the MCU chip is green and the wheel table updates.
5. Write today's prediction in your log before the first test.
6. At the end: MAP stop (saves), copy logs and maps off the Pi, hash them, `sudo poweroff` before cutting power.

16.2 The health check

`rk-check` (`setup/check_robot.sh`) asks the running system, never a file: devices and their symlinks, your `dialout` membership, the service, the nodes (and duplicates), the rates of `/wheel_states`, `/odom` and `/scan_filtered`, whether `odom → base_link` exists, live parameter values from the running nodes, CPU temperature and disk space. It exits non-zero if anything failed, so it can gate a script.

16.3 Seeing the robot from a laptop

Install Foxglove on the laptop, set `use_foxglove:=true` (`ros2 launch robot_base base.launch.py use_foxglove:=true`, or edit the service's launch line), and connect Foxglove to `ws://10.42.0.1:8765`. It shows TF, the URDF, scans, the map, costmaps and plots, without loading the Pi with RViz.

16.4 Command reference

```
# build and deploy after any change to the kit or robot_config.yaml
rk-build

# service
rk-start; rk-stop; rk-restart; rk-status; rk-log

# by hand
ros2 launch robot_base base.launch.py
ros2 launch robot_base lidar.launch.py
ros2 launch robot_base mapping.launch.py
ros2 launch robot_base nav2.launch.py

# look
ros2 node list
ros2 topic hz /wheel_states
ros2 topic echo /odom --field pose.pose
ros2 run tf2_ros tf2_echo map base_link
ros2 param get /wheel_odometry lateral_scale

# act
ros2 topic pub --once /odom/reset std_msgs/msg/Empty
ros2 topic pub --once /mcu/command std_msgs/msg/String "data: '<S>'"
ros2 topic pub --once /mcu/command std_msgs/msg/String "data: '<E1>'"

# calibrate (rk-stop first)
./tools/encoder_logger.py --test plant
./tools/encoder_logger.py --test staircase
./tools/encoder_logger.py --test steps
./tools/encoder_logger.py --test sweep --gains "..."
python3 tools/analyze_log.py ~/robot_logs/<file>.csv

# maps
ros2 run nav2_map_server map_saver_cli -f ~/maps/<name> --ros-args -p
map_subscribe_transient_local:=true
```

Appendix A. Serial protocol reference

All commands are ASCII framed by `<` and `>`, fields separated by commas, no line ending needed. N is the number of motors; per-motor values are in wheel order.

Command	Reply	Meaning
`<V,w0,...,wN-1>`	none	wheel velocity targets, rad/s, clamped to the speed limit. Leaves open-loop mode. Feeds the watchdog.
`<B,vx,vy,wz>`	none	body twist (m/s, m/s, rad/s); the MCU applies the inverse kinematics generated into the header, scaled so no wheel exceeds the limit
`<M,p0,...,pN-1>`	`[OK,M,...]`	raw PWM, ±PWM_MAX, open loop until the next `V`, `B` or `T`
`<T,i,w>`	`[OK,T]`	one motor closed loop, others keep their targets
`<S>`	`[OK,ESTOP_LATCHED]`	E-STOP; latches until `<E1>`
`<E1>` / `<E0>`	`[OK,ENABLED]` / `[OK,DISABLED]`	enable and clear E-STOP / disable
`<W1>` / `<W0>`	`[OK,WDOG=1/0]`	command watchdog on/off
`<Y1>` / `<Y0>`	`[OK,TRIPS=1/0]`	runaway, stall and overspeed trips on/off
`<G,kp,ki,kd>`	`[OK,GAINS,...]`	PID gains; resets the integrators
`<F,f0,...>`	`[OK,FF,name=..,...]`	Kff per motor
`<K,k0,...>`	`[OK,KSTAT,name=..,...]`	Kstat per motor
`<A,accel>`	`[OK,ACCEL=..]`	setpoint slew limit, rad/s², 0 = off
`<X,vmax>`	`[OK,VMAX=..]`	speed limit, up to twice the compiled value
`<L0>`, `<L1[,hz]>`, `<L2[,hz]>`	`[OK,LOG=n,HZ=..]`	telemetry off / basic / extended; up to 100 Hz
`<O>`	`[ODOM,a0,...]`	accumulated wheel angles, rad
`<R>`	`[OK,ODOM_ZEROED]`	zero them
`<P>`	`[PONG]`	ping
`<I>`	`[INFO]`, `[MOTORS,N,names]`, `[GAINS]`, `[FF]`, `[KSTAT]`, `[CPR]`, `[LIMITS]`, `[CONFIG]`	configuration
`<?>`	`[STATE,...]` and one line per motor	live state
`<H>`	`[HELP,...]` lines	help

Unsolicited lines: `[BOOT,...]` and `[READY]` at reset, `[WARN,WATCHDOG,...]` when the watchdog stops the wheels, `[TRIP,REASON,motor,v1,v2]` when a trip latches, `[ERR,...]` for a malformed command.

Telemetry (`<L1>`): `ms,` then for each motor `target,actual,pwm`. With `<L2>`, then for each motor `error,ff,iterm,pos_rad`. `target` is the slew-limited setpoint the PID is chasing, `actual` the filtered measured speed (rad/s), `ff` and `iterm` in PWM units, `pos_rad` the unfiltered accumulated wheel angle.

Appendix B. How this kit was checked

`./run_tests.sh` runs every check below. All of them passed on the version this manual describes.

Test file	What it proves
`test_kinematics.py`	differential and mecanum match the textbook formulas; mecanum reproduces NarrowAisleBot's asymmetric equations from `aislebot_esp32.ino` exactly; forward kinematics inverts inverse kinematics for every drive type; non-holonomic bases ignore vy; wheel limits keep ratios; the integrator closes a square. 26 checks.
`test_generate.py`	the generator runs on all three example robots; every YAML parses; every double parameter is written as a double; no empty lists; the URDF has the right joints; every firmware array has one entry per motor; the Nav2 controller and motion model match the drive type; nine bad configs are rejected with a message.
`test_scan_and_tools.py`	the scan mirror puts each beam at exactly the negated angle; masks wrap across ±180°; the protocol parsers; MCU-clock alignment removes delivery jitter; `list_usb_serial.py` on a fake `/sys` including two identical CP2102 adapters.
`test_firmware_sim.py`	compiles the real sketch with `-Wall -Werror` for three robots (2 and 4 motors; all three driver types; x2 and x4 decoding; 50 and 100 Hz) and runs it on simulated motors: targets reached within 5 %, telemetry column counts, open-loop speed matches the plant, watchdog timing, E-STOP latching, runaway trip on a backwards encoder (and overspeed when the motor can outrun the limit), `<B>`, `<I>`, `<O>`, `<R>`, bad input. 60 checks.
`test_logger_sim.py`	the encoder logger against the simulator over a pseudo-terminal, in real time: plant test recovers Kff and tau; staircase recovers Kff and Kstat; steps with fitted feedforward hold every setpoint within 0.1 rad/s; sweep writes one file per gain set; record rate; raw mode with derived columns.
`test_dashboard_browser.js`	the dashboard in headless Chromium (mock mode): connects, applies config, joystick moves and stops the robot, map and scan render, RE-ZERO refused while mapping, a dragged goal carries the right heading and is reached, E-STOP blocks the joystick and CLEAR releases it, REC writes rows, the MCU console round-trips, MAP stop saves a map, and no JavaScript error at any point.

Not checked here, and to be checked on the first real robot: the ROS 2 nodes and launch files under an actual ROS 2 Jazzy install (they compile and pass a linter); the ESP32-only firmware paths (PCNT, ledc, FreeRTOS), which are carried over from NarrowAisleBot's running firmware; flashing with arduino-cli; the install scripts end to end on a fresh Pi (each was syntax-checked and linted); the Wi-Fi access point script; the generated Nav2 parameters on any robot.

Appendix C. Source listings

Every file you need to run the robot, as shipped. The tests (`tests/`) and the files `generate.py` writes are in the kit folder but not printed here; `robot_config.h` is printed as generated from the example `robot_config.yaml`, so you can see what the generator produces. If you are copying from this PDF rather than the folder, check the line counts against the headings.

Your robot

`robot_config.yaml` (183 lines)

```yaml
# robot_config.yaml
#
# The one file you edit. Everything hardware-specific in this kit is generated
# from it:
#
#     python3 generate.py            (from the robot_kit folder)
#
# writes the URDF, the ROS 2 parameter files, the Nav2 and slam_toolbox
# configs, the udev rule and the microcontroller header (robot_config.h).
# Never hand-edit a generated file. Change this file and run generate.py again.
#
# The values below are EXAMPLE values for a small two-wheel differential robot
# on an ESP32. Replace every one of them with your own. The manual (chapter 3)
# says how to measure each.
#
# Units: metres, radians, seconds. Degrees only where a key ends in _deg.
# Axes follow REP-103 everywhere: +x forward, +y left, +z up, and positive yaw
# is counter-clockwise seen from above. Do not invent your own convention.
# We did, on NarrowAisleBot, and it cost us weeks (manual section 2.3).

robot:
  name: myrobot                   # lowercase, no spaces. Used for the URDF and file names.
  drive_type: differential        # differential | skid_steer | mecanum | omni
  wheel_radius: 0.0500            # m. Measure it loaded: roll 10 turns, distance / (20 * pi).
  track_width: 0.300              # m, centre of a left wheel to centre of the matching right wheel
  wheelbase: 0.000                # m, front axle to rear axle. 0 for a 2-wheel differential.

  # Measured corrections. Leave both at 1.0 until the odometry tests in the
  # manual (section 10.4) give you a number.
  track_width_scale: 1.0          # differential / skid_steer: effective track / geometric track
  lateral_scale: 1.0              # mecanum / omni only: true strafe distance / reported distance

  body:                           # the box that must never touch anything
    length: 0.40                  # m, along x
    width: 0.34                   # m, along y, at the widest point (wheels included)
    height: 0.20                  # m, only used to draw the URDF
    center_x: 0.00                # m, x of the box centre in base_link. base_link is the
                                  # point the robot turns about (the axle centre for diff drive).
    footprint_margin: 0.03        # m of padding Nav2 adds on every side

wheels:
  # One entry per DRIVEN wheel, in the order the microcontroller numbers its
  # motors (index 0 first). This order is the order of every <V,...> command
  # and every telemetry column, so get it right once and never change it.
  #
  # x, y       wheel contact point in base_link, metres. "auto" takes them from
  #            track_width / wheelbase using the wheel name: left, right,
  #            front_left, front_right, rear_left, rear_right, middle_left,
  #            middle_right (or fl, fr, rl, rr, ml, mr).
  # counts_per_rev   encoder counts per WHEEL revolution, as the firmware will
  #            count them. "auto" = encoder_ppr * encoder_decoding * gear_ratio.
  #            Measure it anyway (manual 7.6): datasheets lie about PPR vs CPR.
  # motor_dir  +1, or -1 if a positive PWM turns this wheel backwards.
  # encoder_dir +1, or -1 if rolling this wheel forward makes its count go down.
  # mecanum_sign / drive_angle_deg: only for mecanum / omni, see the manual.
  #
  # pins: what each pin means depends on driver.type below.
  #   pwm_dir      pwm, dir
  #   pwm_in1_in2  pwm, in1, in2
  #   dual_pwm     pwm_fwd, pwm_rev
  # plus enc_a and enc_b for the encoder, for every driver type.
```

```yaml
  - name: left
    x: auto
    y: auto
    counts_per_rev: auto
    encoder_ppr: 11
    encoder_decoding: 4
    gear_ratio: 30.0
    motor_dir: 1
    encoder_dir: 1
    pins: {pwm: 25, dir: 26, enc_a: 34, enc_b: 35}

  - name: right
    x: auto
    y: auto
    counts_per_rev: auto
    encoder_ppr: 11
    encoder_decoding: 4
    gear_ratio: 30.0
    motor_dir: -1                # the right motor is usually mounted mirrored
    encoder_dir: -1
    pins: {pwm: 32, dir: 33, enc_a: 36, enc_b: 39}

driver:
  type: pwm_dir                  # pwm_dir | pwm_in1_in2 | dual_pwm
  # pwm_dir      one PWM pin + one direction pin (Cytron MDD10A / MDD20A, most "smart" drivers)
  # pwm_in1_in2  enable PWM + two direction pins (L298N, TB6612FNG)
  # dual_pwm     two PWM pins, one per direction (BTS7960 RPWM/LPWM, DRV8833 IN1/IN2)

mcu:
  board: esp32                   # esp32 | avr | rp2040 | teensy | other
  encoder_backend: pcnt          # pcnt = ESP32 hardware counters (classic ESP32 only)
                                 # isr  = pin-change interrupts, any board, lower edge-rate limit
  baud: 921600                   # 921600 on ESP32. 115200 on AVR (Uno, Mega, Nano).
  device_name: robot_mcu         # becomes /dev/robot_mcu

  # USB identity, for the udev rule. Run  python3 setup/list_usb_serial.py
  # with the board plugged in and copy the values it prints.
  usb_vendor_id: "10c4"
  usb_product_id: "ea60"
  usb_serial: ""                 # fill in if the adapter has a unique serial number
  usb_port_path: ""              # fill in ONLY if two devices share VID, PID and serial.
                                 # Then the physical USB socket is the only thing that
                                 # tells them apart, and the cable must never move.

  control_hz: 100                # PID rate. 100 on ESP32, 50 on AVR.
  telemetry_hz: 20               # default telemetry rate
  watchdog_ms: 750               # no command for this long: wheels ramp to stop
  max_wheel_speed: 10.0          # rad/s. The firmware clamps to this. See manual 9.8 for
                                 # how to pick it (leave PWM headroom for the PID).
  max_wheel_accel: 20.0          # rad/s^2 setpoint slew limit, 0 disables it
  pwm_bits: 8                    # PWM resolution. 8 gives 0..255.
  pwm_freq_hz: 20000             # above hearing. Ignored on AVR (fixed by the core).
  min_pwm_out: 0                 # PWM below this is sent as 0 (kills motor hum)

  # Starting gains. These are deliberately soft. Run the plant test (manual
  # chapter 9) and replace them with numbers you measured.
  pid: {kp: 10.0, ki: 50.0, kd: 0.0}
  kff: [0.0, 0.0]                # PWM per rad/s, one per wheel, from the staircase test
  kstat: [0.0, 0.0]              # PWM to break static friction, one per wheel

lidar:
  model: ydlidar                 # ydlidar | rplidar | none | custom
  device_name: lidar             # becomes /dev/lidar
  usb_vendor_id: "10c4"
  usb_product_id: "ea60"
  usb_serial: ""
  usb_port_path: ""
  baud: 128000                   # YDLIDAR X4 / X4 Pro 128000, X2 115200, G4 230400,
                                 # RPLIDAR A1/A2M8 115200, A2M12/A3/S1 256000
  frame_id: laser_frame
  mount: {x: 0.10, y: 0.00, z: 0.18, yaw_deg: 0.0}   # laser centre in base_link
  mirror: false                  # true if objects on the left show up on the right (manual 12.4)
  mask_deg: []                   # arcs to blank, e.g. [[-135, -45]] for a mast behind the laser.
                                 # Degrees in base_link, 0 = forward, +90 = left.
  range_min: 0.10
  range_max: 10.0
  # Extra driver parameters, merged over the defaults for the model.
  # For ydlidar the defaults are our X4 Pro values (manual 12.2).
  driver_overrides: {}
  # Only for model: custom
  custom: {package: "", executable: "", params_file: ""}
```

```yaml
teleop:
  max_linear: 0.20                # m/s from the dashboard joystick and the keyboard
  max_angular: 0.80               # rad/s

dashboard:
  port: 8080
  log_dir: ~/robot_logs
  map_dir: ~/maps

slam:
  resolution: 0.05
  max_laser_range: 6.0            # what SLAM uses. The driver keeps the full range.
  use_scan_matching: true         # slam_toolbox default. We run false on NarrowAisleBot
                                  # because our odometry beat the matcher (manual 13.4).
  minimum_travel_distance: 0.2
  minimum_travel_heading: 0.2

nav:
  controller: auto                # auto | rpp | mppi. auto = rpp for differential and
                                  # skid_steer, mppi for mecanum and omni.
  max_vx: 0.15                    # m/s forward
  min_vx: -0.10                   # m/s reverse (negative)
  max_vy: 0.15                    # m/s sideways, ignored unless mecanum / omni
  max_wz: 0.60                    # rad/s
  accel: [0.3, 0.3, 0.8]          # vx, vy, wz
  decel: [-0.5, -0.5, -1.0]
  inflation_radius: auto          # auto = circumscribed radius of the padded footprint + 0.10
  goal_xy_tolerance: 0.10
  goal_yaw_tolerance: 0.15
  controller_hz: 10.0             # what a Pi 5 actually sustains with SLAM running

system:
  workspace: ~/ros2_ws
  ros_domain_id: 42
  autostart: true                 # start the base stack at boot with systemd
  wifi_ap:
    enable: false                 # true = the Pi hosts its own Wi-Fi (manual chapter 6)
    ssid: MyRobot-Pi
    password: change-me-please    # 8 characters minimum
    country: IN                   # two-letter Wi-Fi regulatory domain
```

`examples/nab_mecanum_reference.yaml` (137 lines)

```yaml
# NarrowAisleBot itself, written in this kit's format. A reference, not a
# template: it shows how per-wheel overrides express a robot the "auto"
# layouts cannot (asymmetric mecanum, two different encoders, a LiDAR that
# shares its USB identity with the MCU).
#
# Values are from aislebot_esp32.ino v3.0, docs/PID_Calibration.md and
# system/ydlidar_params.yaml at commit 2bf14e0.
#
# Two things differ from what actually runs on NarrowAisleBot today:
#   1. Axes. NarrowAisleBot's base_link is +x RIGHT, +y FORWARD. This file is
#      REP-103 (+x forward, +y left), which is what we would choose again.
#   2. Its LiDAR mirror and 270 deg yaw offset are expressed here as
#      mirror: true plus the mount yaw, not inside a relay node.

robot:
  name: narrowaislebot
  drive_type: mecanum
  wheel_radius: 0.0762            # DekuPro 6 inch SR mecanum
  track_width: 0.31538            # 2 x 0.15769
  wheelbase: 0.0                  # not used: every wheel has explicit x, y below
  track_width_scale: 1.0
  lateral_scale: 0.92             # measured on the zero-mark floor (0.80 on another floor)
  body: {length: 1.00, width: 0.36, height: 0.30, center_x: 0.0, footprint_margin: 0.06}

wheels:
  # Wire order FR, FL, RR, RL, exactly as <V,fr,fl,rr,rl>.
  # Outer pair (FR, RL) sits 0.403 m from centre, inner pair 0.333 m.
  - name: FR
    x: 0.403
    y: -0.15769
    mecanum_sign: 1
    counts_per_rev: 186264        # GTK08, 1000 PPR x 4 x 46.566
    encoder_decoding: 4
    motor_dir: -1
    encoder_dir: -1
    pins: {pwm: 4, dir: 16, enc_a: 36, enc_b: 39}
```

```yaml
  - name: FL
    x: 0.333
    y: 0.15769
    mecanum_sign: -1
    counts_per_rev: 186264
    encoder_decoding: 4
    motor_dir: 1
    encoder_dir: 1
    pins: {pwm: 17, dir: 18, enc_a: 34, enc_b: 35}
  - name: RR
    x: -0.333
    y: -0.15769
    mecanum_sign: -1
    counts_per_rev: 93132         # RMCS-2086 optical, 500 lines x 4 x 46.566
    encoder_decoding: 4
    motor_dir: -1
    encoder_dir: -1
    pins: {pwm: 19, dir: 21, enc_a: 32, enc_b: 33}
  - name: RL
    x: -0.403
    y: 0.15769
    mecanum_sign: 1
    counts_per_rev: 93132
    encoder_decoding: 4
    motor_dir: 1
    encoder_dir: 1
    pins: {pwm: 22, dir: 23, enc_a: 25, enc_b: 26}

driver:
  type: pwm_dir                   # 2 x Cytron MDD20A

mcu:
  board: esp32
  encoder_backend: pcnt
  baud: 921600
  device_name: esp32
  usb_vendor_id: "10c4"           # CP2102 ...
  usb_product_id: "ea60"
  usb_serial: "0001"              # ... with the same serial as the LiDAR's adapter,
  usb_port_path: "4-1"            # so only the physical port separates them
  control_hz: 100
  telemetry_hz: 20
  watchdog_ms: 750
  max_wheel_speed: 5.20           # air value; drop to ~4.2 on the ground
  max_wheel_accel: 12.0
  pwm_bits: 8
  pwm_freq_hz: 5000
  min_pwm_out: 5
  pid: {kp: 45.0, ki: 250.0, kd: 0.5}
  kff: [37.3, 38.4, 38.3, 38.0]
  kstat: [8.0, 8.0, 8.0, 8.0]

lidar:
  model: ydlidar
  device_name: ydlidar
  usb_vendor_id: "10c4"
  usb_product_id: "ea60"
  usb_serial: "0001"
  usb_port_path: "2-2"
  baud: 128000
  frame_id: laser_frame
  # 0.27 m ahead of base_link, 0.275 m up. scan_relay's rule on the real robot
  # is true = 270 deg - reported, in its +x-right frame. In REP-103 that is
  # true = 180 deg - reported: a mirror plus a 180 deg mount yaw. Derived from
  # the relay's rule, not re-measured in this form.
  mount: {x: 0.27, y: 0.0, z: 0.275, yaw_deg: 180.0}
  mirror: true
  mask_deg: [[135, 225]]          # the mast behind the scanner (107 beams)
  range_min: 0.1
  range_max: 10.0
  driver_overrides: {frequency: 6.0}   # inert on this unit: the head free-runs at ~11.4 Hz
  custom: {package: "", executable: "", params_file: ""}

teleop: {max_linear: 0.15, max_angular: 0.30}
dashboard: {port: 8080, log_dir: ~/aislebot_logs}

slam:
  resolution: 0.05
  max_laser_range: 5.0
  use_scan_matching: false        # Stage G: wheel odometry beat the matcher 3x
  minimum_travel_distance: 0.2
  minimum_travel_heading: 0.2
```

```
nav:
  controller: mppi
  max_vx: 0.12
  min_vx: -0.12
  max_vy: 0.12
  max_wz: 0.30
  accel: [0.3, 0.3, 0.6]
  decel: [-0.5, -0.5, -1.0]
  inflation_radius: 0.65
  goal_xy_tolerance: 0.05
  goal_yaw_tolerance: 0.05
  controller_hz: 10.0

system:
  workspace: ~/ros2_ws
  ros_domain_id: 42
  autostart: true
  wifi_ap: {enable: true, ssid: AisleBot-Pi, password: change-me-please, country: IN}
```

Generator

`generate.py` (1024 lines)

```
#!/usr/bin/env python3
"""
generate.py - turn robot_config.yaml into every robot-specific file.

    python3 generate.py                     # uses ./robot_config.yaml
    python3 generate.py --config other.yaml
    python3 generate.py --check             # validate only, write nothing

Writes (and overwrites, every time):

    ros2/robot_base/config/robot.yaml          parameters for the kit's ROS nodes
    ros2/robot_base/config/nav2_params.yaml    Nav2, REP-103, your footprint and limits
    ros2/robot_base/config/slam_toolbox.yaml   slam_toolbox, online async mapping
    ros2/robot_base/config/lidar.yaml          LiDAR driver parameters
    ros2/robot_base/config/twist_mux.yaml      manual beats autonomous
    ros2/robot_base/config/kit.yaml            which LiDAR driver the launch files start
    ros2/robot_base/urdf/robot.urdf            body, wheels, laser mount
    firmware/robot_mcu/robot_config.h          constants for the microcontroller
    setup/generated/99-robot.rules             udev rule: /dev/<mcu>, /dev/<lidar>
    setup/generated/robot.env                  shell variables for the setup scripts

Prints a summary you can sanity-check by eye: wheel positions, counts per
revolution, the top speed the wheel limit allows, and any warning.
"""

import argparse
import math
import os
import shlex
import sys
from string import Template

KIT = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.join(KIT, "ros2", "robot_base"))

from robot_base.robot_model import (  # noqa: E402
    ConfigError, DRIVER_PINS, footprint, load_config)
from robot_base.kinematics import HOLONOMIC_TYPES  # noqa: E402

PKG = os.path.join(KIT, "ros2", "robot_base")
HEADER = "# GENERATED by robot_kit/generate.py from robot_config.yaml. Do not edit.\n"


def yflt(v):
    """A float that YAML and ROS 2 both read as a DOUBLE. '10' would load as an
    integer, and rclpy/rclcpp refuse an integer for a parameter declared as a
    double (InvalidParameterTypeException), so there is always a '.' or 'e'."""
    v = float(v)
    s = f"{v:.6g}" if abs(v) >= 1e-4 or v == 0 else f"{v:.6e}"
    if not any(c in s for c in ".en"):          # n covers inf / nan
        s += ".0"
    return s
```

```
def ylist(vals):
    return "[" + ", ".join(yflt(v) for v in vals) + "]"


def slist(vals):
    return "[" + ", ".join(f'"{v}"' for v in vals) + "]"


def write(path, text, written):
    os.makedirs(os.path.dirname(path), exist_ok=True)
    with open(path, "w", newline="\n") as fh:
        fh.write(text)
    written.append(os.path.relpath(path, KIT))


# ═══════════════════════════════════════════════════════════════
#  ROS node parameters
# ═══════════════════════════════════════════════════════════════

def robot_params(cfg):
    rb, mcu, lid, tel, dash = (cfg["robot"], cfg["mcu"], cfg["lidar"],
                               cfg["teleop"], cfg["dashboard"])
    W = cfg["wheels"]
    names = [w["name"] for w in W]
    geom = (
        f"    drive_type: {rb['drive_type']}\n"
        f"    wheel_radius: {yflt(rb['wheel_radius'])}\n"
        f"    wheel_names: {slist(names)}\n"
        f"    wheel_x: {ylist(w['x'] for w in W)}\n"
        f"    wheel_y: {ylist(w['y'] for w in W)}\n"
        f"    wheel_ax: {ylist(w['ax'] for w in W)}\n"
        f"    wheel_ay: {ylist(w['ay'] for w in W)}\n"
        f"    track_width_scale: {yflt(rb.get('track_width_scale', 1.0))}\n"
    )
    yaw = float(lid["mount"].get("yaw_deg", 0.0))
    arcs = lid.get("mask_deg") or []
    flat = []
    for a in arcs:
        if len(a) != 2:
            raise ConfigError("lidar.mask_deg entries must be [from_deg, to_deg]")
        # The config gives arcs in base_link (0 = forward). The filter works
        # on beam angles in the laser frame, so subtract the mount yaw.
        flat += [float(a[0]) - yaw, float(a[1]) - yaw]
    mask_enabled = bool(flat)
    if not flat:
        flat = [0.0, 0.0]          # ROS cannot type an empty list; enabled flag says off
    fp = footprint(cfg, padded=False)
    log_dir = dash.get("log_dir", "~/robot_logs")
    return HEADER + f"""
mcu_bridge:
  ros__parameters:
    port: /dev/{mcu['device_name']}
    baud: {int(mcu['baud'])}
    wheel_names: {slist(names)}
    max_wheel_speed: {yflt(mcu['max_wheel_speed'])}
    telemetry_mode: 2              # 2 = extended, includes wheel angles for odometry
    telemetry_hz: {yflt(mcu.get('telemetry_hz', 20))}
    cmd_timeout: 0.5               # s without /wheel_cmd → send zeros

base_controller:
  ros__parameters:
{geom}    max_wheel_speed: {yflt(mcu['max_wheel_speed'])}
    max_linear: {yflt(max(tel['max_linear'], cfg['nav']['max_vx'], cfg['nav'].get('max_vy', 0)))}
    max_angular: {yflt(max(tel['max_angular'], cfg['nav']['max_wz']))}
    cmd_timeout: 0.5

wheel_odometry:
  ros__parameters:
{geom}    lateral_scale: {yflt(rb.get('lateral_scale', 1.0))}
    odom_frame: odom
    base_frame: base_link
    publish_tf: true

scan_filter:
  ros__parameters:
    input_topic: scan
    output_topic: scan_filtered
    mirror: {str(bool(lid.get('mirror', False))).lower()}
    mask_enabled: {str(mask_enabled).lower()}
    mask_deg: {ylist(flat)}       # laser-frame degrees, pairs [from, to]
    range_min: {yflt(lid.get('range_min', 0.1))}
```

```
    range_max: {yflt(lid.get('range_max', 10.0))}

telemetry_logger:
  ros__parameters:
    wheel_names: {slist(names)}
    log_dir: {log_dir}

dashboard:
  ros__parameters:
    port: {int(dash.get('port', 8080))}
    log_dir: {log_dir}
    map_dir: {dash.get('map_dir', '~/maps')}
    wheel_names: {slist(names)}
    holonomic: {str(rb['drive_type'] in HOLONOMIC_TYPES).lower()}
    max_linear: {yflt(tel['max_linear'])}
    max_angular: {yflt(tel['max_angular'])}
    footprint_x: {ylist(p[0] for p in fp)}
    footprint_y: {ylist(p[1] for p in fp)}
    laser_x: {yflt(lid['mount'].get('x', 0))}
    laser_y: {yflt(lid['mount'].get('y', 0))}
"""


# ============================================================
#  Nav2
# ============================================================

NAV2 = Template(HEADER + r"""#
# REP-103 throughout: +x forward, +y left. Stock Nav2 assumes exactly this,
# so no value below is axis-swapped (NarrowAisleBot's file is, and that file
# calls its swap "the stack's deepest open issue").
#
# Status: a STARTING POINT. The structure is NarrowAisleBot's, which brings
# up cleanly on Nav2 1.3.x (Jazzy). The numbers are derived from your
# robot_config.yaml and have not been driven on your robot. Expect to tune
# the controller (manual chapter 14).

bt_navigator:
  ros__parameters:
    global_frame: map
    robot_base_frame: base_link
    odom_topic: /odom
    bt_loop_duration: 10
    default_server_timeout: 20
    wait_for_service_timeout: 1000

controller_server:
  ros__parameters:
    controller_frequency: ${controller_hz}
    # Set explicitly. Left unset, the controller listens on its default
    # topic and, on NarrowAisleBot, ran for weeks with no measured velocity.
    odom_topic: /odom
    min_x_velocity_threshold: 0.001
    min_y_velocity_threshold: ${min_y_threshold}
    min_theta_velocity_threshold: 0.001
    failure_tolerance: 0.3
    progress_checker_plugins: ["progress_checker"]
    goal_checker_plugins: ["goal_checker"]
    controller_plugins: ["FollowPath"]
    progress_checker:
      plugin: "nav2_controller::PoseProgressChecker"
      required_movement_radius: 0.10
      required_movement_angle: 0.25
      movement_time_allowance: 10.0
    goal_checker:
      plugin: "nav2_controller::SimpleGoalChecker"
      xy_goal_tolerance: ${goal_xy}
      yaw_goal_tolerance: ${goal_yaw}
      stateful: true
${controller_block}
planner_server:
  ros__parameters:
    expected_planner_frequency: 5.0
    planner_plugins: ["GridBased"]
    GridBased:
      plugin: "nav2_navfn_planner::NavfnPlanner"
      tolerance: 0.5
      use_astar: true
      allow_unknown: true

smoother_server:
  ros__parameters:
```

```
    smoother_plugins: ["simple_smoother"]
    simple_smoother:
      plugin: "nav2_smoother::SimpleSmoother"
      tolerance: 1.0e-10
      max_its: 1000
      do_refinement: true

behavior_server:
  ros__parameters:
    local_costmap_topic: local_costmap/costmap_raw
    global_costmap_topic: global_costmap/costmap_raw
    local_footprint_topic: local_costmap/published_footprint
    global_footprint_topic: global_costmap/published_footprint
    cycle_frequency: 10.0
    behavior_plugins: ["spin", "backup", "wait"]
    spin:
      plugin: "nav2_behaviors::Spin"
    backup:
      plugin: "nav2_behaviors::BackUp"
    wait:
      plugin: "nav2_behaviors::Wait"
    local_frame: odom
    global_frame: map
    robot_base_frame: base_link
    transform_tolerance: 0.1
    simulate_ahead_time: 2.0
    max_rotational_vel: ${max_wz}
    min_rotational_vel: 0.2
    rotational_acc_lim: ${accel_wz}

local_costmap:
  local_costmap:
    ros__parameters:
      update_frequency: 5.0
      publish_frequency: 2.0
      global_frame: odom
      robot_base_frame: base_link
      rolling_window: true
      width: 3
      height: 3
      resolution: 0.05
      footprint: "${footprint}"
      plugins: ["obstacle_layer", "inflation_layer"]
      obstacle_layer:
        plugin: "nav2_costmap_2d::ObstacleLayer"
        enabled: true
        observation_sources: scan
        scan:
          topic: /scan_filtered
          max_obstacle_height: 2.0
          clearing: true
          marking: true
          data_type: "LaserScan"
          raytrace_max_range: ${raytrace_range}
          raytrace_min_range: 0.0
          obstacle_max_range: ${obstacle_range}
          obstacle_min_range: 0.0
      inflation_layer:
        plugin: "nav2_costmap_2d::InflationLayer"
        cost_scaling_factor: 5.0
        inflation_radius: ${inflation}

global_costmap:
  global_costmap:
    ros__parameters:
      update_frequency: 1.0
      publish_frequency: 1.0
      global_frame: map
      robot_base_frame: base_link
      resolution: ${resolution}
      track_unknown_space: true
      footprint: "${footprint}"
      plugins: ["static_layer", "obstacle_layer", "inflation_layer"]
      static_layer:
        plugin: "nav2_costmap_2d::StaticLayer"
        map_subscribe_transient_local: true
      obstacle_layer:
        plugin: "nav2_costmap_2d::ObstacleLayer"
        enabled: true
        observation_sources: scan
        scan:
          topic: /scan_filtered
```

```
          max_obstacle_height: 2.0
          clearing: true
          marking: true
          data_type: "LaserScan"
          raytrace_max_range: ${raytrace_range}
          raytrace_min_range: 0.0
          obstacle_max_range: ${obstacle_range}
          obstacle_min_range: 0.0
      inflation_layer:
        plugin: "nav2_costmap_2d::InflationLayer"
        cost_scaling_factor: 5.0
        inflation_radius: ${inflation}

map_saver:
  ros__parameters:
    save_map_timeout: 5.0
    free_thresh_default: 0.25
    occupied_thresh_default: 0.65

waypoint_follower:
  ros__parameters:
    loop_rate: 20
    stop_on_failure: false
    waypoint_task_executor_plugin: "wait_at_waypoint"
    wait_at_waypoint:
      plugin: "nav2_waypoint_follower::WaitAtWaypoint"
      enabled: true
      waypoint_pause_duration: 200

velocity_smoother:
  ros__parameters:
    smoothing_frequency: 20.0
    scale_velocities: false
    feedback: "OPEN_LOOP"
    max_velocity: [${max_vx}, ${max_vy}, ${max_wz}]
    min_velocity: [${min_vx}, ${min_vy}, ${neg_max_wz}]
    max_accel: [${accel_vx}, ${accel_vy}, ${accel_wz}]
    max_decel: [${decel_vx}, ${decel_vy}, ${decel_wz}]
    odom_topic: /odom
    odom_duration: 0.1
    deadband_velocity: [0.0, 0.0, 0.0]
    velocity_timeout: 1.0

# The last safety layer before the wheels. It reads the live scan only, no
# memory, and forward-simulates the real footprint along the command.
collision_monitor:
  ros__parameters:
    base_frame_id: "base_link"
    odom_frame_id: "odom"
    cmd_vel_in_topic: "cmd_vel_smoothed"
    cmd_vel_out_topic: "cmd_vel_nav_out"     # straight into twist_mux
    state_topic: "collision_monitor_state"
    transform_tolerance: 0.2
    source_timeout: 1.0
    base_shift_correction: True
    stop_pub_timeout: 2.0
    polygons: ["FootprintApproach"]
    FootprintApproach:
      type: "polygon"
      action_type: "approach"
      footprint_topic: "/local_costmap/published_footprint"
      time_before_collision: 1.2
      simulation_time_step: 0.1
      min_points: 6
      visualize: True
      polygon_pub_topic: "collision_monitor_polygon"
      enabled: True
    observation_sources: ["scan"]
    scan:
      type: "scan"
      topic: "/scan_filtered"
      enabled: True
""")

RPP_BLOCK = Template(r"""    FollowPath:
      # Regulated Pure Pursuit: simple and predictable on a differential or
      # skid-steer base. Rotates to the path heading first, then follows.
      plugin: "nav2_regulated_pure_pursuit_controller::RegulatedPurePursuitController"
      desired_linear_vel: ${max_vx}
      lookahead_dist: 0.6
      min_lookahead_dist: 0.3
      max_lookahead_dist: 0.9
```

```
      lookahead_time: 1.5
      rotate_to_heading_angular_vel: ${rot_vel}
      transform_tolerance: 0.1
      use_velocity_scaled_lookahead_dist: false
      min_approach_linear_velocity: 0.05
      approach_velocity_scaling_dist: 0.6
      use_collision_detection: true
      max_allowed_time_to_collision_up_to_carrot: 1.0
      use_regulated_linear_velocity_scaling: true
      use_cost_regulated_linear_velocity_scaling: false
      regulated_linear_scaling_min_radius: 0.9
      regulated_linear_scaling_min_speed: 0.05
      use_rotate_to_heading: true
      allow_reversing: false
      rotate_to_heading_min_angle: 0.785
      max_angular_accel: ${accel_wz}
      max_robot_pose_search_dist: 10.0
"""")

MPPI_BLOCK = Template(r"""    FollowPath:
      # MPPI. Samples ${batch} trajectories of ${steps} steps every cycle.
      # On NarrowAisleBot (Pi 5, SLAM running) 300 x 40 at 0.05 s managed
      # 5.5 to 8.9 Hz against 20 requested, and a 0.6 m goal crept at about
      # 4 mm/s (manual 14.5). model_dt here matches controller_frequency.
      plugin: "nav2_mppi_controller::MPPIController"
      motion_model: "${motion_model}"
      vx_max: ${max_vx}
      vx_min: ${min_vx}
      vy_max: ${max_vy}
      wz_max: ${max_wz}
      ax_max: ${accel_vx}
      ax_min: ${decel_vx}
      ay_max: ${accel_vy}
      az_max: ${accel_wz}
      vx_std: 0.2
      vy_std: ${vy_std}
      wz_std: 0.4
      batch_size: ${batch}
      time_steps: ${steps}
      model_dt: ${model_dt}
      iteration_count: 1
      temperature: 0.3
      gamma: 0.015
      retry_attempt_limit: 1
      regenerate_noises: false
      visualize: false
      critics: [${critics}]
      ConstraintCritic:
        enabled: true
        cost_power: 1
        cost_weight: 4.0
      CostCritic:
        enabled: true
        cost_power: 1
        cost_weight: 3.81
        critical_cost: 300.0
        consider_footprint: true
        collision_cost: 1000000.0
        near_goal_distance: 0.5
        trajectory_point_step: 2
      GoalCritic:
        enabled: true
        cost_power: 1
        cost_weight: 5.0
        threshold_to_consider: 1.4
      GoalAngleCritic:
        enabled: true
        cost_power: 1
        cost_weight: 3.0
        threshold_to_consider: 0.5
      PathAlignCritic:
        enabled: true
        cost_power: 1
        cost_weight: 14.0
        max_path_occupancy_ratio: 0.05
        trajectory_point_step: 4
        threshold_to_consider: 0.5
        offset_from_furthest: 20
        use_path_orientations: false
      PathFollowCritic:
        enabled: true
        cost_power: 1
```

```
        cost_weight: 5.0
        offset_from_furthest: 5
        threshold_to_consider: 1.4
      PathAngleCritic:
        enabled: ${diff_only}
        cost_power: 1
        cost_weight: 2.0
        offset_from_furthest: 4
        threshold_to_consider: 0.5
        max_angle_to_furthest: 1.0
        mode: 0
      PreferForwardCritic:
        enabled: ${diff_only}
        cost_power: 1
        cost_weight: 5.0
        threshold_to_consider: 0.5
      TwirlingCritic:
        enabled: true
        cost_power: 1
        cost_weight: 10.0
""")


def nav2_params(cfg):
    nav, rb, lid = cfg["nav"], cfg["robot"], cfg["lidar"]
    holo = rb["drive_type"] in HOLONOMIC_TYPES
    max_vx, min_vx = float(nav["max_vx"]), float(nav.get("min_vx", -0.1))
    max_vy = float(nav.get("max_vy", 0.0)) if holo else 0.0
    max_wz = float(nav["max_wz"])
    acc = [float(a) for a in nav.get("accel", [0.3, 0.3, 0.8])]
    dec = [float(d) for d in nav.get("decel", [-0.5, -0.5, -1.0])]
    if not holo:
        acc[1], dec[1] = 0.0, 0.0
    fp = footprint(cfg, padded=True)
    fp_s = "[" + ", ".join(f"[{x:.3f}, {y:.3f}]" for x, y in fp) + "]"
    ctrl_hz = float(nav.get("controller_hz", 10.0))
    rng = min(float(lid.get("range_max", 10.0)), 8.0)
    v = dict(
        controller_hz=yflt(ctrl_hz), min_y_threshold="0.001" if holo else "0.5",
        goal_xy=yflt(nav["goal_xy_tolerance"]), goal_yaw=yflt(nav["goal_yaw_tolerance"]),
        max_vx=yflt(max_vx), min_vx=yflt(min_vx), max_vy=yflt(max_vy),
        min_vy=yflt(-max_vy), max_wz=yflt(max_wz), neg_max_wz=yflt(-max_wz),
        accel_vx=yflt(acc[0]), accel_vy=yflt(acc[1]), accel_wz=yflt(acc[2]),
        decel_vx=yflt(dec[0]), decel_vy=yflt(dec[1]), decel_wz=yflt(dec[2]),
        footprint=fp_s, inflation=yflt(nav["inflation_radius"]),
        resolution=yflt(cfg["slam"].get("resolution", 0.05)),
        raytrace_range=yflt(rng + 0.5), obstacle_range=yflt(rng),
        rot_vel=yflt(min(max_wz, 1.0)),
    )
    if nav["controller"] == "rpp":
        if holo:
            print("  ! nav.controller rpp on a holonomic base: it will never strafe")
        v["controller_block"] = RPP_BLOCK.substitute(v)
    else:
        steps = max(20, int(round(3.0 * ctrl_hz)))
        critics = ["ConstraintCritic", "CostCritic", "GoalCritic", "GoalAngleCritic",
                   "PathAlignCritic", "PathFollowCritic", "TwirlingCritic"]
        if not holo:
            critics[5:5] = ["PathAngleCritic", "PreferForwardCritic"]
        v["controller_block"] = MPPI_BLOCK.substitute(
            v, motion_model="Omni" if holo else "DiffDrive", batch=500, steps=steps,
            model_dt=yflt(1.0 / ctrl_hz), vy_std="0.2" if holo else "0.0",
            critics=", ".join(f'"{c}"' for c in critics),
            diff_only="false" if holo else "true")
    return NAV2.substitute(v)


# ═══════════════════════════════════════════════════════════════
#  slam_toolbox, LiDAR, twist_mux
# ═══════════════════════════════════════════════════════════════

def slam_params(cfg):
    s = cfg["slam"]
    return HEADER + f"""#
# slam_toolbox online async. Mostly the package's own defaults
# (mapper_params_online_async.yaml), with the choices below taken from
# robot_config.yaml. NarrowAisleBot's measured values are in the manual
# (chapter 13); they suit that robot's sensor, not necessarily yours.
slam_toolbox:
  ros__parameters:
    solver_plugin: solver_plugins::CeresSolver
    ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
```

```
    ceres_preconditioner: SCHUR_JACOBI
    ceres_trust_strategy: LEVENBERG_MARQUARDT
    ceres_dogleg_type: TRADITIONAL_DOGLEG
    ceres_loss_function: None

    odom_frame: odom
    map_frame: map
    base_frame: base_link
    scan_topic: /scan_filtered
    mode: mapping

    debug_logging: false
    throttle_scans: 1
    transform_publish_period: 0.02
    map_update_interval: 1.0
    resolution: {yflt(s.get('resolution', 0.05))}
    min_laser_range: 0.0
    max_laser_range: {yflt(s.get('max_laser_range', 6.0))}
    minimum_time_interval: 0.5
    transform_timeout: 0.2
    tf_buffer_duration: 30.0
    stack_size_to_use: 40000000
    enable_interactive_mode: false

    use_scan_matching: {str(bool(s.get('use_scan_matching', True))).lower()}
    use_scan_barycenter: true
    minimum_travel_distance: {yflt(s.get('minimum_travel_distance', 0.2))}
    minimum_travel_heading: {yflt(s.get('minimum_travel_heading', 0.2))}
    scan_buffer_size: 10
    scan_buffer_maximum_scan_distance: 10.0
    link_match_minimum_response_fine: 0.1
    link_scan_maximum_distance: 1.5
    loop_search_maximum_distance: 3.0
    do_loop_closing: true
    loop_match_minimum_chain_size: 10
    loop_match_maximum_variance_coarse: 3.0
    loop_match_minimum_response_coarse: 0.35
    loop_match_minimum_response_fine: 0.45

    correlation_search_space_dimension: 0.5
    correlation_search_space_resolution: 0.01
    correlation_search_space_smear_deviation: 0.1
    loop_search_space_dimension: 8.0
    loop_search_space_resolution: 0.05
    loop_search_space_smear_deviation: 0.03
    distance_variance_penalty: 0.5
    angle_variance_penalty: 1.0
    fine_search_angle_offset: 0.00349
    coarse_search_angle_offset: 0.349
    coarse_angle_resolution: 0.0349
    minimum_angle_penalty: 0.9
    minimum_distance_penalty: 0.5
    use_response_expansion: true
"""


YDLIDAR_DEFAULTS = {
    # NarrowAisleBot's YDLIDAR X4 Pro values, byte-verified against the live
    # robot. The nesting under ros__parameters is load-bearing: a flat file
    # binds nothing and the driver silently runs on compiled defaults.
    "frame_id": "laser_frame", "ignore_array": "", "baudrate": 128000,
    "lidar_type": 1, "device_type": 0, "isSingleChannel": True,
    "intensity": False, "intensity_bit": 0, "sample_rate": 5,
    "abnormal_check_count": 4, "fixed_resolution": True, "reversion": False,
    "inverted": False, "auto_reconnect": True, "support_motor_dtr": False,
    "angle_max": 180.0, "angle_min": -180.0, "range_max": 10.0, "range_min": 0.1,
    "frequency": 10.0, "invalid_range_is_inf": False, "debug": False,
}


def yval(v):
    if isinstance(v, bool):
        return "true" if v else "false"
    if isinstance(v, (int, float)):
        return yflt(v) if isinstance(v, float) else str(v)
    return f'"{v}"'


def lidar_params(cfg):
    lid = cfg["lidar"]
    model = lid.get("model", "none")
    if model == "ydlidar":
```

```
        p = dict(YDLIDAR_DEFAULTS)
        p.update(port=f"/dev/{lid['device_name']}", frame_id=lid.get("frame_id", "laser_frame"),
                 baudrate=int(lid["baud"]), range_min=float(lid.get("range_min", 0.1)),
                 range_max=float(lid.get("range_max", 10.0)))
        p.update(lid.get("driver_overrides") or {})
        node = "ydlidar_ros2_driver_node"
    elif model == "rplidar":
        p = {"serial_port": f"/dev/{lid['device_name']}", "serial_baudrate": int(lid["baud"]),
             "frame_id": lid.get("frame_id", "laser_frame"), "inverted": False,
             "angle_compensate": True}
        p.update(lid.get("driver_overrides") or {})
        node = "rplidar_node"
    else:
        return HEADER + "# lidar.model is none or custom: no driver parameters generated.\n"
    body = "".join(f"    {k}: {yval(v)}\n" for k, v in p.items())
    return HEADER + f"{node}:\n  ros__parameters:\n{body}"


TWIST_MUX = HEADER + """#
# Manual always beats autonomous while it is publishing, and stops beating it
# 0.5 s after the last manual message. You should never have to fight the
# planner for the robot.
twist_mux:
  ros__parameters:
    # Plain geometry_msgs/Twist everywhere in this kit. twist_mux 4.x can
    # default to TwistStamped; without this line the topics silently fail to
    # match (seen live on NarrowAisleBot, 21 Aug 2026).
    use_stamped: false
    topics:
      manual:
        topic: cmd_vel_manual
        timeout: 0.5
        priority: 100
      navigation:
        topic: cmd_vel_nav_out
        timeout: 0.5
        priority: 10
"""


def kit_yaml(cfg):
    """Facts the LAUNCH files need (not node parameters): which LiDAR driver."""
    lid = cfg["lidar"]
    cust = lid.get("custom") or {}
    return HEADER + f"""lidar:
  model: {lid.get('model', 'none')}
  package: "{cust.get('package', '')}"
  executable: "{cust.get('executable', '')}"
  params_file: "{cust.get('params_file', '')}"
robot:
  name: {cfg['robot']['name']}
  drive_type: {cfg['robot']['drive_type']}
"""


# ═══════════════════════════════════════════════════════════════
#  URDF
# ═══════════════════════════════════════════════════════════════

def urdf(cfg):
    rb, lid = cfg["robot"], cfg["lidar"]
    body, r = rb["body"], float(rb["wheel_radius"])
    L, W, H = float(body["length"]), float(body["width"]), float(body.get("height", 0.2))
    cx = float(body.get("center_x", 0.0))
    m = lid["mount"]
    wheels = ""
    for w in cfg["wheels"]:
        n = w["name"]
        wheels += f"""
  <link name="wheel_{n}">
    <visual>
      <origin xyz="0 0 0" rpy="1.5708 0 0"/>
      <geometry><cylinder radius="{r:.4f}" length="0.04"/></geometry>
      <material name="black"/>
    </visual>
  </link>
  <!-- fixed, not continuous: nothing publishes /joint_states, and a
       continuous joint without it simply vanishes from TF -->
  <joint name="wheel_{n}_joint" type="fixed">
    <parent link="base_link"/>
    <child link="wheel_{n}"/>
    <origin xyz="{w['x']:.4f} {w['y']:.4f} {r:.4f}" rpy="0 0 0"/>
```

```
  </joint>
"""
    return f"""<?xml version="1.0"?>
<!-- GENERATED by robot_kit/generate.py from robot_config.yaml. Do not edit. -->
<!-- base_link sits on the floor, directly under the point the robot turns
     about. +x forward, +y left, +z up (REP-103). Every z here is a height
     above the floor, so you can measure it with a ruler. -->
<robot name="{rb['name']}">
  <material name="grey"><color rgba="0.55 0.55 0.6 1"/></material>
  <material name="black"><color rgba="0.1 0.1 0.1 1"/></material>
  <material name="blue"><color rgba="0.2 0.4 0.9 1"/></material>

  <link name="base_link"/>

  <link name="chassis">
    <visual>
      <geometry><box size="{L:.4f} {W:.4f} {H:.4f}"/></geometry>
      <material name="grey"/>
    </visual>
  </link>
  <joint name="chassis_joint" type="fixed">
    <parent link="base_link"/>
    <child link="chassis"/>
    <origin xyz="{cx:.4f} 0 {r + H / 2:.4f}" rpy="0 0 0"/>
  </joint>
{wheels}
  <link name="{lid.get('frame_id', 'laser_frame')}">
    <visual>
      <geometry><cylinder radius="0.035" length="0.04"/></geometry>
      <material name="blue"/>
    </visual>
  </link>
  <joint name="laser_joint" type="fixed">
    <parent link="base_link"/>
    <child link="{lid.get('frame_id', 'laser_frame')}"/>
    <origin xyz="{float(m.get('x', 0)):.4f} {float(m.get('y', 0)):.4f} {float(m.get('z', 0.2)):.4f}" rpy="0 0
{math.radians(float(m.get('yaw_deg', 0))):.6f}"/>
  </joint>
</robot>
"""


# ═══════════════════════════════════════════════════════════════
#  Firmware header
# ═══════════════════════════════════════════════════════════════

def c_arr(vals, fmt="{:.6f}f"):
    return "{" + ", ".join(fmt.format(v) for v in vals) + "}"


def velocity_quantum(cfg, window):
    """rad/s that one encoder count represents, over `window` control ticks,
    on the coarsest encoder."""
    cpr = min(w["counts_per_rev"] for w in cfg["wheels"])
    dt = 1.0 / float(cfg["mcu"].get("control_hz", 100))
    return 2 * math.pi / (cpr * dt * window)


def velocity_window(cfg, target=0.05, cap=8):
    """Smallest window (in ticks) that gets the quantum to `target` rad/s,
    capped so the lag stays bounded."""
    return max(1, min(cap, math.ceil(velocity_quantum(cfg, 1) / target - 1e-9)))


def firmware_header(cfg):
    mcu, W, dtype = cfg["mcu"], cfg["wheels"], cfg["driver"]["type"]
    model = cfg["_model"]
    n = len(W)
    pa, pb, pc = DRIVER_PINS[dtype]

    def pins(key):
        return [int(w["pins"][key]) if key else -1 for w in W]

    # Inverse kinematics coefficients for <B,vx,vy,wz>: w_i = cvx*vx + cvy*vy + cwz*wz
    ik = [model.inverse(1, 0, 0), model.inverse(0, 1, 0), model.inverse(0, 0, 1)]
    driver_const = {"pwm_dir": "DRIVER_PWM_DIR", "pwm_in1_in2": "DRIVER_PWM_IN1_IN2",
                    "dual_pwm": "DRIVER_DUAL_PWM"}[dtype]
    enc_const = "ENC_PCNT" if mcu["encoder_backend"] == "pcnt" else "ENC_ISR"
    pid = mcu.get("pid", {})
    bits = int(mcu.get("pwm_bits", 8))
    win = velocity_window(cfg)
    return f"""// GENERATED by robot_kit/generate.py from robot_config.yaml. Do not edit.
```

```
// Robot: {cfg['robot']['name']}  drive: {cfg['robot']['drive_type']}  driver: {dtype}
#pragma once

#define ROBOT_NAME "{cfg['robot']['name']}"
#define NUM_MOTORS {n}

// Motor order = wheel order in robot_config.yaml = order on the wire.
static const char* const MOTOR_NAMES[NUM_MOTORS] = {{{", ".join(f'"{w["name"]}"' for w in W)}}};

// ── Motor driver ─────────────────────────────────────────────
#define DRIVER_PWM_DIR      1   // PIN_A = PWM, PIN_B = DIR
#define DRIVER_PWM_IN1_IN2  2   // PIN_A = PWM/EN, PIN_B = IN1, PIN_C = IN2
#define DRIVER_DUAL_PWM     3   // PIN_A = PWM forward, PIN_B = PWM reverse
#ifndef DRIVER_TYPE
#define DRIVER_TYPE {driver_const}
#endif
static const int PIN_A[NUM_MOTORS] = {c_arr(pins(pa), "{}")};   // {pa}
static const int PIN_B[NUM_MOTORS] = {c_arr(pins(pb), "{}")};   // {pb}
static const int PIN_C[NUM_MOTORS] = {c_arr(pins(pc), "{}")};   // {pc or "unused"}

// ── Encoders ─────────────────────────────────────────────────
#define ENC_PCNT 1              // ESP32 hardware pulse counters
#define ENC_ISR  2              // pin-change interrupts, any board
#ifndef ENCODER_BACKEND
#define ENCODER_BACKEND {enc_const}
#endif
#define ENCODER_DECODING {W[0]['encoder_decoding']}      // 4 = every edge of A and B, 2 = edges of A only
static const int ENC_A[NUM_MOTORS] = {c_arr(pins('enc_a'), "{}")};
static const int ENC_B[NUM_MOTORS] = {c_arr(pins('enc_b'), "{}")};
// Counts per WHEEL revolution, as this firmware counts them.
static const float ENCODER_CPR[NUM_MOTORS] = {c_arr([w['counts_per_rev'] for w in W], "{:.1f}f")};

// +1 / -1. Set independently by two bench tests (manual 7.5 and 7.6).
static const int MOTOR_DIR[NUM_MOTORS] = {c_arr([w['motor_dir'] for w in W], "{:d}")};
static const int ENC_DIR[NUM_MOTORS]   = {c_arr([w['encoder_dir'] for w in W], "{:d}")};

// ── Kinematics for <B,vx,vy,wz> (the Pi normally sends <V> instead) ──
static const float IK_VX[NUM_MOTORS] = {c_arr(ik[0])};
static const float IK_VY[NUM_MOTORS] = {c_arr(ik[1])};
static const float IK_WZ[NUM_MOTORS] = {c_arr(ik[2])};

// ── Link and timing ──────────────────────────────────────────
#ifndef SERIAL_BAUD
#define SERIAL_BAUD {int(mcu['baud'])}
#endif
#define CONTROL_HZ {int(mcu.get('control_hz', 100))}
// Velocity is measured over the last VEL_WINDOW control ticks. One count in
// one tick is {velocity_quantum(cfg, 1):.4f} rad/s on the coarsest encoder; over {win} tick(s)
// it is {velocity_quantum(cfg, win):.4f} rad/s. Coarse encoders need a window, at the
// cost of {(win - 1) * 500.0 / int(mcu.get('control_hz', 100)):.0f} ms of extra lag (manual 4.3).
#define VEL_WINDOW {win}
#define TELEMETRY_HZ_DEFAULT {float(mcu.get('telemetry_hz', 20)):.1f}f
#define WATCHDOG_MS {int(mcu.get('watchdog_ms', 750))}

// ── Limits ───────────────────────────────────────────────────
#define MAX_WHEEL_SPEED {float(mcu['max_wheel_speed']):.3f}f     // rad/s
#define MAX_WHEEL_ACCEL {float(mcu.get('max_wheel_accel', 0)):.3f}f     // rad/s^2, 0 = off
#define PWM_BITS {bits}
#define PWM_MAX {(1 << bits) - 1}
#define PWM_FREQ_HZ {int(mcu.get('pwm_freq_hz', 20000))}
#define MIN_PWM_OUT {int(mcu.get('min_pwm_out', 0))}

// ── Controller (replace with measured values, manual chapter 9) ─────
#define KP_INIT {float(pid.get('kp', 10)):.4f}f
#define KI_INIT {float(pid.get('ki', 50)):.4f}f
#define KD_INIT {float(pid.get('kd', 0)):.4f}f
static const float KFF_INIT[NUM_MOTORS]   = {c_arr(mcu['kff'], "{:.4f}f")};
static const float KSTAT_INIT[NUM_MOTORS] = {c_arr(mcu['kstat'], "{:.4f}f")};
"""


# ═══════════════════════════════════════════════════════════════
#  udev and shell environment
# ═══════════════════════════════════════════════════════════════

def udev_line(dev, comment):
    vid, pid = str(dev.get("usb_vendor_id", "")).lower(), str(dev.get("usb_product_id", "")).lower()
    if not vid or not pid:
        return f"# {comment}: no usb_vendor_id / usb_product_id in robot_config.yaml\n"
    m = ['SUBSYSTEM=="tty"', f'ATTRS{{idVendor}}=="{vid}"', f'ATTRS{{idProduct}}=="{pid}"']
    if dev.get("usb_serial"):
        m.append(f'ATTRS{{serial}}=="{dev["usb_serial"]}"')
```

```
    if dev.get("usb_port_path"):
        m.append(f'KERNELS=="{dev["usb_port_path"]}"')
    m += [f'SYMLINK+="{dev["device_name"]}"', 'MODE="0666"']
    return f"# {comment}\n" + ", ".join(m) + "\n"


def identity(dev):
    return (str(dev.get("usb_vendor_id", "")).lower(), str(dev.get("usb_product_id", "")).lower(),
            dev.get("usb_serial") or "", dev.get("usb_port_path") or "")


def udev_rules(cfg, warnings):
    mcu, lid = cfg["mcu"], cfg["lidar"]
    out = "# GENERATED by robot_kit/generate.py. Install with setup/05_system.sh.\n"
    out += udev_line(mcu, f"microcontroller → /dev/{mcu['device_name']}")
    if lid.get("model") not in ("none", None):
        out += udev_line(lid, f"LiDAR → /dev/{lid['device_name']}")
        if identity(mcu) == identity(lid):
            warnings.append(
                "The MCU and the LiDAR have the SAME USB identity (VID, PID, serial, port path). "
                "udev cannot tell them apart. Fill usb_port_path for both (manual 8.2). "
                "This is exactly what bit NarrowAisleBot: two CP2102 adapters, same serial 0001.")
    return out


def robot_env(cfg):
    s, mcu, lid, ap = cfg["system"], cfg["mcu"], cfg["lidar"], cfg["system"].get("wifi_ap", {})
    vals = {
        "ROBOT_NAME": cfg["robot"]["name"],
        "ROBOT_WS": s.get("workspace", "~/ros2_ws").replace("~", "$HOME", 1),
        "ROBOT_DOMAIN_ID": str(int(s.get("ros_domain_id", 42))),
        "ROBOT_AUTOSTART": "true" if s.get("autostart", True) else "false",
        "MCU_DEVICE": mcu["device_name"], "MCU_BAUD": str(int(mcu["baud"])),
        "LIDAR_MODEL": lid.get("model", "none"), "LIDAR_DEVICE": lid.get("device_name", "lidar"),
        "DASHBOARD_PORT": str(int(cfg["dashboard"].get("port", 8080))),
        "LOG_DIR": cfg["dashboard"].get("log_dir", "~/robot_logs").replace("~", "$HOME", 1),
        "AP_ENABLE": "true" if ap.get("enable") else "false",
        "AP_SSID": ap.get("ssid", "MyRobot-Pi"), "AP_PASSWORD": ap.get("password", ""),
        "AP_COUNTRY": ap.get("country", "IN"),
    }
    lines = ["# GENERATED by robot_kit/generate.py. Sourced by the setup scripts."]
    for k, v in vals.items():
        if "$HOME" in v:
            lines.append(f'{k}="{v}"')
        else:
            lines.append(f"{k}={shlex.quote(v)}")
    return "\n".join(lines) + "\n"


# ═══════════════════════════════════════════════════════════════
#  Summary and sanity checks
# ═══════════════════════════════════════════════════════════════

def sanity(cfg, warnings):
    mcu, W, rb = cfg["mcu"], cfg["wheels"], cfg["robot"]
    vmax = float(mcu["max_wheel_speed"])
    model = cfg["_model"]
    print(f"\n  robot '{rb['name']}': {rb['drive_type']}, {len(W)} wheels, "
          f"r = {rb['wheel_radius']} m, controller {cfg['nav']['controller']}")
    print("    #  name            x (m)    y (m)   counts/rev   edges/s at max speed")
    worst = 0.0
    for i, w in enumerate(W):
        edges = w["counts_per_rev"] * vmax / (2 * math.pi)
        worst = max(worst, edges)
        print(f"    {i}  {w['name']:<14} {w['x']:+.4f}  {w['y']:+.4f}   {w['counts_per_rev']:10.1f}"
              f"   {edges:10.0f}")
    top = vmax * float(rb["wheel_radius"])
    print(f"    wheel limit {vmax} rad/s = {top:.3f} m/s straight-line top speed")
    win = velocity_window(cfg)
    qn = velocity_quantum(cfg, win)
    print(f"    velocity resolution {qn:.4f} rad/s (window {win} tick(s) at "
          f"{mcu.get('control_hz', 100)} Hz)")
    if qn > 0.1:
        warnings.append(f"encoders are coarse: even over {win} ticks one count is {qn:.3f} rad/s. "
                        f"Use x4 decoding, a lower control_hz, or encoders on the motor shaft "
                        f"rather than the wheel (manual 4.3).")
    if cfg["nav"]["max_vx"] > top:
        warnings.append(f"nav.max_vx {cfg['nav']['max_vx']} m/s is above what the wheel limit "
                        f"allows ({top:.3f} m/s)")
    if cfg["teleop"]["max_linear"] > top:
        warnings.append(f"teleop.max_linear {cfg['teleop']['max_linear']} m/s is above the "
                        f"wheel limit's {top:.3f} m/s")
```

```
    if mcu["encoder_backend"] == "isr":
        total = worst * len(W)
        limit = 40000 if mcu.get("board") == "avr" else 300000
        if total > limit:
            warnings.append(f"ISR encoders: about {total:.0f} interrupts/s at top speed, more than "
                            f"this board handles comfortably (~{limit}). Lower max_wheel_speed, use "
                            f"encoder_decoding 2, or move to an ESP32 with pcnt (manual 4.2).")
    if mcu.get("board") == "avr" and int(mcu.get("baud", 0)) > 250000:
        warnings.append("AVR boards are unreliable above 250000 baud; use 115200")
    if any(k > 0 for k in mcu["kff"]):
        for i, w in enumerate(W):
            need = mcu["kff"][i] * vmax + mcu["kstat"][i]
            pmax = (1 << int(mcu.get("pwm_bits", 8))) - 1
            if need > 0.85 * pmax:
                warnings.append(f"wheel {w['name']}: feedforward alone needs {need:.0f} of {pmax} "
                                f"PWM at max_wheel_speed. Leave the PID ~15 % headroom (manual 9.8).")
    q = model.forward(model.inverse(0.1, 0.05, 0.3))
    print(f"    kinematics round trip (0.1, 0.05, 0.3) → ({q[0]:.4f}, {q[1]:.4f}, {q[2]:.4f})")


def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                 formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--config", default=os.path.join(KIT, "robot_config.yaml"))
    ap.add_argument("--check", action="store_true", help="validate only, write nothing")
    ap.add_argument("--out", default=KIT, help="root to write into (tests use a temp dir)")
    args = ap.parse_args()

    try:
        cfg = load_config(args.config)
    except (ConfigError, OSError) as e:
        print(f"\n  CONFIG ERROR: {e}\n")
        return 2
    warnings = []
    sanity(cfg, warnings)
    try:
        files = {
            "ros2/robot_base/config/robot.yaml": robot_params(cfg),
            "ros2/robot_base/config/nav2_params.yaml": nav2_params(cfg),
            "ros2/robot_base/config/slam_toolbox.yaml": slam_params(cfg),
            "ros2/robot_base/config/lidar.yaml": lidar_params(cfg),
            "ros2/robot_base/config/twist_mux.yaml": TWIST_MUX,
            "ros2/robot_base/config/kit.yaml": kit_yaml(cfg),
            "ros2/robot_base/urdf/robot.urdf": urdf(cfg),
            "firmware/robot_mcu/robot_config.h": firmware_header(cfg),
            "setup/generated/99-robot.rules": udev_rules(cfg, warnings),
            "setup/generated/robot.env": robot_env(cfg),
        }
    except ConfigError as e:
        print(f"\n  CONFIG ERROR: {e}\n")
        return 2

    for w in warnings:
        print(f"\n  WARNING: {w}")
    if args.check:
        print("\n  config OK (--check: nothing written)")
        return 0
    written = []
    for rel, text in files.items():
        write(os.path.join(args.out, rel), text, written)
    print("\n  wrote:")
    for p in written:
        print(f"    {p}")
    print("\n  next: rebuild the workspace (setup/04_workspace.sh) and reflash the MCU "
          "if robot_config.h changed.\n")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

Setup scripts

`setup/install_all.sh` (23 lines)

```
#!/bin/bash
# install_all.sh - a fresh Pi 5 (Ubuntu 24.04) to a working robot stack.
```

```
#
#   cd ~/robot_kit
#   nano robot_config.yaml          # your hardware, FIRST
#   python3 generate.py             # must print no CONFIG ERROR
#   ./setup/install_all.sh
#
# Runs steps 01 to 05, and 06 (Wi-Fi AP) last because it can drop your
# connection. Each step is safe to re-run. Full log: ~/robot_kit_install.log
set -e
D="$(cd "$(dirname "$0")" && pwd)"
python3 "$D/../generate.py" --check
"$D/01_base_system.sh"
"$D/02_ros_jazzy.sh"
"$D/03_lidar_driver.sh"
"$D/04_workspace.sh"
"$D/05_system.sh"
echo
echo "Steps 1-5 done. Log out and back in (serial permissions), then:"
echo "    rk-start        # or: sudo reboot, if autostart is on"
echo "    rk-check        # health check"
echo "Wi-Fi access point (optional, can disconnect you):  $D/06_wifi_ap.sh"
```

`setup/common.sh` (19 lines)

```
# shellcheck shell=bash
# common.sh - sourced by every setup script. Not run on its own.
set -eo pipefail
KIT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
LOG="$HOME/robot_kit_install.log"
G='\033[0;32m'; Y='\033[1;33m'; R='\033[0;31m'; C='\033[0;36m'; B='\033[1m'; N='\033[0m'
ok()   { echo -e "${G}  ok    $*${N}"; echo "ok    $*" >> "$LOG"; }
warn() { echo -e "${Y}  WARN  $*${N}"; echo "WARN  $*" >> "$LOG"; }
die()  { echo -e "${R}  FAIL  $*${N}"; echo "FAIL  $*" >> "$LOG"; exit 1; }
step() { echo -e "\n${C}${B}== $*${N}"; echo "== $* ($(date))" >> "$LOG"; }
run()  { echo "+ $*" >> "$LOG"; "$@" >> "$LOG" 2>&1 || die "command failed: $*  (see $LOG)"; }

[ "$(id -u)" -ne "0" ] || die "run as your normal user, not root (the scripts call sudo themselves)"

load_env() {
    [ -f "$KIT/setup/generated/robot.env" ] || die "no setup/generated/robot.env: run  python3 $KIT/generate.py  first"
    # shellcheck disable=SC1091
    source "$KIT/setup/generated/robot.env"
}
```

`setup/01_base_system.sh` (49 lines)

```
#!/bin/bash
# 01_base_system.sh - OS packages, locale, serial-port permissions.
# Safe to run more than once.
source "$(dirname "$0")/common.sh"

step "1. Base system"
. /etc/os-release
[ "$VERSION_CODENAME" = "noble" ] || die "needs Ubuntu 24.04 (noble); this is $PRETTY_NAME"
ok "Ubuntu $VERSION_ID ($VERSION_CODENAME)"
[ "$(uname -m)" = "aarch64" ] || warn "not aarch64 ($(uname -m)); fine on a PC, unexpected on a Pi 5"
curl -sSf -o /dev/null https://github.com || die "no internet. Connect the Pi (Wi-Fi or Ethernet) first."
ok "internet reachable"

run sudo apt-get update
run sudo apt-get -y upgrade
run sudo apt-get install -y git curl wget rsync build-essential cmake pkg-config \
    python3-pip python3-venv python3-serial python3-yaml python3-numpy python3-matplotlib \
    software-properties-common locales i2c-tools openssh-server net-tools iw
ok "base packages"

run sudo locale-gen en_US en_US.UTF-8
run sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
ok "locale en_US.UTF-8"

# Two Ubuntu desktop services that grab USB serial adapters:
#  brltty (braille displays) claims CH340 and CP210x adapters and the
#  /dev/ttyUSB* node disappears seconds after you plug the board in.
#  ModemManager probes new ttyACM devices with modem AT commands, which an
#  Arduino receives as garbage, and holds the port while it does.
if dpkg -s brltty >/dev/null 2>&1; then
    run sudo apt-get remove -y brltty
```

```
    ok "removed brltty (it steals USB serial adapters)"
fi
if systemctl list-unit-files | grep -q '^ModemManager'; then
    run sudo systemctl disable --now ModemManager
    ok "disabled ModemManager (it probes serial ports)"
fi

run sudo usermod -aG dialout "$USER"
getent group i2c >/dev/null && run sudo usermod -aG i2c "$USER"
ok "$USER in dialout (serial ports). Takes effect at the next login."

run sudo systemctl enable --now ssh
ok "ssh enabled"

# The dashboard's web server. pip into the user's own site-packages, pinned
# to the versions the kit was tested with.
run pip3 install --user --break-system-packages "fastapi==0.162.5" "uvicorn==0.54.0" "websockets==17.4"
ok "fastapi, uvicorn, websockets"
```

`setup/02_ros_jazzy.sh` (33 lines)

```
#!/bin/bash
# 02_ros_jazzy.sh - ROS 2 Jazzy and every ROS package the kit uses.
#   ROS_PKG=desktop ./02_ros_jazzy.sh    for RViz on the Pi itself (bigger, slower)
source "$(dirname "$0")/common.sh"
ROS_PKG="${ROS_PKG:-ros-base}"

step "2. ROS 2 Jazzy ($ROS_PKG)"
if [ ! -f /etc/apt/sources.list.d/ros2.sources ] && [ ! -f /etc/apt/sources.list.d/ros2.list ]; then
    run sudo add-apt-repository -y universe
    # The current official method: the ros2-apt-source package installs the
    # repository and its signing key, and keeps the key up to date.
    VER=$(curl -s https://api.github.com/repos/ros-infrastructure/ros-apt-source/releases/latest \
          | grep -F '"tag_name"' | awk -F'"' '{print $4}')
    [ -n "$VER" ] || die "could not read the ros-apt-source version from GitHub"
    run curl -L -o /tmp/ros2-apt-source.deb \
        "https://github.com/ros-infrastructure/ros-apt-source/releases/download/${VER}/ros2-apt-
source_${VER}.noble_all.deb"
    run sudo dpkg -i /tmp/ros2-apt-source.deb
    ok "ROS 2 apt repository ($VER)"
fi
run sudo apt-get update
echo "  installing ros-jazzy-$ROS_PKG and add-ons: the slow part, 10 to 25 minutes"
run sudo apt-get install -y "ros-jazzy-$ROS_PKG" \
    ros-jazzy-navigation2 ros-jazzy-nav2-bringup ros-jazzy-slam-toolbox \
    ros-jazzy-twist-mux ros-jazzy-robot-state-publisher ros-jazzy-xacro \
    ros-jazzy-tf2-tools ros-jazzy-teleop-twist-keyboard ros-jazzy-foxglove-bridge \
    ros-jazzy-rmw-cyclonedds-cpp ros-jazzy-robot-localization \
    python3-colcon-common-extensions python3-rosdep
ok "ROS 2 Jazzy and add-ons"
if [ ! -f /etc/ros/rosdep/sources.list.d/20-default.list ]; then
    run sudo rosdep init
fi
run rosdep update
ok "rosdep"
```

`setup/03_lidar_driver.sh` (31 lines)

```
#!/bin/bash
# 03_lidar_driver.sh - the driver for lidar.model in robot_config.yaml.
source "$(dirname "$0")/common.sh"
load_env
step "3. LiDAR driver: $LIDAR_MODEL"
case "$LIDAR_MODEL" in
  ydlidar)
    if [ ! -d "$HOME/YDLidar-SDK" ]; then
        run git clone https://github.com/YDLIDAR/YDLidar-SDK.git "$HOME/YDLidar-SDK"
    fi
    mkdir -p "$HOME/YDLidar-SDK/build"
    ( cd "$HOME/YDLidar-SDK/build" && cmake .. >> "$LOG" 2>&1 && make -j"$(nproc)" >> "$LOG" 2>&1 \
      && sudo make install >> "$LOG" 2>&1 ) || die "YDLidar-SDK build failed (see $LOG)"
    ok "YDLidar-SDK built and installed"
    mkdir -p "$ROBOT_WS/src"
    if [ ! -d "$ROBOT_WS/src/ydlidar_ros2_driver" ]; then
        # The humble branch builds on Jazzy (NarrowAisleBot runs it).
        run git clone -b humble https://github.com/YDLIDAR/ydlidar_ros2_driver.git \
            "$ROBOT_WS/src/ydlidar_ros2_driver"
```

```
    fi
    ok "ydlidar_ros2_driver in $ROBOT_WS/src (built by 04_workspace.sh)"
    ;;
  rplidar)
    run sudo apt-get install -y ros-jazzy-rplidar-ros
    ok "rplidar_ros from apt"
    ;;
  none|custom)
    ok "nothing to install for '$LIDAR_MODEL'. For custom, put your driver in $ROBOT_WS/src."
    ;;
  *) die "unknown LIDAR_MODEL '$LIDAR_MODEL'" ;;
esac
```

`setup/04_workspace.sh` (20 lines)

```
#!/bin/bash
# 04_workspace.sh - regenerate configs, copy the kit's ROS package into the
# workspace and build it. Run it again after ANY change to the kit or to
# robot_config.yaml. It is the one "deploy" command.
source "$(dirname "$0")/common.sh"
step "4. Workspace"
python3 "$KIT/generate.py" || die "generate.py rejected robot_config.yaml (message above)"
load_env
source /opt/ros/jazzy/setup.bash
mkdir -p "$ROBOT_WS/src"
run rsync -a --delete --exclude '__pycache__' "$KIT/ros2/robot_base/" "$ROBOT_WS/src/robot_base/"
ok "robot_base copied to $ROBOT_WS/src/robot_base"
( cd "$ROBOT_WS" && rosdep install --from-paths src --ignore-src -y -r >> "$LOG" 2>&1 ) \
    || warn "rosdep could not resolve everything (see $LOG); the build may still work"
( cd "$ROBOT_WS" && colcon build --symlink-install --cmake-args -DCMAKE_BUILD_TYPE=Release \
    >> "$LOG" 2>&1 ) || die "colcon build failed: tail -50 $LOG"
ok "colcon build"
echo
echo "  sha256 of what was deployed (compare with the kit if in doubt):"
( cd "$ROBOT_WS/src/robot_base" && sha256sum config/*.yaml urdf/*.urdf | sed 's/^/    /' )
```

`setup/05_system.sh` (62 lines)

```
#!/bin/bash
# 05_system.sh - udev rule, shell environment, systemd service.
source "$(dirname "$0")/common.sh"
load_env
step "5. System integration"

run sudo cp "$KIT/setup/generated/99-robot.rules" /etc/udev/rules.d/99-robot.rules
run sudo udevadm control --reload-rules
run sudo udevadm trigger
ok "udev rule installed: /dev/$MCU_DEVICE, /dev/$LIDAR_DEVICE"

add() { grep -qxF "$1" "$HOME/.bashrc" || echo "$1" >> "$HOME/.bashrc"; }
add ""
add "# ── robot kit ──────────────────────────────────────────"
add "source /opt/ros/jazzy/setup.bash"
add "[ -f $ROBOT_WS/install/setup.bash ] && source $ROBOT_WS/install/setup.bash"
add "export ROS_DOMAIN_ID=$ROBOT_DOMAIN_ID"
add "export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp"
add "export CYCLONEDDS_URI='<CycloneDDS><Domain><General><Interfaces><NetworkInterface name=\"lo\" priority=\"default\"
multicast=\"false\"/></Interfaces></General></Domain></CycloneDDS>'"
add "alias rk-gen='python3 $KIT/generate.py'"
add "alias rk-build='$KIT/setup/04_workspace.sh && source $ROBOT_WS/install/setup.bash'"
add "alias rk-start='sudo systemctl start robot'"
add "alias rk-stop='sudo systemctl stop robot'"
add "alias rk-restart='sudo systemctl restart robot'"
add "alias rk-status='systemctl status robot --no-pager'"
add "alias rk-log='tail -f $HOME/robot_boot.log'"
add "alias rk-check='$KIT/setup/check_robot.sh'"
add "alias rk-ports='ls -l /dev/$MCU_DEVICE /dev/$LIDAR_DEVICE 2>&1'"
ok "~/.bashrc: ROS environment and rk-* aliases"

sudo tee /etc/systemd/system/robot.service > /dev/null <<UNIT
[Unit]
Description=Robot kit base stack (bridge, controller, odometry, twist_mux, dashboard)
After=network.target systemd-udev-settle.service
Wants=systemd-udev-settle.service

[Service]
Type=simple
```

```
User=$USER
WorkingDirectory=$HOME
ExecStart=/bin/bash $KIT/setup/start_robot.sh
Restart=on-failure
RestartSec=10s
# mixed: SIGINT/SIGTERM to the launch process first, so the dashboard can
# save an open map and stop SLAM cleanly; everything else is killed after
# the timeout. (NarrowAisleBot once lost a 24-minute map to a restart.)
KillMode=mixed
KillSignal=SIGINT
TimeoutStopSec=30s

[Install]
WantedBy=multi-user.target
UNIT
run sudo systemctl daemon-reload
if [ "$ROBOT_AUTOSTART" = "true" ]; then
    run sudo systemctl enable robot.service
    ok "robot.service enabled at boot (start now with: sudo systemctl start robot)"
else
    ok "robot.service installed, not enabled (system.autostart is false)"
fi
mkdir -p "$LOG_DIR" "$HOME/maps"
ok "log dir $LOG_DIR, maps in ~/maps"
```

`setup/06_wifi_ap.sh` (29 lines)

```
#!/bin/bash
# 06_wifi_ap.sh - make the Pi its own Wi-Fi access point, so the robot works
# anywhere with no router: join the SSID, open http://10.42.0.1:8080.
#
# Runs only if system.wifi_ap.enable is true. Needs NetworkManager (Ubuntu
# Desktop has it; on Ubuntu Server see manual 6.4 first).
# If you are connected over Wi-Fi, THIS DROPS YOUR SSH SESSION. That is
# expected: join the new network and ssh to 10.42.0.1.
source "$(dirname "$0")/common.sh"
load_env
step "6. Wi-Fi access point"
[ "$AP_ENABLE" = "true" ] || { ok "system.wifi_ap.enable is false: skipped"; exit 0; }
systemctl is-active --quiet NetworkManager || die "NetworkManager is not running (manual 6.4)"
[ ${#AP_PASSWORD} -ge 8 ] || die "wifi_ap.password must be at least 8 characters"
sudo iw reg set "$AP_COUNTRY" >> "$LOG" 2>&1 || warn "could not set Wi-Fi country $AP_COUNTRY"
if nmcli -t -f NAME con show | grep -qx robot-ap; then
    run sudo nmcli con delete robot-ap
fi
run sudo nmcli con add type wifi ifname wlan0 con-name robot-ap autoconnect yes ssid "$AP_SSID" \
    802-11-wireless.mode ap 802-11-wireless.band bg 802-11-wireless.channel 6 \
    ipv4.method shared ipv6.method ignore \
    wifi-sec.key-mgmt wpa-psk wifi-sec.psk "$AP_PASSWORD"
# Higher than any client network, so the AP wins at boot whatever is in range.
run sudo nmcli con modify robot-ap connection.autoconnect-priority 10
ok "connection robot-ap created (SSID $AP_SSID)"
echo "  Bringing it up now. Over Wi-Fi SSH this disconnects you: join $AP_SSID, then ssh $USER@10.42.0.1"
sleep 2
sudo nmcli con up robot-ap >> "$LOG" 2>&1 && ok "AP up: http://10.42.0.1:$DASHBOARD_PORT" \
    || die "nmcli con up robot-ap failed (see $LOG)"
```

`setup/start_robot.sh` (25 lines)

```
#!/bin/bash
# start_robot.sh - what robot.service runs. Waits for the microcontroller,
# then starts base.launch.py. Output goes to ~/robot_boot.log.
KIT="$(cd "$(dirname "$0")/.." && pwd)"
exec >> "$HOME/robot_boot.log" 2>&1
echo; echo "==== robot start $(date)"
source "$KIT/setup/generated/robot.env"
source /opt/ros/jazzy/setup.bash
source "$ROBOT_WS/install/setup.bash"
export ROS_DOMAIN_ID="$ROBOT_DOMAIN_ID"
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
# Loopback-only DDS: every node is on this Pi, and without a LAN cable
# CycloneDDS otherwise fails to pick an interface on a headless boot.
export CYCLONEDDS_URI='<CycloneDDS><Domain><General><Interfaces><NetworkInterface name="lo" priority="default"
multicast="false"/></Interfaces></General></Domain></CycloneDDS>'
for i in $(seq 1 30); do
    [ -e "/dev/$MCU_DEVICE" ] && break
    echo "  waiting for /dev/$MCU_DEVICE ($i/30)"
```

```
    sleep 1
done
if [ ! -e "/dev/$MCU_DEVICE" ]; then
    echo "[ERROR] /dev/$MCU_DEVICE never appeared. Cable? udev rule? (setup/list_usb_serial.py)"
    exit 1        # systemd retries in 10 s
fi
echo "  /dev/$MCU_DEVICE present, launching"
exec ros2 launch robot_base base.launch.py
```

`setup/check_robot.sh` (72 lines)

```
#!/bin/bash
# check_robot.sh - is the robot actually running what you think it is?
# Read-only. Asks the LIVE system, never a file: a value in a YAML file is
# not a value on the robot (NarrowAisleBot ran three days of drives on SLAM
# defaults because a tuned file never reached the Pi).
#
#   ~/robot_kit/setup/check_robot.sh          (alias: rk-check)
KIT="$(cd "$(dirname "$0")/.." && pwd)"
# shellcheck disable=SC1091
source "$KIT/setup/generated/robot.env"
# shellcheck disable=SC1091
source /opt/ros/jazzy/setup.bash 2>/dev/null
# shellcheck disable=SC1091
[ -f "$ROBOT_WS/install/setup.bash" ] && source "$ROBOT_WS/install/setup.bash"
export ROS_DOMAIN_ID="$ROBOT_DOMAIN_ID" RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
export CYCLONEDDS_URI='<CycloneDDS><Domain><General><Interfaces><NetworkInterface name="lo" priority="default"
multicast="false"/></Interfaces></General></Domain></CycloneDDS>'
pass=0; fail=0
p() { echo "  ok    $*"; pass=$((pass+1)); }
f() { echo "  FAIL  $*"; fail=$((fail+1)); }
hz() { timeout 6 ros2 topic hz "$1" 2>/dev/null | grep -m1 "average rate" | awk '{print $3}'; }

echo "== devices"
[ -e "/dev/$MCU_DEVICE" ] && p "/dev/$MCU_DEVICE → $(readlink -f "/dev/$MCU_DEVICE")" || f "/dev/$MCU_DEVICE missing"
if [ "$LIDAR_MODEL" != "none" ]; then
  [ -e "/dev/$LIDAR_DEVICE" ] && p "/dev/$LIDAR_DEVICE → $(readlink -f "/dev/$LIDAR_DEVICE")" || f "/dev/$LIDAR_DEVICE
missing"
fi
groups | grep -qw dialout && p "user in dialout" || f "user not in dialout (log out and in after 01_base_system.sh)"

echo "== service"
systemctl is-active --quiet robot && p "robot.service active" || f "robot.service not active (rk-start; rk-log)"

echo "== graph"
ros2 daemon stop >/dev/null 2>&1   # stale daemon caches show nodes that are long gone
nodes=$(timeout 10 ros2 node list 2>/dev/null)
for n in mcu_bridge base_controller wheel_odometry twist_mux robot_state_publisher; do
  echo "$nodes" | grep -qx "/$n" && p "node /$n" || f "node /$n not running"
done
dups=$(echo "$nodes" | sort | uniq -d)
[ -z "$dups" ] && p "no duplicate nodes" || f "duplicate nodes (two launches?): $dups"

echo "== data"
r=$(hz /wheel_states); [ -n "$r" ] && p "/wheel_states at $r Hz" || f "/wheel_states silent: no telemetry from the MCU"
r=$(hz /odom); [ -n "$r" ] && p "/odom at $r Hz" || f "/odom silent"
timeout 5 ros2 run tf2_ros tf2_echo odom base_link 2>/dev/null | grep -q Translation \
  && p "TF odom → base_link exists" || f "no TF odom → base_link (SLAM will hang at Activating)"
if echo "$nodes" | grep -q scan_filter; then
  r=$(hz /scan_filtered); [ -n "$r" ] && p "/scan_filtered at $r Hz" || f "/scan_filtered silent"
fi

echo "== live parameters (from the running nodes, not from files)"
for q in "mcu_bridge port" "mcu_bridge wheel_names" "base_controller drive_type" \
         "wheel_odometry lateral_scale" "wheel_odometry track_width_scale"; do
  set -- $q
  v=$(timeout 5 ros2 param get "/$1" "$2" 2>/dev/null)
  [ -n "$v" ] && p "$1.$2: ${v#*: }" || f "$1.$2: no answer"
done
if echo "$nodes" | grep -qx /slam_toolbox; then
  for q in use_scan_matching max_laser_range scan_topic; do
    v=$(timeout 5 ros2 param get /slam_toolbox "$q" 2>/dev/null); p "slam_toolbox.$q: ${v#*: }"
  done
fi
if echo "$nodes" | grep -qx /controller_server; then
  v=$(timeout 5 ros2 param get /controller_server odom_topic 2>/dev/null); p "controller_server.odom_topic: ${v#*: }"
fi

echo "== Pi"
```

```
t=$(cat /sys/class/thermal/thermal_zone0/temp 2>/dev/null); [ -n "$t" ] && p "CPU $((t/1000)) C"
command -v vcgencmd >/dev/null && p "throttled: $(vcgencmd get_throttled)"
df -h "$HOME" | awk 'NR==2 {print "  info  disk: " $4 " free of " $2}'
echo
echo "$pass ok, $fail failed"
[ "$fail" -eq 0 ]
```

`setup/list_usb_serial.py` (112 lines)

```
#!/usr/bin/env python3
"""
list_usb_serial.py - which USB serial device is which, ready to paste into
robot_config.yaml. Standard library only.

    python3 setup/list_usb_serial.py

Plug in ONE board at a time and run it each time: the new entry is that
board. For each /dev/ttyUSB* and /dev/ttyACM* it prints the USB vendor and
product id, the serial number, the physical port path, and any /dev symlink
already pointing at it.

If two devices print the same vendor, product AND serial (NarrowAisleBot's
ESP32 and its YDLIDAR adapter did: two CP2102s, both serial "0001"), udev can
only tell them apart by the USB socket they are plugged into. Then fill
usb_port_path for BOTH, and never swap the cables.
"""
import argparse
import glob
import os


def read(path):
    try:
        with open(path) as fh:
            return fh.read().strip()
    except OSError:
        return ""


def usb_parent(sys_dev):
    """Walk up from the tty's device directory to the USB device (the
    directory that has idVendor)."""
    d = os.path.realpath(sys_dev)
    while d and d != "/":
        if os.path.exists(os.path.join(d, "idVendor")):
            return d
        d = os.path.dirname(d)
    return None


def scan(root="/"):
    out = []
    ttys = sorted(glob.glob(os.path.join(root, "sys/class/tty/ttyUSB*")) +
                  glob.glob(os.path.join(root, "sys/class/tty/ttyACM*")))
    links = {}
    for l in glob.glob(os.path.join(root, "dev/*")):
        if os.path.islink(l):
            links.setdefault(os.path.basename(os.path.realpath(l)), []).append(os.path.basename(l))
    for t in ttys:
        name = os.path.basename(t)
        usb = usb_parent(os.path.join(t, "device"))
        if not usb:
            continue
        out.append({
            "dev": "/dev/" + name,
            "vid": read(os.path.join(usb, "idVendor")),
            "pid": read(os.path.join(usb, "idProduct")),
            "serial": read(os.path.join(usb, "serial")),
            "manufacturer": read(os.path.join(usb, "manufacturer")),
            "product": read(os.path.join(usb, "product")),
            "port_path": os.path.basename(usb),
            "symlinks": links.get(name, []),
        })
    return out


KNOWN = {("10c4", "ea60"): "Silicon Labs CP210x (many ESP32 boards, YDLIDAR and RPLIDAR adapters)",
         ("1a86", "7523"): "WCH CH340 (cheap Arduino clones, some ESP32 boards)",
         ("1a86", "55d4"): "WCH CH9102 (newer ESP32 boards)",
```

```
        ("0403", "6001"): "FTDI FT232",
        ("2341", "0042"): "Arduino Mega 2560 (genuine)",
        ("2341", "0043"): "Arduino Uno (genuine)",
        ("303a", "1001"): "ESP32-S3 / C3 native USB",
        ("2e8a", "000a"): "Raspberry Pi Pico (RP2040)",
        ("16c0", "0483"): "Teensy"}


def main():
    ap = argparse.ArgumentParser(description="list USB serial devices")
    ap.add_argument("--root", default="/", help="filesystem root (for testing)")
    a = ap.parse_args()
    devs = scan(a.root)
    if not devs:
        print("No /dev/ttyUSB* or /dev/ttyACM* devices. Plug the board in (data cable, not a\n"
              "charge-only one) and check  dmesg | tail  for what the kernel saw.")
        return 1
    ids = {}
    for d in devs:
        ids.setdefault((d["vid"], d["pid"], d["serial"]), []).append(d["dev"])
    for d in devs:
        print(f"\n{d['dev']}   {d['manufacturer']} {d['product']}".rstrip())
        guess = KNOWN.get((d["vid"], d["pid"]))
        if guess:
            print(f"  (looks like: {guess})")
        if d["symlinks"]:
            print(f"  already linked as: {', '.join('/dev/' + s for s in d['symlinks'])}")
        print("  paste into robot_config.yaml (mcu: or lidar:):")
        print(f'    usb_vendor_id: "{d["vid"]}"')
        print(f'    usb_product_id: "{d["pid"]}"')
        print(f'    usb_serial: "{d["serial"]}"')
        print(f'    usb_port_path: ""          # this socket is "{d["port_path"]}"')
    clashes = [v for v in ids.values() if len(v) > 1]
    for c in clashes:
        print(f"\n!! {', '.join(c)} have the SAME vendor, product and serial. udev cannot tell\n"
              f"   them apart. Set usb_port_path for each (the socket values above) and do not\n"
              f"   move those cables.")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Microcontroller firmware

`firmware/robot_mcu/robot_mcu.ino` (1037 lines)

```
/*
 * robot_mcu.ino - wheel velocity controller for the robot kit
 * ============================================================
 *
 * The Raspberry Pi sends wheel velocity targets over USB serial. This sketch
 * closes a PID + feedforward loop per wheel on encoder feedback, enforces
 * the safety trips, and streams telemetry back. It is NarrowAisleBot's ESP32
 * v3.0 firmware generalised:
 *
 *   - any number of motors from 1 to 6           (NUM_MOTORS)
 *   - three motor driver styles                  (DRIVER_TYPE)
 *   - ESP32 hardware counters or interrupts      (ENCODER_BACKEND)
 *   - ESP32, AVR (Uno/Mega/Nano), RP2040, Teensy and other Arduino cores
 *
 * Every robot-specific number lives in robot_config.h, which generate.py
 * writes from robot_config.yaml. Do not put constants in this file.
 *
 * Board notes
 *   ESP32 (classic WROOM/WROVER): best choice. Arduino-ESP32 core 2.x or 3.x.
 *     The control loop runs in its own FreeRTOS task on core 1, serial on
 *     core 0, so a long telemetry line can never delay a control tick.
 *     PCNT counts encoder edges in silicon, no interrupt load at all.
 *   ESP32-C3: has no PCNT. Use encoder_backend: isr.
 *   AVR: everything runs from loop(). Use 115200 baud and control_hz 50.
 *     Every encoder pin that needs an interrupt must be an external-interrupt
 *     pin (Uno: 2, 3. Mega: 2, 3, 18, 19, 20, 21). The sketch checks this
 *     at boot and prints [ERR,ENC_PIN,...] if one is not.
 *
 * Wire protocol: see robot_base/protocol.py, or send <H>.
 */
```

```
#include <Arduino.h>
#include "robot_config.h"

#if defined(ARDUINO_ARCH_ESP32)
  #define KIT_ESP32 1
  #if defined(__has_include)
    #if __has_include("esp_arduino_version.h")
      #include "esp_arduino_version.h"
    #endif
  #endif
#endif

#if ENCODER_BACKEND == ENC_PCNT
  #ifndef KIT_ESP32
    #error "encoder_backend pcnt needs an ESP32. Set mcu.encoder_backend: isr and regenerate."
  #endif
  #include "driver/pcnt.h"
#endif

#if defined(__AVR__) && PWM_BITS != 8
  #error "AVR analogWrite is 8-bit. Set mcu.pwm_bits: 8."
#endif

#if NUM_MOTORS < 1 || NUM_MOTORS > 6
  #error "NUM_MOTORS must be 1 to 6"
#endif

#define FIRMWARE_VERSION "robot_kit_mcu 1.0"

// ============================================================
//  1. Tunables that are not robot-specific
// ============================================================

static const float CTRL_DT = 1.0f / (float)CONTROL_HZ;
static const float VEL_FILTER_ALPHA = 0.4f;   // EMA on measured velocity
static const float D_FILTER_ALPHA = 0.25f;    // heavier, derivative amplifies noise
static const float KSTAT_FADE_LO = 0.05f;     // rad/s: no static term below this
static const float KSTAT_FADE_HI = 0.20f;     // rad/s: full static term above this

// Safety trips. All latch the E-STOP and print [TRIP,...].
static const float OVERSPEED_FACTOR = 1.30f;              // x max_wheel_speed
static const unsigned long OVERSPEED_TRIP_MS = 300;
static const int RUNAWAY_PWM_SAT = (PWM_MAX * 94) / 100;  // counts as saturated
static const float RUNAWAY_DECEL_TOL = 0.20f;             // rad/s over the window
static const unsigned long RUNAWAY_WINDOW_MS = 500;
static const unsigned long RUNAWAY_TRIP_MS = 500;
static const int STALL_PWM = (PWM_MAX * 98) / 100;
static const float STALL_SPEED = 0.15f;                   // rad/s
static const unsigned long STALL_TRIP_MS = 2000;

static const int LINE_MAX = 40 + NUM_MOTORS * 72;
#if defined(__AVR__)
static const int TX_RING = (LINE_MAX * 2 < 384) ? 384 : LINE_MAX * 2;
#else
static const int TX_RING = 2048;
#endif
static const int RX_MAX = 160;

// AVR's libm has no guaranteed fabsf/lroundf, so the sketch carries its own.
static inline float absf(float x) { return x < 0 ? -x : x; }
static inline long roundToLong(float x) { return (long)(x >= 0 ? x + 0.5f : x - 0.5f); }

// ============================================================
//  2. State
// ============================================================

struct Motor {
  float target;          // rad/s, commanded
  float slewed;          // rad/s, what the PID chases
  float actual;          // rad/s, filtered
  float prev_actual;
  float error;
  float integral;
  float dfilt;
  float ff;
  int pwm;               // -PWM_MAX .. +PWM_MAX
  int64_t total_counts;  // since boot or <R>, for odometry
  int32_t win[VEL_WINDOW];
  uint8_t win_idx;
  int32_t win_sum;
  unsigned long overspeed_since, runaway_since, stall_since, checkpoint_ms;
```

```
  float vel_checkpoint;
};

static Motor motors[NUM_MOTORS];
static float Kp = KP_INIT, Ki = KI_INIT, Kd = KD_INIT;
static float Kff[NUM_MOTORS], Kstat[NUM_MOTORS];
static float max_wheel_speed = MAX_WHEEL_SPEED;
static float max_wheel_accel = MAX_WHEEL_ACCEL;

enum EstopWhy { ES_NONE, ES_COMMAND, ES_OVERSPEED, ES_RUNAWAY, ES_STALL };
static volatile bool enabled = true;
static volatile bool estop = false;
static volatile uint8_t estop_why = ES_NONE;
static volatile bool watchdog_on = true;
static volatile bool trips_on = true;
static volatile bool open_loop = false;
static volatile unsigned long last_cmd_ms = 0;
static volatile bool watchdog_fired = false;
static volatile bool watchdog_report = false;
static volatile uint8_t telemetry_mode = 0;
static volatile unsigned long telemetry_interval_ms = (unsigned long)(1000.0f / TELEMETRY_HZ_DEFAULT);

// A trip is detected in the control step and printed by the serial side,
// so the two never write to Serial at the same time.
struct TripReport { volatile bool pending; uint8_t why; uint8_t motor; float v1, v2; };
static TripReport trip_report = {false, 0, 0, 0, 0};

#ifdef KIT_ESP32
static portMUX_TYPE cmd_mux = portMUX_INITIALIZER_UNLOCKED;
#define CMD_LOCK()   portENTER_CRITICAL(&cmd_mux)
#define CMD_UNLOCK() portEXIT_CRITICAL(&cmd_mux)
#else
#define CMD_LOCK()
#define CMD_UNLOCK()
#endif

// ============================================================================
//  3. PWM output, portable
// ============================================================================

#if defined(KIT_ESP32) && !(defined(ESP_ARDUINO_VERSION_MAJOR) && ESP_ARDUINO_VERSION_MAJOR >= 3)
// Arduino-ESP32 2.x: channels must be assigned by hand.
static int pwm_pin_of_ch[16];
static int pwm_channels = 0;
static int esp32ChannelFor(int pin) {
  for (int c = 0; c < pwm_channels; c++) if (pwm_pin_of_ch[c] == pin) return c;
  return -1;
}
#endif

static void pwmInit(int pin) {
  if (pin < 0) return;
#if defined(KIT_ESP32)
  #if defined(ESP_ARDUINO_VERSION_MAJOR) && ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcAttach(pin, PWM_FREQ_HZ, PWM_BITS);
  ledcWrite(pin, 0);
  #else
  int ch = pwm_channels++;
  pwm_pin_of_ch[ch] = pin;
  ledcSetup(ch, PWM_FREQ_HZ, PWM_BITS);
  ledcAttachPin(pin, ch);
  ledcWrite(ch, 0);
  #endif
#elif defined(ARDUINO_ARCH_RP2040)
  pinMode(pin, OUTPUT);
  analogWriteFreq(PWM_FREQ_HZ);
  analogWriteRange(PWM_MAX);
  analogWrite(pin, 0);
#elif defined(TEENSYDUINO)
  pinMode(pin, OUTPUT);
  analogWriteResolution(PWM_BITS);
  analogWriteFrequency(pin, PWM_FREQ_HZ);
  analogWrite(pin, 0);
#else
  pinMode(pin, OUTPUT);   // AVR and the rest: 8-bit, frequency fixed by the core
  analogWrite(pin, 0);
#endif
}

static void pwmWrite(int pin, int duty) {
  if (pin < 0) return;
#if defined(KIT_ESP32)
```

```
  #if defined(ESP_ARDUINO_VERSION_MAJOR) && ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcWrite(pin, duty);
  #else
  ledcWrite(esp32ChannelFor(pin), duty);
  #endif
#else
  analogWrite(pin, duty);
#endif
}

static void setupMotorPins() {
  for (int i = 0; i < NUM_MOTORS; i++) {
#if DRIVER_TYPE == DRIVER_PWM_DIR
    pwmInit(PIN_A[i]);
    pinMode(PIN_B[i], OUTPUT); digitalWrite(PIN_B[i], LOW);
#elif DRIVER_TYPE == DRIVER_PWM_IN1_IN2
    pwmInit(PIN_A[i]);
    pinMode(PIN_B[i], OUTPUT); digitalWrite(PIN_B[i], LOW);
    pinMode(PIN_C[i], OUTPUT); digitalWrite(PIN_C[i], LOW);
#elif DRIVER_TYPE == DRIVER_DUAL_PWM
    pwmInit(PIN_A[i]);
    pwmInit(PIN_B[i]);
#else
  #error "unknown DRIVER_TYPE"
#endif
  }
}

// pwm is in the WHEEL's sense: positive = this wheel rolls the robot forward.
static void setMotorOutput(int i, int pwm) {
  int v = pwm * MOTOR_DIR[i];
  bool rev = v < 0;
  int a = v < 0 ? -v : v;
  if (a < MIN_PWM_OUT) a = 0;
  if (a > PWM_MAX) a = PWM_MAX;
#if DRIVER_TYPE == DRIVER_PWM_DIR
  digitalWrite(PIN_B[i], rev ? HIGH : LOW);
  pwmWrite(PIN_A[i], a);
#elif DRIVER_TYPE == DRIVER_PWM_IN1_IN2
  if (a == 0) {                       // both low = coast
    digitalWrite(PIN_B[i], LOW);
    digitalWrite(PIN_C[i], LOW);
  } else {
    digitalWrite(PIN_B[i], rev ? LOW : HIGH);
    digitalWrite(PIN_C[i], rev ? HIGH : LOW);
  }
  pwmWrite(PIN_A[i], a);
#elif DRIVER_TYPE == DRIVER_DUAL_PWM
  if (rev) { pwmWrite(PIN_A[i], 0); pwmWrite(PIN_B[i], a); }
  else     { pwmWrite(PIN_B[i], 0); pwmWrite(PIN_A[i], a); }
#endif
}

// ============================================================
//  4. Encoders
// ============================================================

#if ENCODER_BACKEND == ENC_PCNT

static void setupEncoders() {
  for (int i = 0; i < NUM_MOTORS; i++) {
    pcnt_unit_t unit = (pcnt_unit_t)i;
    // Channel 0: count A edges, direction from B.
    pcnt_config_t c0 = {};
    c0.pulse_gpio_num = ENC_A[i];
    c0.ctrl_gpio_num = ENC_B[i];
    c0.channel = PCNT_CHANNEL_0;
    c0.unit = unit;
    c0.pos_mode = PCNT_COUNT_INC;
    c0.neg_mode = PCNT_COUNT_DEC;
    c0.lctrl_mode = PCNT_MODE_REVERSE;
    c0.hctrl_mode = PCNT_MODE_KEEP;
    c0.counter_h_lim = 32767;
    c0.counter_l_lim = -32768;
    pcnt_unit_config(&c0);
  #if ENCODER_DECODING == 4
    // Channel 1: count B edges, direction from A. Together: x4.
    pcnt_config_t c1 = c0;
    c1.pulse_gpio_num = ENC_B[i];
    c1.ctrl_gpio_num = ENC_A[i];
    c1.channel = PCNT_CHANNEL_1;
    c1.pos_mode = PCNT_COUNT_DEC;
```

```
    c1.neg_mode = PCNT_COUNT_INC;
    pcnt_unit_config(&c1);
  #endif
    // ~1.25 us glitch filter (100 APB cycles). Raise it only if you see noise.
    pcnt_set_filter_value(unit, 100);
    pcnt_filter_enable(unit);
    pcnt_counter_pause(unit);
    pcnt_counter_clear(unit);
    pcnt_counter_resume(unit);
  }
}

// Counts since the last call, in the wheel's forward sense. The int16
// hardware counter is read and cleared every tick; at 100 Hz it would take
// 3.2 million counts per second to overflow it.
static int32_t readEncoderDelta(int i) {
  int16_t c = 0;
  pcnt_get_counter_value((pcnt_unit_t)i, &c);
  pcnt_counter_clear((pcnt_unit_t)i);
  return (int32_t)c * ENC_DIR[i];
}

#else  // ENC_ISR

static volatile int32_t enc_count[NUM_MOTORS];
static volatile uint8_t enc_state[NUM_MOTORS];
// Quadrature state table: index = (old AB << 2) | new AB. A leading B = +1.
static const int8_t QEM[16] = {0, -1, 1, 0, 1, 0, 0, -1, -1, 0, 0, 1, 0, 1, -1, 0};

#ifdef KIT_ESP32
static portMUX_TYPE enc_mux = portMUX_INITIALIZER_UNLOCKED;
  #define ENC_LOCK()       portENTER_CRITICAL(&enc_mux)
  #define ENC_UNLOCK()     portEXIT_CRITICAL(&enc_mux)
  #define ENC_LOCK_ISR()   portENTER_CRITICAL_ISR(&enc_mux)
  #define ENC_UNLOCK_ISR() portEXIT_CRITICAL_ISR(&enc_mux)
  #define ISR_ATTR IRAM_ATTR
#else
  #define ENC_LOCK()       noInterrupts()
  #define ENC_UNLOCK()     interrupts()
  #define ENC_LOCK_ISR()
  #define ENC_UNLOCK_ISR()
  #define ISR_ATTR
#endif

static inline void encStep(int i) {
  uint8_t a = digitalRead(ENC_A[i]) ? 1 : 0;
  uint8_t b = digitalRead(ENC_B[i]) ? 1 : 0;
  ENC_LOCK_ISR();
#if ENCODER_DECODING == 4
  uint8_t s = (uint8_t)((a << 1) | b);
  enc_count[i] += QEM[(enc_state[i] << 2) | s];
  enc_state[i] = s;
#else
  enc_count[i] += (a == b) ? 1 : -1;      // x2: only A interrupts
#endif
  ENC_UNLOCK_ISR();
}

#define ENC_ISR_DEF(I) static void ISR_ATTR encIsr##I() { encStep(I); }
ENC_ISR_DEF(0)
#if NUM_MOTORS > 1
ENC_ISR_DEF(1)
#endif
#if NUM_MOTORS > 2
ENC_ISR_DEF(2)
#endif
#if NUM_MOTORS > 3
ENC_ISR_DEF(3)
#endif
#if NUM_MOTORS > 4
ENC_ISR_DEF(4)
#endif
#if NUM_MOTORS > 5
ENC_ISR_DEF(5)
#endif

typedef void (*IsrFn)();
static const IsrFn ENC_ISRS[NUM_MOTORS] = {
  encIsr0,
#if NUM_MOTORS > 1
  encIsr1,
#endif
```

```
#if NUM_MOTORS > 2
  encIsr2,
#endif
#if NUM_MOTORS > 3
  encIsr3,
#endif
#if NUM_MOTORS > 4
  encIsr4,
#endif
#if NUM_MOTORS > 5
  encIsr5,
#endif
};

static bool enc_pin_error = false;
static int enc_bad_pin = -1;

static void attachEnc(int pin, IsrFn fn) {
  int irq = digitalPinToInterrupt(pin);
  if (irq < 0) { enc_pin_error = true; enc_bad_pin = pin; return; }
  attachInterrupt(irq, fn, CHANGE);
}

static void setupEncoders() {
  for (int i = 0; i < NUM_MOTORS; i++) {
    pinMode(ENC_A[i], INPUT_PULLUP);
    pinMode(ENC_B[i], INPUT_PULLUP);
    enc_state[i] = (uint8_t)(((digitalRead(ENC_A[i]) ? 1 : 0) << 1) | (digitalRead(ENC_B[i]) ? 1 : 0));
    enc_count[i] = 0;
    attachEnc(ENC_A[i], ENC_ISRS[i]);
  #if ENCODER_DECODING == 4
    attachEnc(ENC_B[i], ENC_ISRS[i]);
  #endif
  }
}

static int32_t readEncoderDelta(int i) {
  ENC_LOCK();
  int32_t c = enc_count[i];
  enc_count[i] = 0;
  ENC_UNLOCK();
  return c * ENC_DIR[i];
}

#endif

// ============================================================================
//  5. Controller: two-term feedforward + PID with headroom anti-windup
// ============================================================================

static void resetMotorState(int i) {
  Motor& m = motors[i];
  m.target = m.slewed = 0;
  m.error = 0;
  m.integral = m.dfilt = 0;
  m.ff = 0;
  m.pwm = 0;
  m.overspeed_since = m.runaway_since = m.stall_since = 0;
}

static void stopAll() {
  open_loop = false;
  for (int i = 0; i < NUM_MOTORS; i++) {
    resetMotorState(i);
    setMotorOutput(i, 0);
  }
}

// pwm_ff = Kff * w + Kstat * sgn(w), with Kstat faded in at low speed so a
// tiny command does not produce a step in PWM.
static float feedforward(int i, float w) {
  float aw = absf(w);
  float ff = Kff[i] * w;
  if (aw > KSTAT_FADE_LO) {
    float fade = (aw - KSTAT_FADE_LO) / (KSTAT_FADE_HI - KSTAT_FADE_LO);
    if (fade > 1.0f) fade = 1.0f;
    ff += Kstat[i] * fade * (w > 0 ? 1.0f : -1.0f);
  }
  return ff;
}

static void computePID(int i, float dt) {
```

```
  Motor& m = motors[i];
  if (absf(m.slewed) < 0.01f) {        // at rest: coast, hold nothing
    m.error = -m.actual;
    m.integral = 0;
    m.dfilt = 0;
    m.ff = 0;
    m.pwm = 0;
    return;
  }
  m.ff = feedforward(i, m.slewed);
  m.error = m.slewed - m.actual;
  float p = Kp * m.error;
  // Derivative on measurement, not error: no kick when the Pi steps the target.
  float raw_d = -(m.actual - m.prev_actual) / dt;
  m.dfilt = D_FILTER_ALPHA * raw_d + (1.0f - D_FILTER_ALPHA) * m.dfilt;
  float d = Kd * m.dfilt;

  // Integral, clamped to the PWM headroom actually left after FF + P + D.
  // Windup is impossible and recovery from saturation is immediate.
  float base = m.ff + p + d;
  float iterm = 0;
  if (Ki > 1e-6f) {
    m.integral += m.error * dt;
    float hi = ((float)PWM_MAX - base) / Ki;
    float lo = (-(float)PWM_MAX - base) / Ki;
    if (lo > hi) { float t = lo; lo = hi; hi = t; }
    if (m.integral > hi) m.integral = hi;
    if (m.integral < lo) m.integral = lo;
    iterm = Ki * m.integral;
  } else {
    m.integral = 0;
  }
  float out = base + iterm;
  if (out > PWM_MAX) out = PWM_MAX;
  if (out < -PWM_MAX) out = -PWM_MAX;
  m.pwm = (int)roundToLong(out);
}

static void slewTarget(int i, float target, float dt) {
  Motor& m = motors[i];
  if (max_wheel_accel <= 0) { m.slewed = target; return; }
  float step = max_wheel_accel * dt;
  float diff = target - m.slewed;
  if (diff > step) m.slewed += step;
  else if (diff < -step) m.slewed -= step;
  else m.slewed = target;
}

static void latchTrip(uint8_t why, int i, float v1, float v2) {
  estop = true;
  estop_why = why;
  stopAll();
  trip_report.why = why;
  trip_report.motor = (uint8_t)i;
  trip_report.v1 = v1;
  trip_report.v2 = v2;
  trip_report.pending = true;
}

// The failure the runaway trip exists for: MOTOR_DIR and ENC_DIR disagree,
// the loop becomes positive feedback, and the wheel pins itself at full
// PWM the wrong way. "PWM one way, wheel the other" is also what a hard
// deceleration looks like, so it only trips if the wheel is NOT slowing.
static void checkTrips() {
  if (!trips_on) return;
  unsigned long now = millis();
  float over = max_wheel_speed * OVERSPEED_FACTOR;
  float runaway_min = max_wheel_speed * 0.2f < 1.0f ? max_wheel_speed * 0.2f : 1.0f;
  for (int i = 0; i < NUM_MOTORS; i++) {
    Motor& m = motors[i];
    float v = m.actual, av = absf(v);
    if (now - m.checkpoint_ms >= RUNAWAY_WINDOW_MS) { m.vel_checkpoint = av; m.checkpoint_ms = now; }

    if (av > over) {
      if (!m.overspeed_since) m.overspeed_since = now;
      else if (now - m.overspeed_since > OVERSPEED_TRIP_MS) { latchTrip(ES_OVERSPEED, i, v, over); return; }
    } else m.overspeed_since = 0;

    bool sat = abs(m.pwm) >= RUNAWAY_PWM_SAT;
    bool opposing = (v * (float)m.pwm < 0) && av > runaway_min;
    bool not_slowing = av >= m.vel_checkpoint - RUNAWAY_DECEL_TOL;
    if (sat && opposing && not_slowing) {
```

```
      if (!m.runaway_since) m.runaway_since = now;
      else if (now - m.runaway_since > RUNAWAY_TRIP_MS) { latchTrip(ES_RUNAWAY, i, (float)m.pwm, v); return; }
    } else m.runaway_since = 0;

    if (abs(m.pwm) >= STALL_PWM && av < STALL_SPEED) {
      if (!m.stall_since) m.stall_since = now;
      else if (now - m.stall_since > STALL_TRIP_MS) { latchTrip(ES_STALL, i, (float)m.pwm, v); return; }
    } else m.stall_since = 0;
  }
}

// One control tick. dt is the measured interval, so loop jitter on boards
// without an RTOS does not turn into a velocity error.
static void controlStep(float dt) {
  for (int i = 0; i < NUM_MOTORS; i++) {
    Motor& m = motors[i];
    int32_t d = readEncoderDelta(i);
    m.total_counts += d;
    m.win_sum += d - m.win[m.win_idx];
    m.win[m.win_idx] = d;
    m.win_idx = (uint8_t)((m.win_idx + 1) % VEL_WINDOW);
    float raw = ((float)m.win_sum / ENCODER_CPR[i]) * (2.0f * PI) / (dt * VEL_WINDOW);
    m.prev_actual = m.actual;
    m.actual = VEL_FILTER_ALPHA * raw + (1.0f - VEL_FILTER_ALPHA) * m.actual;
  }

  if (estop || !enabled) {
    for (int i = 0; i < NUM_MOTORS; i++) { resetMotorState(i); setMotorOutput(i, 0); }
    return;
  }

  if (watchdog_on && (millis() - last_cmd_ms > WATCHDOG_MS)) {
    if (!watchdog_fired) { watchdog_fired = true; watchdog_report = true; }
    for (int i = 0; i < NUM_MOTORS; i++) motors[i].target = 0;   // closed loop ramps down
    if (open_loop) for (int i = 0; i < NUM_MOTORS; i++) motors[i].pwm = 0;
  }

  if (open_loop) {
    for (int i = 0; i < NUM_MOTORS; i++) setMotorOutput(i, motors[i].pwm);
  } else {
    float tgt[NUM_MOTORS];
    CMD_LOCK();                          // all wheels from the same command
    for (int i = 0; i < NUM_MOTORS; i++) tgt[i] = motors[i].target;
    CMD_UNLOCK();
    for (int i = 0; i < NUM_MOTORS; i++) {
      slewTarget(i, tgt[i], dt);
      computePID(i, dt);
      setMotorOutput(i, motors[i].pwm);
    }
  }
  checkTrips();
}

// ============================================================
//  6. Non-blocking serial output
//
//  On AVR the hardware TX buffer is 64 bytes. A blocking 200-byte
//  telemetry line would stall loop(), and with it the control loop, for
//  ~17 ms at 115200 baud. Everything goes through this ring instead and is
//  drained as fast as the UART accepts it.
// ============================================================

static char tx_ring[TX_RING];
static uint16_t tx_head = 0, tx_tail = 0;
static unsigned long tx_stall_since = 0;
static uint16_t tx_dropped = 0;

static uint16_t txUsed() { return (uint16_t)((tx_head + TX_RING - tx_tail) % TX_RING); }
static uint16_t txFree() { return (uint16_t)(TX_RING - 1 - txUsed()); }

static void txPump() {
  if (tx_head == tx_tail) { tx_stall_since = 0; return; }
  int room = Serial.availableForWrite();
  if (room <= 0) {
    // Some cores never report room. Do not wait forever: after 20 ms,
    // write a small blocking chunk.
    if (!tx_stall_since) tx_stall_since = millis();
    if (millis() - tx_stall_since < 20) return;
    room = 16;
  }
  tx_stall_since = 0;
  while (room > 0 && tx_tail != tx_head) {
```

```
    uint16_t end = (tx_head > tx_tail) ? tx_head : TX_RING;
    uint16_t n = end - tx_tail;
    if (n > (uint16_t)room) n = (uint16_t)room;
    Serial.write((const uint8_t*)&tx_ring[tx_tail], n);
    tx_tail = (uint16_t)((tx_tail + n) % TX_RING);
    room -= n;
  }
}

// Queue a whole line or nothing. Telemetry may be dropped under load;
// replies wait for room (they matter more).
static bool txLine(const char* s, uint16_t n, bool must) {
  unsigned long t0 = millis();
  while (txFree() < n) {
    if (!must) { tx_dropped++; return false; }
    txPump();
    if (millis() - t0 > 200) { tx_dropped++; return false; }
  }
  for (uint16_t k = 0; k < n; k++) {
    tx_ring[tx_head] = s[k];
    tx_head = (uint16_t)((tx_head + 1) % TX_RING);
  }
  return true;
}

// A tiny line builder. Integer arithmetic for floats, because AVR's printf
// and scanf have no %f.
struct Line {
  char b[LINE_MAX];
  uint16_t n;
  Line() : n(0) {}
  void c(char ch) { if (n < LINE_MAX - 2) b[n++] = ch; }
  void s(const char* str) { while (*str) c(*str++); }
  void i(long v) {
    char t[12]; int k = 0;
    bool neg = v < 0;
    unsigned long u = neg ? (unsigned long)(-v) : (unsigned long)v;
    do { t[k++] = (char)('0' + (u % 10)); u /= 10; } while (u && k < 11);
    if (neg) c('-');
    while (k) c(t[--k]);
  }
  void f(float v, int dp) {
    if (v != v) { s("nan"); return; }
    long scale = 1;
    for (int k = 0; k < dp; k++) scale *= 10;
    if (v > 2.0e9f / scale) v = 2.0e9f / scale;
    if (v < -2.0e9f / scale) v = -2.0e9f / scale;
    long q = roundToLong(v * (float)scale);
    if (q < 0) { c('-'); q = -q; }
    i(q / scale);
    if (dp > 0) {
      c('.');
      long frac = q % scale;
      for (long d = scale / 10; d > 0; d /= 10) { c((char)('0' + (frac / d) % 10)); }
    }
  }
  void send(bool must) { c('\n'); txLine(b, n, must); n = 0; }
};

static void reply(const char* s) { Line l; l.s(s); l.send(true); }

// ============================================================
//  7. Telemetry
// ============================================================

static void sendTelemetry() {
  if (telemetry_mode == 0) return;
  Line l;
  l.i((long)millis());
  for (int i = 0; i < NUM_MOTORS; i++) {
    l.c(','); l.f(motors[i].slewed, 3);
    l.c(','); l.f(motors[i].actual, 3);
    l.c(','); l.i(motors[i].pwm);
  }
  if (telemetry_mode >= 2) {
    for (int i = 0; i < NUM_MOTORS; i++) {
      Motor& m = motors[i];
      l.c(','); l.f(m.error, 3);
      l.c(','); l.f(m.ff, 1);
      l.c(','); l.f(Ki * m.integral, 1);
      l.c(','); l.f((float)((double)m.total_counts * (2.0 * PI) / (double)ENCODER_CPR[i]), 4);
    }
```

```
  }
  l.send(false);
}

// ============================================================
//  8. Command parser
// ============================================================

static void setTargets(const float* v) {
  CMD_LOCK();
  if (open_loop) {       // leaving open loop: the old integral means nothing now
    open_loop = false;
    for (int i = 0; i < NUM_MOTORS; i++) { motors[i].integral = 0; motors[i].dfilt = 0; motors[i].slewed = 0; }
  }
  for (int i = 0; i < NUM_MOTORS; i++) {
    float t = v[i];
    if (t > max_wheel_speed) t = max_wheel_speed;
    if (t < -max_wheel_speed) t = -max_wheel_speed;
    motors[i].target = t;
  }
  CMD_UNLOCK();
}

static void printList(const char* tag, const float* vals, int dp) {
  Line l; l.c('['); l.s(tag);
  for (int i = 0; i < NUM_MOTORS; i++) { l.c(','); l.s(MOTOR_NAMES[i]); l.c('='); l.f(vals[i], dp); }
  l.c(']'); l.send(true);
}

static void printInfo() {
  Line l;
  l.s("[INFO,"); l.s(FIRMWARE_VERSION); l.c(','); l.s(ROBOT_NAME); l.c(']'); l.send(true);
  l.s("[MOTORS,"); l.i(NUM_MOTORS);
  for (int i = 0; i < NUM_MOTORS; i++) { l.c(','); l.s(MOTOR_NAMES[i]); }
  l.c(']'); l.send(true);
  l.s("[GAINS,Kp="); l.f(Kp, 3); l.s(",Ki="); l.f(Ki, 3); l.s(",Kd="); l.f(Kd, 3);
  l.s(",Hz="); l.i(CONTROL_HZ); l.c(']'); l.send(true);
  printList("FF", Kff, 3);
  printList("KSTAT", Kstat, 3);
  printList("CPR", ENCODER_CPR, 1);
  l.s("[LIMITS,Vmax="); l.f(max_wheel_speed, 3); l.s(",Accel="); l.f(max_wheel_accel, 3);
  l.s(",PWM_MAX="); l.i(PWM_MAX); l.s(",WD_MS="); l.i(WATCHDOG_MS); l.c(']'); l.send(true);
  l.s("[CONFIG,driver="); l.i(DRIVER_TYPE); l.s(",encoder="); l.s(ENCODER_BACKEND == ENC_PCNT ? "pcnt" : "isr");
  l.s(",decoding=x"); l.i(ENCODER_DECODING); l.s(",vel_window="); l.i(VEL_WINDOW);
  l.s(",baud="); l.i((long)SERIAL_BAUD); l.c(']'); l.send(true);
#if ENCODER_BACKEND == ENC_ISR
  if (enc_pin_error) { l.s("[ERR,ENC_PIN,"); l.i(enc_bad_pin); l.s(",not an interrupt pin]"); l.send(true); }
#endif
}

static void printState() {
  static const char* WHY[] = {"NONE", "COMMAND", "OVERSPEED", "RUNAWAY", "STALL"};
  Line l;
  l.s("[STATE,EN="); l.i(enabled); l.s(",WD="); l.i(watchdog_on); l.s(",TRIPS="); l.i(trips_on);
  l.s(",ESTOP="); l.i(estop); l.s(",WHY="); l.s(WHY[estop_why]);
  l.s(",MODE="); l.s(open_loop ? "OPENLOOP" : "PID"); l.s(",TXDROP="); l.i(tx_dropped); l.c(']');
  l.send(true);
  for (int i = 0; i < NUM_MOTORS; i++) {
    Motor& m = motors[i];
    l.c('['); l.s(MOTOR_NAMES[i]); l.s(",tgt="); l.f(m.target, 3); l.s(",slew="); l.f(m.slewed, 3);
    l.s(",act="); l.f(m.actual, 3); l.s(",pwm="); l.i(m.pwm); l.s(",err="); l.f(m.error, 3);
    l.s(",i="); l.f(m.integral, 4); l.s(",counts="); l.i((long)m.total_counts); l.c(']');
    l.send(true);
  }
}

static void printHelp() {
  static const char* const H[] = {
    "[HELP,<V,w0..wN-1>  wheel velocity targets rad/s (closed loop)]",
    "[HELP,<B,vx,vy,wz>  body twist; the MCU runs the inverse kinematics]",
    "[HELP,<M,p0..pN-1>  raw PWM; open loop until the next V/B/T. Send <W0> first for long holds]",
    "[HELP,<T,i,w>  one motor closed loop]",
    "[HELP,<S> E-STOP latches; <E1> enable and clear; <E0> disable]",
    "[HELP,<W1>/<W0> watchdog; <Y1>/<Y0> safety trips]",
    "[HELP,<G,kp,ki,kd>  <F,kff..>  <K,kstat..>  <A,accel>  <X,vmax>]",
    "[HELP,<L0> off  <L1[,hz]> basic  <L2[,hz]> extended telemetry]",
    "[HELP,<O> wheel angles  <R> zero them  <P> ping  <I> config  <?> state]",
  };
  for (unsigned k = 0; k < sizeof(H) / sizeof(H[0]); k++) reply(H[k]);
}

// Split "X,a,b,c" in place. Returns the number of fields.
```

```
static int splitFields(char* s, char** f, int maxf) {
  int n = 0;
  f[n++] = s;
  for (char* p = s; *p && n < maxf; p++) {
    if (*p == ',') { *p = 0; f[n++] = p + 1; }
  }
  return n;
}

static bool parseFloats(char** f, int count, float* out) {
  for (int k = 0; k < count; k++) {
    char* end;
    out[k] = (float)strtod(f[k], &end);
    if (end == f[k]) return false;
  }
  return true;
}

static void feedWatchdog() {
  last_cmd_ms = millis();
  watchdog_fired = false;
}

// Only motion commands (V, B, M, T) feed the watchdog. A status query or a
// telemetry change must never keep a stale velocity alive. (NarrowAisleBot's
// v3.0 fed it on every command; this is stricter on purpose.)
static void processCommand(char* cmd) {
  char* f[NUM_MOTORS + 4];
  int nf = splitFields(cmd, f, NUM_MOTORS + 4);
  char c0 = cmd[0], c1 = cmd[1];
  float v[NUM_MOTORS + 3];

  switch (c0) {
    case 'V':
      if (nf == NUM_MOTORS + 1 && parseFloats(f + 1, NUM_MOTORS, v)) { feedWatchdog(); setTargets(v); }
      else reply("[ERR,BAD_V]");
      break;

    case 'B':
      if (nf == 4 && parseFloats(f + 1, 3, v)) {
        float w[NUM_MOTORS], peak = 0;
        for (int i = 0; i < NUM_MOTORS; i++) {
          w[i] = IK_VX[i] * v[0] + IK_VY[i] * v[1] + IK_WZ[i] * v[2];
          if (absf(w[i]) > peak) peak = absf(w[i]);
        }
        if (peak > max_wheel_speed) for (int i = 0; i < NUM_MOTORS; i++) w[i] *= max_wheel_speed / peak;
        feedWatchdog();
        setTargets(w);
      } else reply("[ERR,BAD_B]");
      break;

    case 'M':
      if (nf == NUM_MOTORS + 1 && parseFloats(f + 1, NUM_MOTORS, v)) {
        feedWatchdog();
        CMD_LOCK();
        open_loop = true;
        for (int i = 0; i < NUM_MOTORS; i++) {
          int p = (int)roundToLong(v[i]);
          if (p > PWM_MAX) p = PWM_MAX;
          if (p < -PWM_MAX) p = -PWM_MAX;
          motors[i].target = motors[i].slewed = 0;
          motors[i].integral = 0;
          motors[i].pwm = p;
        }
        CMD_UNLOCK();
        Line l; l.s("[OK,M");
        for (int i = 0; i < NUM_MOTORS; i++) { l.c(','); l.i(motors[i].pwm); }
        l.c(']'); l.send(true);
      } else reply("[ERR,BAD_M]");
      break;

    case 'T':
      if (nf == 3 && parseFloats(f + 1, 2, v) && (int)v[0] >= 0 && (int)v[0] < NUM_MOTORS) {
        float w[NUM_MOTORS];
        for (int i = 0; i < NUM_MOTORS; i++) w[i] = open_loop ? 0 : motors[i].target;
        w[(int)v[0]] = v[1];
        feedWatchdog();
        setTargets(w);
        reply("[OK,T]");
      } else reply("[ERR,BAD_T]");
      break;
```

```
    case 'S':
      estop = true;
      estop_why = ES_COMMAND;
      stopAll();
      reply("[OK,ESTOP_LATCHED]");
      break;

    case 'E':
      if (c1 == '1') {
        enabled = true;
        estop = false;
        estop_why = ES_NONE;
        for (int i = 0; i < NUM_MOTORS; i++) resetMotorState(i);
        reply("[OK,ENABLED]");
      } else {
        enabled = false;
        stopAll();
        reply("[OK,DISABLED]");
      }
      break;

    case 'W': watchdog_on = (c1 == '1'); reply(watchdog_on ? "[OK,WDOG=1]" : "[OK,WDOG=0]"); break;
    case 'Y': trips_on = (c1 == '1'); reply(trips_on ? "[OK,TRIPS=1]" : "[OK,TRIPS=0]"); break;

    case 'G':
      if (nf == 4 && parseFloats(f + 1, 3, v)) {
        Kp = v[0]; Ki = v[1]; Kd = v[2];
        for (int i = 0; i < NUM_MOTORS; i++) motors[i].integral = 0;   // stale integral would kick
        Line l; l.s("[OK,GAINS,"); l.f(Kp, 3); l.c(','); l.f(Ki, 3); l.c(','); l.f(Kd, 3); l.c(']'); l.send(true);
      } else reply("[ERR,BAD_G]");
      break;

    case 'F':
    case 'K':
      if (nf == NUM_MOTORS + 1 && parseFloats(f + 1, NUM_MOTORS, v)) {
        float* dst = (c0 == 'F') ? Kff : Kstat;
        for (int i = 0; i < NUM_MOTORS; i++) dst[i] = v[i];
        printList(c0 == 'F' ? "OK,FF" : "OK,KSTAT", dst, 3);
      } else reply(c0 == 'F' ? "[ERR,BAD_F]" : "[ERR,BAD_K]");
      break;

    case 'A':
      if (nf == 2 && parseFloats(f + 1, 1, v) && v[0] >= 0) {
        max_wheel_accel = v[0];
        Line l; l.s("[OK,ACCEL="); l.f(max_wheel_accel, 3); l.c(']'); l.send(true);
      } else reply("[ERR,BAD_A]");
      break;

    case 'X':
      if (nf == 2 && parseFloats(f + 1, 1, v) && v[0] > 0 && v[0] <= 2.0f * MAX_WHEEL_SPEED) {
        max_wheel_speed = v[0];
        Line l; l.s("[OK,VMAX="); l.f(max_wheel_speed, 3); l.c(']'); l.send(true);
      } else reply("[ERR,BAD_X]");
      break;

    case 'L': {
      telemetry_mode = (c1 >= '0' && c1 <= '2') ? (uint8_t)(c1 - '0') : 0;
      if (nf >= 2 && parseFloats(f + 1, 1, v) && v[0] > 0) {
        unsigned long iv = (unsigned long)(1000.0f / v[0] + 0.5f);
        if (iv < 10) iv = 10;                       // 100 Hz ceiling
        telemetry_interval_ms = iv;
      }
      Line l; l.s("[OK,LOG="); l.i(telemetry_mode); l.s(",HZ="); l.f(1000.0f / telemetry_interval_ms, 1);
      l.c(']'); l.send(true);
      break;
    }

    case 'O': {
      Line l; l.s("[ODOM");
      for (int i = 0; i < NUM_MOTORS; i++) {
        l.c(','); l.f((float)((double)motors[i].total_counts * (2.0 * PI) / (double)ENCODER_CPR[i]), 4);
      }
      l.c(']'); l.send(true);
      break;
    }

    case 'R':
      for (int i = 0; i < NUM_MOTORS; i++) motors[i].total_counts = 0;
      reply("[OK,ODOM_ZEROED]");
      break;

    case 'P': reply("[PONG]"); break;
```

```
    case 'I': printInfo(); break;
    case '?': printState(); break;
    case 'H': printHelp(); break;
    default: {
      Line l; l.s("[ERR,UNKNOWN:"); l.c(c0); l.c(']'); l.send(true);
    }
  }
}

static char rx_buf[RX_MAX];
static int rx_len = 0;
static bool rx_in_frame = false;

static void readSerial() {
  int budget = 128;                    // bound the time spent here per call
  while (Serial.available() && budget-- > 0) {
    char ch = (char)Serial.read();
    if (ch == '<') { rx_in_frame = true; rx_len = 0; }
    else if (ch == '>' && rx_in_frame) {
      rx_in_frame = false;
      rx_buf[rx_len] = 0;
      if (rx_len > 0) processCommand(rx_buf);
    } else if (rx_in_frame) {
      if (rx_len < RX_MAX - 1) rx_buf[rx_len++] = ch;
      else { rx_in_frame = false; rx_len = 0; }      // overlong frame: drop it
    }
  }
}

// Everything the serial side does, once per pass.
static unsigned long last_tel_ms = 0;
static void serviceSerial() {
  readSerial();
  if (trip_report.pending) {
    static const char* WHY[] = {"NONE", "COMMAND", "OVERSPEED", "RUNAWAY", "STALL"};
    Line l; l.s("[TRIP,"); l.s(WHY[trip_report.why]); l.c(','); l.s(MOTOR_NAMES[trip_report.motor]);
    l.c(','); l.f(trip_report.v1, 2); l.c(','); l.f(trip_report.v2, 2); l.c(']'); l.send(true);
    if (trip_report.why == ES_RUNAWAY) reply("[TRIP,check motor_dir and encoder_dir for this wheel]");
    trip_report.pending = false;
  }
  if (watchdog_report) { watchdog_report = false; reply("[WARN,WATCHDOG,no command; stopping]"); }
  unsigned long now = millis();
  if (now - last_tel_ms >= telemetry_interval_ms) { last_tel_ms = now; sendTelemetry(); }
  txPump();
}

// ============================================================
//  9. Scheduling
// ============================================================

#ifdef KIT_ESP32
static void controlTask(void*) {
  TickType_t wake = xTaskGetTickCount();
  const TickType_t period = pdMS_TO_TICKS(1000 / CONTROL_HZ);
  unsigned long last_us = micros();
  for (;;) {
    vTaskDelayUntil(&wake, period);
    unsigned long now = micros();
    float dt = (now - last_us) * 1e-6f;
    last_us = now;
    if (dt < 0.5f * CTRL_DT || dt > 2.0f * CTRL_DT) dt = CTRL_DT;
    controlStep(dt);
  }
}
static void commsTask(void*) {
  for (;;) { serviceSerial(); vTaskDelay(1); }
}
#endif

void setup() {
  Serial.begin(SERIAL_BAUD);
  unsigned long t0 = millis();
  while (!Serial && millis() - t0 < 1500) {}   // native-USB boards
  memset(motors, 0, sizeof(motors));
  for (int i = 0; i < NUM_MOTORS; i++) { Kff[i] = KFF_INIT[i]; Kstat[i] = KSTAT_INIT[i]; }
  setupMotorPins();
  stopAll();
  setupEncoders();
  last_cmd_ms = millis();                 // do not trip the watchdog before the Pi speaks
  reply("[BOOT,robot_kit_mcu]");
  printInfo();
  reply("[READY]");
```

```
#ifdef KIT_ESP32
  xTaskCreatePinnedToCore(controlTask, "ctrl", 4096, NULL, 3, NULL, 1);
  xTaskCreatePinnedToCore(commsTask, "comms", 4096, NULL, 1, NULL, 0);
#endif
}

#ifdef KIT_ESP32
void loop() { vTaskDelay(1000); }
#else
static unsigned long last_ctrl_us = 0;
void loop() {
  unsigned long now = micros();
  const unsigned long period_us = 1000000UL / CONTROL_HZ;
  if (last_ctrl_us == 0) last_ctrl_us = now;
  if (now - last_ctrl_us >= period_us) {
    float dt = (now - last_ctrl_us) * 1e-6f;
    if (now - last_ctrl_us > 5 * period_us) dt = CTRL_DT;   // we fell far behind: do not integrate the gap
    last_ctrl_us = now;
    controlStep(dt);
  }
  serviceSerial();
}
#endif
```

`firmware/robot_mcu/robot_config.h` (69 lines)

```
// GENERATED by robot_kit/generate.py from robot_config.yaml. Do not edit.
// Robot: myrobot  drive: differential  driver: pwm_dir
#pragma once

#define ROBOT_NAME "myrobot"
#define NUM_MOTORS 2

// Motor order = wheel order in robot_config.yaml = order on the wire.
static const char* const MOTOR_NAMES[NUM_MOTORS] = {"left", "right"};

// ── Motor driver ──────────────────────────────────────────────
#define DRIVER_PWM_DIR      1   // PIN_A = PWM, PIN_B = DIR
#define DRIVER_PWM_IN1_IN2  2   // PIN_A = PWM/EN, PIN_B = IN1, PIN_C = IN2
#define DRIVER_DUAL_PWM     3   // PIN_A = PWM forward, PIN_B = PWM reverse
#ifndef DRIVER_TYPE
#define DRIVER_TYPE DRIVER_PWM_DIR
#endif
static const int PIN_A[NUM_MOTORS] = {25, 32};   // pwm
static const int PIN_B[NUM_MOTORS] = {26, 33};   // dir
static const int PIN_C[NUM_MOTORS] = {-1, -1};   // unused

// ── Encoders ──────────────────────────────────────────────────
#define ENC_PCNT 1              // ESP32 hardware pulse counters
#define ENC_ISR  2              // pin-change interrupts, any board
#ifndef ENCODER_BACKEND
#define ENCODER_BACKEND ENC_PCNT
#endif
#define ENCODER_DECODING 4      // 4 = every edge of A and B, 2 = edges of A only
static const int ENC_A[NUM_MOTORS] = {34, 36};
static const int ENC_B[NUM_MOTORS] = {35, 39};
// Counts per WHEEL revolution, as this firmware counts them.
static const float ENCODER_CPR[NUM_MOTORS] = {1320.0f, 1320.0f};

// +1 / -1. Set independently by two bench tests (manual 7.5 and 7.6).
static const int MOTOR_DIR[NUM_MOTORS] = {1, -1};
static const int ENC_DIR[NUM_MOTORS]   = {1, -1};

// ── Kinematics for <B,vx,vy,wz> (the Pi normally sends <V> instead) ──
static const float IK_VX[NUM_MOTORS] = {20.000000f, 20.000000f};
static const float IK_VY[NUM_MOTORS] = {0.000000f, 0.000000f};
static const float IK_WZ[NUM_MOTORS] = {-3.000000f, 3.000000f};

// ── Link and timing ───────────────────────────────────────────
#ifndef SERIAL_BAUD
#define SERIAL_BAUD 921600
#endif
#define CONTROL_HZ 100
// Velocity is measured over the last VEL_WINDOW control ticks. One count in
// one tick is 0.4760 rad/s on the coarsest encoder; over 8 tick(s)
// it is 0.0595 rad/s. Coarse encoders need a window, at the
// cost of 35 ms of extra lag (manual 4.3).
#define VEL_WINDOW 8
#define TELEMETRY_HZ_DEFAULT 20.0f
```

```
#define WATCHDOG_MS 750

// ── Limits ──────────────────────────────────────────────────────
#define MAX_WHEEL_SPEED 10.000f     // rad/s
#define MAX_WHEEL_ACCEL 20.000f     // rad/s^2, 0 = off
#define PWM_BITS 8
#define PWM_MAX 255
#define PWM_FREQ_HZ 20000
#define MIN_PWM_OUT 0

// ── Controller (replace with measured values, manual chapter 9) ─────
#define KP_INIT 10.0000f
#define KI_INIT 50.0000f
#define KD_INIT 0.0000f
static const float KFF_INIT[NUM_MOTORS]   = {0.0000f, 0.0000f};
static const float KSTAT_INIT[NUM_MOTORS] = {0.0000f, 0.0000f};
```

ROS 2 package robot_base

`ros2/robot_base/package.xml` (26 lines)

```
<?xml version="1.0"?>
<package format="3">
  <name>robot_base</name>
  <version>1.0.0</version>
  <description>Robot kit base stack: MCU bridge, kinematics, wheel odometry, scan filter, dashboard, launch files.
</description>
  <maintainer email="robot@example.com">robot kit</maintainer>
  <license>MIT</license>
  <buildtool_depend>ament_python</buildtool_depend>
  <exec_depend>rclpy</exec_depend>
  <exec_depend>std_msgs</exec_depend>
  <exec_depend>geometry_msgs</exec_depend>
  <exec_depend>sensor_msgs</exec_depend>
  <exec_depend>nav_msgs</exec_depend>
  <exec_depend>action_msgs</exec_depend>
  <exec_depend>builtin_interfaces</exec_depend>
  <exec_depend>tf2_ros</exec_depend>
  <exec_depend>robot_state_publisher</exec_depend>
  <exec_depend>twist_mux</exec_depend>
  <exec_depend>slam_toolbox</exec_depend>
  <exec_depend>nav2_map_server</exec_depend>
  <exec_depend>python3-serial</exec_depend>
  <exec_depend>python3-yaml</exec_depend>
  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

`ros2/robot_base/setup.py` (35 lines)

```
import os
from glob import glob
from setuptools import setup

package_name = "robot_base"

setup(
    name=package_name,
    version="1.0.0",
    packages=[package_name, package_name + ".dashboard"],
    package_data={package_name + ".dashboard": ["static/*"]},
    data_files=[
        ("share/ament_index/resource_index/packages", ["resource/" + package_name]),
        ("share/" + package_name, ["package.xml"]),
        (os.path.join("share", package_name, "launch"), glob("launch/*.launch.py")),
        (os.path.join("share", package_name, "config"), glob("config/*.yaml")),
        (os.path.join("share", package_name, "urdf"), glob("urdf/*.urdf")),
    ],
    install_requires=["setuptools"],
    zip_safe=False,
    maintainer="robot kit",
    maintainer_email="robot@example.com",
    description="Robot kit base stack",
    license="MIT",
```

```
    entry_points={
        "console_scripts": [
            "mcu_bridge = robot_base.mcu_bridge:main",
            "base_controller = robot_base.base_controller:main",
            "wheel_odometry = robot_base.wheel_odometry:main",
            "scan_filter = robot_base.scan_filter:main",
            "telemetry_logger = robot_base.telemetry_logger:main",
            "dashboard = robot_base.dashboard.server:main",
        ],
    },
)
```

`ros2/robot_base/robot_base/kinematics.py` (221 lines)

```
"""
kinematics.py - wheel kinematics for any planar wheeled base.

Pure Python, no ROS, no numpy. The ROS nodes, the generator and the tests all
use this one file, so the robot's geometry is defined in exactly one place.

The model
---------
Every driven wheel i sits at (x_i, y_i) in base_link (REP-103: +x forward,
+y left). When the body moves with twist (vx, vy, wz), the contact point of
wheel i moves with velocity

    vcx = vx - wz * y_i
    vcy = vy + wz * x_i

A wheel only drives along one direction. Write that as a coupling vector
(ax_i, ay_i). The wheel's surface speed is the contact velocity projected on
it, and dividing by the radius gives the wheel's angular rate:

    w_i = (ax_i * vcx + ay_i * vcy) / r

That single line covers every drive type in this kit:

    differential, skid_steer   a = (1, 0)        rolls along x, cannot strafe
    mecanum                    a = (1, s_i)      s_i = +1 or -1 per wheel (45 deg rollers)
    omni                       a = (cos t, sin t) t = the wheel's drive direction

Check it against the textbook mecanum result: front_left at (lx, ly) with
s = -1 gives w = (vx - vy - (lx + ly) wz) / r. Same formula.

Forward kinematics (wheel rates -> body twist) is the least-squares inverse of
that map. For a holonomic base it solves for (vx, vy, wz); for a base that
cannot strafe it solves for (vx, wz) and reports vy = 0. With four mecanum
wheels the least-squares answer is the usual "sum the four wheels / 4" form.

Two measured corrections, both 1.0 until you have measured them:

    track_width_scale   multiplies every y_i (and x_i) lever arm in BOTH
                        directions. Skid-steer bases scrub when they turn and
                        behave as if the wheels were further apart. Same idea
                        as diff_drive_controller's wheel_separation_multiplier.
    lateral_scale       multiplies the vy that forward kinematics reports.
                        Mecanum rollers scrub sideways, so wheels over-report
                        strafe. Odometry only; commands are not changed.
                        NarrowAisleBot measured 0.80 on one floor and 0.92 on
                        another, same robot, same day.
"""

import math

HOLONOMIC_TYPES = ("mecanum", "omni")
DRIVE_TYPES = ("differential", "skid_steer", "mecanum", "omni")


class Wheel:
    __slots__ = ("name", "x", "y", "ax", "ay")

    def __init__(self, name, x, y, ax=1.0, ay=0.0):
        self.name = name
        self.x = float(x)
        self.y = float(y)
        self.ax = float(ax)
        self.ay = float(ay)

    def __repr__(self):
```

```
        return (f"Wheel({self.name!r}, x={self.x:+.4f}, y={self.y:+.4f}, "
                f"a=({self.ax:+.3f}, {self.ay:+.3f}))")


def _solve(m, b):
    """Solve m x = b for a small square system by Gaussian elimination with
    partial pivoting. Raises ValueError if m is singular."""
    n = len(m)
    a = [list(map(float, row)) + [float(b[i])] for i, row in enumerate(m)]
    for col in range(n):
        piv = max(range(col, n), key=lambda r: abs(a[r][col]))
        if abs(a[piv][col]) < 1e-12:
            raise ValueError("wheel layout cannot determine the body motion "
                             "(singular kinematics); check wheel positions")
        a[col], a[piv] = a[piv], a[col]
        for r in range(n):
            if r != col:
                f = a[r][col] / a[col][col]
                for c in range(col, n + 1):
                    a[r][c] -= f * a[col][c]
    return [a[i][n] / a[i][i] for i in range(n)]


class DriveModel:
    """Inverse and forward kinematics for a list of Wheel objects."""

    def __init__(self, wheels, wheel_radius, holonomic,
                 track_width_scale=1.0, lateral_scale=1.0):
        if not wheels:
            raise ValueError("need at least one wheel")
        if wheel_radius <= 0:
            raise ValueError("wheel_radius must be positive")
        self.wheels = list(wheels)
        self.r = float(wheel_radius)
        self.holonomic = bool(holonomic)
        self.track_width_scale = float(track_width_scale)
        self.lateral_scale = float(lateral_scale)
        self._rows = self._build_rows()
        # Precompute the least-squares solver: (J^T J)^-1 J^T, as a matrix.
        self._pinv = self._pseudo_inverse()

    @property
    def names(self):
        return [w.name for w in self.wheels]

    def _build_rows(self):
        """Rows of J such that w = J * q, where q = (vx, vy, wz) for a
        holonomic base and (vx, wz) otherwise."""
        rows = []
        s = self.track_width_scale
        for w in self.wheels:
            x, y = w.x * s, w.y * s
            c_vx = w.ax / self.r
            c_vy = w.ay / self.r
            c_wz = (-w.ax * y + w.ay * x) / self.r
            rows.append((c_vx, c_vy, c_wz) if self.holonomic else (c_vx, c_wz))
        return rows

    def _pseudo_inverse(self):
        rows = self._rows
        k = len(rows[0])
        jtj = [[sum(r[i] * r[j] for r in rows) for j in range(k)] for i in range(k)]
        # Columns of (J^T J)^-1, one unit vector at a time.
        inv_cols = [_solve(jtj, [1.0 if i == c else 0.0 for i in range(k)])
                    for c in range(k)]
        inv = [[inv_cols[c][r] for c in range(k)] for r in range(k)]
        # pinv = inv * J^T, shape k x n
        return [[sum(inv[i][m] * rows[w][m] for m in range(k))
                 for w in range(len(rows))] for i in range(k)]

    # ── body twist → wheel rates ──────────────────────────────────────
    def inverse(self, vx, vy, wz):
        """Wheel angular rates (rad/s), in wheel order. vy is ignored for a
        base that cannot strafe."""
        out = []
        for row in self._rows:
            if self.holonomic:
                out.append(row[0] * vx + row[1] * vy + row[2] * wz)
            else:
                out.append(row[0] * vx + row[1] * wz)
        return out
```

```
    # ── wheel rates (or angle increments) → body twist ──────────────
    def forward(self, rates):
        """Least-squares body twist (vx, vy, wz) from wheel rates. Works the
        same on angle increments: pass d_phi and get (dx, dy, dtheta) in the
        body frame for that interval."""
        if len(rates) != len(self.wheels):
            raise ValueError(f"expected {len(self.wheels)} wheel values, got {len(rates)}")
        q = [sum(p * w for p, w in zip(row, rates)) for row in self._pinv]
        if self.holonomic:
            vx, vy, wz = q
            vy *= self.lateral_scale
        else:
            vx, wz = q
            vy = 0.0
        return vx, vy, wz

    @staticmethod
    def scale_to_limit(rates, max_rate):
        """If any wheel would exceed max_rate, scale ALL of them down by the
        same factor. Keeps the direction of motion and the curvature; a
        per-wheel clamp would turn a straight line into a curve."""
        peak = max((abs(v) for v in rates), default=0.0)
        if max_rate > 0 and peak > max_rate:
            k = max_rate / peak
            return [v * k for v in rates]
        return list(rates)

    def sign_pattern(self, vx, vy, wz):
        """Signs (+1/0/-1) of each wheel for a given motion. Used by the
        encoder logger to spin the robot in place for open-loop tests."""
        out = []
        for v in self.inverse(vx, vy, wz):
            out.append(0 if abs(v) < 1e-9 else (1 if v > 0 else -1))
        return out


class OdometryIntegrator:
    """Integrates body-frame increments into a pose in the odom frame.

    Uses the midpoint heading for each step (second order), which is what
    NarrowAisleBot's odometry_publisher does and what closed a 9.5 m,
    three-lap route to 1.9 mm on that robot.
    """

    def __init__(self):
        self.x = 0.0
        self.y = 0.0
        self.theta = 0.0

    def reset(self):
        self.x = self.y = self.theta = 0.0

    def step(self, dx, dy, dtheta):
        mid = self.theta + 0.5 * dtheta
        c, s = math.cos(mid), math.sin(mid)
        self.x += dx * c - dy * s
        self.y += dx * s + dy * c
        self.theta = math.atan2(math.sin(self.theta + dtheta),
                                math.cos(self.theta + dtheta))
        return self.x, self.y, self.theta


def model_from_params(names, xs, ys, axs, ays, wheel_radius, drive_type,
                      track_width_scale=1.0, lateral_scale=1.0):
    """Build a DriveModel from the flat parameter arrays the ROS nodes get
    (see config/robot.yaml, which generate.py writes)."""
    n = len(names)
    if not (len(xs) == len(ys) == len(axs) == len(ays) == n):
        raise ValueError("wheel parameter arrays must all have the same length")
    if drive_type not in DRIVE_TYPES:
        raise ValueError(f"drive_type must be one of {DRIVE_TYPES}, got {drive_type!r}")
    wheels = [Wheel(names[i], xs[i], ys[i], axs[i], ays[i]) for i in range(n)]
    return DriveModel(wheels, wheel_radius, drive_type in HOLONOMIC_TYPES,
                      track_width_scale, lateral_scale)
```

`ros2/robot_base/robot_base/robot_model.py` (244 lines)

```
"""
robot_model.py - read robot_config.yaml, fill in every "auto", and check it.
```

```
Used by generate.py and by tools/encoder_logger.py. The ROS nodes never read
the YAML themselves; they get flat parameters that generate.py writes into
config/robot.yaml. That keeps one source of truth and one place that
interprets it (this file).

load_config(path) returns the config dict with:
  - every wheel's x, y, ax, ay, counts_per_rev resolved to numbers
  - nav.controller and nav.inflation_radius resolved
  - lists (kff, kstat) checked against the number of wheels
and raises ConfigError with a plain-English message on anything wrong.
"""

import math
import os

try:
    import yaml
except ImportError:  # pragma: no cover
    yaml = None

from .kinematics import DRIVE_TYPES, HOLONOMIC_TYPES, Wheel, DriveModel


class ConfigError(ValueError):
    pass


ALIASES = {
    "l": "left", "r": "right",
    "fl": "front_left", "fr": "front_right",
    "rl": "rear_left", "rr": "rear_right",
    "ml": "middle_left", "mr": "middle_right",
    "bl": "rear_left", "br": "rear_right",
    "back_left": "rear_left", "back_right": "rear_right",
}

# Which pins each driver type needs, in the order the firmware stores them
# (PIN_A, PIN_B, PIN_C).
DRIVER_PINS = {
    "pwm_dir": ("pwm", "dir", None),
    "pwm_in1_in2": ("pwm", "in1", "in2"),
    "dual_pwm": ("pwm_fwd", "pwm_rev", None),
}

# Mecanum roller handedness for the standard mounting. With these signs,
# +vy (left) spins front_left and rear_right backwards and the other two
# forwards, which is the usual layout. Verify on the robot (manual 10.3).
MECANUM_AUTO_SIGN = {
    "front_left": -1, "front_right": +1,
    "rear_left": +1, "rear_right": -1,
}


def canonical(name):
    n = str(name).strip().lower().replace("-", "_").replace(" ", "_")
    return ALIASES.get(n, n)


def _auto_xy(name, drive, track, base):
    n = canonical(name)
    half_t, half_b = track / 2.0, base / 2.0
    side = None
    if n.endswith("left") or n == "left":
        side = +1
    elif n.endswith("right") or n == "right":
        side = -1
    if side is None:
        raise ConfigError(
            f"wheel '{name}': cannot place it automatically. Name it left/right, "
            f"front_left/front_right/rear_left/rear_right (or fl/fr/rl/rr), or give x and y.")
    if n.startswith("front"):
        x = half_b
    elif n.startswith("rear"):
        x = -half_b
    else:
        x = 0.0
    if drive in ("skid_steer", "mecanum") and n in ("left", "right") and base > 0:
        raise ConfigError(f"wheel '{name}': with a wheelbase, name wheels front_/rear_/middle_ "
                          f"or give x and y explicitly.")
    return x, side * half_t
```

```
def _num(v, what):
    try:
        return float(v)
    except (TypeError, ValueError):
        raise ConfigError(f"{what} must be a number, got {v!r}")


def resolve(cfg):
    """Resolve and validate a config dict in place. Returns it."""
    robot = cfg.get("robot") or {}
    drive = robot.get("drive_type")
    if drive not in DRIVE_TYPES:
        raise ConfigError(f"robot.drive_type must be one of {DRIVE_TYPES}, got {drive!r}")
    r = _num(robot.get("wheel_radius"), "robot.wheel_radius")
    if not 0.005 < r < 1.0:
        raise ConfigError(f"robot.wheel_radius {r} m looks wrong (expect 0.005 to 1.0)")
    track = _num(robot.get("track_width", 0), "robot.track_width")
    base = _num(robot.get("wheelbase", 0), "robot.wheelbase")
    robot.setdefault("track_width_scale", 1.0)
    robot.setdefault("lateral_scale", 1.0)

    wheels = cfg.get("wheels") or []
    if not wheels:
        raise ConfigError("wheels: list at least one driven wheel")
    if len(wheels) > 6:
        raise ConfigError("the firmware template supports up to 6 motors")
    names = [str(w.get("name", "")).strip() for w in wheels]
    if any(not n for n in names) or len(set(names)) != len(names):
        raise ConfigError("every wheel needs a unique name")
    if drive == "differential" and len(wheels) != 2:
        raise ConfigError("drive_type differential needs exactly 2 wheels "
                          "(use skid_steer for 4 or 6)")
    if drive == "mecanum" and len(wheels) != 4:
        raise ConfigError("drive_type mecanum needs exactly 4 wheels")

    dtype = (cfg.get("driver") or {}).get("type")
    if dtype not in DRIVER_PINS:
        raise ConfigError(f"driver.type must be one of {tuple(DRIVER_PINS)}, got {dtype!r}")

    for i, w in enumerate(wheels):
        tag = f"wheels[{i}] ({w['name']})"
        x, y = w.get("x", "auto"), w.get("y", "auto")
        if x == "auto" or y == "auto" or x is None or y is None:
            if drive == "omni":
                raise ConfigError(f"{tag}: omni wheels need explicit x, y and drive_angle_deg")
            ax_, ay_ = _auto_xy(w["name"], drive, track, base)
            x = ax_ if x in ("auto", None) else x
            y = ay_ if y in ("auto", None) else y
        w["x"], w["y"] = _num(x, f"{tag}.x"), _num(y, f"{tag}.y")

        if drive in ("differential", "skid_steer"):
            w["ax"], w["ay"] = 1.0, 0.0
        elif drive == "mecanum":
            s = w.get("mecanum_sign", "auto")
            if s in ("auto", None):
                s = MECANUM_AUTO_SIGN.get(canonical(w["name"]))
                if s is None:
                    raise ConfigError(f"{tag}: name mecanum wheels front_left/front_right/"
                                      f"rear_left/rear_right, or set mecanum_sign")
            s = int(s)
            if s not in (-1, 1):
                raise ConfigError(f"{tag}.mecanum_sign must be +1 or -1")
            w["mecanum_sign"] = s
            w["ax"], w["ay"] = 1.0, float(s)
        else:  # omni
            ang = math.radians(_num(w.get("drive_angle_deg"), f"{tag}.drive_angle_deg"))
            w["ax"], w["ay"] = math.cos(ang), math.sin(ang)

        cpr = w.get("counts_per_rev", "auto")
        if cpr in ("auto", None):
            ppr = _num(w.get("encoder_ppr"), f"{tag}.encoder_ppr")
            dec = _num(w.get("encoder_decoding", 4), f"{tag}.encoder_decoding")
            gear = _num(w.get("gear_ratio", 1), f"{tag}.gear_ratio")
            cpr = ppr * dec * gear
        cpr = _num(cpr, f"{tag}.counts_per_rev")
        if cpr <= 0:
            raise ConfigError(f"{tag}: counts_per_rev must be positive")
        w["counts_per_rev"] = cpr
        dec = int(_num(w.get("encoder_decoding", 4), f"{tag}.encoder_decoding"))
        if dec not in (2, 4):
            raise ConfigError(f"{tag}.encoder_decoding must be 2 or 4 for the firmware template")
```

```
        w["encoder_decoding"] = dec
        for k in ("motor_dir", "encoder_dir"):
            v = int(w.get(k, 1))
            if v not in (-1, 1):
                raise ConfigError(f"{tag}.{k} must be +1 or -1")
            w[k] = v

        pins = w.get("pins") or {}
        for pin_name in [p for p in DRIVER_PINS[dtype] if p] + ["enc_a", "enc_b"]:
            if pin_name not in pins:
                raise ConfigError(f"{tag}.pins needs '{pin_name}' for driver.type {dtype}")

    decs = {w["encoder_decoding"] for w in wheels}
    if len(decs) != 1:
        raise ConfigError("all wheels must use the same encoder_decoding")

    # The model has to be solvable, or odometry is meaningless.
    model = build_model(cfg)

    mcu = cfg.setdefault("mcu", {})
    n = len(wheels)
    for k in ("kff", "kstat"):
        v = mcu.get(k)
        if v in (None, "auto"):
            v = [0.0] * n
        if not isinstance(v, (list, tuple)) or len(v) != n:
            raise ConfigError(f"mcu.{k} needs exactly {n} values (one per wheel)")
        mcu[k] = [float(x) for x in v]
    if mcu.get("encoder_backend") not in ("pcnt", "isr"):
        raise ConfigError("mcu.encoder_backend must be pcnt or isr")
    if mcu.get("encoder_backend") == "pcnt" and mcu.get("board") != "esp32":
        raise ConfigError("encoder_backend pcnt only exists on the classic ESP32; use isr")

    nav = cfg.setdefault("nav", {})
    ctrl = nav.get("controller", "auto")
    if ctrl in ("auto", None):
        ctrl = "mppi" if drive in HOLONOMIC_TYPES else "rpp"
    if ctrl not in ("rpp", "mppi"):
        raise ConfigError("nav.controller must be auto, rpp or mppi")
    nav["controller"] = ctrl

    fp = footprint(cfg)
    circ = max(math.hypot(px, py) for px, py in fp)
    if nav.get("inflation_radius", "auto") in ("auto", None):
        nav["inflation_radius"] = round(circ + 0.10, 3)
    nav["circumscribed_radius"] = round(circ, 4)
    cfg["_model"] = model
    return cfg


def build_model(cfg):
    robot = cfg["robot"]
    wheels = [Wheel(w["name"], w["x"], w["y"], w["ax"], w["ay"]) for w in cfg["wheels"]]
    return DriveModel(wheels, robot["wheel_radius"],
                      robot["drive_type"] in HOLONOMIC_TYPES,
                      robot.get("track_width_scale", 1.0),
                      robot.get("lateral_scale", 1.0))


def footprint(cfg, padded=True):
    """Rectangle corners in base_link, counter-clockwise from front-left."""
    body = cfg["robot"]["body"]
    L, W = _num(body["length"], "body.length"), _num(body["width"], "body.width")
    cx = _num(body.get("center_x", 0.0), "body.center_x")
    m = _num(body.get("footprint_margin", 0.0), "body.footprint_margin") if padded else 0.0
    hx, hy = L / 2 + m, W / 2 + m
    return [(cx + hx, hy), (cx - hx, hy), (cx - hx, -hy), (cx + hx, -hy)]


def load_config(path):
    if yaml is None:
        raise ConfigError("PyYAML missing: sudo apt install python3-yaml")
    path = os.path.expanduser(path)
    with open(path) as fh:
        cfg = yaml.safe_load(fh)
    if not isinstance(cfg, dict):
        raise ConfigError(f"{path} is empty or not a YAML mapping")
    return resolve(cfg)
```

`ros2/robot_base/robot_base/protocol.py` (291 lines)

```
"""
protocol.py - the Pi ⟷ microcontroller serial protocol, in one place.

Shared by the ROS bridge (mcu_bridge.py) and the encoder logger
(tools/encoder_logger.py). Nothing here imports ROS.

Wire format (identical in spirit to NarrowAisleBot's ESP32 v3.0 protocol,
generalised from 4 motors to N):

  Pi → MCU, every command framed as <...>:
    <V,w0,...,wN-1>   wheel velocity targets, rad/s, closed loop
    <M,p0,...,pN-1>   raw PWM, open loop, until the next V/B/T
    <B,vx,vy,wz>      body twist, the MCU runs the inverse kinematics
    <T,i,w>           one motor closed loop
    <S>               E-STOP, latches until <E1>
    <E1> / <E0>       enable (and clear E-STOP) / disable
    <W1> / <W0>       command watchdog on / off
    <Y1> / <Y0>       runaway / stall / overspeed trips on / off
    <G,kp,ki,kd>      PID gains
    <F,...N>          feedforward slope Kff per motor
    <K,...N>          static friction Kstat per motor
    <A,accel>         setpoint slew limit rad/s^2, 0 = off
    <X,vmax>          wheel speed clamp rad/s
    <L0> <L1[,hz]> <L2[,hz]>   telemetry off / basic / extended
    <O> <R>           print / zero accumulated wheel angles
    <P> <I> <?> <H>   ping / config dump / live state / help

  MCU → Pi:
    telemetry lines, plain CSV, first field is the MCU's millis():
      L1:  ms, then per motor: target, actual, pwm            (1 + 3N fields)
      L2:  L1, then per motor: error, ff, iterm, pos_rad      (1 + 7N fields)
    replies, always in square brackets: [PONG] [OK,...] [ERR,...]
      [WARN,...] [TRIP,...] [INFO,...] [MOTORS,N,name0,...] ...

Any line that is neither is ignored, so a boot banner never breaks parsing.
"""

import threading
import time

BASIC_FIELDS = ("target", "actual", "pwm")
EXT_FIELDS = ("error", "ff", "iterm", "pos_rad")


def frame(*parts):
    return "<" + ",".join(str(p) for p in parts) + ">"


def fmt(v, nd=3):
    return f"{v:.{nd}f}"


def cmd_velocity(rates):
    return frame("V", *(fmt(r) for r in rates))


def cmd_pwm(pwms):
    return frame("M", *(str(int(round(p))) for p in pwms))


def cmd_gains(kp, ki, kd):
    return frame("G", fmt(kp, 4), fmt(ki, 4), fmt(kd, 4))


def cmd_per_motor(letter, values):
    return frame(letter, *(fmt(v, 4) for v in values))


class Telemetry:
    __slots__ = ("mcu_ms", "target", "actual", "pwm",
                 "error", "ff", "iterm", "pos_rad", "extended", "values")

    def __init__(self, mcu_ms, values, n):
        self.mcu_ms = mcu_ms
        self.values = values                        # everything after the stamp
        self.target = [values[3 * i] for i in range(n)]
        self.actual = [values[3 * i + 1] for i in range(n)]
        self.pwm = [values[3 * i + 2] for i in range(n)]
```

```
        self.extended = len(values) >= 7 * n
        if self.extended:
            b = 3 * n
            self.error = [values[b + 4 * i] for i in range(n)]
            self.ff = [values[b + 4 * i + 1] for i in range(n)]
            self.iterm = [values[b + 4 * i + 2] for i in range(n)]
            self.pos_rad = [values[b + 4 * i + 3] for i in range(n)]
        else:
            self.error = self.ff = self.iterm = self.pos_rad = None


def parse_telemetry(line, n):
    """Return Telemetry or None. Accepts exactly 1+3N or 1+7N fields."""
    if not line or not line[0].isdigit() or "," not in line:
        return None
    parts = line.split(",")
    if len(parts) not in (1 + 3 * n, 1 + 7 * n):
        return None
    try:
        vals = [float(p) for p in parts]
    except ValueError:
        return None
    return Telemetry(int(vals[0]), vals[1:], n)


def parse_reply(line):
    """'[OK,GAINS,1,2,3]' -> ('OK', ['GAINS', '1', '2', '3']). None if not a reply."""
    if not (line.startswith("[") and line.endswith("]")):
        return None
    body = line[1:-1].split(",")
    return body[0], body[1:]


def align_to_mcu_clock(rows):
    """Re-time rows [(pi_time, Telemetry)] on the microcontroller's clock.

    USB delivery jitters the Pi-side arrival time by milliseconds to tens of
    milliseconds. The MCU's millis() stamp is taken when the sample is
    made. Delays only ever make a line arrive LATER, so the smallest
    (arrival - mcu_time) over the run is the true offset between the two
    clocks; adding it to every mcu_time gives sample times on the Pi's
    clock without the transport jitter. Crystal drift (~50 ppm) is a few
    ms over a minute and is ignored.
    """
    if not rows:
        return []
    offset = min(t - tel.mcu_ms / 1000.0 for t, tel in rows)
    return [(tel.mcu_ms / 1000.0 + offset, tel) for t, tel in rows]


def csv_header(names, extended):
    cols = ["pi_time_s", "mcu_ms"]
    for nm in names:
        cols += [f"{nm}_{f}" for f in ("target_rads", "actual_rads", "pwm")]
    if extended:
        for nm in names:
            cols += [f"{nm}_{f}" for f in EXT_FIELDS]
    return cols


class SerialLink:
    """A serial port with a background reader thread.

    Every line is timestamped on arrival in the reader thread (so writer
    activity never adds jitter to the timestamps), then handed to
    on_line(t, line) if given. Telemetry rows can also be collected into a
    list between start_collect() and stop_collect().
    """

    def __init__(self, port, baud, n_motors=None, on_line=None,
                 open_delay=2.0, verbose=False):
        import serial  # imported here so the pure helpers work without pyserial
        self._serial_mod = serial
        self.port = port
        self.ser = serial.Serial(port, baud, timeout=0.1, write_timeout=1.0)
        # Opening the port resets most boards (DTR). Give it time to boot.
        time.sleep(open_delay)
        self.ser.reset_input_buffer()
        self.n = n_motors
        self.on_line = on_line
        self.verbose = verbose
        self.rows = []
```

```
        self.replies = []
        self._collect = False
        self._lock = threading.Lock()
        self._wlock = threading.Lock()
        self.running = True
        self.last_rx = 0.0
        self.error = None
        self._thread = threading.Thread(target=self._reader, daemon=True)
        self._thread.start()

    def _reader(self):
        buf = b""
        while self.running:
            try:
                # Read what is waiting, or block for ONE byte. ser.read(512)
                # with a timeout waits until 512 bytes pile up, so lines
                # arrive in bursts and get timestamped up to ~100 ms late.
                # NarrowAisleBot's nab_pid_logger.py does exactly that.
                chunk = self.ser.read(self.ser.in_waiting or 1)
            except Exception as e:  # serial.SerialException, OSError
                self.error = str(e)
                self.running = False
                break
            if not chunk:
                continue
            buf += chunk
            while b"\n" in buf:
                raw, buf = buf.split(b"\n", 1)
                line = raw.decode("ascii", "ignore").strip()
                if line:
                    self._handle(time.time(), line)
            if len(buf) > 4096:          # garbage with no newline: drop it
                buf = b""

    def _handle(self, t, line):
        self.last_rx = t
        if self.on_line:
            try:
                self.on_line(t, line)
            except Exception:
                pass
        if line.startswith("["):
            with self._lock:
                self.replies.append((t, line))
                del self.replies[:-200]
            if self.verbose:
                print(f"    {line}")
            return
        if self._collect and self.n:
            tel = parse_telemetry(line, self.n)
            if tel is not None:
                with self._lock:
                    self.rows.append((t, tel))

    def send(self, cmd, settle=0.02):
        with self._wlock:
            self.ser.write(cmd.encode("ascii"))
            self.ser.flush()
        if settle:
            time.sleep(settle)

    def query(self, cmd, prefix, timeout=1.0):
        """Send cmd, wait for the first reply starting with '[' + prefix."""
        self.drain_replies()
        self.send(cmd, 0)
        t_end = time.time() + timeout
        while time.time() < t_end:
            with self._lock:
                for _, r in self.replies:
                    if r.startswith("[" + prefix):
                        return r
            time.sleep(0.01)
        return None

    def start_collect(self):
        with self._lock:
            self.rows = []
            self._collect = True

    def stop_collect(self):
        with self._lock:
            self._collect = False
```

```
            return list(self.rows)

    def drain_replies(self):
        with self._lock:
            r = list(self.replies)
            self.replies.clear()
        return r

    def close(self, stop=True):
        if stop:
            for c in ("<S>", "<L0>", "<E0>"):
                try:
                    self.send(c, 0.03)
                except Exception:
                    pass
        self.running = False
        time.sleep(0.15)
        try:
            self.ser.close()
        except Exception:
            pass


def read_identity(link, timeout=1.5):
    """Ask the firmware who it is. Returns dict with 'info', 'n', 'names'
    (n/names None if the firmware does not print [MOTORS,...])."""
    link.drain_replies()
    link.send("<I>", 0)
    t_end = time.time() + timeout
    out = {"info": None, "n": None, "names": None, "lines": []}
    while time.time() < t_end:
        time.sleep(0.05)
        for _, r in link.drain_replies():
            out["lines"].append(r)
            p = parse_reply(r)
            if not p:
                continue
            tag, fields = p
            if tag == "INFO":
                out["info"] = ",".join(fields)
            elif tag == "MOTORS" and fields:
                try:
                    out["n"] = int(fields[0])
                    out["names"] = fields[1:1 + out["n"]]
                except ValueError:
                    pass
        if out["n"] is not None and out["info"] is not None:
            break
    return out
```

`ros2/robot_base/robot_base/scan_math.py` (71 lines)

```
"""
scan_math.py - the LaserScan arithmetic behind scan_filter.py. No ROS.

Two corrections, both learnt the hard way on NarrowAisleBot:

mirror
    Some scanners report bearings in the opposite rotational sense to
    REP-103 (counter-clockwise positive). TF cannot express a reflection, so
    no URDF rotation fixes it: the array has to be reversed. Reversing it
    and swapping the angle limits (new min = -old max, new max = -old min)
    puts every beam at exactly the negated angle, symmetric limits or not.

mask
    Arcs where the laser sees the robot's own body (a mast, a bracket, a
    cable) are set to NaN. NaN, not 0 and not inf: 0 would mark an obstacle
    on top of the sensor, inf would CLEAR the costmap along that ray; NaN is
    "no information" to both slam_toolbox and the costmaps.
"""

import math


def in_arc(theta_deg, a_deg, b_deg):
    """True if theta lies on the counter-clockwise arc from a to b
    (inclusive). Works across +/-180: in_arc(170, 135, 225) is True."""
    span = (b_deg - a_deg) % 360.0
    d = (theta_deg - a_deg) % 360.0
```

```
    return d <= span + 1e-9


def mirror(ranges, intensities, angle_min, angle_max):
    """Reverse the beam order. Returns (ranges, intensities, new_min, new_max)."""
    r = list(reversed(ranges))
    i = list(reversed(intensities)) if intensities else intensities
    return r, i, -angle_max, -angle_min


def apply_mask(ranges, angle_min, angle_increment, arcs_deg):
    """Set every beam whose angle falls in any [from, to] arc (degrees,
    laser frame) to NaN. Returns (ranges, number masked)."""
    out = list(ranges)
    n = 0
    if not arcs_deg:
        return out, 0
    for k in range(len(out)):
        th = math.degrees(angle_min + k * angle_increment)
        for a, b in arcs_deg:
            if in_arc(th, a, b):
                out[k] = float("nan")
                n += 1
                break
    return out, n


def pairs(flat):
    """[a0, b0, a1, b1, ...] -> [(a0, b0), (a1, b1), ...]"""
    if len(flat) % 2:
        raise ValueError("mask_deg needs an even number of values (pairs of from, to)")
    return [(float(flat[k]), float(flat[k + 1])) for k in range(0, len(flat), 2)]


def scan_to_points(ranges, angle_min, angle_increment, range_min, range_max, step=1):
    """(x, y) in the laser frame for every valid beam, every `step`th beam."""
    pts = []
    for k in range(0, len(ranges), step):
        r = ranges[k]
        if r is None or not (range_min <= r <= range_max) or math.isnan(r):
            continue
        a = angle_min + k * angle_increment
        pts.append((r * math.cos(a), r * math.sin(a)))
    return pts
```

`ros2/robot_base/robot_base/telemetry_csv.py` (44 lines)

```
"""
telemetry_csv.py - write mcu_bridge telemetry rows to CSV. No ROS, so the
dashboard's mock mode and the ROS logger share it.
"""

import csv
import os
from datetime import datetime

from robot_base.protocol import csv_header


class TelemetryCsv:
    """Shared by this node and the dashboard."""

    def __init__(self, log_dir, names, tag="ros"):
        d = os.path.expanduser(log_dir)
        os.makedirs(d, exist_ok=True)
        self.path = os.path.join(d, f"{tag}_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv")
        self.names = list(names)
        self.n = len(self.names)
        self.fh = None
        self.w = None
        self.rows = 0

    def add(self, t, data):
        """data = [mcu_ms, values...] as mcu_bridge publishes it."""
        vals = list(data[1:])
        if len(vals) not in (3 * self.n, 7 * self.n):
            return
        if self.fh is None:
            self.fh = open(self.path, "w", newline="")
```

```
            self.w = csv.writer(self.fh)
            self.w.writerow(csv_header(self.names, len(vals) == 7 * self.n))
        self.w.writerow([f"{t:.4f}", int(data[0])] + [f"{v:g}" for v in vals])
        self.rows += 1
        if self.rows % 50 == 0:
            self.fh.flush()

    def close(self):
        if self.fh:
            self.fh.close()
            self.fh = None
        return self.path if self.rows else None
```

`ros2/robot_base/robot_base/mcu_bridge.py` (231 lines)

```
#!/usr/bin/env python3
"""
mcu_bridge - the only ROS node that talks to the microcontroller.

  subscribes  /wheel_cmd      Float64MultiArray  wheel targets, rad/s, wheel order
              /mcu/command    String             raw protocol command, e.g. "<S>"
  publishes   /wheel_states   JointState         name, position (rad), velocity (rad/s),
                                                 effort (PWM), stamped at sample time
              /mcu/telemetry  Float64MultiArray  [mcu_ms, then every telemetry value]
              /mcu/status     String             every [..] line the firmware prints

Behaviour worth knowing before you debug it:

  * On connect it sends <E1> (enable) and <L2,hz> (extended telemetry), and
    re-sends <L2> every 5 s. NarrowAisleBot lost telemetry silently after a
    brownout reset the ESP32 without the port closing; odometry stopped,
    slam_toolbox sat at "Activating" with no error. Re-sending heals it.
  * It asks the firmware for [MOTORS,N,names] and REFUSES to drive if the
    names differ from wheel_names. A wheel order mismatch sends the right
    command to the wrong wheel; that should be impossible, not just unlikely.
  * No /wheel_cmd for cmd_timeout seconds: it sends zeros once. The
    firmware's own watchdog (750 ms) is the layer below that.
  * On shutdown it sends <S> then <E0>.
  * Only this process may open the port. The encoder logger and this
    bridge cannot run at the same time.
"""

import threading
import time

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState
from std_msgs.msg import Float64MultiArray, String
from builtin_interfaces.msg import Time

from robot_base import protocol as P


class McuBridge(Node):

    def __init__(self):
        super().__init__("mcu_bridge")
        self.declare_parameter("port", "/dev/robot_mcu")
        self.declare_parameter("baud", 921600)
        self.declare_parameter("wheel_names", ["left", "right"])
        self.declare_parameter("max_wheel_speed", 10.0)
        self.declare_parameter("telemetry_mode", 2)
        self.declare_parameter("telemetry_hz", 20.0)
        self.declare_parameter("cmd_timeout", 0.5)
        self.declare_parameter("open_delay", 2.0)

        self.port = self.get_parameter("port").value
        self.baud = int(self.get_parameter("baud").value)
        self.names = list(self.get_parameter("wheel_names").value)
        self.n = len(self.names)
        self.vmax = float(self.get_parameter("max_wheel_speed").value)
        self.tmode = int(self.get_parameter("telemetry_mode").value)
        self.thz = float(self.get_parameter("telemetry_hz").value)
        self.cmd_timeout = float(self.get_parameter("cmd_timeout").value)
        self.open_delay = float(self.get_parameter("open_delay").value)

        self.link = None
        self.ready = False              # connected AND identity checked
```

```
        self.last_cmd = 0.0
        self.last_sent_nonzero = False
        self.offset = None              # arrival - mcu time, see _stamp()
        self.last_mcu_ms = None
        self.rx_rows = 0

        self.pub_states = self.create_publisher(JointState, "wheel_states", 20)
        self.pub_tel = self.create_publisher(Float64MultiArray, "mcu/telemetry", 20)
        self.pub_status = self.create_publisher(String, "mcu/status", 20)
        self.create_subscription(Float64MultiArray, "wheel_cmd", self._on_cmd, 10)
        self.create_subscription(String, "mcu/command", self._on_raw, 10)
        self.create_timer(0.05, self._watchdog)
        self.create_timer(5.0, self._keepalive)
        self.create_timer(10.0, self._report)

        self._stop = False
        threading.Thread(target=self._connection_loop, daemon=True).start()
        self.get_logger().info(f"mcu_bridge: {self.port} @ {self.baud}, wheels {self.names}")

    # ── connection management (own thread: opening a port can take seconds)
    def _connection_loop(self):
        while not self._stop:
            if self.link is None or not self.link.running:
                self.ready = False
                if self.link is not None:
                    self.get_logger().warn(f"serial link lost: {self.link.error}")
                    self.link = None
                try:
                    link = P.SerialLink(self.port, self.baud, n_motors=self.n,
                                        on_line=self._on_line, open_delay=self.open_delay)
                except Exception as e:  # noqa: BLE001
                    self.get_logger().warn(f"cannot open {self.port}: {e}",
                                           throttle_duration_sec=10.0)
                    time.sleep(2.0)
                    continue
                self.link = link
                self.offset = None
                ident = P.read_identity(link)
                if ident["names"] is not None and ident["names"] != self.names:
                    self.get_logger().error(
                        f"firmware motors {ident['names']} != wheel_names {self.names}. "
                        "NOT driving. Regenerate robot_config.h and reflash, or fix "
                        "robot_config.yaml.")
                    time.sleep(5.0)
                    link.close(stop=True)
                    continue
                if ident["names"] is None:
                    self.get_logger().warn("firmware did not report [MOTORS,...]; "
                                           "trusting wheel_names")
                link.send("<E1>", 0.05)
                link.send(f"<L{self.tmode},{self.thz:g}>", 0.05)
                self.ready = True
                self.get_logger().info(f"connected: {ident['info'] or 'unknown firmware'}")
            time.sleep(1.0)

    def _send(self, cmd):
        if self.link is None or not self.link.running:
            return False
        try:
            self.link.send(cmd, 0)
            return True
        except Exception as e:  # noqa: BLE001
            self.get_logger().warn(f"write failed: {e}", throttle_duration_sec=5.0)
            self.link.running = False
            return False

    # ── commands in ──────────────────────────────────────────────
    def _on_cmd(self, msg):
        if not self.ready:
            return
        if len(msg.data) != self.n:
            self.get_logger().warn(f"/wheel_cmd has {len(msg.data)} values, expected {self.n}",
                                   throttle_duration_sec=5.0)
            return
        w = [max(-self.vmax, min(self.vmax, float(v))) for v in msg.data]
        if self._send(P.cmd_velocity(w)):
            self.last_cmd = time.time()
            self.last_sent_nonzero = any(abs(v) > 1e-6 for v in w)

    def _on_raw(self, msg):
        cmd = msg.data.strip()
        if not (cmd.startswith("<") and cmd.endswith(">")):
```

```
            self.get_logger().warn(f"ignoring /mcu/command without <...> framing: {cmd!r}")
            return
        self.get_logger().info(f"raw command: {cmd}")
        self._send(cmd)

    def _watchdog(self):
        if self.last_sent_nonzero and time.time() - self.last_cmd > self.cmd_timeout:
            if self._send(P.cmd_velocity([0.0] * self.n)):
                self.last_sent_nonzero = False

    def _keepalive(self):
        if self.ready:
            self._send(f"<L{self.tmode},{self.thz:g}>")

    def _report(self):
        if self.ready and self.rx_rows == 0:
            self.get_logger().warn("connected but no telemetry for 10 s: is the firmware "
                                   "the kit's (send <I>), and the baud right?")
        self.rx_rows = 0

    # ── telemetry out ──────────────────────────────────────────────
    def _stamp(self, arrival, mcu_ms):
        """Sample time on the ROS clock. USB delivery adds a variable delay;
        the smallest (arrival - mcu time) seen is the true clock offset, so
        delay = (arrival - mcu) - offset. The offset relaxes slowly upward so
        a later USB hiccup or crystal drift cannot pin it forever, and it
        resets if the MCU reboots (millis goes backwards)."""
        mcu = mcu_ms / 1000.0
        if self.last_mcu_ms is not None and mcu_ms < self.last_mcu_ms:
            self.offset = None
        self.last_mcu_ms = mcu_ms
        d = arrival - mcu
        if self.offset is None or d < self.offset:
            self.offset = d
        else:
            self.offset += 1e-5
        delay = max(0.0, d - self.offset)
        now = self.get_clock().now().nanoseconds * 1e-9 - delay
        return Time(sec=int(now), nanosec=int((now % 1.0) * 1e9))

    def _on_line(self, arrival, line):
        if line.startswith("["):
            m = String()
            m.data = line
            self.pub_status.publish(m)
            if line.startswith(("[TRIP", "[ERR", "[WARN")):
                self.get_logger().warn(f"firmware: {line}")
            return
        tel = P.parse_telemetry(line, self.n)
        if tel is None:
            return
        self.rx_rows += 1
        raw = Float64MultiArray()
        raw.data = [float(tel.mcu_ms)] + [float(v) for v in tel.values]
        self.pub_tel.publish(raw)
        js = JointState()
        js.header.stamp = self._stamp(arrival, tel.mcu_ms)
        js.name = self.names
        js.velocity = [float(v) for v in tel.actual]
        js.effort = [float(v) for v in tel.pwm]
        js.position = [float(v) for v in tel.pos_rad] if tel.extended else []
        self.pub_states.publish(js)

    def shutdown(self):
        self._stop = True
        if self.link is not None:
            self.link.close(stop=True)


def main(args=None):
    rclpy.init(args=args)
    node = McuBridge()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.shutdown()
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()
```

```
if __name__ == "__main__":
    main()
```

`ros2/robot_base/robot_base/base_controller.py` (102 lines)

```
#!/usr/bin/env python3
"""
base_controller - body velocity in, wheel velocities out.

  subscribes  /cmd_vel     Twist             REP-103: linear.x forward, linear.y left,
                                             angular.z counter-clockwise
  publishes   /wheel_cmd   Float64MultiArray  rad/s per wheel, in wheel order

The geometry comes from config/robot.yaml (generate.py). linear.y is used
only on a holonomic base (mecanum, omni) and ignored otherwise. If any wheel
would exceed max_wheel_speed, all wheels are scaled down together, so a
curve stays the same curve, just slower.

No /cmd_vel for cmd_timeout seconds: publish zeros once. twist_mux has its
own 0.5 s timeouts upstream and mcu_bridge another downstream; three layers
on purpose.
"""

import time

import rclpy
from geometry_msgs.msg import Twist
from rclpy.node import Node
from std_msgs.msg import Float64MultiArray

from robot_base.kinematics import DriveModel, model_from_params


class BaseController(Node):

    def __init__(self):
        super().__init__("base_controller")
        p = self.declare_parameter
        p("drive_type", "differential")
        p("wheel_radius", 0.05)
        p("wheel_names", ["left", "right"])
        p("wheel_x", [0.0, 0.0])
        p("wheel_y", [0.15, -0.15])
        p("wheel_ax", [1.0, 1.0])
        p("wheel_ay", [0.0, 0.0])
        p("track_width_scale", 1.0)
        p("max_wheel_speed", 10.0)
        p("max_linear", 0.2)
        p("max_angular", 0.8)
        p("cmd_timeout", 0.5)
        g = lambda k: self.get_parameter(k).value  # noqa: E731

        self.model = model_from_params(list(g("wheel_names")), list(g("wheel_x")), list(g("wheel_y")),
                                       list(g("wheel_ax")), list(g("wheel_ay")), float(g("wheel_radius")),
                                       g("drive_type"), float(g("track_width_scale")))
        self.vmax_wheel = float(g("max_wheel_speed"))
        self.max_lin = float(g("max_linear"))
        self.max_ang = float(g("max_angular"))
        self.timeout = float(g("cmd_timeout"))
        self.last = 0.0
        self.nonzero = False

        self.pub = self.create_publisher(Float64MultiArray, "wheel_cmd", 10)
        self.create_subscription(Twist, "cmd_vel", self._on_twist, 10)
        self.create_timer(0.1, self._timeout_check)
        self.get_logger().info(
            f"base_controller: {g('drive_type')}, wheels {self.model.names}, "
            f"r={self.model.r}, holonomic={self.model.holonomic}")

    @staticmethod
    def _clamp(v, lim):
        return max(-lim, min(lim, v))

    def _on_twist(self, msg):
        vx = self._clamp(msg.linear.x, self.max_lin)
        vy = self._clamp(msg.linear.y, self.max_lin) if self.model.holonomic else 0.0
        wz = self._clamp(msg.angular.z, self.max_ang)
        w = DriveModel.scale_to_limit(self.model.inverse(vx, vy, wz), self.vmax_wheel)
```

```
        out = Float64MultiArray()
        out.data = [float(v) for v in w]
        self.pub.publish(out)
        self.last = time.time()
        self.nonzero = any(abs(v) > 1e-9 for v in w)

    def _timeout_check(self):
        if self.nonzero and time.time() - self.last > self.timeout:
            out = Float64MultiArray()
            out.data = [0.0] * len(self.model.wheels)
            self.pub.publish(out)
            self.nonzero = False


def main(args=None):
    rclpy.init(args=args)
    node = BaseController()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()


if __name__ == "__main__":
    main()
```

`ros2/robot_base/robot_base/wheel_odometry.py` (167 lines)

```
#!/usr/bin/env python3
"""
wheel_odometry - wheel encoders in, /odom and TF odom → base_link out.

  subscribes  /wheel_states  JointState  from mcu_bridge
              /odom/reset    Empty       zero the pose where the robot stands
  publishes   /odom          Odometry    REP-103, frame odom, child base_link
              TF             odom → base_link

How it integrates. When the firmware sends extended telemetry (<L2>, the
bridge's default), each message carries every wheel's accumulated ANGLE, an
exact integral of encoder counts. This node differences those angles, runs
them through the forward kinematics to get the body displacement for that
interval, and integrates with the midpoint heading. Nothing is filtered and
no timestamp enters the pose, so neither the firmware's velocity filter nor
USB jitter can bias it. NarrowAisleBot integrated filtered 20 Hz velocity
against Pi arrival time; this is the improvement its own firmware inventory
asked for (section 16, item 4).

Without angles (basic <L1> telemetry) it falls back to velocity x dt.

lateral_scale and track_width_scale are read on every message, so
    ros2 param set /wheel_odometry lateral_scale 0.92
takes effect immediately, without a restart that would lose the pose.

Re-zero ORDER matters once mapping runs: slam_toolbox fixes map → odom at
its first scan, so zero odometry BEFORE starting a map, never during one.
The dashboard refuses /odom/reset while mapping or navigation is running.
"""

import math

import rclpy
from geometry_msgs.msg import TransformStamped
from nav_msgs.msg import Odometry
from rclpy.node import Node
from sensor_msgs.msg import JointState
from std_msgs.msg import Empty
from tf2_ros import TransformBroadcaster

from robot_base.kinematics import OdometryIntegrator, model_from_params


class WheelOdometry(Node):

    def __init__(self):
        super().__init__("wheel_odometry")
```

```
        p = self.declare_parameter
        p("drive_type", "differential")
        p("wheel_radius", 0.05)
        p("wheel_names", ["left", "right"])
        p("wheel_x", [0.0, 0.0])
        p("wheel_y", [0.15, -0.15])
        p("wheel_ax", [1.0, 1.0])
        p("wheel_ay", [0.0, 0.0])
        p("track_width_scale", 1.0)
        p("lateral_scale", 1.0)
        p("odom_frame", "odom")
        p("base_frame", "base_link")
        p("publish_tf", True)
        p("max_wheel_jump", 50.0)    # rad between two messages: anything bigger is a reset
        self.names = list(self.get_parameter("wheel_names").value)
        self.odom_frame = self.get_parameter("odom_frame").value
        self.base_frame = self.get_parameter("base_frame").value
        self.publish_tf = bool(self.get_parameter("publish_tf").value)
        self._scales = None
        self.model = None
        self._rebuild()

        self.odo = OdometryIntegrator()
        self.prev_pos = None
        self.prev_t = None
        self.pub = self.create_publisher(Odometry, "odom", 20)
        self.tf = TransformBroadcaster(self) if self.publish_tf else None
        self.create_subscription(JointState, "wheel_states", self._on_states, 20)
        self.create_subscription(Empty, "odom/reset", self._on_reset, 10)
        self.get_logger().info(f"wheel_odometry: {self.get_parameter('drive_type').value}, "
                               f"wheels {self.names}")

    def _rebuild(self):
        g = lambda k: self.get_parameter(k).value  # noqa: E731
        scales = (float(g("track_width_scale")), float(g("lateral_scale")))
        if scales == self._scales:
            return
        self._scales = scales
        self.model = model_from_params(self.names, list(g("wheel_x")), list(g("wheel_y")),
                                       list(g("wheel_ax")), list(g("wheel_ay")),
                                       float(g("wheel_radius")), g("drive_type"), *scales)
        self.get_logger().info(f"scales: track_width {scales[0]}, lateral {scales[1]}")

    def _on_reset(self, _msg):
        self.odo.reset()
        self.prev_pos = None
        self.get_logger().info("odometry re-zeroed: this spot is now odom (0, 0, 0)")

    def _on_states(self, msg):
        if list(msg.name) != self.names:
            self.get_logger().warn(f"/wheel_states names {list(msg.name)} != {self.names}",
                                   throttle_duration_sec=10.0)
            return
        self._rebuild()
        t = msg.header.stamp.sec + msg.header.stamp.nanosec * 1e-9
        vx, vy, wz = self.model.forward(list(msg.velocity))

        if msg.position:
            pos = list(msg.position)
            if self.prev_pos is not None:
                dphi = [a - b for a, b in zip(pos, self.prev_pos)]
                lim = float(self.get_parameter("max_wheel_jump").value)
                if all(abs(d) < lim for d in dphi):
                    self.odo.step(*self.model.forward(dphi))
                else:
                    self.get_logger().warn("wheel angle jumped (MCU reset or <R>?); "
                                           "skipping one step")
            self.prev_pos = pos
        elif self.prev_t is not None:
            dt = t - self.prev_t
            if 0.0 < dt < 0.5:
                self.odo.step(vx * dt, vy * dt, wz * dt)
        self.prev_t = t

        x, y, th = self.odo.x, self.odo.y, self.odo.theta
        qz, qw = math.sin(th / 2.0), math.cos(th / 2.0)
        od = Odometry()
        od.header.stamp = msg.header.stamp
        od.header.frame_id = self.odom_frame
        od.child_frame_id = self.base_frame
        od.pose.pose.position.x = x
        od.pose.pose.position.y = y
```

```
        od.pose.pose.orientation.z = qz
        od.pose.pose.orientation.w = qw
        od.twist.twist.linear.x = vx
        od.twist.twist.linear.y = vy
        od.twist.twist.angular.z = wz
        # Fixed covariances. Nothing in this stack fuses them yet; an EKF
        # (robot_localization) would, and they should then be measured.
        for k, v in ((0, 0.01), (7, 0.01 if self.model.holonomic else 1e-3), (35, 0.03)):
            od.pose.covariance[k] = v
            od.twist.covariance[k] = v
        self.pub.publish(od)

        if self.tf:
            tf = TransformStamped()
            tf.header = od.header
            tf.child_frame_id = self.base_frame
            tf.transform.translation.x = x
            tf.transform.translation.y = y
            tf.transform.rotation.z = qz
            tf.transform.rotation.w = qw
            self.tf.sendTransform(tf)


def main(args=None):
    rclpy.init(args=args)
    node = WheelOdometry()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()


if __name__ == "__main__":
    main()
```

`ros2/robot_base/robot_base/scan_filter.py` (87 lines)

```
#!/usr/bin/env python3
"""
scan_filter - /scan (driver, best effort) → /scan_filtered (reliable).

Three jobs, all in scan_math.py:

  1. QoS. LiDAR drivers publish BEST_EFFORT. slam_toolbox and the Nav2
     costmaps subscribe RELIABLE, and a reliable subscriber never connects
     to a best-effort publisher: no error, just no data. Republishing as
     reliable is the whole reason NarrowAisleBot's scan_relay first existed.
  2. mirror (parameter), for a scanner that reports bearings backwards.
  3. mask_deg, arcs blanked to NaN where the laser sees the robot itself.

Everything downstream (SLAM, both costmaps, the collision monitor, the
dashboard) reads /scan_filtered, never /scan. If you add a consumer, point
it here too. (NarrowAisleBot's AMCL block was left on /scan by default and
would have read the raw, mirrored scan the day it first ran.)
"""

import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy, qos_profile_sensor_data
from sensor_msgs.msg import LaserScan

from robot_base import scan_math


class ScanFilter(Node):

    def __init__(self):
        super().__init__("scan_filter")
        self.declare_parameter("input_topic", "scan")
        self.declare_parameter("output_topic", "scan_filtered")
        self.declare_parameter("mirror", False)
        self.declare_parameter("mask_enabled", False)
        self.declare_parameter("mask_deg", [0.0, 0.0])
        self.declare_parameter("range_min", 0.1)
        self.declare_parameter("range_max", 10.0)
```

```
        self.mirror = bool(self.get_parameter("mirror").value)
        self.arcs = scan_math.pairs(list(self.get_parameter("mask_deg").value)) \
            if self.get_parameter("mask_enabled").value else []
        self.rmin = float(self.get_parameter("range_min").value)
        self.rmax = float(self.get_parameter("range_max").value)
        out_qos = QoSProfile(depth=10, reliability=ReliabilityPolicy.RELIABLE,
                             history=HistoryPolicy.KEEP_LAST)
        self.pub = self.create_publisher(LaserScan, self.get_parameter("output_topic").value, out_qos)
        self.create_subscription(LaserScan, self.get_parameter("input_topic").value,
                                 self._on_scan, qos_profile_sensor_data)
        self.masked = 0
        self.get_logger().info(f"scan_filter: mirror={self.mirror} mask={self.arcs} "
                               f"range {self.rmin}..{self.rmax} m")

    def _on_scan(self, msg):
        ranges, inten = list(msg.ranges), list(msg.intensities)
        amin, amax = msg.angle_min, msg.angle_max
        if self.mirror:
            ranges, inten, amin, amax = scan_math.mirror(ranges, inten, amin, amax)
        ranges, self.masked = scan_math.apply_mask(ranges, amin, msg.angle_increment, self.arcs)
        out = LaserScan()
        out.header = msg.header
        out.angle_min = amin
        out.angle_max = amax
        out.angle_increment = msg.angle_increment
        out.time_increment = msg.time_increment
        out.scan_time = msg.scan_time
        out.range_min = max(msg.range_min, self.rmin)
        out.range_max = min(msg.range_max, self.rmax) if msg.range_max > 0 else self.rmax
        out.ranges = ranges
        out.intensities = inten
        self.pub.publish(out)


def main(args=None):
    rclpy.init(args=args)
    node = ScanFilter()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()


if __name__ == "__main__":
    main()
```

`ros2/robot_base/robot_base/telemetry_logger.py` (59 lines)

```
#!/usr/bin/env python3
"""
telemetry_logger - write /mcu/telemetry to CSV while the robot runs under ROS.

The encoder logger (tools/encoder_logger.py) needs the serial port to
itself, so it cannot log while ROS drives. This node can: it records what
mcu_bridge already publishes, in the same CSV format, so analyze_log.py reads
either.

    ros2 run robot_base telemetry_logger                    # until Ctrl-C
    ros2 run robot_base telemetry_logger --ros-args -p duration:=60.0

The dashboard's REC button does the same thing from the phone.
"""

import rclpy
from rclpy.node import Node
from std_msgs.msg import Float64MultiArray

from robot_base.telemetry_csv import TelemetryCsv


class TelemetryLogger(Node):
    def __init__(self):
        super().__init__("telemetry_logger")
        self.declare_parameter("wheel_names", ["left", "right"])
        self.declare_parameter("log_dir", "~/robot_logs")
```

```
        self.declare_parameter("duration", 0.0)
        names = list(self.get_parameter("wheel_names").value)
        self.csv = TelemetryCsv(self.get_parameter("log_dir").value, names)
        self.t0 = self.get_clock().now()
        self.duration = float(self.get_parameter("duration").value)
        self.create_subscription(Float64MultiArray, "mcu/telemetry", self._on, 50)
        self.get_logger().info(f"logging /mcu/telemetry to {self.csv.path}")

    def _on(self, msg):
        now = self.get_clock().now()
        self.csv.add(now.nanoseconds * 1e-9, msg.data)
        if self.duration > 0 and (now - self.t0).nanoseconds * 1e-9 > self.duration:
            raise KeyboardInterrupt


def main(args=None):
    rclpy.init(args=args)
    node = TelemetryLogger()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        path = node.csv.close()
        print(f"{node.csv.rows} rows → {path}")
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()


if __name__ == "__main__":
    main()
```

`ros2/robot_base/launch/base.launch.py` (65 lines)

```
#!/usr/bin/env python3
"""
base.launch.py - everything that runs whenever the robot is on.

    ros2 launch robot_base base.launch.py
    ros2 launch robot_base base.launch.py use_dashboard:=false use_foxglove:=true

Starts:
    robot_state_publisher   URDF → /robot_description and the fixed TFs
                            (base_link → laser_frame, wheels, chassis)
    mcu_bridge              /wheel_cmd → serial; serial → /wheel_states
    base_controller         /cmd_vel → /wheel_cmd
    wheel_odometry          /wheel_states → /odom, TF odom → base_link
    twist_mux               /cmd_vel_manual (priority 100) and
                            /cmd_vel_nav_out (priority 10) → /cmd_vel
    dashboard               phone UI on http://<pi>:8080        (use_dashboard)
    foxglove_bridge         ws://<pi>:8765 for Foxglove Studio   (use_foxglove)

The LiDAR, SLAM and Nav2 are NOT started here. They start on demand
(lidar.launch.py, mapping.launch.py, nav2.launch.py), usually from the
dashboard, so that restarting a map never restarts the motor stack.
"""
import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.conditions import IfCondition
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node


def generate_launch_description():
    share = get_package_share_directory("robot_base")
    params = os.path.join(share, "config", "robot.yaml")
    with open(os.path.join(share, "urdf", "robot.urdf")) as fh:
        urdf = fh.read()

    args = [
        DeclareLaunchArgument("params_file", default_value=params),
        DeclareLaunchArgument("use_dashboard", default_value="true"),
        DeclareLaunchArgument("use_foxglove", default_value="false"),
        DeclareLaunchArgument("foxglove_port", default_value="8765"),
    ]
```

```
    p = LaunchConfiguration("params_file")

    return LaunchDescription(args + [
        Node(package="robot_state_publisher", executable="robot_state_publisher",
             parameters=[{"robot_description": urdf}], output="screen"),
        Node(package="robot_base", executable="mcu_bridge", name="mcu_bridge",
             parameters=[p], output="screen"),
        Node(package="robot_base", executable="base_controller", name="base_controller",
             parameters=[p], output="screen"),
        Node(package="robot_base", executable="wheel_odometry", name="wheel_odometry",
             parameters=[p], output="screen"),
        Node(package="twist_mux", executable="twist_mux", name="twist_mux",
             parameters=[os.path.join(share, "config", "twist_mux.yaml")],
             remappings=[("cmd_vel_out", "cmd_vel")], output="screen"),
        Node(package="robot_base", executable="dashboard", name="dashboard",
             parameters=[p], output="screen",
             condition=IfCondition(LaunchConfiguration("use_dashboard"))),
        Node(package="foxglove_bridge", executable="foxglove_bridge", name="foxglove_bridge",
             parameters=[{"port": LaunchConfiguration("foxglove_port")}], output="screen",
             condition=IfCondition(LaunchConfiguration("use_foxglove"))),
    ])
```

`ros2/robot_base/launch/lidar.launch.py` (53 lines)

```
#!/usr/bin/env python3
"""
lidar.launch.py - the LiDAR driver plus scan_filter. Nothing else.

    ros2 launch robot_base lidar.launch.py

Which driver runs comes from config/kit.yaml (generate.py writes it from
robot_config.yaml lidar.model). The driver's own launch file is NOT used:
vendor launch files (YDLIDAR's, for one) also publish a hard-coded
base_link -> laser_frame transform, which then fights the URDF's measured
one. Here the URDF is the only source of that transform.

EXACTLY ONE driver may own the serial port. Before launching by hand:
    ros2 node list | grep -i -e lidar -e rplidar      # must be empty
"""
import os

import yaml
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import LogInfo
from launch_ros.actions import LifecycleNode, Node


def generate_launch_description():
    share = get_package_share_directory("robot_base")
    kit = yaml.safe_load(open(os.path.join(share, "config", "kit.yaml")))["lidar"]
    lidar_params = os.path.join(share, "config", "lidar.yaml")
    robot_params = os.path.join(share, "config", "robot.yaml")
    model = kit.get("model", "none")

    actions = []
    if model == "ydlidar":
        # Lifecycle node with namespace "/", exactly as the vendor launch file
        # declares it (NarrowAisleBot runs it this way on Jazzy).
        actions.append(LifecycleNode(
            package="ydlidar_ros2_driver", executable="ydlidar_ros2_driver_node",
            name="ydlidar_ros2_driver_node", namespace="/", output="screen",
            emulate_tty=True, parameters=[lidar_params]))
    elif model == "rplidar":
        actions.append(Node(package="rplidar_ros", executable="rplidar_node",
                            name="rplidar_node", output="screen", parameters=[lidar_params]))
    elif model == "custom":
        params = [kit["params_file"]] if kit.get("params_file") else []
        actions.append(Node(package=kit["package"], executable=kit["executable"],
                            output="screen", parameters=params,
                            remappings=[("scan", "/scan")]))
    else:
        actions.append(LogInfo(msg="lidar.model is 'none': no driver started"))

    actions.append(Node(package="robot_base", executable="scan_filter", name="scan_filter",
                        parameters=[robot_params], output="screen"))
    return LaunchDescription(actions)
```

`ros2/robot_base/launch/mapping.launch.py` (37 lines)

```
#!/usr/bin/env python3
"""
mapping.launch.py - LiDAR + scan_filter + slam_toolbox (online async).

    ros2 launch robot_base mapping.launch.py

Needs base.launch.py already running: slam_toolbox takes the robot's motion
from TF odom → base_link, which wheel_odometry publishes only while
telemetry flows. Without it slam_toolbox sits at "Activating" and prints
nothing at all (NarrowAisleBot lost an afternoon to that silence). Check:
    ros2 run tf2_ros tf2_echo odom base_link

The params file path is absolute on purpose. A "~" in slam_params_file
fails silently and slam_toolbox runs on its defaults.

Start mapping BEFORE Nav2, never after: Nav2's global costmap rejects the
missing /map once a second until one arrives.
"""
import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource


def generate_launch_description():
    share = get_package_share_directory("robot_base")
    slam_share = get_package_share_directory("slam_toolbox")
    return LaunchDescription([
        IncludeLaunchDescription(PythonLaunchDescriptionSource(
            os.path.join(share, "launch", "lidar.launch.py"))),
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(os.path.join(slam_share, "launch", "online_async_launch.py")),
            launch_arguments={"slam_params_file": os.path.join(share, "config", "slam_toolbox.yaml"),
                              "use_sim_time": "false"}.items()),
    ])
```

`ros2/robot_base/launch/nav2.launch.py` (69 lines)

```
#!/usr/bin/env python3
"""
nav2.launch.py - Nav2 while slam_toolbox maps: send a goal, the robot drives
there, and the map keeps growing on the way.

    ros2 launch robot_base nav2.launch.py

Order: base.launch.py, then mapping.launch.py (wait ~30 s for /map), then this.

Nodes are started explicitly instead of including nav2_bringup. Recent
nav2_bringup also starts route_server and docking_server; docking_server
refuses to configure without dock plugins, and because lifecycle_manager
brings nodes up all-or-nothing, one node nobody needed aborted the whole
bringup on NarrowAisleBot. Start what you use.

The command chain, every autonomous command takes this path:
    controller_server / behavior_server → /cmd_vel_nav
      → velocity_smoother → /cmd_vel_smoothed
        → collision_monitor → /cmd_vel_nav_out
          → twist_mux (priority 10, manual wins) → /cmd_vel → base_controller
behavior_server is remapped too, so recovery Spin and BackUp pass the
smoother and the collision monitor like everything else. (Left on its
default, it publishes straight to /cmd_vel and bypasses both.)

The behaviour tree XML paths are resolved here and passed in, because an
empty string in the YAML does not fall back to the default on this Nav2
build ("Empty Tree", first goal on NarrowAisleBot).
"""
import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node

LIFECYCLE = ["controller_server", "smoother_server", "planner_server", "behavior_server",
             "velocity_smoother", "collision_monitor", "bt_navigator", "waypoint_follower"]
```

```
def generate_launch_description():
    share = get_package_share_directory("robot_base")
    params = os.path.join(share, "config", "nav2_params.yaml")
    bt = os.path.join(get_package_share_directory("nav2_bt_navigator"), "behavior_trees")
    common = [params, {"use_sim_time": False}]
    to_smoother = [("cmd_vel", "cmd_vel_nav")]
    return LaunchDescription([
        Node(package="nav2_controller", executable="controller_server", name="controller_server",
             output="screen", parameters=common, remappings=to_smoother),
        Node(package="nav2_smoother", executable="smoother_server", name="smoother_server",
             output="screen", parameters=common),
        Node(package="nav2_planner", executable="planner_server", name="planner_server",
             output="screen", parameters=common),
        Node(package="nav2_behaviors", executable="behavior_server", name="behavior_server",
             output="screen", parameters=common, remappings=to_smoother),
        Node(package="nav2_velocity_smoother", executable="velocity_smoother",
             name="velocity_smoother", output="screen", parameters=common, remappings=to_smoother),
        Node(package="nav2_collision_monitor", executable="collision_monitor",
             name="collision_monitor", output="screen", parameters=common),
        Node(package="nav2_bt_navigator", executable="bt_navigator", name="bt_navigator",
             output="screen", parameters=common + [{
                 "default_nav_to_pose_bt_xml": os.path.join(
                     bt, "navigate_to_pose_w_replanning_and_recovery.xml"),
                 "default_nav_through_poses_bt_xml": os.path.join(
                     bt, "navigate_through_poses_w_replanning_and_recovery.xml")}]),
        Node(package="nav2_waypoint_follower", executable="waypoint_follower",
             name="waypoint_follower", output="screen", parameters=common),
        Node(package="nav2_lifecycle_manager", executable="lifecycle_manager",
             name="lifecycle_manager_navigation", output="screen",
             parameters=[{"use_sim_time": False, "autostart": True, "node_names": LIFECYCLE}]),
    ])
```

Dashboard

`ros2/robot_base/robot_base/dashboard/server.py` (730 lines)

```
#!/usr/bin/env python3
"""
dashboard - the phone UI. Drive, E-STOP, map, navigate, log, talk to the MCU.

    ros2 run robot_base dashboard                   (normally started by base.launch.py)
    python3 -m robot_base.dashboard.server --mock    (no ROS: a simulated robot,
                                                      for trying the page on a laptop)

Open http://<pi-address>:8080 on a phone on the same network.

Layout of the code
    static/index.html, app.js, style.css   the page. Plain files, served as they
                                           are on disk. NOT a string inside this
                                           Python file: on 15 Sep 2026 a single
                                           escaped apostrophe inside such a string
                                           took NarrowAisleBot's whole dashboard
                                           offline while every server-side check
                                           said "fine" (CLAUDE.md). Keep it out.
    RosIO      the ROS 2 side: subscribers, publishers, TF, process control
    MockIO     the same interface over a simulated robot
    the app    FastAPI + one WebSocket (/ws) carrying JSON both ways

What the buttons do
    E-STOP      zero /cmd_vel_manual, send <S> (latches in the firmware), cancel
                every Nav2 goal. Mapping keeps running: a stop should not cost
                you the map. CLEAR sends <E1>.
    MAP         start/stop mapping.launch.py. Stop saves the map first.
    SAVE        save the map now, keep mapping.
    NAV         start/stop nav2.launch.py (needs MAP running).
    GOAL        tap the map for a goal; drag to set the heading.
    RE-ZERO     odometry (0, 0, 0) where the robot stands. Refused while MAP or
                NAV runs: slam_toolbox pinned map -> odom at its first scan.
    REC         log /mcu/telemetry to <log_dir>/dash_<stamp>.csv
    console     send a raw command to the MCU, e.g. <?> or <G,20,100,0>
"""

import argparse
import asyncio
import base64
```

```
import json
import math
import os
import signal
import subprocess
import threading
import time
from collections import deque
from datetime import datetime

STATIC = os.path.join(os.path.dirname(os.path.abspath(__file__)), "static")


def yaw_of(q):
    return math.atan2(2.0 * (q.w * q.z + q.x * q.y), 1.0 - 2.0 * (q.y * q.y + q.z * q.z))


# ═══════════════════════════════════════════════════════════════
#  Process control (shared by RosIO and MockIO)
# ═══════════════════════════════════════════════════════════════

class Procs:
    """Start/stop launch files in their own process groups, with their
    output in a log file (not /dev/null: you will want it)."""

    def __init__(self, log_dir, notify, mock=False):
        self.log_dir = os.path.expanduser(log_dir)
        os.makedirs(self.log_dir, exist_ok=True)
        self.notify = notify
        self.mock = mock
        self.procs = {}

    def running(self, name):
        p = self.procs.get(name)
        if self.mock:
            return bool(p)
        return p is not None and p.poll() is None

    def start(self, name, launch_file):
        if self.running(name):
            self.notify("info", f"{name} already running")
            return False
        if self.mock:
            self.procs[name] = True
            self.notify("info", f"[mock] started {launch_file}")
            return True
        log = os.path.join(self.log_dir, f"{name}_{datetime.now().strftime('%Y%m%d_%H%M%S')}.log")
        fh = open(log, "w")
        self.procs[name] = subprocess.Popen(["ros2", "launch", "robot_base", launch_file],
                                            stdout=fh, stderr=subprocess.STDOUT,
                                            start_new_session=True)
        self.notify("info", f"started {launch_file} (log: {log})")
        return True

    def stop(self, name, timeout=10.0):
        p = self.procs.pop(name, None)
        if self.mock:
            if p:
                self.notify("info", f"[mock] stopped {name}")
            return
        if p is None or p.poll() is not None:
            return
        try:
            os.killpg(p.pid, signal.SIGINT)
            p.wait(timeout)
        except subprocess.TimeoutExpired:
            os.killpg(p.pid, signal.SIGKILL)
            self.notify("warn", f"{name} did not stop in {timeout:.0f} s; killed")
        except ProcessLookupError:
            pass
        self.notify("info", f"stopped {name}")


def write_pgm(grid, path_base):
    """Fallback map save from the cached OccupancyGrid (trinary, like
    map_saver_cli). grid = dict(width, height, resolution, origin, data[int])."""
    w, h = grid["width"], grid["height"]
    px = bytearray(w * h)
    for j in range(h):
        for i in range(w):
            v = grid["data"][j * w + i]
            px[(h - 1 - j) * w + i] = 205 if v < 0 else (0 if v ≥ 65 else (254 if v ⩽ 25 else 205))
```

```
    with open(path_base + ".pgm", "wb") as fh:
        fh.write(f"P5\n{w} {h}\n255\n".encode() + bytes(px))
    ox, oy, oyaw = grid["origin"]
    with open(path_base + ".yaml", "w") as fh:
        fh.write(f"image: {os.path.basename(path_base)}.pgm\nmode: trinary\n"
                 f"resolution: {grid['resolution']}\norigin: [{ox}, {oy}, {oyaw}]\n"
                 f"negate: 0\noccupied_thresh: 0.65\nfree_thresh: 0.25\n")


# ═══════════════════════════════════════════════════════════════
#  Common state
# ═══════════════════════════════════════════════════════════════

class BaseIO:
    def __init__(self, cfg):
        self.cfg = cfg
        self.lock = threading.Lock()
        self.notices = deque(maxlen=200)
        self.notice_seq = 0
        self.pose = None            # (x, y, yaw, frame)
        self.odom = None            # (x, y, yaw)
        self.wheels = None          # dict(target, actual, pwm)
        self.scan = []              # [(x, y)] in the pose frame
        self.grid = None            # dict(width, height, resolution, origin, data)
        self.map_version = 0
        self.estop = False
        self.mcu_seen = 0.0
        self.nav_status = ""
        self.goal = None
        self.rec = None
        self.procs = Procs(cfg["log_dir"], self.notify, mock=isinstance(self, MockIO))
        self.last_drive = 0.0
        self.drive_nonzero = False

    def notify(self, level, text):
        with self.lock:
            self.notice_seq += 1
            self.notices.append((self.notice_seq, level, text))
        print(f"[dashboard] {level}: {text}", flush=True)

    # ── commands from the page ──────────────────────────────────
    def drive_from_stick(self, x, y, r):
        """x, y: joystick, -1..1, right and up. r: turn input, -1..1, + = left."""
        c = self.cfg
        x, y, r = (max(-1.0, min(1.0, float(v))) for v in (x, y, r))
        if c["holonomic"]:
            vx, vy, wz = y * c["max_linear"], -x * c["max_linear"], r * c["max_angular"]
        else:
            vx, vy, wz = y * c["max_linear"], 0.0, (r - x) * c["max_angular"]
            wz = max(-c["max_angular"], min(c["max_angular"], wz))
        if self.estop:
            vx = vy = wz = 0.0
        self.publish_twist(vx, vy, wz)
        self.last_drive = time.time()
        self.drive_nonzero = any(abs(v) > 1e-6 for v in (vx, vy, wz))

    def deadman(self):
        """Called often. A phone that loses the WebSocket mid-drive must not
        leave the last command standing (twist_mux times out too, at 0.5 s)."""
        if self.drive_nonzero and time.time() - self.last_drive > 0.3:
            self.publish_twist(0.0, 0.0, 0.0)
            self.drive_nonzero = False

    def handle(self, msg):
        t = msg.get("type")
        if t == "drive":
            self.drive_from_stick(msg.get("x", 0), msg.get("y", 0), msg.get("r", 0))
        elif t == "estop":
            self.publish_twist(0.0, 0.0, 0.0)
            self.drive_nonzero = False
            self.mcu("<S>")
            self.cancel_goals()
            self.estop = True
            self.notify("warn", "E-STOP latched. CLEAR to drive again.")
        elif t == "estop_clear":
            self.mcu("<E1>")
            self.estop = False
            self.notify("info", "E-STOP cleared")
        elif t == "map_start":
            self.procs.start("mapping", "mapping.launch.py")
        elif t == "map_stop":
            if self.procs.running("nav"):
```

```
                self.procs.stop("nav")
            self.save_map()
            self.procs.stop("mapping")
        elif t == "map_save":
            self.save_map()
        elif t == "nav_start":
            if not self.procs.running("mapping"):
                self.notify("warn", "start MAP first: Nav2 needs a map and the map → odom transform")
            else:
                self.procs.start("nav", "nav2.launch.py")
        elif t == "nav_stop":
            self.cancel_goals()
            self.procs.stop("nav")
        elif t == "goal":
            if not self.procs.running("nav"):
                self.notify("warn", "NAV is not running")
            else:
                self.send_goal(float(msg["x"]), float(msg["y"]), float(msg["yaw"]))
        elif t == "goal_cancel":
            self.cancel_goals()
        elif t == "rezero":
            if self.procs.running("mapping") or self.procs.running("nav"):
                self.notify("warn", "re-zero refused while MAP or NAV runs. Stop them, re-zero, "
                                    "then start MAP again.")
            else:
                self.rezero()
                self.notify("info", "odometry re-zeroed here")
        elif t == "rec_start":
            self.rec_start()
        elif t == "rec_stop":
            self.rec_stop()
        elif t == "mcu":
            cmd = str(msg.get("cmd", "")).strip()
            if cmd.startswith("<") and cmd.endswith(">") and len(cmd) < 120:
                self.mcu(cmd)
                self.notify("mcu", f"> {cmd}")
            else:
                self.notify("warn", "MCU commands look like <P> or <G,20,100,0>")

    def save_map(self):
        if not self.procs.running("mapping"):
            self.notify("warn", "no map to save: MAP is not running")
            return None
        base = os.path.join(os.path.expanduser(self.cfg["map_dir"]),
                            f"map_{datetime.now().strftime('%Y%m%d_%H%M%S')}")
        ok = self.save_map_to(base)
        if not ok and self.grid:
            write_pgm(self.grid, base)
            ok = True
            self.notify("warn", "map_saver_cli failed; wrote the dashboard's own copy instead")
        if ok:
            self.notify("info", f"map saved: {base}.pgm / .yaml")
        else:
            self.notify("error", "map NOT saved: no map received yet")
        return base if ok else None

    def rec_start(self):
        from robot_base.telemetry_csv import TelemetryCsv
        if self.rec:
            return
        self.rec = TelemetryCsv(self.cfg["log_dir"], self.cfg["wheel_names"], tag="dash")
        self.notify("info", f"recording to {self.rec.path}")

    def rec_stop(self):
        if not self.rec:
            return
        r, self.rec = self.rec, None
        path = r.close()
        self.notify("info", f"recorded {r.rows} rows" + (f" → {path}" if path else ""))

    def shutdown(self):
        self.publish_twist(0.0, 0.0, 0.0)
        self.rec_stop()
        if self.procs.running("nav"):
            self.procs.stop("nav")
        if self.procs.running("mapping"):
            self.notify("info", "shutting down: saving the map before stopping SLAM")
            self.save_map()
            self.procs.stop("mapping")

    # ── what the page sees ──────────────────────────────────────
    def snapshot(self):
```

```
        with self.lock:
            return {
                "type": "state",
                "pose": self.pose, "odom": self.odom, "wheels": self.wheels,
                "estop": self.estop, "mcu_ok": time.time() - self.mcu_seen < 1.5,
                "mapping": self.procs.running("mapping"), "nav": self.procs.running("nav"),
                "nav_status": self.nav_status, "goal": self.goal,
                "rec": self.rec.rows if self.rec else None,
            }

    def scan_msg(self):
        with self.lock:
            pts = self.scan
        step = max(1, len(pts) // 360)
        return {"type": "scan", "points": [[round(x, 3), round(y, 3)] for x, y in pts[::step]]}

    def map_msg(self):
        with self.lock:
            g = self.grid
            v = self.map_version
        if not g:
            return None, v
        data = bytes(255 if c < 0 else min(100, c) for c in g["data"])
        return {"type": "map", "width": g["width"], "height": g["height"],
                "resolution": g["resolution"], "origin": g["origin"],
                "data": base64.b64encode(data).decode("ascii")}, v


# ═══════════════════════════════════════════════════════════════
#  ROS 2
# ═══════════════════════════════════════════════════════════════

class RosIO(BaseIO):

    def __init__(self, node, cfg):
        super().__init__(cfg)
        from action_msgs.msg import GoalStatusArray
        from action_msgs.srv import CancelGoal
        from geometry_msgs.msg import PoseStamped, Twist
        from nav_msgs.msg import OccupancyGrid, Odometry
        from rclpy.qos import (DurabilityPolicy, QoSProfile, ReliabilityPolicy,
                               qos_profile_sensor_data)
        from sensor_msgs.msg import LaserScan
        from std_msgs.msg import Empty, Float64MultiArray, String
        import tf2_ros

        self.node = node
        self.Twist, self.PoseStamped, self.String, self.Empty = Twist, PoseStamped, String, Empty
        self.CancelGoal = CancelGoal
        self.pub_twist = node.create_publisher(Twist, "cmd_vel_manual", 10)
        self.pub_mcu = node.create_publisher(String, "mcu/command", 10)
        self.pub_reset = node.create_publisher(Empty, "odom/reset", 10)
        self.pub_goal = node.create_publisher(PoseStamped, "goal_pose", 10)
        self.cancel_cli = node.create_client(CancelGoal, "navigate_to_pose/_action/cancel_goal")
        latched = QoSProfile(depth=1, reliability=ReliabilityPolicy.RELIABLE,
                             durability=DurabilityPolicy.TRANSIENT_LOCAL)
        node.create_subscription(OccupancyGrid, "map", self._on_map, latched)
        node.create_subscription(LaserScan, "scan_filtered", self._on_scan, qos_profile_sensor_data)
        node.create_subscription(Float64MultiArray, "mcu/telemetry", self._on_tel, 20)
        node.create_subscription(String, "mcu/status", self._on_status, 20)
        node.create_subscription(Odometry, "odom", self._on_odom, 10)
        node.create_subscription(GoalStatusArray, "navigate_to_pose/_action/status", self._on_nav, 10)
        self.tf_buf = tf2_ros.Buffer()
        self.tf_listener = tf2_ros.TransformListener(self.tf_buf, node)
        node.create_timer(0.1, self._update_pose)
        self.n = len(cfg["wheel_names"])

    def _tf2d(self, parent, child):
        import rclpy.time
        try:
            t = self.tf_buf.lookup_transform(parent, child, rclpy.time.Time())
        except Exception:  # noqa: BLE001  (frame does not exist yet)
            return None
        tr = t.transform
        return tr.translation.x, tr.translation.y, yaw_of(tr.rotation)

    def _update_pose(self):
        for frame in ("map", "odom"):
            p = self._tf2d(frame, "base_link")
            if p:
                with self.lock:
                    self.pose = [round(p[0], 4), round(p[1], 4), round(p[2], 4), frame]
```

```
            return

    def _on_map(self, msg):
        info = msg.info
        g = {"width": info.width, "height": info.height, "resolution": info.resolution,
             "origin": [info.origin.position.x, info.origin.position.y, yaw_of(info.origin.orientation)],
             "data": list(msg.data)}
        with self.lock:
            self.grid = g
            self.map_version += 1

    def _on_scan(self, msg):
        frame = "map" if self.pose and self.pose[3] == "map" else "odom"
        tf = self._tf2d(frame, msg.header.frame_id)
        if not tf:
            return
        tx, ty, tyaw = tf
        c, s = math.cos(tyaw), math.sin(tyaw)
        pts = []
        a = msg.angle_min
        for r in msg.ranges:
            if msg.range_min <= r <= msg.range_max:
                lx, ly = r * math.cos(a), r * math.sin(a)
                pts.append((tx + c * lx - s * ly, ty + s * lx + c * ly))
            a += msg.angle_increment
        with self.lock:
            self.scan = pts

    def _on_tel(self, msg):
        d = list(msg.data)
        n = self.n
        vals = d[1:]
        if len(vals) < 3 * n:
            return
        with self.lock:
            self.mcu_seen = time.time()
            self.wheels = {"target": [round(vals[3 * i], 3) for i in range(n)],
                           "actual": [round(vals[3 * i + 1], 3) for i in range(n)],
                           "pwm": [int(vals[3 * i + 2]) for i in range(n)]}
            rec = self.rec
        if rec:
            rec.add(time.time(), d)

    def _on_status(self, msg):
        line = msg.data
        if "ESTOP_LATCHED" in line or line.startswith("[TRIP"):
            self.estop = True
        elif line == "[OK,ENABLED]":
            self.estop = False
        if not line.startswith("[OK,LOG"):        # the bridge's keep-alive, every 5 s
            self.notify("mcu", line)

    def _on_odom(self, msg):
        p = msg.pose.pose
        with self.lock:
            self.odom = [round(p.position.x, 4), round(p.position.y, 4), round(yaw_of(p.orientation), 4)]

    def _on_nav(self, msg):
        names = {1: "accepted", 2: "driving", 3: "canceling", 4: "SUCCEEDED", 5: "CANCELED", 6: "ABORTED"}
        if msg.status_list:
            st = names.get(msg.status_list[-1].status, "")
            if st != self.nav_status:
                self.nav_status = st
                if st in ("SUCCEEDED", "CANCELED", "ABORTED"):
                    self.notify("info" if st == "SUCCEEDED" else "warn", f"goal {st}")

    # ── outputs ──────────────────────────────────────────────────
    def publish_twist(self, vx, vy, wz):
        m = self.Twist()
        m.linear.x, m.linear.y, m.angular.z = float(vx), float(vy), float(wz)
        self.pub_twist.publish(m)

    def mcu(self, cmd):
        m = self.String()
        m.data = cmd
        self.pub_mcu.publish(m)

    def rezero(self):
        self.pub_reset.publish(self.Empty())

    def send_goal(self, x, y, yaw):
        m = self.PoseStamped()
```

```
        m.header.frame_id = "map"
        m.header.stamp = self.node.get_clock().now().to_msg()
        m.pose.position.x, m.pose.position.y = x, y
        m.pose.orientation.z, m.pose.orientation.w = math.sin(yaw / 2), math.cos(yaw / 2)
        self.pub_goal.publish(m)
        self.goal = [round(x, 3), round(y, 3), round(yaw, 3)]
        self.notify("info", f"goal ({x:.2f}, {y:.2f}) heading {math.degrees(yaw):.0f} deg")

    def cancel_goals(self):
        self.goal = None
        if self.cancel_cli.service_is_ready():
            # A zero goal id and zero stamp means "cancel every goal".
            self.cancel_cli.call_async(self.CancelGoal.Request())

    def save_map_to(self, base):
        try:
            r = subprocess.run(["ros2", "run", "nav2_map_server", "map_saver_cli", "-f", base,
                                "--ros-args", "-p", "map_subscribe_transient_local:=true"],
                               capture_output=True, text=True, timeout=25)
            return r.returncode == 0 and os.path.exists(base + ".yaml")
        except (subprocess.TimeoutExpired, FileNotFoundError):
            return False


# ═══════════════════════════════════════════════════════════════
#  Mock robot (no ROS): a 6 x 4 m room with a box, a differential or
#  holonomic base, a 180-beam scanner and a map that fills in as it looks.
# ═══════════════════════════════════════════════════════════════

class MockIO(BaseIO):
    RES = 0.05

    def __init__(self, cfg):
        super().__init__(cfg)
        self.walls = [((-1, -2), (5, -2)), ((5, -2), (5, 2)), ((5, 2), (-1, 2)), ((-1, 2), (-1, -2)),
                      ((2, -0.5), (2.6, -0.5)), ((2.6, -0.5), (2.6, 0.3)), ((2.6, 0.3), (2, 0.3)),
                      ((2, 0.3), (2, -0.5))]
        self.x = self.y = self.th = 0.0
        self.cmd = (0.0, 0.0, 0.0)
        self.w, self.h = int(6.4 / self.RES), int(4.4 / self.RES)
        self.origin = [-1.2, -2.2, 0.0]
        self.cells = [-1] * (self.w * self.h)
        self.mcu_seen = time.time()
        threading.Thread(target=self._run, daemon=True).start()

    def publish_twist(self, vx, vy, wz):
        self.cmd = (vx, vy if self.cfg["holonomic"] else 0.0, wz)

    def mcu(self, cmd):
        if cmd == "<S>":
            self.estop = True
        elif cmd == "<E1>":
            self.estop = False
        self.notify("mcu", "[OK,MOCK]" if cmd not in ("<S>", "<E1>") else
                    ("[OK,ESTOP_LATCHED]" if cmd == "<S>" else "[OK,ENABLED]"))

    def rezero(self):
        self.x = self.y = self.th = 0.0

    def send_goal(self, x, y, yaw):
        self.goal = [round(x, 3), round(y, 3), round(yaw, 3)]
        self.nav_status = "driving"
        self.notify("info", f"goal ({x:.2f}, {y:.2f}) heading {math.degrees(yaw):.0f} deg")

    def cancel_goals(self):
        if self.goal:
            self.nav_status = "CANCELED"
        self.goal = None

    def save_map_to(self, base):
        with self.lock:
            g = dict(width=self.w, height=self.h, resolution=self.RES, origin=self.origin,
                     data=list(self.cells))
        write_pgm(g, base)
        return True

    def _ray(self, ox, oy, a, rmax=6.0):
        dx, dy = math.cos(a), math.sin(a)
        best = rmax
        for (x1, y1), (x2, y2) in self.walls:
            ex, ey = x2 - x1, y2 - y1
            den = dx * ey - dy * ex
```

```
            if abs(den) < 1e-9:
                continue
            t = ((x1 - ox) * ey - (y1 - oy) * ex) / den
            u = ((x1 - ox) * dy - (y1 - oy) * dx) / den
            if 0 <= u <= 1 and 0 < t < best:
                best = t
        return best

    def _mark(self, x, y, v):
        i = int((x - self.origin[0]) / self.RES)
        j = int((y - self.origin[1]) / self.RES)
        if 0 <= i < self.w and 0 <= j < self.h:
            k = j * self.w + i
            if self.cells[k] != 100 or v == 100:
                self.cells[k] = v

    def _run(self):
        dt, k = 0.05, 0
        while True:
            time.sleep(dt)
            k += 1
            vx, vy, wz = (0.0, 0.0, 0.0) if self.estop else self.cmd
            if self.goal and self.procs.running("nav") and not self.estop:
                gx, gy, gyaw = self.goal
                d = math.hypot(gx - self.x, gy - self.y)
                if d > 0.05:
                    head = math.atan2(gy - self.y, gx - self.x)
                    err = math.atan2(math.sin(head - self.th), math.cos(head - self.th))
                    wz = max(-0.6, min(0.6, 2.0 * err))
                    vx = 0.15 if abs(err) < 0.3 else 0.0
                else:
                    err = math.atan2(math.sin(gyaw - self.th), math.cos(gyaw - self.th))
                    wz = max(-0.6, min(0.6, 2.0 * err))
                    if abs(err) < 0.05:
                        self.goal = None
                        self.nav_status = "SUCCEEDED"
                        self.notify("info", "goal SUCCEEDED")
            c, s = math.cos(self.th), math.sin(self.th)
            self.x += (vx * c - vy * s) * dt
            self.y += (vx * s + vy * c) * dt
            self.th = math.atan2(math.sin(self.th + wz * dt), math.cos(self.th + wz * dt))
            lx, ly = self.x + 0.10 * c, self.y + 0.10 * s
            pts = []
            for b in range(180):
                a = self.th - math.pi + b * 2 * math.pi / 180
                r = self._ray(lx, ly, a)
                if r < 6.0:
                    pts.append((lx + r * math.cos(a), ly + r * math.sin(a)))
                if self.procs.running("mapping") and k % 4 == 0:
                    for q in range(int(r / self.RES)):
                        self._mark(lx + q * self.RES * math.cos(a), ly + q * self.RES * math.sin(a), 0)
                    if r < 6.0:
                        self._mark(lx + r * math.cos(a), ly + r * math.sin(a), 100)
            n = len(self.cfg["wheel_names"])
            wl = (vx - wz * 0.15) / 0.05
            wr = (vx + wz * 0.15) / 0.05
            tg = [wl if i % 2 == 0 else wr for i in range(n)]
            with self.lock:
                frame = "map" if self.procs.running("mapping") else "odom"
                self.pose = [round(self.x, 4), round(self.y, 4), round(self.th, 4), frame]
                self.odom = list(self.pose[:3])
                self.scan = pts
                self.mcu_seen = time.time()
                self.wheels = {"target": [round(v, 3) for v in tg], "actual": [round(v * 0.98, 3) for v in tg],
                               "pwm": [int(v * 16) for v in tg]}
                if self.procs.running("mapping") and k % 20 == 0:
                    self.grid = dict(width=self.w, height=self.h, resolution=self.RES,
                                     origin=self.origin, data=list(self.cells))
                    self.map_version += 1
            if self.rec:
                vals = []
                for i in range(n):
                    vals += [tg[i], tg[i] * 0.98, tg[i] * 16]
                self.rec.add(time.time(), [k * 50.0] + vals)


# ═══════════════════════════════════════════════════════════════
#  Web app
# ═══════════════════════════════════════════════════════════════

def make_app(io):
    from fastapi import FastAPI, WebSocket, WebSocketDisconnect
```

```
    from fastapi.responses import FileResponse
    from fastapi.staticfiles import StaticFiles

    app = FastAPI()
    app.mount("/static", StaticFiles(directory=STATIC), name="static")

    @app.get("/")
    def index():
        return FileResponse(os.path.join(STATIC, "index.html"),
                            headers={"Cache-Control": "no-store"})

    @app.get("/health")
    def health():
        s = io.snapshot()
        return {"ok": True, "mcu_ok": s["mcu_ok"], "mapping": s["mapping"], "nav": s["nav"]}

    @app.on_event("shutdown")
    def _shutdown():
        io.shutdown()

    @app.websocket("/ws")
    async def ws(sock: WebSocket):
        await sock.accept()
        c = io.cfg
        await sock.send_text(json.dumps({"type": "hello", "config": {
            "name": c["name"], "wheel_names": c["wheel_names"], "holonomic": c["holonomic"],
            "max_linear": c["max_linear"], "max_angular": c["max_angular"],
            "footprint": list(zip(c["footprint_x"], c["footprint_y"])),
            "laser": [c["laser_x"], c["laser_y"]], "mock": isinstance(io, MockIO)}}))
        seen_map, seen_notice = -1, io.notice_seq - 10

        async def sender():
            nonlocal seen_map, seen_notice
            k = 0
            while True:
                io.deadman()
                await sock.send_text(json.dumps(io.snapshot()))
                if k % 2 == 0:
                    await sock.send_text(json.dumps(io.scan_msg()))
                if k % 10 == 0:
                    m, v = io.map_msg()
                    if m and v != seen_map:
                        seen_map = v
                        await sock.send_text(json.dumps(m))
                with io.lock:
                    new = [n for n in io.notices if n[0] > seen_notice]
                for seq, level, text in new:
                    seen_notice = seq
                    await sock.send_text(json.dumps({"type": "log", "level": level, "text": text}))
                k += 1
                await asyncio.sleep(0.1)

        task = asyncio.create_task(sender())
        try:
            while True:
                msg = json.loads(await sock.receive_text())
                await asyncio.get_running_loop().run_in_executor(None, io.handle, msg)
        except (WebSocketDisconnect, RuntimeError, json.JSONDecodeError):
            pass
        finally:
            task.cancel()
            io.publish_twist(0.0, 0.0, 0.0)      # page gone: stop

    return app


def config_from_args(args):
    return {"name": args.name, "wheel_names": args.wheels.split(","), "holonomic": args.holonomic,
            "max_linear": 0.2, "max_angular": 0.8, "footprint_x": [0.2, -0.2, -0.2, 0.2],
            "footprint_y": [0.17, 0.17, -0.17, -0.17], "laser_x": 0.1, "laser_y": 0.0,
            "log_dir": args.log_dir, "map_dir": args.log_dir}


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--mock", action="store_true", help="no ROS: simulated robot")
    ap.add_argument("--port", type=int, default=8080)
    ap.add_argument("--name", default="mock")
    ap.add_argument("--wheels", default="left,right")
    ap.add_argument("--holonomic", action="store_true")
    ap.add_argument("--log-dir", default="~/robot_logs")
    args, ros_args = ap.parse_known_args()
```

```
    import uvicorn

    if args.mock:
        io = MockIO(config_from_args(args))
        uvicorn.run(make_app(io), host="0.0.0.0", port=args.port, log_level="warning")
        return

    import rclpy
    from rclpy.node import Node
    rclpy.init(args=ros_args)
    node = Node("dashboard")
    for k, v in (("port", 8080), ("log_dir", "~/robot_logs"), ("map_dir", "~/maps"),
                 ("wheel_names", ["left", "right"]), ("holonomic", False), ("max_linear", 0.2),
                 ("max_angular", 0.8), ("footprint_x", [0.2, -0.2, -0.2, 0.2]),
                 ("footprint_y", [0.17, 0.17, -0.17, -0.17]), ("laser_x", 0.0), ("laser_y", 0.0)):
        node.declare_parameter(k, v)
    g = lambda k: node.get_parameter(k).value  # noqa: E731
    cfg = {k: g(k) for k in ("log_dir", "map_dir", "holonomic", "max_linear", "max_angular",
                             "laser_x", "laser_y")}
    cfg.update(name=os.uname().nodename, wheel_names=list(g("wheel_names")),
               footprint_x=list(g("footprint_x")), footprint_y=list(g("footprint_y")))
    io = RosIO(node, cfg)
    threading.Thread(target=rclpy.spin, args=(node,), daemon=True).start()
    node.get_logger().info(f"dashboard on http://0.0.0.0:{g('port')}")
    try:
        uvicorn.run(make_app(io), host="0.0.0.0", port=int(g("port")), log_level="warning")
    finally:
        io.shutdown()
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()


if __name__ == "__main__":
    main()
```

`ros2/robot_base/robot_base/dashboard/static/index.html` (72 lines)

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1, user-scalable=no">
  <title>Robot Dashboard</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
  <header>
    <div class="title">
      <span id="conn" class="dot" title="connection"></span>
      <span id="name">robot</span>
      <span id="mock" class="chip hidden">MOCK</span>
    </div>
    <div class="chips">
      <span id="chip-mcu" class="chip">MCU</span>
      <span id="chip-map" class="chip">MAP</span>
      <span id="chip-nav" class="chip">NAV</span>
      <span id="chip-rec" class="chip">REC</span>
    </div>
    <button id="estop" class="estop">E-STOP</button>
  </header>

  <main>
    <section id="map-wrap">
      <canvas id="map"></canvas>
      <div class="map-tools">
        <button id="zoom-in" title="zoom in">+</button>
        <button id="zoom-out" title="zoom out">&minus;</button>
        <button id="follow" class="on" title="follow the robot">&#9678;</button>
        <button id="goal-mode" title="tap the map to set a goal, drag for heading">GOAL</button>
      </div>
      <div id="pose" class="pose">pose: waiting</div>
    </section>

    <section id="drive">
      <div id="stick" class="stick"><div id="knob" class="knob"></div></div>
      <div id="turn-wrap" class="turn-wrap">
        <div class="turn-label">turn</div>
        <div id="turn" class="turn"><div id="turn-knob" class="turn-knob"></div></div>
```

```
      </div>
    </section>

    <section id="buttons">
      <button id="map-toggle">MAP start</button>
      <button id="map-save">SAVE map</button>
      <button id="nav-toggle">NAV start</button>
      <button id="goal-cancel">CANCEL goal</button>
      <button id="rezero">RE-ZERO</button>
      <button id="rec-toggle">REC start</button>
    </section>

    <section id="wheels">
      <table>
        <thead><tr><th>wheel</th><th>target</th><th>actual</th><th>pwm</th></tr></thead>
        <tbody id="wheel-rows"></tbody>
      </table>
    </section>

    <section id="console">
      <form id="mcu-form" autocomplete="off">
        <input id="mcu-cmd" placeholder="MCU command, e.g. <?> or <G,20,100,0>" spellcheck="false">
        <button type="submit">send</button>
      </form>
      <div id="log" class="log"></div>
    </section>
  </main>

  <script src="/static/app.js"></script>
</body>
</html>
```

`ros2/robot_base/robot_base/dashboard/static/app.js` (370 lines)

```
// Robot dashboard client. Plain JavaScript, no build step, no libraries.
// Talks JSON over one WebSocket (/ws). See dashboard/server.py for the messages.
//
// Map view: screen right = map +x, screen up = map +y (REP-103 seen from above,
// the same picture RViz and Foxglove draw). Yaw is counter-clockwise from +x.
"use strict";

const $ = (id) => document.getElementById(id);
const cfg = { name: "robot", wheel_names: [], holonomic: false, footprint: [], laser: [0, 0], mock: false };
const st = { pose: null, odom: null, wheels: null, estop: false, mcu_ok: false, mapping: false,
             nav: false, nav_status: "", goal: null, rec: null };
let scanPts = [];
let mapImg = null;          // offscreen canvas
let mapInfo = null;         // {width, height, resolution, origin}
let ws = null;
let connected = false;

// ── WebSocket ──────────────────────────────────────────────────────
function send(obj) {
  if (ws && ws.readyState === WebSocket.OPEN) {
    ws.send(JSON.stringify(obj));
    return true;
  }
  return false;
}

function connect() {
  const proto = location.protocol === "https:" ? "wss" : "ws";
  ws = new WebSocket(proto + "://" + location.host + "/ws");
  ws.onopen = () => { connected = true; $("conn").classList.add("on"); log("info", "connected"); };
  ws.onclose = () => {
    connected = false;
    $("conn").classList.remove("on");
    setTimeout(connect, 1000);
  };
  ws.onerror = () => { try { ws.close(); } catch (e) { /* closing anyway */ } };
  ws.onmessage = (ev) => {
    let m;
    try { m = JSON.parse(ev.data); } catch (e) { return; }
    if (m.type === "hello") applyConfig(m.config);
    else if (m.type === "state") { Object.assign(st, m); renderState(); }
    else if (m.type === "scan") scanPts = m.points;
    else if (m.type === "map") buildMap(m);
    else if (m.type === "log") log(m.level, m.text);
  };
```

```
}

function applyConfig(c) {
  Object.assign(cfg, c);
  $("name").textContent = cfg.name;
  $("mock").classList.toggle("hidden", !cfg.mock);
  $("turn-wrap").classList.toggle("hidden", !cfg.holonomic);
  const rows = $("wheel-rows");
  rows.innerHTML = "";
  cfg.wheel_names.forEach((n, i) => {
    const tr = document.createElement("tr");
    tr.innerHTML = "<td></td><td id=\"wt" + i + "\">-</td><td id=\"wa" + i + "\">-</td><td id=\"wp" + i + "\">-</td>";
    tr.firstChild.textContent = n;
    rows.appendChild(tr);
  });
}

// ── Log panel ──────────────────────────────────────────────────
function log(level, text) {
  const box = $("log");
  const line = document.createElement("div");
  line.className = level;
  line.textContent = new Date().toLocaleTimeString() + "  " + text;
  box.appendChild(line);
  while (box.childNodes.length > 200) box.removeChild(box.firstChild);
  box.scrollTop = box.scrollHeight;
}

// ── State → page ───────────────────────────────────────────────
function chip(id, cls, label) {
  const el = $(id);
  el.className = "chip " + cls;
  if (label) el.textContent = label;
}

function renderState() {
  chip("chip-mcu", st.estop ? "bad" : (st.mcu_ok ? "on" : "warn"), st.estop ? "ESTOP" : "MCU");
  chip("chip-map", st.mapping ? "on" : "", "MAP");
  chip("chip-nav", st.nav ? "on" : "", st.nav_status ? "NAV " + st.nav_status : "NAV");
  chip("chip-rec", st.rec !== null ? "bad" : "", st.rec !== null ? "REC " + st.rec : "REC");
  const es = $("estop");
  es.textContent = st.estop ? "CLEAR" : "E-STOP";
  es.classList.toggle("latched", st.estop);
  $("map-toggle").textContent = st.mapping ? "MAP stop" : "MAP start";
  $("map-toggle").classList.toggle("on", st.mapping);
  $("nav-toggle").textContent = st.nav ? "NAV stop" : "NAV start";
  $("nav-toggle").classList.toggle("on", st.nav);
  $("rec-toggle").textContent = st.rec !== null ? "REC stop" : "REC start";
  $("rec-toggle").classList.toggle("on", st.rec !== null);
  if (st.wheels) {
    cfg.wheel_names.forEach((n, i) => {
      $("wt" + i).textContent = st.wheels.target[i].toFixed(2);
      $("wa" + i).textContent = st.wheels.actual[i].toFixed(2);
      $("wp" + i).textContent = st.wheels.pwm[i];
    });
  }
  const p = st.pose;
  const o = st.odom;
  let txt = p ? (p[3] + ": x " + p[0].toFixed(3) + "  y " + p[1].toFixed(3) + "  yaw " +
                 (p[2] * 180 / Math.PI).toFixed(1) + "°") : "pose: no TF yet";
  if (o && p && p[3] !== "odom") txt += "\nodom: x " + o[0].toFixed(3) + "  y " + o[1].toFixed(3);
  $("pose").textContent = txt;
}

// ── Map view ───────────────────────────────────────────────────
const canvas = $("map");
const ctx = canvas.getContext("2d");
const view = { cx: 0, cy: 0, scale: 80, follow: true };   // scale in px per metre
let goalMode = false;
let goalDraft = null;      // {x, y, yaw}

function cssVar(name) { return getComputedStyle(document.documentElement).getPropertyValue(name).trim(); }

function resize() {
  const r = canvas.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  canvas.width = Math.max(1, Math.round(r.width * dpr));
  canvas.height = Math.max(1, Math.round(r.height * dpr));
}
window.addEventListener("resize", resize);

function w2s(x, y) {
```

```
  const dpr = window.devicePixelRatio || 1;
  return [canvas.width / 2 + (x - view.cx) * view.scale * dpr,
          canvas.height / 2 - (y - view.cy) * view.scale * dpr];
}
function s2w(px, py) {
  const dpr = window.devicePixelRatio || 1;
  return [view.cx + (px * dpr - canvas.width / 2) / (view.scale * dpr),
          view.cy - (py * dpr - canvas.height / 2) / (view.scale * dpr)];
}

function buildMap(m) {
  const raw = atob(m.data);
  const off = document.createElement("canvas");
  off.width = m.width;
  off.height = m.height;
  const c = off.getContext("2d");
  const img = c.createImageData(m.width, m.height);
  const hex = (h) => [parseInt(h.slice(1, 3), 16), parseInt(h.slice(3, 5), 16), parseInt(h.slice(5, 7), 16)];
  const free = hex(cssVar("--map-free")), unk = hex(cssVar("--map-unknown")), occ = hex(cssVar("--map-occ"));
  for (let j = 0; j < m.height; j++) {
    for (let i = 0; i < m.width; i++) {
      const v = raw.charCodeAt(j * m.width + i);
      // Row j of the grid is world y = origin.y + j*res. Store it so that the
      // canvas transform below (negative y scale) puts row 0 at the bottom.
      const k = 4 * (j * m.width + i);
      let col = unk;
      if (v === 255) col = v >= 65 ? occ : (v <= 25 ? free : unk);
      img.data[k] = col[0]; img.data[k + 1] = col[1]; img.data[k + 2] = col[2]; img.data[k + 3] = 255;
    }
  }
  c.putImageData(img, 0, 0);
  mapImg = off;
  mapInfo = m;
}

function drawArrow(x, y, yaw, len, color, width) {
  const [sx, sy] = w2s(x, y);
  const [ex, ey] = w2s(x + len * Math.cos(yaw), y + len * Math.sin(yaw));
  ctx.strokeStyle = color; ctx.fillStyle = color; ctx.lineWidth = width;
  ctx.beginPath(); ctx.moveTo(sx, sy); ctx.lineTo(ex, ey); ctx.stroke();
  const a = Math.atan2(ey - sy, ex - sx);
  const h = 10 * (window.devicePixelRatio || 1);
  ctx.beginPath();
  ctx.moveTo(ex, ey);
  ctx.lineTo(ex - h * Math.cos(a - 0.4), ey - h * Math.sin(a - 0.4));
  ctx.lineTo(ex - h * Math.cos(a + 0.4), ey - h * Math.sin(a + 0.4));
  ctx.closePath(); ctx.fill();
}

function draw() {
  const dpr = window.devicePixelRatio || 1;
  if (view.follow && st.pose) { view.cx = st.pose[0]; view.cy = st.pose[1]; }
  ctx.setTransform(1, 0, 0, 1, 0, 0);
  ctx.fillStyle = cssVar("--map-unknown");
  ctx.fillRect(0, 0, canvas.width, canvas.height);

  if (mapImg && mapInfo) {
    const [ox, oy] = w2s(mapInfo.origin[0], mapInfo.origin[1]);
    const s = mapInfo.resolution * view.scale * dpr;
    ctx.save();
    ctx.imageSmoothingEnabled = false;
    ctx.setTransform(s, 0, 0, -s, ox, oy);
    ctx.drawImage(mapImg, 0, 0);
    ctx.restore();
  }

  // 1 m grid around the view centre, faint
  ctx.strokeStyle = cssVar("--line"); ctx.lineWidth = 1;
  const span = canvas.width / (view.scale * dpr);
  if (span < 40) {
    for (let g = Math.floor(view.cx - span); g <= view.cx + span; g++) {
      const [a] = w2s(g, 0); ctx.beginPath(); ctx.moveTo(a, 0); ctx.lineTo(a, canvas.height); ctx.stroke();
    }
    for (let g = Math.floor(view.cy - span); g <= view.cy + span; g++) {
      const [, b] = w2s(0, g); ctx.beginPath(); ctx.moveTo(0, b); ctx.lineTo(canvas.width, b); ctx.stroke();
    }
  }
  // origin axes: x red, y green
  drawArrow(0, 0, 0, 0.3, "#d23b2f", 2 * dpr);
  drawArrow(0, 0, Math.PI / 2, 0.3, "#1f9d55", 2 * dpr);

  // scan
```

```
  ctx.fillStyle = cssVar("--scan");
  const r = Math.max(1.5, 1.5 * dpr);
  for (const p of scanPts) {
    const [sx, sy] = w2s(p[0], p[1]);
    ctx.fillRect(sx - r, sy - r, 2 * r, 2 * r);
  }

  // goal
  if (st.goal) drawArrow(st.goal[0], st.goal[1], st.goal[2], 0.4, cssVar("--goal"), 3 * dpr);
  if (goalDraft) drawArrow(goalDraft.x, goalDraft.y, goalDraft.yaw, 0.4, cssVar("--goal"), 2 * dpr);

  // robot footprint + heading
  if (st.pose) {
    const [x, y, yaw] = st.pose;
    const c = Math.cos(yaw), s = Math.sin(yaw);
    const fp = cfg.footprint.length ? cfg.footprint : [[0.2, 0.15], [-0.2, 0.15], [-0.2, -0.15], [0.2, -0.15]];
    ctx.beginPath();
    fp.forEach((q, i) => {
      const [sx, sy] = w2s(x + c * q[0] - s * q[1], y + s * q[0] + c * q[1]);
      if (i === 0) ctx.moveTo(sx, sy); else ctx.lineTo(sx, sy);
    });
    ctx.closePath();
    ctx.fillStyle = cssVar("--robot") + "55";
    ctx.fill();
    ctx.strokeStyle = cssVar("--robot"); ctx.lineWidth = 2 * dpr; ctx.stroke();
    drawArrow(x, y, yaw, 0.35, cssVar("--robot"), 2 * dpr);
  }
  requestAnimationFrame(draw);
}

// pan, pinch, wheel zoom, goal taps
const pointers = new Map();
let pinch0 = null;
canvas.addEventListener("pointerdown", (e) => {
  canvas.setPointerCapture(e.pointerId);
  pointers.set(e.pointerId, [e.offsetX, e.offsetY]);
  if (goalMode && pointers.size === 1) {
    const [x, y] = s2w(e.offsetX, e.offsetY);
    const yaw = st.pose ? Math.atan2(y - st.pose[1], x - st.pose[0]) : 0;
    goalDraft = { x: x, y: y, yaw: yaw, px: e.offsetX, py: e.offsetY, dragged: false };
  }
  if (pointers.size === 2) {
    const [a, b] = [...pointers.values()];
    pinch0 = { d: Math.hypot(a[0] - b[0], a[1] - b[1]), scale: view.scale };
  }
});
canvas.addEventListener("pointermove", (e) => {
  if (!pointers.has(e.pointerId)) return;
  const prev = pointers.get(e.pointerId);
  pointers.set(e.pointerId, [e.offsetX, e.offsetY]);
  if (pointers.size === 2 && pinch0) {
    const [a, b] = [...pointers.values()];
    const d = Math.hypot(a[0] - b[0], a[1] - b[1]);
    view.scale = Math.min(400, Math.max(10, pinch0.scale * d / Math.max(1, pinch0.d)));
    return;
  }
  if (goalDraft) {
    const dx = e.offsetX - goalDraft.px, dy = e.offsetY - goalDraft.py;
    if (Math.hypot(dx, dy) > 12) {
      goalDraft.dragged = true;
      goalDraft.yaw = Math.atan2(-dy, dx);     // screen y is down, map y is up
    }
    return;
  }
  view.follow = false;
  $("follow").classList.remove("on");
  view.cx -= (e.offsetX - prev[0]) / view.scale;
  view.cy += (e.offsetY - prev[1]) / view.scale;
});
function endPointer(e) {
  pointers.delete(e.pointerId);
  if (pointers.size < 2) pinch0 = null;
  if (goalDraft && pointers.size === 0) {
    const g = goalDraft;
    goalDraft = null;
    setGoalMode(false);
    if (send({ type: "goal", x: g.x, y: g.y, yaw: g.yaw })) {
      log("info", "goal sent (" + g.x.toFixed(2) + ", " + g.y.toFixed(2) + ") " +
          (g.yaw * 180 / Math.PI).toFixed(0) + "°" + (g.dragged ? "" : " (heading: facing the goal)"));
    } else {
      log("error", "goal NOT sent: no connection");
    }
```

```
  }
}
canvas.addEventListener("pointerup", endPointer);
canvas.addEventListener("pointercancel", endPointer);
canvas.addEventListener("wheel", (e) => {
  e.preventDefault();
  view.scale = Math.min(400, Math.max(10, view.scale * (e.deltaY < 0 ? 1.15 : 1 / 1.15)));
}, { passive: false });
$("zoom-in").onclick = () => { view.scale = Math.min(400, view.scale * 1.3); };
$("zoom-out").onclick = () => { view.scale = Math.max(10, view.scale / 1.3); };
$("follow").onclick = () => { view.follow = !view.follow; $("follow").classList.toggle("on", view.follow); };
function setGoalMode(on) { goalMode = on; $("goal-mode").classList.toggle("on", on); }
$("goal-mode").onclick = () => setGoalMode(!goalMode);

// ── Joystick and turn slider ──────────────────────────────────
const drive = { x: 0, y: 0, r: 0, active: false };
function attachPad(el, knob, onMove, horizontalOnly) {
  let id = null;
  const move = (e) => {
    const rect = el.getBoundingClientRect();
    const cx = rect.left + rect.width / 2, cy = rect.top + rect.height / 2;
    const rad = horizontalOnly ? rect.width / 2 - 23 : rect.width / 2 - 32;
    let dx = e.clientX - cx, dy = horizontalOnly ? 0 : e.clientY - cy;
    const d = Math.hypot(dx, dy);
    if (d > rad) { dx *= rad / d; dy *= rad / d; }
    knob.style.transform = "translate(" + dx + "px," + dy + "px)";
    onMove(dx / rad, -dy / rad);
  };
  const reset = () => { id = null; knob.style.transform = ""; onMove(0, 0); };
  el.addEventListener("pointerdown", (e) => { id = e.pointerId; el.setPointerCapture(id); move(e); });
  el.addEventListener("pointermove", (e) => { if (e.pointerId === id) move(e); });
  el.addEventListener("pointerup", (e) => { if (e.pointerId === id) reset(); });
  el.addEventListener("pointercancel", (e) => { if (e.pointerId === id) reset(); });
}
const dz = (v) => (Math.abs(v) < 0.08 ? 0 : v);
attachPad($("stick"), $("knob"), (x, y) => { drive.x = dz(x); drive.y = dz(y); }, false);
attachPad($("turn"), $("turn-knob"), (x) => { drive.r = dz(-x); }, true);   // slide left = turn left

let lastSentZero = true;
setInterval(() => {
  const moving = drive.x !== 0 || drive.y !== 0 || drive.r !== 0;
  if (moving) {
    send({ type: "drive", x: drive.x, y: drive.y, r: drive.r });
    lastSentZero = false;
  } else if (!lastSentZero) {
    send({ type: "drive", x: 0, y: 0, r: 0 });     // one explicit zero on release
    lastSentZero = true;
  }
}, 66);

// ── Buttons ──────────────────────────────────────────────────
$("estop").onclick = () => send({ type: st.estop ? "estop_clear" : "estop" });
$("map-toggle").onclick = () => {
  if (st.mapping && !confirm("Stop mapping? The map is saved first.")) return;
  send({ type: st.mapping ? "map_stop" : "map_start" });
};
$("map-save").onclick = () => send({ type: "map_save" });
$("nav-toggle").onclick = () => send({ type: st.nav ? "nav_stop" : "nav_start" });
$("goal-cancel").onclick = () => send({ type: "goal_cancel" });
$("rezero").onclick = () => {
  if (confirm("Make the robot's current spot odometry (0, 0, 0)?")) send({ type: "rezero" });
};
$("rec-toggle").onclick = () => send({ type: st.rec !== null ? "rec_stop" : "rec_start" });
$("mcu-form").onsubmit = (e) => {
  e.preventDefault();
  const v = $("mcu-cmd").value.trim();
  if (v) { send({ type: "mcu", cmd: v }); $("mcu-cmd").value = ""; }
};
// Space bar = E-STOP when typing is not in progress (laptop use).
document.addEventListener("keydown", (e) => {
  if (e.code === "Space" && document.activeElement !== $("mcu-cmd")) {
    e.preventDefault();
    send({ type: "estop" });
  }
});

resize();
connect();
requestAnimationFrame(draw);
```

`ros2/robot_base/robot_base/dashboard/static/style.css` (138 lines)

```
:root {
  --bg: #f4f5f7;
  --panel: #ffffff;
  --ink: #1d2330;
  --muted: #6b7385;
  --line: #d9dde5;
  --accent: #2f6fdf;
  --ok: #1f9d55;
  --warn: #c98a00;
  --bad: #d23b2f;
  --map-free: #ffffff;
  --map-unknown: #c9ccd3;
  --map-occ: #1d2330;
  --scan: #e0453a;
  --robot: #2f6fdf;
  --goal: #1f9d55;
}
@media (prefers-color-scheme: dark) {
  :root {
    --bg: #11141a;
    --panel: #1a1e26;
    --ink: #e7e9ee;
    --muted: #9aa1b1;
    --line: #2c323e;
    --accent: #5b8ff0;
    --map-free: #2a2f3a;
    --map-unknown: #161a21;
    --map-occ: #e7e9ee;
  }
}
* { box-sizing: border-box; }
html, body { margin: 0; height: 100%; }
body {
  background: var(--bg);
  color: var(--ink);
  font: 14px/1.35 system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
  -webkit-user-select: none;
  user-select: none;
  overscroll-behavior: none;
}
header {
  position: sticky; top: 0; z-index: 5;
  display: flex; align-items: center; flex-wrap: wrap; gap: 6px 8px;
  padding: 8px 12px;
  background: var(--panel);
  border-bottom: 1px solid var(--line);
}
.title { display: flex; align-items: center; gap: 6px; font-weight: 600; min-width: 0; flex: 1; }
#name { overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }
.chips { display: flex; gap: 4px; flex-wrap: wrap; order: 3; width: 100%; }
@media (min-width: 900px) { .chips { order: 0; width: auto; flex: 1; } .title { flex: 0 1 auto; } }
.chip {
  font-size: 11px; font-weight: 600; letter-spacing: 0.03em;
  padding: 2px 7px; border-radius: 999px;
  border: 1px solid var(--line); color: var(--muted);
}
.chip.on { background: var(--ok); border-color: var(--ok); color: #fff; }
.chip.warn { background: var(--warn); border-color: var(--warn); color: #fff; }
.chip.bad { background: var(--bad); border-color: var(--bad); color: #fff; }
.hidden { display: none !important; }
.dot { width: 10px; height: 10px; border-radius: 50%; background: var(--bad); flex: none; }
.dot.on { background: var(--ok); }
button {
  font: inherit; font-weight: 600;
  padding: 10px 12px; border-radius: 10px;
  border: 1px solid var(--line);
  background: var(--panel); color: var(--ink);
  touch-action: manipulation;
}
button:active { transform: translateY(1px); }
button.on { background: var(--accent); border-color: var(--accent); color: #fff; }
.estop {
  background: var(--bad); color: #fff; border: none;
  padding: 12px 16px; font-size: 16px; letter-spacing: 0.05em;
  box-shadow: 0 2px 6px rgba(0, 0, 0, 0.25);
}
.estop.latched { background: var(--warn); }
main {
```

```
  display: grid; gap: 10px; padding: 10px 12px 24px;
  grid-template-columns: 1fr;
}
section { background: var(--panel); border: 1px solid var(--line); border-radius: 12px; }
#map-wrap { position: relative; height: 52vh; min-height: 280px; overflow: hidden; touch-action: none; }
#map { width: 100%; height: 100%; display: block; }
.map-tools { position: absolute; top: 8px; right: 8px; display: flex; flex-direction: column; gap: 6px; }
.map-tools button { width: 44px; height: 40px; padding: 0; opacity: 0.92; }
.pose {
  position: absolute; left: 8px; bottom: 8px;
  font: 12px/1.3 ui-monospace, Menlo, Consolas, monospace; white-space: pre;
  background: var(--panel); border: 1px solid var(--line);
  padding: 3px 7px; border-radius: 8px; opacity: 0.92;
}
#drive { display: flex; align-items: center; justify-content: space-around; gap: 16px; padding: 14px; touch-action:
none; }
.stick {
  position: relative; width: 170px; height: 170px; border-radius: 50%;
  background: radial-gradient(circle, var(--bg) 0%, var(--line) 100%);
  border: 1px solid var(--line); touch-action: none; flex: none;
}
.knob {
  position: absolute; left: 50%; top: 50%; width: 64px; height: 64px;
  margin: -32px 0 0 -32px; border-radius: 50%;
  background: var(--accent); box-shadow: 0 2px 8px rgba(0, 0, 0, 0.3);
}
.turn-wrap { display: flex; flex-direction: column; align-items: center; gap: 6px; }
.turn-label { color: var(--muted); font-size: 12px; }
.turn {
  position: relative; width: 170px; height: 54px; border-radius: 27px;
  background: var(--bg); border: 1px solid var(--line); touch-action: none;
}
.turn-knob {
  position: absolute; left: 50%; top: 50%; width: 46px; height: 46px;
  margin: -23px 0 0 -23px; border-radius: 50%; background: var(--accent);
}
#buttons { display: grid; grid-template-columns: repeat(3, 1fr); gap: 8px; padding: 10px; }
#wheels { padding: 6px 10px; }
table { width: 100%; border-collapse: collapse; font-variant-numeric: tabular-nums; }
th { text-align: left; color: var(--muted); font-weight: 500; font-size: 12px; padding: 4px; }
td { padding: 4px; border-top: 1px solid var(--line); }
#console { padding: 10px; }
#mcu-form { display: flex; gap: 8px; }
#mcu-cmd {
  flex: 1; min-width: 0; font: 14px ui-monospace, Menlo, Consolas, monospace;
  padding: 10px; border-radius: 10px; border: 1px solid var(--line);
  background: var(--bg); color: var(--ink); -webkit-user-select: text; user-select: text;
}
.log {
  margin-top: 8px; height: 150px; overflow-y: auto;
  font: 12px/1.4 ui-monospace, Menlo, Consolas, monospace; color: var(--muted);
  -webkit-user-select: text; user-select: text;
}
.log .warn { color: var(--warn); }
.log .error { color: var(--bad); }
.log .mcu { color: var(--ink); }
@media (min-width: 900px) {
  main { grid-template-columns: 1.6fr 1fr; }
  #map-wrap { grid-row: span 4; height: calc(100vh - 90px); }
  #drive, #buttons, #wheels, #console { grid-column: 2; }
}
```

Tools

`tools/encoder_logger.py` (751 lines)

```
#!/usr/bin/env python3
"""
encoder_logger.py - log encoder data from the microcontroller and run the
PID calibration tests. Runs on the Pi (or any PC with the board plugged in).
No ROS needed: stop the base stack first, the serial port can only have one
reader (sudo systemctl stop robot.service).

    >>> WHEELS IN THE AIR for plant, staircase, steps and sweep. <<<

The robot kit firmware (firmware/robot_mcu) speaks the protocol these tests
need. If your microcontroller runs your OWN firmware, use --test raw: it logs
```

```
whatever CSV your board prints, timestamps it on the Pi, and can turn tick
counts into rad/s for you.

TESTS
  plant      Open-loop PWM steps, PID bypassed. Measures each motor's time
             constant tau and gain K, and prints the Kff, Ki and Kp they
             imply. Run this FIRST.
  staircase  PWM raised in small steps from rest. Fits Kff (slope) and
             Kstat (breakaway) per motor by least squares. Run once in the
             air, once on the floor: the difference is the ground load.
             On the floor this DRIVES. --mode rotate spins in place instead.
  steps      Closed-loop velocity steps. Rise, overshoot, settling time,
             steady-state error, PWM saturation: the verdict on a gain set.
  sweep      steps, repeated for several gain sets. One CSV each.
  record     Log telemetry for --duration seconds while nothing else drives
             (use the ROS telemetry logger to log while driving under ROS).
  raw        Log any CSV the board prints. For non-kit firmware.

EXAMPLES
  ./encoder_logger.py --test plant
  ./encoder_logger.py --test plant --only left           # one wheel at a time
  ./encoder_logger.py --test staircase --max-pwm 90
  ./encoder_logger.py --test steps --setpoints 1,2,4 --kff 16.1,16.6 --kstat 7.7,7.7
  ./encoder_logger.py --test sweep --gains "20,120,0 30,120,0 20,200,0"
  ./encoder_logger.py --test raw --port /dev/ttyACM0 --baud 115200 \\
        --columns t_ms,left_ticks,right_ticks --ticks left_ticks:1320,right_ticks:1320 \\
        --time-col t_ms --duration 30

Every run writes <logdir>/<test>_<timestamp>.csv plus a .json beside it with
what was commanded and when, so tools/analyze_log.py can redo the maths later.

Ctrl-C at any point sends <S>, <L0>, <E0>: motors stop. So does a crash.
"""

import argparse
import csv
import json
import math
import os
import signal
import subprocess
import sys
import time
from datetime import datetime

HERE = os.path.dirname(os.path.abspath(__file__))
KIT = os.path.dirname(HERE)
sys.path.insert(0, os.path.join(KIT, "ros2", "robot_base"))

from robot_base import protocol as P  # noqa: E402
from robot_base.protocol import SerialLink  # noqa: E402

try:
    from robot_base.robot_model import load_config, ConfigError  # noqa: E402
except Exception:  # pragma: no cover  (PyYAML missing)
    load_config, ConfigError = None, Exception


# ═══════════════════════════════════════════════════════════════════
#  Small numerical helpers (pure Python, so this runs with pyserial only)
# ═══════════════════════════════════════════════════════════════════

def mean(xs):
    xs = list(xs)
    return sum(xs) / len(xs) if xs else float("nan")


def window(series, t0, t1):
    return [v for t, v in series if t0 <= t <= t1]


def estimate_tau(series, t_step, v_final):
    """Time from the step to 63.2 % of the final value (first-order lag),
    interpolated between the two samples either side of the crossing so the
    telemetry period does not round it up. Includes the dead time between
    the command and the first control tick that acts on it (<= 1 tick)."""
    if abs(v_final) < 1e-6:
        return None
    target = 0.632 * v_final
    prev = None
    for t, v in series:
        if t < t_step:
```

```
            continue
        if (v_final > 0 and v ≥ target) or (v_final < 0 and v ≤ target):
            if prev is not None and v ≠ prev[1]:
                t = prev[0] + (target - prev[1]) * (t - prev[0]) / (v - prev[1])
            return max(0.0, t - t_step)
        prev = (t, v)
    return None


def settling_time(series, t_step, target, band=0.05, hold=0.2):
    """First moment after the step from which the signal stays within
    +/-band*|target| for the rest of THIS step's window."""
    if abs(target) < 1e-6:
        return None
    thr = abs(target) * band
    pts = [(t, v) for t, v in series if t ≥ t_step]
    for i, (t, _) in enumerate(pts):
        if all(abs(v - target) ≤ thr for _, v in pts[i:]):
            return (t - t_step) if pts[-1][0] - t ≥ hold else None
    return None


def centered_rate(series_, half=1):
    """d/dt by a centred difference: (p[k+h] - p[k-h]) / (t[k+h] - t[k-h]).
    Symmetric, so it adds no lag. Used on the wheel ANGLE, which the
    firmware reports unfiltered, to see the motor without the firmware's
    velocity filter in the way."""
    out = []
    for k in range(half, len(series_) - half):
        (t0, p0), (t1, p1) = series_[k - half], series_[k + half]
        if t1 > t0:
            out.append((series_[k][0], (p1 - p0) / (t1 - t0)))
    return out


def ols(xs, ys):
    n = len(xs)
    if n < 2:
        return None
    sx, sy = sum(xs), sum(ys)
    sxx, sxy = sum(x * x for x in xs), sum(x * y for x, y in zip(xs, ys))
    d = n * sxx - sx * sx
    if abs(d) < 1e-12:
        return None
    m = (n * sxy - sx * sy) / d
    return m, (sy - m * sx) / n


# ═══════════════════════════════════════════════════════════════
#  Setup: who is on the other end of the cable
# ═══════════════════════════════════════════════════════════════

class Robot:
    """What the tests need to know about the robot, from the firmware's own
    <I> reply first and robot_config.yaml second."""

    def __init__(self, link, cfg):
        self.link = link
        ident = P.read_identity(link)
        cfg_names = [w["name"] for w in cfg["wheels"]] if cfg else None
        if ident["names"]:
            self.names = ident["names"]
            if cfg_names and cfg_names ≠ self.names:
                print(f"  ! firmware says motors {self.names}, robot_config.yaml says {cfg_names}.\n"
                      f"    Using the firmware's. Regenerate and reflash if that is wrong.")
        elif cfg_names:
            self.names = cfg_names
            print("  ! firmware did not report [MOTORS,...]; using robot_config.yaml")
        else:
            raise RuntimeError("cannot tell how many motors there are: the firmware does not "
                               "report [MOTORS,...] and no robot_config.yaml was loaded")
        self.n = len(self.names)
        link.n = self.n
        self.info = ident["info"]
        self.lines = ident["lines"]
        self.pwm_max = 255
        self.vmax = None
        for ln in ident["lines"]:
            if ln.startswith("[LIMITS"):
                for kv in ln.strip("[]").split(",")[1:]:
                    k, _, v = kv.partition("=")
                    if k == "PWM_MAX":
```

```
                        self.pwm_max = int(float(v))
                    elif k == "Vmax":
                        self.vmax = float(v)
        if self.vmax is None and cfg:
            self.vmax = float(cfg["mcu"]["max_wheel_speed"])
        if self.vmax is None:
            self.vmax = 5.0
        self.model = cfg["_model"] if cfg and cfg_names == self.names else None


def pattern(robot, mode, only):
    """Per-motor sign for open-loop tests. translate = every wheel forward.
    rotate = the sign each wheel gets for a pure spin, from the kinematics."""
    if mode == "rotate":
        if robot.model is None:
            raise RuntimeError("--mode rotate needs robot_config.yaml matching the firmware")
        signs = robot.model.sign_pattern(0.0, 0.0, 1.0)
    else:
        signs = [1] * robot.n
    if only:
        signs = [s if robot.names[i] in only else 0 for i, s in enumerate(signs)]
    return signs


# ═══════════════════════════════════════════════════════════════════
#  Output
# ═══════════════════════════════════════════════════════════════════

def out_path(args, tag):
    d = os.path.expanduser(args.logdir)
    os.makedirs(d, exist_ok=True)
    return os.path.join(d, f"{tag}_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv")


def write_run(path, robot, rows, meta):
    """rows must already be on the MCU clock (P.align_to_mcu_clock): the
    pi_time_s column is the MCU's sample time expressed on the Pi's clock."""
    ext = any(tel.extended for _, tel in rows)
    header = P.csv_header(robot.names, ext)
    with open(path, "w", newline="") as fh:
        w = csv.writer(fh)
        w.writerow(header)
        for t, tel in rows:
            vals = tel.values if ext else tel.values[:3 * robot.n]
            w.writerow([f"{t:.4f}", tel.mcu_ms] + [f"{v:g}" for v in vals])
    meta = dict(meta, motors=robot.names, firmware=robot.info, pwm_max=robot.pwm_max,
                csv=os.path.basename(path), columns=header, samples=len(rows))
    with open(path[:-4] + ".json", "w") as fh:
        json.dump(meta, fh, indent=2)
    print(f"\n    {len(rows)} samples → {path}")
    if rows:
        span = rows[-1][0] - rows[0][0]
        print(f"    {span:.1f} s, {len(rows) / span if span > 0 else 0:.1f} Hz achieved")
    return path


def series(rows, i, field):
    return [(t, getattr(tel, field)[i]) for t, tel in rows if getattr(tel, field) is not None]


# ═══════════════════════════════════════════════════════════════════
#  Tests
# ═══════════════════════════════════════════════════════════════════

def hold_pwm(link, robot, signs, pwm, seconds):
    link.send(P.cmd_pwm([s * pwm for s in signs]), 0)
    time.sleep(seconds)


def hold_velocity(link, robot, signs, w, seconds):
    """Resend the target at 20 Hz so the firmware watchdog stays fed, the
    same way the ROS bridge feeds it."""
    cmd = P.cmd_velocity([s * w for s in signs])
    t_end = time.time() + seconds
    while time.time() < t_end:
        link.send(cmd, 0)
        time.sleep(0.05)


def open_loop_prelude(link, hz):
    link.send("<E1>")
    link.send("<W0>")          # <M> holds longer than the watchdog
```

```
    link.send("<Y0>")          # trips assume a velocity target; open loop has none
    link.send(f"<L2,{hz}>")


def open_loop_postlude(link, robot):
    link.send(P.cmd_pwm([0] * robot.n))
    time.sleep(0.5)
    rows = P.align_to_mcu_clock(link.stop_collect())
    link.send("<L0>")
    link.send("<W1>")
    link.send("<Y1>")
    return rows


def test_plant(link, robot, args):
    print("\n=== PLANT IDENTIFICATION (open loop, PID bypassed) ===")
    signs = pattern(robot, args.mode, args.only)
    levels = args.plant_pwm or [int(robot.pwm_max * f) for f in (0.30, 0.45, 0.65)]
    open_loop_prelude(link, 50)
    link.start_collect()
    marks = []
    t0 = time.time()
    for pwm in levels:
        for sgn in (+1, -1):
            hold_pwm(link, robot, signs, 0, 1.5)
            t_step = time.time()
            link.send(P.cmd_pwm([sgn * s * pwm for s in signs]), 0)
            marks.append((t_step, sgn * pwm))
            print(f"    step → PWM {sgn * pwm:+5d}   ({time.time() - t0:5.1f} s)")
            time.sleep(args.plant_hold)
    rows = open_loop_postlude(link, robot)
    path = write_run(out_path(args, "plant"), robot, rows,
                     dict(test="plant", mode=args.mode, signs=signs, hold_s=args.plant_hold,
                          steps=[{"t": t, "pwm": p} for t, p in marks]))

    print("\n    tau(angle) is the motor itself, from the unfiltered wheel angle.")
    print("    tau(filtered) is what the PID sees, through the firmware's velocity filter.")
    print("\n    motor         PWM    v_ss(rad/s)   K=v/pwm   tau(angle)  tau(filtered)")
    result = {}
    for i, m in enumerate(robot.names):
        if signs[i] == 0:
            continue
        act = series(rows, i, "actual")
        rate = centered_rate(series(rows, i, "pos_rad"))
        ks, taus, ftaus, pts = [], [], [], []
        for t_step, pwm in marks:
            p = pwm * signs[i]
            v_ss = mean(window(act, t_step + args.plant_hold - 0.5, t_step + args.plant_hold))
            if math.isnan(v_ss) or abs(v_ss) < 0.05:
                print(f"    {m:<12} {p:+5d}   no motion")
                continue
            ftau = estimate_tau(act, t_step, v_ss)
            tau = estimate_tau(rate, t_step, v_ss) if rate else None
            ks.append(abs(v_ss / p))
            pts.append((abs(p), abs(v_ss)))
            if tau:
                taus.append(tau)
            if ftau:
                ftaus.append(ftau)
            print(f"    {m:<12} {p:+5d}   {v_ss:11.3f}   {abs(v_ss / p):8.5f}   "
                  f"{tau if tau else float('nan'):9.3f}  {ftau if ftau else float('nan'):12.3f}")
        result[m] = (ks, taus, pts, ftaus)

    print("\n    === WHAT TO DO WITH THIS (lambda = %.2f s) ===" % args.lam)
    print("    Kff = PWM per rad/s (line fit through the steps). Ki = Kff / lambda.")
    print("    Kp = tau * Ki is the textbook start; verify it with --test sweep.")
    for m, (ks, taus, pts, ftaus) in result.items():
        if not ks:
            print(f"    {m}: no usable steps. Check the wiring, motor_dir, and that the "
                  f"PWM levels are above breakaway.")
            continue
        fit = ols([p for p, _ in pts], [v for _, v in pts]) if len(pts) >= 2 else None
        if fit and fit[0] > 0:
            kff, kstat = 1.0 / fit[0], -fit[1] / fit[0]
        else:
            kff, kstat = 1.0 / mean(ks), 0.0
        tau = mean(taus) if taus else None
        ki = kff / args.lam
        line = f"    {m}: Kff {kff:6.2f}  Kstat {kstat:5.1f}  Ki {ki:6.1f}"
        if tau:
            line += f"  tau {tau:.3f} s  Kp {tau * ki:6.2f}"
            if ftaus:
```

```
                line += f"   (filter adds {mean(ftaus) - tau:+.3f} s)"
        else:
            line += "  tau not resolved (longer --plant-hold, or 50 Hz telemetry)"
        print(line)
    print("\n    Live:  <F,...> <K,...> <G,kp,ki,kd>.  Permanent: robot_config.yaml "
          "mcu.kff / kstat / pid, then generate.py and reflash.")
    return path


def test_staircase(link, robot, args):
    print("\n═══ STATIC FRICTION STAIRCASE (open loop) ═══")
    signs = pattern(robot, args.mode, args.only)
    lo = args.min_pwm if args.min_pwm is not None else max(1, int(robot.pwm_max * 0.01))
    hi = args.max_pwm if args.max_pwm is not None else int(robot.pwm_max * 0.35)
    step = args.pwm_step if args.pwm_step is not None else max(1, int(robot.pwm_max * 0.01))
    print(f"    PWM {lo} → {hi} step {step}, {args.dwell:.1f} s each, mode {args.mode}")
    open_loop_prelude(link, 50)
    link.start_collect()
    marks = []
    for pwm in range(lo, hi + 1, step):
        hold_pwm(link, robot, signs, 0, 0.6)
        t_step = time.time()
        link.send(P.cmd_pwm([s * pwm for s in signs]), 0)
        marks.append((t_step, pwm))
        time.sleep(args.dwell)
        print(f"    PWM {pwm:4d}", end="\r", flush=True)
    rows = open_loop_postlude(link, robot)
    path = write_run(out_path(args, "staircase"), robot, rows,
                     dict(test="staircase", mode=args.mode, signs=signs, dwell_s=args.dwell,
                          steps=[{"t": t, "pwm": p} for t, p in marks]))

    active = [i for i in range(robot.n) if signs[i] != 0]
    print("\n     PWM " + "".join(f"{robot.names[i]:>12}" for i in active))
    table = {i: [] for i in active}
    for t_step, pwm in marks:
        cells = []
        for i in active:
            v = mean(window(series(rows, i, "actual"), t_step + args.dwell - 0.4, t_step + args.dwell))
            table[i].append((pwm, v * signs[i]))
            cells.append(f"{v:12.3f}" if not math.isnan(v) else f"{'-':>12}")
        print(f"    {pwm:4d} " + "".join(cells))

    print("\n    ═══ LINE FIT  v = m*pwm + c   →   Kff = 1/m,  Kstat = -c/m ═══")
    kff, kstat = {}, {}
    for i in active:
        moving = [(p, v) for p, v in table[i] if not math.isnan(v) and v > 0.05]
        if len(moving) < 3:
            print(f"    {robot.names[i]}: only {len(moving)} moving points. Raise --max-pwm.")
            continue
        fit = ols([p for p, _ in moving], [v for _, v in moving])
        if not fit or fit[0] <= 0:
            print(f"    {robot.names[i]}: could not fit a positive slope.")
            continue
        kff[i], kstat[i] = 1.0 / fit[0], -fit[1] / fit[0]
        first = moving[0][0]
        print(f"    {robot.names[i]}: Kff {kff[i]:7.2f} PWM/(rad/s)   Kstat {kstat[i]:6.1f} PWM   "
              f"({len(moving)} points, first motion at PWM {first})")
    if len(kff) == len(active) == robot.n:
        print(f"\n    Live:  <F,{','.join(f'{kff[i]:.2f}' for i in range(robot.n))}>")
        print(f"           <K,{','.join(f'{max(0.0, kstat[i]):.1f}' for i in range(robot.n))}>")
        print("    Permanent: mcu.kff / mcu.kstat in robot_config.yaml, generate.py, reflash.")
    if args.mode == "rotate":
        print("    (rotate mode: each wheel still rolls on the floor, but rollers and tyres are\n"
              "     loaded at a slightly different angle than in a straight run.)")
    return path


def test_steps(link, robot, args, gains=None, tag="steps"):
    label = f"  Kp={gains[0]:g} Ki={gains[1]:g} Kd={gains[2]:g}" if gains else ""
    print(f"\n═══ CLOSED-LOOP STEP RESPONSE{label} ═══")
    signs = pattern(robot, "translate", args.only)
    setpoints = args.setpoints or [round(robot.vmax * f, 2) for f in (0.2, 0.4, 0.6, 0.85)]
    link.send("<E1>")
    link.send("<W1>")
    link.send("<Y1>")
    apply_feedforward(link, robot, args)
    if gains:
        link.send(P.cmd_gains(*gains))
    if args.no_slew:
        link.send("<A,0>")
        print("    slew limiter OFF: a true step into the controller")
    link.send("<L2,50>")
```

```
    link.start_collect()
    marks = []
    for sp in setpoints:
        hold_velocity(link, robot, signs, 0.0, args.rest)
        t_step = time.time()
        hold_velocity(link, robot, signs, sp, args.dwell)
        marks.append((t_step, sp))
        print(f"    setpoint {sp:.2f} rad/s")
    hold_velocity(link, robot, signs, 0.0, 1.0)
    rows = P.align_to_mcu_clock(link.stop_collect())
    link.send("<L0>")
    path = write_run(out_path(args, tag), robot, rows,
                     dict(test="steps", gains=gains, signs=signs, dwell_s=args.dwell,
                          no_slew=args.no_slew, steps=[{"t": t, "target": s} for t, s in marks]))

    sat_level = 0.98 * robot.pwm_max
    print("\n    setpt  motor          ss      err   overshoot  settle(s)  sat%")
    worst = 0.0
    summary = []
    for t_step, sp in marks:
        t_end = t_step + args.dwell
        for i, m in enumerate(robot.names):
            if signs[i] == 0:
                continue
            act = series(rows, i, "actual")
            pwm = series(rows, i, "pwm")
            ss = mean(window(act, t_end - 0.5, t_end))
            pk = max((abs(v) for v in window(act, t_step, t_end)), default=float("nan"))
            st = settling_time([(t, v) for t, v in act if t <= t_end], t_step, sp)
            win = window(pwm, t_step, t_end)
            sat = 100.0 * sum(1 for p in win if abs(p) >= sat_level) / len(win) if win else 0.0
            err = ss - sp
            worst = max(worst, abs(err))
            summary.append(dict(setpoint=sp, motor=m, ss=ss, overshoot=pk - sp, settle=st, sat=sat))
            print(f"    {sp:5.2f}  {m:<12} {ss:6.3f}  {err:+6.3f}  {pk - sp:+9.3f}  "
                  f"{(f'{st:.2f}' if st else 'none'):>9}  {sat:4.0f}")
    print(f"\n    worst steady-state error: {worst:.3f} rad/s")
    if worst > 0.15:
        print("    -> feedforward is off or Ki is too low. Run --test plant and refit.")
    if any(s["sat"] > 20 for s in summary):
        print("    -> PWM saturates on some steps: the top setpoint is beyond what this "
              "motor can hold. Lower max_wheel_speed (manual 9.8).")
    return path, summary


def apply_feedforward(link, robot, args):
    """--kff / --kstat: try fitted values live before making them permanent."""
    for letter, vals in (("F", args.kff), ("K", args.kstat)):
        if vals is None:
            continue
        if len(vals) != robot.n:
            raise RuntimeError(f"--{'kff' if letter == 'F' else 'kstat'} needs {robot.n} values")
        reply = link.query(P.cmd_per_motor(letter, vals), "OK," + ("FF" if letter == "F" else "KSTAT"))
        print(f"    {reply or 'no reply to <' + letter + ',...>'}")


def test_sweep(link, robot, args):
    results = []
    for g in args.gains:
        path, summary = test_steps(link, robot, args, gains=g,
                                   tag=f"sweep_kp{g[0]:g}_ki{g[1]:g}_kd{g[2]:g}")
        results.append((g, path, summary))
        hold_velocity(link, robot, [0] * robot.n, 0.0, 1.5)
    print("\n=== SWEEP SUMMARY ===")
    print("    Kp       Ki       Kd     mean overshoot   mean settle   worst |err|")
    for g, _, s in results:
        ov = mean(x["overshoot"] for x in s)
        st = [x["settle"] for x in s if x["settle"]]
        we = max(abs(x["ss"] - x["setpoint"]) for x in s)
        print(f"    {g[0]:<8g} {g[1]:<8g} {g[2]:<6g} {ov:14.3f}   "
              f"{(mean(st) if st else float('nan')):11.3f}   {we:11.3f}")
    print("\n    Prefer the lowest overshoot with settle under ~0.3 s and no saturation.")
    print("    On NarrowAisleBot the 'textbook' Kp = tau*Ki LOST to the existing Kp on\n"
          "    overshoot in 16 of 16 rows (manual 9.6). Let the sweep decide.")
    return [p for _, p, _ in results]


def bridge_running():
    try:
        out = subprocess.run(["pgrep", "-af", "mcu_bridge"], capture_output=True, text=True).stdout
        return [ln for ln in out.splitlines() if "pgrep" not in ln]
    except FileNotFoundError:
```

```
        return []


def test_record(link, robot, args):
    print("\n=== RECORD ===  (Ctrl-C to stop)")
    link.send(f"<L2,{args.hz:g}>")
    link.start_collect()
    t0 = time.time()
    try:
        while args.duration <= 0 or time.time() - t0 < args.duration:
            time.sleep(0.5)
            print(f"\r    {time.time() - t0:6.1f} s   {len(link.rows)} samples", end="", flush=True)
    except KeyboardInterrupt:
        print()
    rows = P.align_to_mcu_clock(link.stop_collect())
    link.send("<L0>")
    return write_run(out_path(args, "record"), robot, rows, dict(test="record"))


# ============================================================
#  raw mode: any firmware that prints CSV
# ============================================================

def test_raw(args):
    """Log every numeric CSV line. No protocol assumed."""
    import serial
    ticks = {}
    for item in (args.ticks or "").split(","):
        if item.strip():
            name, _, cpr = item.partition(":")
            ticks[name.strip()] = float(cpr)
    path = out_path(args, "raw")
    print(f"\n=== RAW LOG === {args.port} @ {args.baud}  ->  {path}")
    ser = serial.Serial(args.port, args.baud, timeout=0.1)
    time.sleep(args.open_delay)
    ser.reset_input_buffer()
    for c in args.send or []:
        ser.write(c.encode("ascii"))
        time.sleep(0.05)
    columns = args.columns.split(",") if args.columns else None
    ncol = len(columns) if columns else None
    rows, skipped = [], 0
    t0 = time.time()
    buf = b""
    try:
        while args.duration <= 0 or time.time() - t0 < args.duration:
            buf += ser.read(512)
            while b"\n" in buf:
                raw, buf = buf.split(b"\n", 1)
                t = time.time()
                line = raw.decode("ascii", "ignore").strip()
                if not line:
                    continue
                if line.startswith("#") and columns is None:
                    columns = [c.strip() for c in line[1:].split(",")]
                    ncol = len(columns)
                    print(f"    header from the board: {columns}")
                    continue
                try:
                    vals = [float(x) for x in line.split(",")]
                except ValueError:
                    skipped += 1
                    continue
                if ncol is None:
                    ncol = len(vals)
                    columns = [f"col{k}" for k in range(ncol)]
                if len(vals) != ncol:
                    skipped += 1
                    continue
                rows.append((t, vals))
            print(f"\r    {time.time() - t0:6.1f} s   {len(rows)} rows   {skipped} skipped",
                  end="", flush=True)
    except KeyboardInterrupt:
        pass
    finally:
        ser.close()
    print()
    missing = [k for k in ticks if not columns or k not in columns]
    if missing:
        print(f"    ! --ticks names not in the columns: {missing}")
    tcol = columns.index(args.time_col) if args.time_col and columns and args.time_col in columns else None
    derived = [k for k in ticks if columns and k in columns]
```

```
    header = ["pi_time_s"] + (columns or [])
    for k in derived:
        header += [f"{k}_rad", f"{k}_rads"]
    prev = None
    with open(path, "w", newline="") as fh:
        w = csv.writer(fh)
        w.writerow(header)
        for t, vals in rows:
            extra = []
            tt = vals[tcol] / 1000.0 if tcol is not None else t
            for k in derived:
                ang = vals[columns.index(k)] * 2 * math.pi / ticks[k]
                vel = ""
                if prev is not None and tt > prev[0]:
                    vel = f"{(ang - prev[1][k]) / (tt - prev[0]):.4f}"
                extra += [f"{ang:.5f}", vel]
            w.writerow([f"{t:.4f}"] + [f"{v:g}" for v in vals] + extra)
            prev = (tt, {k: vals[columns.index(k)] * 2 * math.pi / ticks[k] for k in derived})
    print(f"    {len(rows)} rows → {path}")
    return path


# ═══════════════════════════════════════════════════════════════
#  Main
# ═══════════════════════════════════════════════════════════════

def parse_gain_sets(text):
    out = []
    for chunk in text.split():
        parts = chunk.split(",")
        if len(parts) != 3:
            raise argparse.ArgumentTypeError(f"gain set '{chunk}' must be Kp,Ki,Kd")
        out.append(tuple(float(p) for p in parts))
    return out


def floats(text):
    return [float(x) for x in text.split(",") if x.strip()]


def ints(text):
    return [int(float(x)) for x in text.split(",") if x.strip()]


def main():
    ap = argparse.ArgumentParser(description="Encoder logger and PID calibration",
                                 formatter_class=argparse.RawDescriptionHelpFormatter,
                                 epilog=__doc__)
    ap.add_argument("--test", required=True,
                    choices=["plant", "staircase", "steps", "sweep", "record", "raw"])
    ap.add_argument("--config", default=os.path.join(KIT, "robot_config.yaml"),
                    help="robot_config.yaml (motor names, port, limits). 'none' to skip.")
    ap.add_argument("--port", default=None, help="default: /dev/<mcu.device_name> from the config")
    ap.add_argument("--baud", type=int, default=None)
    ap.add_argument("--logdir", default=None, help="default: dashboard.log_dir, else ~/robot_logs")
    ap.add_argument("--only", default=None, help="comma list of motor names to test; others stay at 0")
    ap.add_argument("--mode", choices=["translate", "rotate"], default="translate",
                    help="plant/staircase: rotate spins in place using the kinematics")
    ap.add_argument("--yes", action="store_true", help="do not ask before moving")
    ap.add_argument("--open-delay", type=float, default=2.0,
                    help="seconds to wait after opening the port (most boards reset)")
    ap.add_argument("-v", "--verbose", action="store_true")
    ap.add_argument("--lam", type=float, default=0.15, help="closed-loop time constant for Ki")

    ap.add_argument("--plant-pwm", type=ints, default=None, help="e.g. 70,110,160")
    ap.add_argument("--plant-hold", type=float, default=2.5)
    ap.add_argument("--min-pwm", type=int, default=None)
    ap.add_argument("--max-pwm", type=int, default=None)
    ap.add_argument("--pwm-step", type=int, default=None)
    ap.add_argument("--dwell", type=float, default=1.5)
    ap.add_argument("--setpoints", type=floats, default=None, help="rad/s, e.g. 1,2,3")
    ap.add_argument("--rest", type=float, default=1.5)
    ap.add_argument("--no-slew", action="store_true")
    ap.add_argument("--gains", type=parse_gain_sets, default=None,
                    help='sweep: "kp,ki,kd kp,ki,kd ..."; steps: one set')
    ap.add_argument("--kff", type=floats, default=None, help="steps/sweep: send <F,...> first")
    ap.add_argument("--kstat", type=floats, default=None, help="steps/sweep: send <K,...> first")
    ap.add_argument("--duration", type=float, default=0, help="record/raw: seconds, 0 = until Ctrl-C")
    ap.add_argument("--hz", type=float, default=50, help="record: telemetry rate")

    ap.add_argument("--columns", default=None, help="raw: column names, comma separated")
    ap.add_argument("--ticks", default=None, help="raw: name:counts_per_rev,... to add rad and rad/s")
```

```
    ap.add_argument("--time-col", default=None, help="raw: a millisecond time column from the board")
    ap.add_argument("--send", action="append", help="raw: a command to send first (repeatable)")
    args = ap.parse_args()

    cfg = None
    if args.config != "none" and load_config and os.path.exists(os.path.expanduser(args.config)):
        try:
            cfg = load_config(args.config)
        except ConfigError as e:
            print(f"  ! robot_config.yaml: {e}  (continuing without it)")
    if args.port is None:
        args.port = f"/dev/{cfg['mcu']['device_name']}" if cfg else "/dev/ttyUSB0"
    if args.baud is None:
        args.baud = int(cfg["mcu"]["baud"]) if cfg else 115200
    if args.logdir is None:
        args.logdir = (cfg or {}).get("dashboard", {}).get("log_dir", "~/robot_logs")
    if args.only:
        args.only = [s.strip() for s in args.only.split(",")]
    if args.test == "sweep" and not args.gains:
        ap.error('--test sweep needs --gains "Kp,Ki,Kd Kp,Ki,Kd ..."')

    if args.test == "raw":
        return 0 if test_raw(args) else 1

    if args.test == "record":
        others = bridge_running()
        if others:
            print("  ! the ROS bridge seems to be running:\n    " + "\n    ".join(others))
            print("    Two programs reading one serial port corrupt each other's data.\n"
                  "    Stop it (sudo systemctl stop robot.service), or log under ROS with\n"
                  "    the dashboard's REC button instead.")
            return 1

    if not args.yes and args.test != "record":
        print("\n" + "=" * 64)
        if args.test in ("plant", "staircase") and args.mode == "translate":
            print("  Open loop. WHEELS IN THE AIR. On the floor this drives in a")
            print("  straight line and keeps going: clear a runway or use --mode rotate.")
        else:
            print("  Wheels will turn. Confirm they are clear and the battery")
            print("  disconnect is in reach.")
        print("  Ctrl-C stops everything (<S>, <L0>, <E0>).")
        print("=" * 64)
        try:
            input("  Enter to start, Ctrl-C to abort: ")
        except (KeyboardInterrupt, EOFError):
            print("\n  aborted.")
            return 1

    link = None
    try:
        link = SerialLink(args.port, args.baud, open_delay=args.open_delay, verbose=args.verbose)
        robot = Robot(link, cfg)
        print(f"\n    connected: {args.port} @ {args.baud}   firmware: {robot.info or 'unknown'}")
        print(f"    motors: {', '.join(robot.names)}   PWM_MAX {robot.pwm_max}   vmax {robot.vmax} rad/s")
        if args.only:
            bad = [m for m in args.only if m not in robot.names]
            if bad:
                raise RuntimeError(f"--only {bad}: not a motor name ({robot.names})")

        def bail(*_):
            raise KeyboardInterrupt
        signal.signal(signal.SIGTERM, bail)

        {"plant": test_plant, "staircase": test_staircase,
         "steps": lambda l, r, a: test_steps(l, r, a, gains=a.gains[0] if a.gains else None)[0],
         "sweep": test_sweep, "record": test_record}[args.test](link, robot, args)
    except KeyboardInterrupt:
        print("\n\n    interrupted: stopping motors.")
    except Exception as e:  # noqa: BLE001
        print(f"\n\n    FAILED: {e}", file=sys.stderr)
        return 1
    finally:
        if link:
            for r in link.drain_replies():
                if "TRIP" in r[1] or "ERR" in r[1]:
                    print(f"    firmware said: {r[1]}")
            link.close()
            print("    motors stopped, port closed.")
    return 0
```

```
if __name__ == "__main__":
    sys.exit(main())
```

`tools/analyze_log.py` (134 lines)

```
#!/usr/bin/env python3
"""
analyze_log.py - summarise and plot a telemetry CSV.

Works on any CSV written by tools/encoder_logger.py (plant, staircase,
steps, sweep, record) or by the dashboard's REC button. Both use the same
header: pi_time_s, mcu_ms, then <motor>_target_rads, _actual_rads, _pwm per
motor, then optionally _error, _ff, _iterm, _pos_rad per motor.

    python3 analyze_log.py ~/robot_logs/steps_20261112_101500.csv
    python3 analyze_log.py run.csv --no-plot          # numbers only
    python3 analyze_log.py run.csv --pwm-max 1023     # 10-bit PWM

Prints, per motor:
    RMS tracking error   while the target is non-zero
    saturation %         samples at ≥ 98 % of PWM_MAX: no headroom left
    sign mismatch %      wheel turning against its target (wiring or slip)
    distance             integral of |actual| in radians and metres
Writes <csv>_plot.png beside the CSV if matplotlib is installed
(sudo apt install python3-matplotlib).
"""
import argparse
import csv
import json
import math
import os
import sys


def load(path):
    with open(path) as fh:
        r = csv.reader(fh)
        header = next(r)
        rows = [[float(x) if x not in ("", None) else float("nan") for x in row] for row in r if row]
    motors = [h[:-len("_target_rads")] for h in header if h.endswith("_target_rads")]
    col = {h: i for i, h in enumerate(header)}
    t0 = rows[0][0] if rows else 0.0
    data = {"t": [row[0] - t0 for row in rows]}
    for m in motors:
        for f in ("target_rads", "actual_rads", "pwm", "error", "ff", "iterm", "pos_rad"):
            k = f"{m}_{f}"
            if k in col:
                data[k] = [row[col[k]] if col[k] < len(row) else float("nan") for row in rows]
    return motors, data, t0


def summarise(motors, data, pwm_max, radius):
    t = data["t"]
    out = {}
    print(f"\n    {len(t)} samples over {t[-1] - t[0] if t else 0:.1f} s "
          f"({((len(t) - 1) / (t[-1] - t[0]) if len(t) > 1 and t[-1] > t[0] else 0:.1f} Hz)")
    print("\n    motor          RMS err   sat %   mismatch %   distance")
    for m in motors:
        tg, ac, pw = data[f"{m}_target_rads"], data[f"{m}_actual_rads"], data[f"{m}_pwm"]
        moving = [(a - b) ** 2 for a, b in zip(ac, tg) if abs(b) > 0.05]
        rms = math.sqrt(sum(moving) / len(moving)) if moving else float("nan")
        sat = 100.0 * sum(1 for p in pw if abs(p) ≥ 0.98 * pwm_max) / len(pw)
        mism = [1 for a, b in zip(ac, tg) if abs(b) > 0.2 and a * b < 0]
        mis = 100.0 * len(mism) / max(1, sum(1 for b in tg if abs(b) > 0.2))
        dist = sum(abs(ac[k]) * (t[k] - t[k - 1]) for k in range(1, len(t)))
        dist_s = f"{dist:7.2f} rad" + (f" = {dist * radius:.2f} m" if radius else "")
        print(f"    {m:<12} {rms:9.3f}   {sat:5.1f}   {mis:10.1f}   {dist_s}")
        out[m] = dict(rms=rms, sat=sat, mismatch=mis, dist_rad=dist)
        if sat > 10:
            print(f"      ! {m} saturates {sat:.0f} % of the time: not enough PWM for what was asked")
        if mis > 5:
            print(f"      ! {m} turns against its target {mis:.0f} % of the time: check motor_dir / encoder_dir")
    return out


def plot(path, motors, data, meta, t0):
    try:
        import matplotlib
        matplotlib.use("Agg")
```

```
        import matplotlib.pyplot as plt
    except ImportError:
        print("\n    (no matplotlib: sudo apt install python3-matplotlib for the plot)")
        return None
    n = len(motors)
    fig, axes = plt.subplots(n, 2, figsize=(12, 2.6 * n), sharex=True, squeeze=False)
    t = data["t"]
    marks = [s.get("t", 0) - t0 for s in (meta or {}).get("steps", [])]
    for i, m in enumerate(motors):
        a, b = axes[i]
        a.plot(t, data[f"{m}_target_rads"], color="#888888", lw=1.2, label="target")
        a.plot(t, data[f"{m}_actual_rads"], color="#1f5fbf", lw=1.0, label="actual")
        a.set_ylabel(f"{m}\nrad/s")
        b.plot(t, data[f"{m}_pwm"], color="#c0392b", lw=0.9, label="pwm")
        if f"{m}_ff" in data:
            b.plot(t, data[f"{m}_ff"], color="#27ae60", lw=0.8, label="feedforward")
            b.plot(t, data[f"{m}_iterm"], color="#8e44ad", lw=0.8, label="I term")
        for x in marks:
            a.axvline(x, color="#cccccc", lw=0.6, zorder=0)
            b.axvline(x, color="#cccccc", lw=0.6, zorder=0)
        if i == 0:
            a.legend(loc="upper right", fontsize=8)
            b.legend(loc="upper right", fontsize=8)
        a.grid(alpha=0.3)
        b.grid(alpha=0.3)
    axes[-1][0].set_xlabel("time (s)")
    axes[-1][1].set_xlabel("time (s)")
    fig.suptitle(os.path.basename(path), fontsize=10)
    fig.tight_layout()
    out = path[:-4] + "_plot.png"
    fig.savefig(out, dpi=110)
    print(f"\n    plot → {out}")
    return out


def main():
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("csv")
    ap.add_argument("--pwm-max", type=float, default=None, help="default: from the .json sidecar, else 255")
    ap.add_argument("--radius", type=float, default=None, help="wheel radius in m, to print metres")
    ap.add_argument("--no-plot", action="store_true")
    args = ap.parse_args()
    meta = None
    side = args.csv[:-4] + ".json"
    if os.path.exists(side):
        meta = json.load(open(side))
    pwm_max = args.pwm_max or (meta or {}).get("pwm_max", 255)
    motors, data, t0 = load(args.csv)
    if not motors:
        print("no <motor>_target_rads columns: is this a raw-mode CSV? Open it in a spreadsheet instead.")
        return 1
    print(f"\n  {args.csv}" + (f"   ({meta.get('test')} test)" if meta else ""))
    summarise(motors, data, pwm_max, args.radius)
    if not args.no_plot:
        plot(args.csv, motors, data, meta, t0)
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

`tools/firmware_sim/sim.cpp` (284 lines)

```
// sim.cpp - run robot_mcu.ino against simulated motors and encoders.
//
// The firmware source is compiled unchanged (with the mock Arduino.h next to
// this file). Each motor is a first-order plant with static friction:
//
//     steady state:  pwm = Kff * w + Kstat * sgn(w)      (no motion below Kstat)
//     dynamics:      dw/dt = (w_ss(pwm) - w) / tau
//
// and its encoder produces real quadrature edges that call the sketch's own
// interrupt handlers. Defaults give a plant that needs about 65 % PWM at
// MAX_WHEEL_SPEED, tau = 0.09 s (NarrowAisleBot's measured value).
//
// Two modes:
//
//   ./sim --pty
//       Real time. Prints "PTY /dev/pts/N". Point tools/encoder_logger.py or
```

```
//       the ROS bridge at that path and it behaves like a robot on a bench,
//       wheels in the air. Ctrl-C to stop.
//
//   ./sim --script file.txt [--until MS]
//       As fast as possible, for tests. Each script line is
//           <t_ms> <command>                e.g.  0 <E1>
//           <t0>-<t1>/<every> <command>     e.g.  100-3000/50 <V,2,-1>
//           <t_ms> !encflip <motor>         wire one encoder backwards
//       Every line the firmware prints comes out as "<t_ms> <line>".
//
// Options: --kff X --kstat X --tau X --seed N --noise X (fraction of w)

#include "Arduino.h"
#include "robot_config.h"

#include <fcntl.h>
#include <signal.h>
#include <stdio.h>
#include <termios.h>
#include <time.h>
#include <unistd.h>

#include <deque>
#include <random>
#include <string>
#include <vector>

SimSerial Serial;
void setup();
void loop();

namespace sim {
unsigned long now_us = 0;
int pin_level[64];
int pin_pwm[64];
void (*pin_isr[64])() = {nullptr};
std::deque<uint8_t> rx;
std::string tx_line;
std::vector<std::string> out_lines;
int pty_master = -1;
bool realtime = false;

void serial_write(const uint8_t* b, size_t n) {
  if (pty_master >= 0) {
    ssize_t w = ::write(pty_master, b, n);
    (void)w;
    return;
  }
  for (size_t k = 0; k < n; k++) {
    char c = (char)b[k];
    if (c == '\n') {
      printf("%lu %s\n", now_us / 1000UL, tx_line.c_str());
      tx_line.clear();
    } else if (c != '\r') {
      tx_line += c;
    }
  }
}
int serial_available() { return (int)rx.size(); }
int serial_read() {
  if (rx.empty()) return -1;
  int c = rx.front();
  rx.pop_front();
  return c;
}
}  // namespace sim

struct Plant {
  double w = 0, phi = 0;        // rad/s, rad (true wheel angle)
  long counts = 0;              // firmware counts produced so far (wheel sense)
  int gray = 0;                 // index into the Gray sequence
  double kff, kstat, tau;
  int enc_flip = 1;             // -1 = wired backwards (fault injection)
};

static const int GRAY[4][2] = {{0, 0}, {1, 0}, {1, 1}, {0, 1}};  // A leads B = forward

static double hw_pwm(int i) {
#if DRIVER_TYPE == DRIVER_PWM_DIR
  double duty = sim::pin_pwm[PIN_A[i]];
  return sim::pin_level[PIN_B[i]] ? -duty : duty;
#elif DRIVER_TYPE == DRIVER_PWM_IN1_IN2
```

```
  double duty = sim::pin_pwm[PIN_A[i]];
  int a = sim::pin_level[PIN_B[i]], b = sim::pin_level[PIN_C[i]];
  return (a && !b) ? duty : ((!a && b) ? -duty : 0.0);
#else
  return (double)sim::pin_pwm[PIN_A[i]] - (double)sim::pin_pwm[PIN_B[i]];
#endif
}

static void set_enc_pins(int i, int g) {
  sim::pin_level[ENC_A[i]] = GRAY[g][0];
  sim::pin_level[ENC_B[i]] = GRAY[g][1];
}

static void step_plants(std::vector<Plant>& P, double h, std::mt19937& rng, double noise) {
  std::normal_distribution<double> nd(0.0, 1.0);
  for (int i = 0; i < NUM_MOTORS; i++) {
    Plant& p = P[i];
    // Physical motor: the firmware multiplies by MOTOR_DIR before writing the
    // pins, the mounting multiplies by it again. A correct config cancels.
    double u = hw_pwm(i) * MOTOR_DIR[i];
    double au = fabs(u);
    double w_ss = 0;
    if (au > p.kstat) w_ss = (au - p.kstat) / p.kff * (u > 0 ? 1 : -1);
    if (fabs(p.w) < 1e-3 && au <= p.kstat) p.w = 0;      // static friction holds it
    else p.w += (w_ss - p.w) * (h / p.tau);
    double w_meas = p.w * (1.0 + noise * nd(rng));
    p.phi += w_meas * h;

    // Encoder: firmware counts in wheel sense = floor(phi * CPR / 2pi).
    long target = (long)floor(p.phi * ENCODER_CPR[i] / (2.0 * PI));
    int per_count = (ENCODER_DECODING == 4) ? 1 : 2;     // Gray transitions per count
    while (p.counts != target) {
      int dir = target > p.counts ? 1 : -1;
      // Electrical direction seen by the MCU = wheel sense * ENC_DIR * fault.
      int edir = dir * ENC_DIR[i] * p.enc_flip;
      for (int k = 0; k < per_count; k++) {
        int old = p.gray;
        p.gray = (p.gray + (edir > 0 ? 1 : 3)) % 4;
        set_enc_pins(i, p.gray);
        int changed = (GRAY[old][0] != GRAY[p.gray][0]) ? ENC_A[i] : ENC_B[i];
        if (sim::pin_isr[changed]) sim::pin_isr[changed]();
      }
      p.counts += dir;
    }
  }
}

struct ScriptCmd { unsigned long t0, t1, every; std::string cmd; };

static std::vector<ScriptCmd> load_script(const char* path) {
  std::vector<ScriptCmd> out;
  FILE* f = fopen(path, "r");
  if (!f) { perror(path); exit(2); }
  char line[512];
  while (fgets(line, sizeof line, f)) {
    std::string s(line);
    while (!s.empty() && (s.back() == '\n' || s.back() == '\r' || s.back() == ' ')) s.pop_back();
    if (s.empty() || s[0] == '#') continue;
    size_t sp = s.find(' ');
    if (sp == std::string::npos) continue;
    std::string when = s.substr(0, sp), cmd = s.substr(sp + 1);
    ScriptCmd c{0, 0, 0, cmd};
    size_t dash = when.find('-'), slash = when.find('/');
    if (dash != std::string::npos && slash != std::string::npos) {
      c.t0 = strtoul(when.c_str(), nullptr, 10);
      c.t1 = strtoul(when.c_str() + dash + 1, nullptr, 10);
      c.every = strtoul(when.c_str() + slash + 1, nullptr, 10);
    } else {
      c.t0 = c.t1 = strtoul(when.c_str(), nullptr, 10);
    }
    out.push_back(c);
  }
  fclose(f);
  return out;
}

static volatile bool g_stop = false;
static void on_sig(int) { g_stop = true; }

static double mono_s() {
  timespec ts;
  clock_gettime(CLOCK_MONOTONIC, &ts);
```

```
  return ts.tv_sec + ts.tv_nsec * 1e-9;
}

int main(int argc, char** argv) {
  const char* script = nullptr;
  bool pty = false;
  unsigned long until_ms = 10000;
  double kff = -1, kstat = -1, tau = 0.09, noise = 0.003;
  unsigned seed = 1;
  for (int a = 1; a < argc; a++) {
    std::string k = argv[a];
    auto next = [&]() { return a + 1 < argc ? argv[++a] : (char*)"0"; };
    if (k == "--pty") pty = true;
    else if (k == "--script") script = next();
    else if (k == "--until") until_ms = strtoul(next(), nullptr, 10);
    else if (k == "--kff") kff = atof(next());
    else if (k == "--kstat") kstat = atof(next());
    else if (k == "--tau") tau = atof(next());
    else if (k == "--noise") noise = atof(next());
    else if (k == "--seed") seed = (unsigned)atoi(next());
    else { fprintf(stderr, "unknown option %s\n", k.c_str()); return 2; }
  }
  if (!pty && !script) {
    fprintf(stderr, "usage: sim --pty | --script FILE [--until MS]\n");
    return 2;
  }
  if (kff < 0) kff = 0.65 * PWM_MAX / MAX_WHEEL_SPEED;
  if (kstat < 0) kstat = 0.03 * PWM_MAX;

  std::mt19937 rng(seed);
  std::vector<Plant> P(NUM_MOTORS);
  for (int i = 0; i < NUM_MOTORS; i++) {
    double spread = 1.0 + 0.03 * ((i % 3) - 1);       // motors differ by a few percent
    P[i].kff = kff * spread;
    P[i].kstat = kstat;
    P[i].tau = tau;
    set_enc_pins(i, 0);
  }
  for (int i = 0; i < 64; i++) sim::pin_pwm[i] = 0;

  if (pty) {
    sim::pty_master = posix_openpt(O_RDWR | O_NOCTTY);
    if (sim::pty_master < 0 || grantpt(sim::pty_master) || unlockpt(sim::pty_master)) {
      perror("pty");
      return 1;
    }
    const char* name = ptsname(sim::pty_master);
    int slave = open(name, O_RDWR | O_NOCTTY);      // held open: no hangup when a client leaves
    termios t;
    tcgetattr(slave, &t);
    cfmakeraw(&t);
    tcsetattr(slave, TCSANOW, &t);
    fcntl(sim::pty_master, F_SETFL, fcntl(sim::pty_master, F_GETFL) | O_NONBLOCK);
    printf("PTY %s\n", name);
    printf("#PLANT kff=%.3f kstat=%.3f tau=%.3f motors=%d (motor i kff x 0.97/1.00/1.03)\n",
           kff, kstat, tau, NUM_MOTORS);
    fflush(stdout);
    signal(SIGINT, on_sig);
    signal(SIGTERM, on_sig);
    sim::realtime = true;
  } else {
    printf("#PLANT kff=%.3f kstat=%.3f tau=%.3f motors=%d\n", kff, kstat, tau, NUM_MOTORS);
  }

  std::vector<ScriptCmd> cmds;
  std::vector<unsigned long> next_due;
  if (script) {
    cmds = load_script(script);
    for (auto& c : cmds) next_due.push_back(c.t0);
  }

  const unsigned long H_US = 50;
  const double t_start = mono_s();
  setup();
  while (!g_stop) {
    unsigned long ms = sim::now_us / 1000UL;
    if (script) {
      if (ms >= until_ms) break;
      for (size_t k = 0; k < cmds.size(); k++) {
        ScriptCmd& c = cmds[k];
        if (next_due[k] == (unsigned long)-1 || ms < next_due[k]) continue;
        if (c.cmd.rfind("!encflip", 0) == 0) {
```

```
          int m = atoi(c.cmd.c_str() + 9);
          if (m >= 0 && m < NUM_MOTORS) P[m].enc_flip = -P[m].enc_flip;
        } else {
          for (char ch : c.cmd) sim::rx.push_back((uint8_t)ch);
        }
        if (c.every && next_due[k] + c.every <= c.t1) next_due[k] += c.every;
        else next_due[k] = (unsigned long)-1;
      }
    } else {
      uint8_t buf[256];
      ssize_t n = ::read(sim::pty_master, buf, sizeof buf);
      for (ssize_t k = 0; k < n; k++) sim::rx.push_back(buf[k]);
      // Keep pace with the wall clock.
      double ahead = sim::now_us * 1e-6 - (mono_s() - t_start);
      if (ahead > 0.002) usleep((useconds_t)(ahead * 1e6));
    }
    step_plants(P, H_US * 1e-6, rng, noise);
    loop();
    sim::now_us += H_US;
  }
  return 0;
}
```

`tools/firmware_sim/Arduino.h` (56 lines)

```
// Arduino.h - a minimal stand-in for the Arduino core, so robot_mcu.ino can
// be compiled with g++ on a PC or on the Pi and run against simulated motors
// (sim.cpp). Only what the sketch uses is here.
//
// This is NOT ESP32 or AVR. It exercises the portable path of the sketch
// (loop() scheduling, interrupt encoders, analogWrite PWM): the controller,
// the parser, the telemetry, the safety trips. The ESP32-only pieces
// (PCNT, ledc, FreeRTOS tasks) are not compiled here.
#pragma once
#include <math.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include <cstddef>

#define HIGH 1
#define LOW 0
#define INPUT 0
#define OUTPUT 1
#define INPUT_PULLUP 2
#define CHANGE 1
#define NOT_AN_INTERRUPT -1
#ifndef PI
#define PI 3.14159265358979323846
#endif

namespace sim {
extern unsigned long now_us;
extern int pin_level[64];
extern int pin_pwm[64];
extern void (*pin_isr[64])();
void serial_write(const uint8_t* b, size_t n);
int serial_available();
int serial_read();
}

inline unsigned long micros() { return sim::now_us; }
inline unsigned long millis() { return sim::now_us / 1000UL; }
inline void pinMode(int, int) {}
inline void digitalWrite(int p, int v) { if (p >= 0 && p < 64) sim::pin_level[p] = v ? 1 : 0; }
inline int digitalRead(int p) { return (p >= 0 && p < 64) ? sim::pin_level[p] : 0; }
inline void analogWrite(int p, int v) { if (p >= 0 && p < 64) sim::pin_pwm[p] = v; }
inline int digitalPinToInterrupt(int p) { return (p >= 0 && p < 64) ? p : NOT_AN_INTERRUPT; }
inline void attachInterrupt(int irq, void (*fn)(), int) { if (irq >= 0 && irq < 64) sim::pin_isr[irq] = fn; }
inline void noInterrupts() {}
inline void interrupts() {}

struct SimSerial {
  void begin(long) {}
  int available() { return sim::serial_available(); }
  int read() { return sim::serial_read(); }
  size_t write(const uint8_t* b, size_t n) { sim::serial_write(b, n); return n; }
  int availableForWrite() { return 256; }
  explicit operator bool() const { return true; }
```

```
};
extern SimSerial Serial;
```

`tools/firmware_sim/Makefile` (18 lines)

```
# Build the firmware simulator from the real sketch and the generated header.
#
#   make            → ./sim  (uses ../../firmware/robot_mcu)
#   make FW=/path   → a sketch folder elsewhere (tests use this)
#
# Needs only g++ (sudo apt install build-essential).

FW ?= ../../firmware/robot_mcu
CXXFLAGS ?= -std=c++17 -O2 -Wall -Wno-unused-function -Wno-unused-variable

sim: sim.cpp Arduino.h $(FW)/robot_mcu.ino $(FW)/robot_config.h
    g++ $(CXXFLAGS) -DENCODER_BACKEND=ENC_ISR -I. -I$(FW) \
        -x c++ $(FW)/robot_mcu.ino -x c++ sim.cpp -o sim

clean:
    rm -f sim

.PHONY: clean
```